

EXPERT INSIGHT

Learning OpenCV 5 Computer Vision with Python

Tackle computer vision and machine learning
with the newest tools, techniques and algorithms



Fourth Edition



Joseph Howse
Joe Minichino

<packt>

Contents

1. [Learning OpenCV 5 Computer Vision with Python Fourth Edition](#)
[Tackle tools techniques and algorithms for computer vision and machine learning](#)
2. [Join our book community on Discord](#)
3. [Technical requirements](#)
4. [What's new in OpenCV 5](#)
5. [Optimizing OpenCV for specific hardware](#)
6. [Choosing and using the right setup tools](#)
7. [Running the official sample code](#)
8. [Finding documentation help and updates](#)
9. [Finding this book s sample code](#)
10. [Summary](#)
11. [Join our book community on Discord](#)
12. [Technical requirements](#)
13. [Basic I O scripts](#)
14. [Project Cameo \(face tracking and image manipulation\)](#)
15. [Cameo an object-oriented design](#)
16. [Summary](#)
17. [Join our book community on Discord](#)
18. [Technical requirements](#)
19. [Converting images between different color models](#)
20. [Exploring the Fourier transform](#)
21. [Creating modules](#)
22. [Edge detection](#)
23. [Custom kernels getting convoluted](#)
24. [Modifying the application](#)
25. [Edge detection with Canny](#)
26. [Contour detection](#)
27. [Detecting lines circles and other shapes](#)
28. [Summary](#)
29. [Join our book community on Discord](#)
30. [Technical requirements](#)
31. [Conceptualizing Haar cascades](#)
32. [Getting Haar cascade data](#)

33. [Using OpenCV to perform face detection](#)
34. [Performing face recognition](#)
35. [Swapping faces in infrared](#)
36. [Summary](#)
37. [Join our book community on Discord](#)
38. [Technical requirements](#)
39. [Understanding types of feature detection and matching](#)
40. [Detecting Harris corners](#)
41. [Detecting DoG features and extracting SIFT descriptors](#)
42. [Detecting Fast Hessian features and extracting SURF descriptors](#)
43. [Using ORB with FAST features and BRIEF descriptors](#)
44. [Filtering matches using K-Nearest Neighbors and the ratio test](#)
45. [Matching with FLANN](#)
46. [Finding homography with FLANN-based matches](#)
47. [A sample application tattoo forensics](#)
48. [Summary](#)
49. [Join our book community on Discord](#)
50. [Technical requirements](#)
51. [Understanding HOG descriptors](#)
52. [Understanding NMS](#)
53. [Understanding SVMs](#)
54. [Detecting people with HOG descriptors](#)
55. [Creating and training an object detector](#)
56. [Detecting cars](#)
57. [Summary](#)
58. [Join our book community on Discord](#)
59. [Technical requirements](#)
60. [Detecting moving objects with background subtraction](#)
61. [Tracking colorful objects using MeanShift and CamShift](#)
62. [Finding trends in motion using the Kalman filter](#)
63. [Tracking pedestrians](#)
64. [Summary](#)
65. [Join our book community on Discord](#)
66. [Technical requirements](#)
67. [Understanding 3D image tracking and augmented reality](#)
68. [Implementing the demo application](#)
69. [Improving the 3D tracking algorithm](#)

- 70. [Summary](#)
- 71. [Join our book community on Discord](#)
- 72. [Technical requirements](#)
- 73. [Understanding ANNs](#)
- 74. [Training a basic ANN in OpenCV](#)
- 75. [Training an ANN classifier in multiple epochs](#)
- 76. [Recognizing handwritten digits with an ANN](#)
- 77. [Using DNNs from other frameworks in OpenCV](#)
- 78. [Detecting and classifying objects with third-party DNNs](#)
- 79. [Detecting and classifying faces with third-party DNNs](#)
- 80. [Using OpenCV with MediaPipe and Tensorflow to classify gestures and actions](#)
- 81. [Summary](#)
- 82. [Join our book community on Discord](#)
- 83. [What are Containers](#)
- 84. [Docker Basics](#)
- 85. [AWS Serverless \(Fargate Lambda\) and AWS SAM](#)
- 86. [Conclusion](#)

[OceanofPDF.com](#)

Learning OpenCV 5 Computer Vision with Python, Fourth Edition: Tackle tools, techniques, and algorithms for computer vision and machine learning

Welcome to Packt Early Access . We're giving you an exclusive preview of this book before it goes on sale. It can take many months to write a book, but our authors have cutting-edge information to share with you today. Early Access gives you an insight into the latest developments by making chapter drafts available. The chapters may be a little rough around the edges right now, but our authors will update them over time.

You can dip in and out of this book or follow along from start to finish; Early Access is designed to be flexible. We hope you enjoy getting to know more about the process of writing a Packt book.

1. Chapter 1: Setting Up OpenCV
2. Chapter 2: Handling Files, Cameras, and GUIs
3. Chapter 3: Processing Images with OpenCV
4. Chapter 4: Depth Estimation and Segmentation
5. Chapter 5: Detecting and Recognizing Faces
6. Chapter 6: Retrieving Images and Searching Using Image Descriptors
7. Chapter 7: Building Custom Object Detector
8. Chapter 8: Tracking Objects
9. Chapter 9: Camera Models and Augmented Reality
10. Chapter 10: 3D Reconstruction and Navigation
11. Chapter 11: Introduction to Neural Networks with OpenCV
12. Chapter 12: OpenCV Applications at Scale

Join our book community on Discord

<https://discord.gg/djGjeMECxxw>



You've picked up this book, so you may already have an idea of what OpenCV is. Maybe you have heard of capabilities that seem to come straight out of science fiction, such as training an artificial intelligence model to recognize anything that it sees through a camera. If this is your interest, you will not be disappointed! **OpenCV** stands for **Open Source Computer Vision**. It is a free computer vision library that allows you to manipulate images and videos to accomplish a variety of tasks, from displaying frames from a webcam to teaching a robot to recognize real-life objects.

In this book, you will learn how to leverage the immense potential of OpenCV with the Python programming language. Python is an elegant language with a relatively shallow learning curve and very powerful features. This chapter is a quick guide to setting up Python 3, OpenCV 5, and other dependencies. As part of OpenCV, we will set up the `opencv_contrib` modules, which offer additional functionality that is maintained by the OpenCV community rather than the core development team. After the setup, we will also look at OpenCV's Python sample scripts and documentation.

The following related libraries are covered in this chapter:

- **NumPy** : This library is a dependency of OpenCV's Python bindings. It provides numeric computing functionality, including efficient arrays.
- **SciPy** : This library is a scientific computing library that is closely related to NumPy. It is not required by OpenCV, but it is useful if you wish to manipulate data in OpenCV images.
- **OpenNI 2** : This library is an optional dependency of OpenCV. It adds support for certain depth cameras, such as the Asus Xtion PRO.

For this book's purposes, OpenNI 2 can be considered optional. It is used throughout *Chapter 4 , Depth Estimation and Segmentation* , but is not used in the other chapters or appendices.

Optionally, if we build OpenCV with a custom configuration, we can include support for OpenNI 2, and we can even include OpenCV's implementations of some patented algorithms, which OpenCV calls *non-free* . Notably, one of our code samples in *Chapter 6 , Retrieving Images and Searching Using Image Descriptors* , relies on a patented algorithm called SURF. However, the rest of the book does not use any patented algorithms, so this non-free content can be considered optional, and in any case, patent restrictions may make it unsuitable for redistribution in your future projects.

This book focuses on OpenCV 5, the new major release of the OpenCV library. Additional information about OpenCV is available at <https://opencv.org/> , and the official documentation is available at <https://docs.opencv.org/master> .

We will cover the following topics in this chapter:

- What's new in OpenCV 5
- Choosing and using the right setup tools
- Running samples
- Finding documentation, help, and updates

Again, it is worth mentioning that this chapter includes optional instructions on how to build OpenCV from source code, with custom configurations that

are relevant to a few projects in this book. However, configuring and customizing OpenCV is a large topic, and the configuration options sometimes change between minor releases (such as OpenCV 5.1 versus 5.0), so advanced users may wish to refer to additional tutorials online.

One of the book's authors, Joseph Howse, maintains a GitHub repository of additional OpenCV build recipes at <https://github.com/JoeHowse/OpenCVBuildKit> .

Technical requirements

This chapter assumes that you are using one of the following operating systems:

- Windows 10 or a later version
- macOS 10.15 (Catalina) or a later version
- Debian 10 (Buster) or a later version, or a derivative such as the following:
 - Ubuntu 18.04 (Bionic Beaver) or a later version
 - Linux Mint 19 or a later version
- Arch Linux, or a derivative such as Manjaro

For editing Python scripts and other text files, this book's authors simply recommend that you should have a good text editor. **Visual Studio (VS)** Code is the most suitable editor because of its simplicity and almost all OS distributions support this editor. Other examples include the following:

- Notepad++ for Windows
- BBEdit (free version) for macOS
- GEdit for the GNOME desktop environment on Linux
- Kate for the KDE Plasma desktop environment on Linux

We also recommend that you have a Git client installed since this will be the easiest way to obtain the book's latest sample code, which is hosted on GitHub.

Besides the operating system, there are no other prerequisites for this setup chapter.

What's new in OpenCV 5

If you are an OpenCV veteran, you might want to know more about OpenCV 5's changes before you decide to install it. Here are some of the highlights:

- The C++ implementation of OpenCV has been updated to C++14. OpenCV's Python bindings wrap the C++ implementation, so as Python users, we may gain some performance advantages from this update, even though we are not using C++ directly.
- Support for Python 2 has been dropped because this Python version is obsolete and no longer maintained. If you have any old Python 2 projects, you should upgrade them to Python 3.
- For many algorithms, support for named parameters has been added in order to make the initialization code more concise and readable.
- For the `pip` package installer, the `opencv-python` and `opencv-contrib-python` packages are now maintained by the core OpenCV team rather than a third party. The `opencv-contrib-python-nonfree` package has been discontinued due to patent restrictions.
- The SIFT algorithm's patent has expired and SIFT is now included in standard builds of OpenCV.
- Many new optimizations have been implemented. Existing OpenCV 4 projects can take advantage of many of these optimizations without further changes beyond updating the OpenCV version.
- OpenCV's DNN module, in particular, is better optimized. Also, the loading and caching of DNN models are better automated.
- Support for RISC-V processors has been added.
- A new module has been added for 3D tracking and navigation. This module includes some new functionality, as well as old functionality that was previously located in `opencv_contrib`.
- Likewise, a new module has been added for 2D tracking, including some functionality that was previously located in `opencv_contrib`.

Whether or not you have used a previous version of OpenCV, this book will serve as a general guide to OpenCV 5, and some of the new features will

receive special attention in subsequent chapters.

Optimizing OpenCV for specific hardware

Generally, the setup instructions in this chapter are compatible with any hardware architecture that is supported by OpenCV and the operating system. For example, the setup instructions that target Linux operating systems will work equally well on x64, x86, ARM, or RISC-V hardware (except that the optional OpenNI package is unavailable for RISC-V).

That said, OpenCV supports many optional optimizations for specific hardware. By following the instructions in this chapter, you will get an OpenCV installation that uses a default set of optimizations for your hardware. The defaults are generally good, but for some applications, you could get even better performance with a more highly customized installation of OpenCV. Here are a few examples of use cases:

- For x64, x86, or ARM processors, OpenCV can optionally integrate with Intel Thread Building Blocks (TBB), a multithreading library. OpenCV can use TBB to speed up certain parallel algorithms, notably the Haar cascade classifiers that we will study in *Chapter 5 , Detecting and Recognizing Faces* .
- For NVIDIA graphics processors, OpenCV can optionally integrate with CUDA, another multiprocessing library. OpenCV can use CUDA to speed up various deep learning algorithms, as well as any kind of operation on very large blocks of data, such as very large images.
- For AMD graphics processors, OpenCV can optionally integrate with clBLAS and clFFT. These AMD libraries provide fine-tuned versions of certain OpenCL operations. OpenCL is another multiprocessing framework. Already, by default, OpenCV enables OpenCL optimizations on any compatible hardware, yet clBLAS and clFFT may make these optimizations perform even better on AMD hardware. Specifically, OpenCV can use clBLAS and clFFT to speed up two kinds of operations that are relevant to many signal processing applications: **General Matrix Multiply (GEMM)** and **Fast Fourier**

Transform (FFT). For example, GEMM is the fundamental operation in the Kalman filter, which we will use for motion stabilization in *Chapter 8 , Tracking Objects* , and *Chapter 9 , Camera Models and Augmented Reality* .

For examples of how to enable specific optimizations, see Joseph Howse's repository of OpenCV build recipes at <https://github.com/JoeHowse/OpenCVBuildKit> .

For now, let's proceed using default optimizations, which are fine for the purpose of running this book's sample projects on a wide range of hardware. However, bear in mind that many other optimization options are available to you for your future projects, where you may have specific performance requirements on specific hardware.

Choosing and using the right setup tools

We are free to choose various setup tools depending on our operating system and how much configuration we want to do.

Regardless of the choice of operating system, Python offers some built-in tools that are useful for setting up a development environment. These tools include a package manager called `pip` and a virtual environment manager called `venv`. Some of this chapter's instructions will cover `pip` specifically, but if you would like to learn about `venv`, please refer to the official Python documentation at <https://docs.python.org/3/library/venv.html>.

You should consider using `venv` if you plan to maintain a variety of Python projects that might have conflicting dependencies – for example, projects that depend on different versions of OpenCV. Each of `venv`'s virtual environments has its own set of installed libraries, and we can switch between these environments without reinstalling anything. Within a given virtual environment, libraries can be installed using `pip` or, in some cases, other tools.

Let's take an overview of the setup tools available for Windows, macOS, Ubuntu, Arch Linux, and other Unix-like systems.

Installation on Windows

Windows does not come with Python preinstalled. However, an installation wizard is available for Python, and Python provides a package manager called `pip`, which lets us easily install ready-made builds of NumPy, SciPy, and OpenCV. Alternatively, we can build OpenCV from source in order to enable nonstandard features, such as support for depth cameras via OpenNI 2 or support for patented algorithms. OpenCV's build system uses CMake for configuring the system and Visual Studio for compilation.

Before anything else, let's install Python. Go to <https://www.python.org/getit/> and download and run the most recent installer for Python 3.10. You probably want an installer for 64-bit Python, though OpenCV can work with 32-bit Python too.

Once Python has been installed, we can use `pip` to install NumPy and SciPy. Open the Command Prompt and run the following command:

Now, we must decide whether we want a ready-made build of OpenCV (without support for depth cameras) or a custom build (with support for depth cameras). The next two subsections cover these alternatives.

Using a ready-made OpenCV package

OpenCV, including the `opencv_contrib` modules, can be installed as a `pip` package. This is as simple as running the following command:

You might find that this `pip` package offers all the OpenCV features you currently want. On the other hand, if you intend to use depth cameras or patented algorithms, or if you just want to learn about the general process of making a custom build of OpenCV, you should *not* install the OpenCV `pip` package; you should proceed to the next subsection instead.

Building OpenCV from source

If you want support for depth cameras, you should also install OpenNI 2, which is available as a set of precompiled binaries with an installation wizard. Then, we must build OpenCV from source using CMake and Visual Studio.

To obtain OpenNI 2, go to <https://structure.io/openni> and download the latest ZIP for Windows and your system's architecture (x64 or x86). Unzip it to get an installer file, such as `OpenNI-Windows-x64-2.2.msi`. Run the installer. After the installation is complete, log out and log back in (or, alternatively, reboot) so that changes to the environment variables can take effect.

Now, let's set up Visual Studio. To build OpenCV 5, we need Visual Studio 2017 or a later version. If you do not already have a suitable version, go to <https://visualstudio.microsoft.com/downloads/> and download and run one of the installers for one of the following:

- The free Visual Studio 2022 Community edition
- Any of the paid Visual Studio 2022 editions, which have a 30-day trial period

During installation, ensure that any optional C++ components have been selected. After the installation finishes, reboot.

For OpenCV 5, the build configuration process requires CMake 3 or a later version. Go to <https://cmake.org/download/>, download the installer for the latest version of CMake for your architecture (x64 or x86), and run it. During installation, select either **Add CMake to the system PATH for all users** or **Add CMake to the system PATH for the current user**.

At this stage, we have set up the dependencies and build environment for our custom build of OpenCV. Now, we need to obtain the OpenCV source code and configure and build it. We can do this by following these steps:

1. Go to <https://opencv.org/releases/> and get the latest OpenCV download for Windows. It is a self-extracting ZIP. Run it and, when prompted, enter any destination folder, which we will refer to as `<opencv_unzip_destination>`. During extraction, a subfolder is created at `<opencv_unzip_destination>\opencv`.
2. Go to https://github.com/opencv/opencv_contrib/releases and download the latest ZIP of the `opencv_contrib` modules. Unzip this file to any destination folder, which we will refer to as `<opencv_contrib_unzip_destination>`.
3. Open the Command Prompt and run the following command to make another folder where our build will go:

1. Change the directory to the build folder:

1. Now, we are ready to configure our build with CMake's command-line interface. To understand all the options, we can read the code in

<opencv_unzip_destination>\opencv\CMakeLists.txt . However, for this book's purposes, we only need to use the options that will give us a release build with Python bindings, opencv_contrib modules, non-free content, and depth camera support via OpenNI 2. Some options differ slightly, depending on the Visual Studio version and target architecture (x64 or x86). To create a 64-bit (x64) solution for Visual Studio 2022, run the following command (but replace <opencv_contrib_unzip_destination> and <opencv_unzip_destination> with the actual paths):

Alternatively, to create a 32-bit (x86) solution for Visual Studio 2022, run the following command (but replace <opencv_contrib_unzip_destination> and <opencv_unzip_destination> with the actual paths):

As the preceding command runs, it prints information about dependencies that are either found or missing. OpenCV has many optional dependencies, so do not panic (yet) about missing dependencies. However, if the build does not finish successfully, try installing the missing dependencies. (Many are available as prebuilt binaries.) Then, repeat this step.

1. CMake will have generated a Visual Studio solution file in <opencv_build_folder>/OpenCV.sln . Open it in Visual Studio. Ensure that the **Release** configuration (not the **Debug** configuration) is selected in the drop-down list in the toolbar near the top of the Visual Studio window. (OpenCV's Python bindings will probably fail to build in **Debug** configuration because most Python distributions do not contain debug libraries.) Go to the **BUILD** menu and select **Build Solution** . Watch the build messages in the **Output** pane at the bottom of the window, and wait for the build to finish.
2. By this stage, OpenCV has been built, but it hasn't been installed at a location where Python can find it. Before proceeding further, let's ensure that our Python environment does not already contain a conflicting build of OpenCV. Find and delete any OpenCV files in Python's `DLLs` folder and `site_packages` folder. For example, these files might match the following patterns: `C:\Program Files\Python310\DLLs\opencv_*.dll` , `C:\Program`

- Files\Python310\Lib\site-packages\opencv , and C:\Program Files\Python310\Lib\site-packages\cv2.pyd .
3. Finally, let's install our custom build of OpenCV. For a 64-bit build, copy the following files into Python's `site_packages` folder:
- `<build_folder>\lib\python3\Release\*.pyd`
 - `<build_folder>\bin\Release\*.dll`
 - `C:\Program Files\OpenNI2\Redist\OpenNI2.dll`
 - `C:\Program Files\OpenNI2\Redist\OpenNI2` : Copy the whole folder so as to create a subfolder, `site_packages\OpenNI2`
4. Alternatively, for a 32-bit build, copy the following files into Python's `site_packages` folder:
- `<build_folder>\lib\python3\Release\*.pyd`
 - `<build_folder>\bin\Release\*.dll`
 - `C:\Program Files (x86)\OpenNI2\Redist\OpenNI2.dll`
 - `C:\Program Files (x86)\OpenNI2\Redist\OpenNI2` : Copy the whole folder so as to create a subfolder, `site_packages\OpenNI2`
- .

Now, we have completed the OpenCV build process on Windows, and we have a custom build that is suitable for all of this book's Python projects.

In the future, if you want to update to a new version of the OpenCV source code, repeat all the preceding steps, starting from downloading OpenCV.

Installation on macOS

macOS comes with a preinstalled Python distribution that has been customized by Apple for the system's internal needs. To develop our own projects, we should make a separate Python installation to ensure that we do not conflict with the system's Python needs.

For macOS, there are several possible approaches for obtaining standard versions of Python 3, NumPy, SciPy, and OpenCV. All approaches ultimately require OpenCV to be compiled from source using the Xcode command-line tools. However, depending on the approach, this task is automated for us in various ways by third-party tools. We will look at this

kind of approach using a package manager called Homebrew. A package manager can potentially do everything that CMake can, plus it helps us resolve dependencies and separate our development libraries from system libraries.

MacPorts is another popular package manager for macOS. However, at the time of writing, MacPorts does not offer packages for OpenCV 5 or OpenNI 2, so we will not use it in this book.

Before proceeding, let's make sure that the Xcode command-line tools are set up properly. Open a Terminal and run the following command:

Agree to the license agreement and any other prompts. The installation should run to completion. Now, we have the compilers that Homebrew requires.

Using Homebrew with ready-made packages

Starting on a system where Xcode and its command-line tools are already set up, the following steps will give us an OpenCV installation via Homebrew:

1. Open a Terminal and run the following command to install Homebrew:
1. Homebrew does not automatically put its executables in `PATH`. To do so, create or edit the `~/.profile` file and add the following line at the top of the code:

Save the file and run this command to refresh `PATH` :

Note that executables installed by Homebrew now take precedence over executables installed by the system.

1. For Homebrew's self-diagnostic report, run the following command:

Follow any troubleshooting advice it gives.

1. Now, update Homebrew:

1. Run the following command to install Python 3.10:

1. Now, we want to install OpenCV with the `opencv_contrib` modules. At the same time, we want to install dependencies such as NumPy. To accomplish this, run the following command:

Homebrew does not provide an option to install OpenCV with OpenNI 2 support. Homebrew always installs OpenCV with the `opencv_contrib` modules, including non-free content such as the patented SURF algorithm, which we will cover in *Chapter 6 , Retrieving Images and Searching Using Image Descriptors* . If you intend to distribute software that depends on OpenCV's non-free content, you should do your own investigation of how the patent and licensing issues might apply in specific countries and to specific use cases.

1. Similarly, run the following command to install SciPy:

Now, we have all we need to develop OpenCV projects with Python on macOS.

Using Homebrew with your own custom packages

Just in case you ever need to customize a package, Homebrew makes it easy to edit existing package definitions:

The package definitions are actually scripts in the Ruby programming language. Tips on editing them can be found on the Homebrew wiki page at <https://github.com/Homebrew/brew/blob/master/docs/Formula-Cookbook.md> . A script may specify Make or CMake configuration flags, among other things.

To see which CMake configuration flags are relevant to OpenCV, refer to <https://github.com/opencv/opencv/blob/master/CMakeLists.txt> in the official OpenCV repository on GitHub.

After making edits to the Ruby script, save it.

The customized package can be treated as normal. For example, it can be installed as follows:

Installation on Debian, Ubuntu, Linux Mint, and similar systems

Debian, Ubuntu, Linux Mint, and related Linux distributions use the apt package manager. On these systems, it is easy to install packages for Python 3 and many Python modules, including NumPy and SciPy. An OpenCV package is also available via apt , but at the time of writing, this package has not been updated to OpenCV 5. Instead, we can obtain OpenCV 5 (without support for depth cameras) from Python's standard package manager, pip . Alternatively, we can build OpenCV 5 from source. When built from source, OpenCV can support depth cameras via OpenNI 2, which is available as a set of precompiled binaries with an installation script.

Regardless of our approach to obtaining OpenCV, let's begin by updating apt so that we can obtain the latest packages. Open a Terminal and run the following command:

Having updated apt , let's run the following command to install NumPy and SciPy for Python 3:

Equivalently, we could have used the Ubuntu Software Center, which is the apt package manager's graphical frontend.

Now, we must decide whether we want a ready-made build of OpenCV (without support for depth cameras or patented algorithms) or a custom build (with support for depth cameras and patented algorithms). The next two subsections cover these alternatives.

Using a ready-made OpenCV package

OpenCV, including the `opencv_contrib` modules, can be installed as a pip package. This is as simple as running the following command:

You might find that this `pip` package offers all the OpenCV features you currently want. On the other hand, if you intend to use depth cameras or patented algorithms, or if you just want to learn about the general process of making a custom build of OpenCV, you should *not* install the OpenCV `pip` package; you should proceed to the next subsection instead.

Building OpenCV from source

To build OpenCV from source, we need a C++ build environment and the CMake build configuration system. Specifically, we need CMake 3. Run the following commands to ensure that the necessary versions of CMake and other build tools are installed:

As part of the build process for OpenCV, CMake will need to access the internet to download additional dependencies. If your system uses a proxy server, ensure that your environment variables for the proxy server have been configured properly. Specifically, CMake relies on the `http_proxy` and `https_proxy` environment variables. To define them, you can edit your `~/.bash_profile` script and add lines such as the following (but modify them so that they match your own proxy URLs and port numbers):

If you are unsure whether your system uses a proxy server, it probably doesn't, so you can ignore this step.

To build OpenCV's Python bindings, we need an installation of the Python 3 development headers. To install them, run the following command:

To capture frames from typical USB webcams, OpenCV depends on **Video for Linux (V4L)**. On most systems, V4L comes preinstalled, but just in case it is missing, run the following command:

As we mentioned previously, to support depth cameras, OpenCV depends on OpenNI 2. Go to <https://structure.io/openni> and download the latest ZIP of OpenNI 2 for Linux and your system's architecture (x64, x86, or ARM). Unzip it to any destination, which we will refer to as `<openni2_unzip_destination>`. Run the following commands:

The preceding installation script configures the system so that it supports depth cameras as USB devices.

Now that we have the build environment and dependencies installed, we can obtain and build the OpenCV source code. To do so, follow these steps:

1. Go to <https://opencv.org/releases/> and download the latest source package. Unzip it to any destination folder, which we will refer to as `<opencv_unzip_destination>` .
2. Go to https://github.com/opencv/opencv_contrib/releases and download the latest source package for the `opencv_contrib` modules. Unzip it to any destination folder, which we will refer to as `<opencv_contrib_unzip_destination>` .
3. Open a Terminal. Run the following commands to create a directory where we will put our OpenCV build:
 1. Change into the newly created directory:
 1. Run the following commands to define temporary environment variables that will help CMake locate the OpenNI 2 headers and binaries:
 1. Now, we can use CMake to generate a build configuration for OpenCV. The output of this configuration process will be a set of Makefiles, which are scripts we can use to build and install OpenCV. A complete set of CMake configuration options for OpenCV is defined in the `<opencv_unzip_destination>/opencv/sources/CMakeLists.txt` file. For our purposes, we care about the options that relate to OpenNI 2 support, Python bindings, `opencv_contrib` modules, and non-free content. Configure OpenCV by running the following command:
 1. Finally, run the following commands to interpret our newly generated Makefiles, and thereby build and install OpenCV:

So far, we have completed the OpenCV build process on Debian, Ubuntu, or a similar system, and we have a custom build that is suitable for all of this book's Python projects.

Installation on Arch Linux, Manjaro, and similar systems

Arch Linux, Manjaro, and related Linux distributions use the `pacman` package manager. On these systems, it is easy to install packages for Python 3 and many Python modules, including NumPy and SciPy. An OpenCV package is also available via `pacman`, and it is up-to-date with OpenCV 5. As an alternative to using the `pacman` OpenCV package, we can build OpenCV 5 from source. When built from source, OpenCV can support depth cameras via OpenNI 2, which is available as a set of precompiled binaries with an installation script.

Regardless of our approach to obtaining OpenCV, let's begin by updating `pacman` and the whole system. Open a Terminal and run the following command:

Having updated the system, let's run the following command to install NumPy and SciPy for Python 3:

Equivalently, on Manjaro, we could have used Pamac (also known as Add/Remove Software), which is a package manager with a graphical frontend.

Now, we must decide whether we want a ready-made build of OpenCV (without support for depth cameras or patented algorithms) or a custom build (with support for depth cameras and patented algorithms). The next two subsections cover these alternatives.

Using a ready-made OpenCV package

OpenCV, including the `opencv_contrib` modules, can be installed as a `pacman` package. This is as simple as running the following command:

You might find that this `pip` package offers all the OpenCV features you currently want. On the other hand, if you intend to use depth cameras or patented algorithms, or if you just want to learn about the general process of

making a custom build of OpenCV, you should *not* install the OpenCV `pacman` package; you should proceed to the next subsection instead.

Building OpenCV from source

To build OpenCV from source, we need a C++ build environment and the CMake build configuration system. Specifically, we need CMake 3. Run the following commands to ensure that the necessary versions of CMake and other build tools are installed:

As part of the build process for OpenCV, CMake will need to access the internet to download additional dependencies. If your system uses a proxy server, ensure that your environment variables for the proxy server have been configured properly. Specifically, CMake relies on the `http_proxy` and `https_proxy` environment variables. To define them, you can edit your `~/.bash_profile` script and add lines such as the following (but modify them so that they match your own proxy URLs and port numbers):

If you are unsure whether your system uses a proxy server, it probably doesn't, so you can ignore this step.

To capture frames from typical USB webcams, OpenCV depends on **Video for Linux (V4L)**. On most systems, V4L comes preinstalled, but just in case it is missing, run the following command:

As we mentioned previously, to support depth cameras, OpenCV depends on OpenNI 2. Go to <https://structure.io/openni> and download the latest ZIP of OpenNI 2 for Linux and your system's architecture (x64, x86, or ARM). Unzip it to any destination, which we will refer to as `<openni2_unzip_destination>` . Run the following commands:

The preceding installation script configures the system so that it supports depth cameras as USB devices.

Now that we have the build environment and dependencies installed, we can obtain and build the OpenCV source code. To do so, follow these steps:

1. Go to <https://opencv.org/releases/> and download the latest source package. Unzip it to any destination folder, which we will refer to as `<opencv_unzip_destination>` .
2. Go to https://github.com/opencv/opencv_contrib/releases and download the latest source package for the `opencv_contrib` modules. Unzip it to any destination folder, which we will refer to as `<opencv_contrib_unzip_destination>` .
3. Open a Terminal. Run the following commands to create a directory where we will put our OpenCV build:
 1. Change into the newly created directory:
 1. Run the following commands to define temporary environment variables that will help CMake locate the OpenNI 2 headers and binaries:
 1. Now, we can use CMake to generate a build configuration for OpenCV. The output of this configuration process will be a set of Makefiles, which are scripts we can use to build and install OpenCV. A complete set of CMake configuration options for OpenCV is defined in the `<opencv_unzip_destination>/opencv/sources/CMakeLists.txt` file. For our purposes, we care about the options that relate to OpenNI 2 support, Python bindings, `opencv_contrib` modules, and non-free content. Configure OpenCV by running the following command:
 1. Now, run the following commands to interpret our newly generated Makefiles, and thereby build and install OpenCV:
 1. A final step is necessary because OpenCV's installer uses a different `Python site_modules` path than the standard one for Arch Linux or Manjaro. To resolve this difference, create a link as follows:

At this point, we have completed the OpenCV build process on Arch Linux, Manjaro, or a similar system, and we have a custom build that is suitable for all of this book's Python projects.

Installation on other Unix-like systems

On other Unix-like systems, the package manager and available packages may differ. Consult your package manager's documentation and search for packages with `opencv` in their names. Remember that OpenCV and its Python bindings might be split into multiple packages.

Also, look for any installation notes that have been published by the system provider, the repository maintainer, or the community. Since OpenCV uses camera drivers and media codecs, getting all of its functionality to work can be tricky on systems with poor multimedia support. Under some circumstances, system packages might need to be reconfigured or reinstalled for compatibility.

If packages are available for OpenCV, check their version number. OpenCV 5 is recommended for this book's purposes. Also, check whether the packages offer Python bindings, depth camera support via OpenNI 2, and implementations of the non-free algorithms. Finally, check whether anyone in the developer community has reported success or failure when using the packages.

If, instead, you want to do a custom build of OpenCV from source, it might be helpful to refer to the previous sections' steps for Debian or Arch Linux and adapt these steps to the package manager and packages that are present on another system.

Running the official sample code

Running a few sample scripts is a good way to test whether OpenCV has been set up correctly. Some samples are included in OpenCV's source code archive. If you have not already obtained the source code, go to <https://opencv.org/releases/> and download one of the following archives:

- For Windows, download the latest archive, labeled **Windows** . It is a self-extracting ZIP. Run it and, when prompted, enter any destination folder, which we will refer to as `<opencv_unzip_destination>` . Find the Python samples in `<opencv_unzip_destination>/opencv/sources/samples/python` .
- For other systems, download the latest archive, labeled **Sources** . It is a ZIP file. Unzip it to any destination folder, which we will refer to as `<opencv_unzip_destination>` . Find the Python samples in `<opencv_unzip_destination>/samples/python` .

Some of the sample scripts require command-line arguments. However, the following scripts (among others) should work without any arguments:

- `hist.py` : This script displays a photo. Press A , B , C , D , or E to see variations of the photo, along with a corresponding histogram of color or grayscale values.
- `opt_flow.py` : This script displays a webcam feed with a superimposed visualization of the optical flow or, in other words, the direction of motion. Slowly wave your hand at the webcam to see the effect. Press 1 or 2 for alternative visualizations.

To exit a script, press Esc (not the window's close button).

If we encounter the `ImportError: No module named cv2` message, then this means that we are running the script from a Python installation that does not know anything about OpenCV. There are two possible explanations for this:

- Some steps in the OpenCV installation might have failed or been missed. Go back and review the steps.
- If we have multiple Python installations on the machine, we might be using the wrong version of Python to launch the script. For example, on macOS, it might be the case that OpenCV has been installed for Homebrew Python, but we are running the script with the system's version of Python. Go back and review the installation steps about editing the system's `PATH` variable. Also, try launching the script manually from the command line using commands such as the following:
- You can also try the following command:

As another possible means of selecting a different Python installation, try editing the sample script to remove the `#!` lines. These lines might explicitly associate the script with the wrong Python installation (for our particular setup).

Finding documentation, help, and updates

OpenCV's documentation can be found at <http://docs.opencv.org/> , where you can either read it online or download it for offline reading. If you write code on airplanes or other places without internet access, you will definitely want to keep offline copies of the documentation.

The documentation includes a combined API reference for OpenCV's C++ API and its Python API. When you look up a class or function, be sure to read the section under the heading **Python** .

OpenCV's Python module is named `cv2` . The **2** in `cv2` has nothing to do with the version number of OpenCV; we really are using OpenCV 5. Historically, OpenCV had a `cv` Python module that wrapped a now-obsolete C version of the library. The `cv` module was removed in OpenCV 4.

Still, in the official OpenCV 5 samples and in some other authors' samples, you might see valid Python code such as the following, which imports the `cv2` module and gives it a local alias, `cv` :

Here, there is nothing special about the alias `cv` ; countless other aliases would also be valid, and the choosing of an alias will have no effect on the module's version or functionality. For example, the alias `george_eliot_42` would be valid, though not sensible:

You might even observe that the OpenCV 5 documentation follows an unstated convention of using `cv` as an alias for `cv2` . Despite all of this, remember that in OpenCV 5, the Python module's real name is `cv2` .

If the documentation does not seem to answer your questions, try talking to the OpenCV community. Here are some sites where you will find helpful

people:

- The OpenCV forum: <https://forum.opencv.org/>
- Adrian Rosebrock's website: <https://www.pyimagesearch.com/>
- Joseph Howse's website for his books and presentations:
<https://nummist.com/opencv/>

Lastly, if you are an advanced user who wants to try new features, bug fixes, and sample scripts from the latest (unstable) OpenCV source code, have a look at the project's repository at <https://github.com/opencv/opencv/>

.

Finding this book's sample code

We will walk through a set of relevant code samples in each chapter of this book. For now, let's just take note of the samples' location and format.

All of this book's samples can be found online in the Git repository at <https://github.com/PacktPublishing/Learning-OpenCV-5-Computer-Vision-with-Python-Fourth-Edition/> . The contents are organized into various folders by chapter. For some chapters, an interactively editable version of the code is provided as a Jupyter notebook. For example, Chapter 5 has a Jupyter notebook located in the repository at `chapter05/chapter05.ipynb` .

If you are new to Git or want to expand your Git skills, take a look at the free ebook *Pro Git* , by Scott Chacon and Ben Straub. You can find it online at <https://git-scm.com/book/> . The book's second chapter, *Git Basics* , is especially helpful for beginners.

Typically, when you start studying a chapter, you might want to pull down the latest code from GitHub, just in case the authors have made any recent corrections or updates. Also, if the chapter has a Jupyter notebook, you might want to open it online via the Google Colab website, which enables you to experiment with your own modifications to the code in a readymade environment, without the need to set up or save anything on your local machine. Where applicable, the chapter's *Technical requirements* section will specify a Google Colab URL where you can open the chapter's Jupyter notebook. For example, for Chapter 5, the relevant Google Colab URL is <https://colab.research.google.com/github/PacktPublishing/Learning-OpenCV-5-Computer-Vision-with-Python-Fourth-Edition/blob/main/chapter05/chapter05.ipynb> .

Advanced readers might want to experiment with some of the book's sample code right away. Otherwise, the rest of this book will be your guide!

Summary

By now, we should have an OpenCV installation that will serve our needs for the diverse projects described in this book. Depending on which approach we took, we may also have a set of tools and scripts that can be used to reconfigure and rebuild OpenCV for our future needs.

We also know where to find this book's sample code and OpenCV's official Python samples. The official samples cover a different range of functionalities outside this book's scope, but they are useful as additional learning aids.

In the next chapter, we will familiarize ourselves with the most basic functions of the OpenCV API, namely displaying images and videos, capturing videos through a webcam, and handling basic keyboard and mouse inputs.

Join our book community on Discord

<https://discord.gg/djGjeMECxx>



Installing OpenCV and running samples is fun, but at this stage, we want to try things out in our own way. This chapter introduces OpenCV's I/O functionality. We also discuss the concept of a project and the beginnings of an object-oriented design for this project, which we will flesh out in subsequent chapters.

By starting with a look at I/O capabilities and design patterns, we will build our project in the same way we would make a sandwich: from the outside in. Bread slices and spread, or endpoints and glue, come before fillings or algorithms. We choose this approach because computer vision is mostly extroverted, it contemplates the real world outside our computer, and we want to apply all of our subsequent algorithmic work to the real world through a common interface.

Specifically, in this chapter, our code samples and discussions will cover the following tasks:

- Reading images from image files, video files, camera devices, or raw bytes of data in memory

- Writing images to image files or video files
- Manipulating image data in NumPy arrays
- Converting between NumPy arrays and OpenCV's `UMat` format, which supports OpenCL acceleration
- Enabling hardware acceleration of video I/O
- Sending a video stream over a network
- Displaying images in windows
- Handling keyboard and mouse input
- Implementing an application with an object-oriented design

Technical requirements

This chapter uses Python, OpenCV, and NumPy. Please refer back to *Chapter 1 , Setting Up OpenCV* , for installation instructions.

The complete code for this chapter can be found in this book's GitHub repository, <https://github.com/PacktPublishing/Learning-OpenCV-5-Computer-Vision-with-Python-Fourth-Edition> , in the Chapter02 folder.

Basic I/O scripts

Most CV applications need to get images as input. Most also produce images as output. An interactive CV application might require a camera as an input source and a window as an output destination. However, other possible sources and destinations include image files, video files, and raw bytes. For example, raw bytes might be transmitted via a network connection, or they might be generated by an algorithm, if we incorporate procedural graphics into our application. Let's look at each of these possibilities.

Reading/writing an image file

OpenCV provides the `imread` function to load an image from a file and the `imwrite` function to write an image to a file. These functions support various file formats for still images (not videos). The supported formats vary—as formats can be added or removed in a custom build of OpenCV—but normally BMP, PNG, JPEG, and TIFF are among the supported formats.

Let's explore the anatomy of the representation of an image in OpenCV and NumPy. An image is a multidimensional array; it has columns and rows of pixels, and each pixel has a value. For different kinds of image data, the pixel value may be formatted in different ways. For example, we can create a 3x3 square black image from scratch by simply creating a 2D NumPy array:

If we print this image to a console, we obtain the following result:

Here, each pixel is represented by a single 8-bit integer, which means that the values for each pixel are in the 0-255 range, where 0 is black, 255 is white, and the in-between values are shades of gray. This is a grayscale image.

Let's now convert this image into **blue-green-red (BGR)** format using the `cv2.cvtColor` function:

Let's observe how the image has changed:

As you can see, each pixel is now represented by a three-element array, with each integer representing one of the three color channels: B, G, and R, respectively. Other common color models, such as **hue-saturation-value (HSV)**, will be

represented in the same way, albeit with different value ranges. For example, the hue value of the HSV color model has a range of 0-180.

For more information about color models, refer to *Chapter 3 , Processing Images with OpenCV* , specifically the *Converting between different color models* section .

You can check the structure of an image by inspecting the `shape` property, which returns rows, columns, and the number of channels (if there is more than one).

Consider this example:

The preceding code will print `(5, 3)` ; in other words, we have a grayscale image with 5 rows and 3 columns. If you then converted the image into BGR, the shape would be `(5, 3, 3)` , which indicates the presence of three channels per pixel.

Images can be loaded from one file format and saved to another. For example, let's convert an image from PNG into JPEG:

OpenCV's Python module is called `cv2` even though we are using OpenCV 5.x and not OpenCV 2.x. Historically, OpenCV had two Python modules: `cv2` and `cv` . The latter wrapped a legacy version of OpenCV implemented in C. Nowadays, OpenCV has only the `cv2` Python module, which wraps the current version of OpenCV implemented in C++.

If `imread` fails to read the specified image file (for example, if the file does not exist or does not match any supported image file format), then `imread` returns `None` ; otherwise, it returns the image as a NumPy array. If `imwrite` fails to write the specified image file, `imwrite` returns `False` ; otherwise, it returns `True` .

By default, `imread` returns an image in the BGR color format even if the file uses a grayscale format. BGR represents the same color model as the widely-used **red-green-blue (RGB)** , but the byte order is reversed. If you are using OpenCV functions for all your I/O needs, you do not need to worry about byte order differences (OpenCV will handle these internally for you), but bear in mind that other file I/O libraries or GUI libraries may expect image data to be in RGB order instead of BGR. Then, to reverse the byte order, you can use `cv2.cvtColor` with the `cv2.COLOR_BGR2RGB` flag or the `cv2.COLOR_RGB2BGR` flag.

Optionally, we may specify the mode of `imread` . The supported options include the following:

- `cv2.IMREAD_COLOR` : This is the default option, providing a 3-channel BGR image with an 8-bit value (0-255) for each channel.
- `cv2.IMREAD_GRAYSCALE` : This provides an 8-bit grayscale image.
- `cv2.IMREAD_ANYCOLOR` : This provides either an 8-bit-per-channel BGR image or an 8-bit grayscale image, depending on the metadata in the file.
- `cv2.IMREAD_UNCHANGED` : This reads all of the image data, including the alpha or transparency channel (if there is one) as a fourth channel.
- `cv2.IMREAD_ANYDEPTH` : This loads an image in grayscale at its original bit depth. For example, it provides a 16-bit-per-channel grayscale image if the file represents an image in this format.
- `cv2.IMREAD_ANYDEPTH | cv2.IMREAD_COLOR` : This combination loads an image in BGR color at its original bit depth.
- `cv2.IMREAD_REduced_GRAYSCALE_2` : This loads an image in grayscale at half its original resolution. For example, if the file contains a 640 x 480 image, it is loaded as a 320 x 240 image.
- `cv2.IMREAD_REduced_COLOR_2` : This loads an image in 8-bit-per-channel BGR color at half its original resolution.
- `cv2.IMREAD_REduced_GRAYSCALE_4` : This loads an image in grayscale at one-quarter of its original resolution.
- `cv2.IMREAD_REduced_COLOR_4` : This loads an image in 8-bit-per-channel color at one-quarter of its original resolution.
- `cv2.IMREAD_REduced_GRAYSCALE_8` : This loads an image in grayscale at one-eighth of its original resolution.
- `cv2.IMREAD_REduced_COLOR_8` : This loads an image in 8-bit-per-channel color at one-eighth of its original resolution.

As an example, let's load a PNG file as a grayscale image (losing any color information in the process), and then save it as a grayscale PNG image:

The path of an image, unless absolute, is relative to the working directory (which is usually the path from which the Python script is run), so, in the preceding example, `MyPic.png` would have to be in the working directory or the image would not be found. If you prefer to avoid assumptions about the working directory, you can use absolute paths, such as
"C:\\Users\\Joe\\Pictures\\MyPic.png" on Windows,
"/Users/Joe/Pictures/MyPic.png" on Mac, or
"/home/joe/pictures/MyPic.png" on Linux.

The `imwrite` function requires an image to be in the BGR or grayscale format with a certain number of bits per channel that the output format can support. For example, the BMP file format requires 8 bits per channel, while PNG allows either 8 or 16 bits per channel.

Some image file formats, notably **multipage TIFF**, can contain multiple images in one file. OpenCV can work with these kinds of files via the `imreadmulti` and `imwritemulti` functions, as seen in the following example:

Here, note that `grayImages` is a tuple of NumPy arrays; each element in the tuple represents a single image.

Converting between an image and raw bytes

Conceptually, a byte is an integer ranging from 0 to 255. Throughout real-time graphic applications today, a pixel is typically represented by one byte per channel, though other representations are also possible.

An OpenCV image is a 2D or 3D array of the `numpy.array` type. An 8-bit grayscale image is a 2D array containing byte values. A 24-bit BGR image is a 3D array, which also contains byte values. We may access these values by using an expression such as `image[0, 0]` or `image[0, 0, 0]`. The first index is the pixel's *y* coordinate or row, 0 being the top. The second index is the pixel's *x* coordinate or column, 0 being the leftmost. The third index (if applicable) represents a color channel. The array's three dimensions can be visualized in the following Cartesian coordinate system:

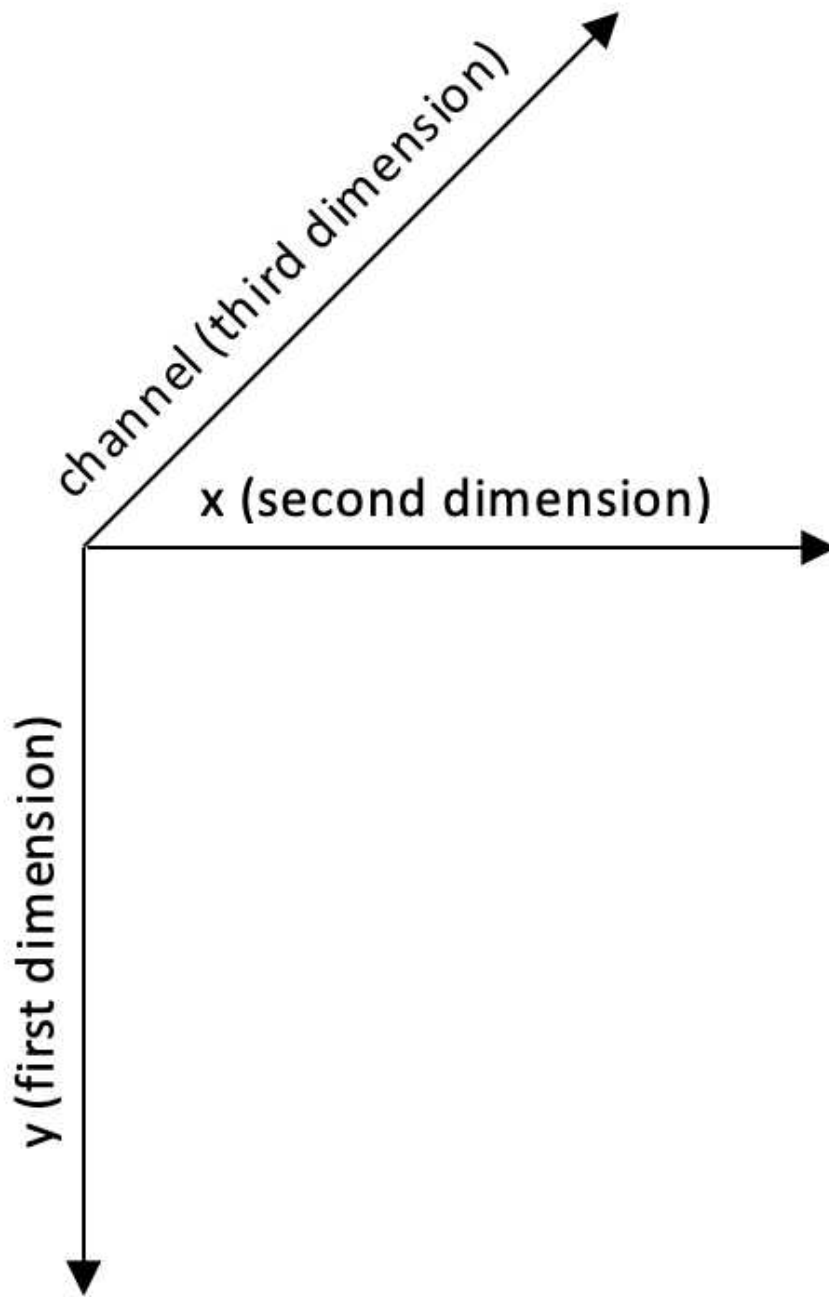


Figure 2.1: The dimensions of an image as a NumPy array

For example, in an 8-bit grayscale image with a white pixel in the upper-left corner, `image[0, 0]` is 255 . For a 24-bit (8-bit-per-channel) BGR image with a blue pixel in the upper-left corner, `image[0, 0]` is `[255, 0, 0]` .

Provided that an image has 8 bits per channel, we can cast it to a standard Python `bytearray` object, which is one-dimensional:

Conversely, provided that `bytearray` contains bytes in an appropriate order, we can cast and then reshape it to get a `numpy.array` type that is an image:

As a more complete example, let's convert a `bytearray` that contains random bytes into a grayscale image and a BGR image:

Here, we use Python's standard `os.urandom` function to generate random raw bytes, which we then convert into a NumPy array. Note that it is also possible to generate a random NumPy array directly (and more efficiently) using a statement such as `numpy.random.randint(0, 256, 120000).reshape(300, 400)`. The only reason we use `os.urandom` is to help to demonstrate conversion from raw bytes.

After running this script, we should have a pair of randomly generated images, `RandomGray.png` and `RandomColor.png`, in the script's directory.

Here is an example of `RandomGray.png` (though yours will almost certainly differ since it is random):

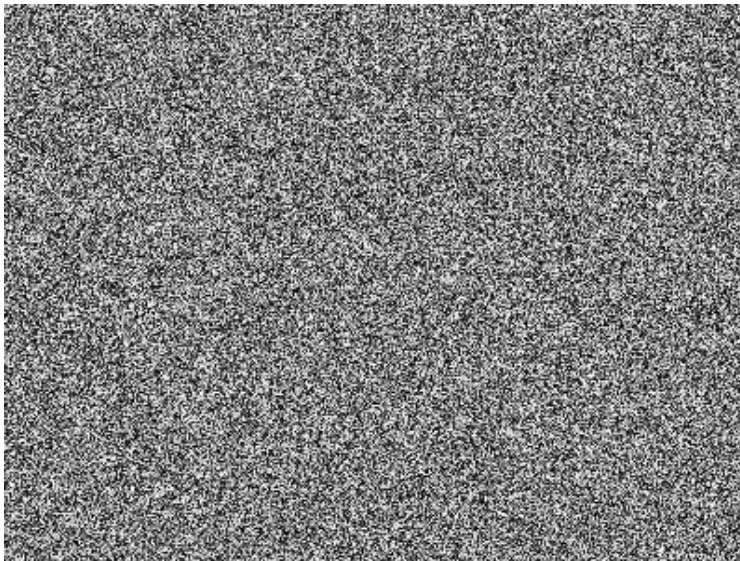


Figure 2.2a: The result of interpreting 120,000 random bytes as a 400x300 grayscale image

Similarly, here is an example of `RandomColor.png` :



Figure 2.2b: The result of interpreting the same 120,000 random bytes as a 400x100 BGR color image

Now that we have a better understanding of how an image is formed from data, we can start performing basic operations on it.

Accessing image data with `numpy.array`

We already know that the easiest (and most common) way to load an image in OpenCV is to use the `imread` function. We also know that this will return an image, which is really an array (either a 2D or 3D one, depending on the parameters you passed to `imread`).

The `numpy.array` class is greatly optimized for array operations, and it allows certain kinds of bulk manipulations that are not available in a plain Python list. These kinds of `numpy.array` type-specific operations come in handy for image manipulations in OpenCV. However, let's explore image manipulations step by step, starting with a basic example. Say you want to manipulate a pixel at coordinates (0, 0) in a BGR image and turn it into a white pixel:

If you then save the modified image to file and view it, you will see a white dot in the top-left corner of the image. Naturally, this modification is not very useful, but it begins to show the possibilities. Now, let's leverage the capabilities of `numpy.array` to perform transformations on an array much faster than we could do with a plain Python list.

Let's say that you want to change the blue value of a particular pixel, say, the pixel at coordinates, (150, 120). The `numpy.array` type provides a handy method, `item` , which takes three parameters: the *x* (or left) position, the *y* (or top) position, and the index within the array at the (*x* , *y*) position (remember that in a BGR image, the data at a certain position is a three-element array containing the B, G, and R values in this order) and returns the value at the index position. Another method, `itemset` , sets the value of a particular channel of a particular pixel to a specified value. `itemset` takes two arguments: a three-element tuple (*x* , *y* , and index) and the new value.

In the following example, we change the value of the blue channel at (150, 120) from its current value to an arbitrary 255 :

For modifying a single element in an array, the `itemset` method is somewhat faster than the indexing syntax that we saw in the first example in this section.

Again, modifying an element of an array does not do much in itself, but it does open a world of possibilities. However, for performance reasons, this is only suitable for small regions of interest. When you need to manipulate an entire image or a large region of interest, it is advisable that you utilize either OpenCV's functions or NumPy's **array slicing** . The latter allows you to specify a range of indices. Let's consider an example of using array slicing to manipulate color channels. Setting all G (green) values of an image to 0 is as simple as the following code:

This piece of code performs a fairly significant operation and is easy to understand. The relevant line is the last one, which basically instructs the program to take all pixels from all rows and columns and set the green value (at index one of the three-element BGR array) to 0 . If you display this image, you will notice a complete absence of green.

There are several interesting things we can do by accessing raw pixels with NumPy's array slicing; one of them is defining a **region of interest (ROI)** . An ROI is simply a selected rectangular subsection of an image; we may process or modify this subsection without affecting the rest of the image. Once the region is defined, we can perform a number of operations. For example, we can bind this region to a variable, then define a second region, and finally assign the value of the first region to the second (hence, copying a portion of the image over to another position in the image):

It is important to make sure that the two regions correspond in terms of size. If not, NumPy will (rightly) complain that the two shapes are mismatched.

Finally, we can access the properties of `numpy.array` , as shown in the following code:

These three properties are defined as follows:

- `shape` : This is a tuple describing the shape of the array. For an image, it contains (in order) the height, width, and—if the image is in color—the number of channels. The length of the `shape` tuple is a useful way to

determine whether an image is grayscale or color. For a grayscale image, we have `len(shape) == 2` , and for a color image, `len(shape) == 3` .

- `size` : This is the number of elements in the array. In the case of a grayscale image, this is the same as the number of pixels. In the case of a BGR image, it is three times the number of pixels because each pixel is represented by three elements (B, G, and R).
- `dtype` : This is the datatype of the array's elements. For an 8-bit-per-channel image, the datatype is `numpy.uint8` .

All in all, it is strongly advised that you familiarize yourself with NumPy in general, and `numpy.array` in particular, when working with OpenCV. This class is the foundation of OpenCV's Python interface.

Converting between `numpy.array` and `cv2.UMat`

A NumPy array keeps its data in main memory – never in GPU memory. This means that OpenCV cannot apply its optional GPU optimizations to images that are represented as NumPy arrays. If we want to allow OpenCV to move data into GPU memory and apply GPU optimizations (where supported), then we can convert a NumPy array to a different format called an **OpenCV Universal Matrix** or `cv2.UMat` .

Currently, `cv2.UMat` supports GPU acceleration on systems that have OpenCL-compliant GPU drivers, with a minimum version of OpenCL 1.2. Most modern desktop or laptop GPUs, as well as some high-end smartphone or tablet GPUs, have this level of OpenCL support.

The following code initializes a `UMat` by copying data from a NumPy array:

Most OpenCV functions will accept a `UMat` argument in place of NumPy array and, then, will return a `UMat` instead of NumPy array. Here is an example:

NumPy functions will not accept a `UMat` argument, a `UMat` does not have the same methods as a `numpy.array` , and we cannot perform slicing or other Pythonic operations on `UMat` data. However, we can convert a `UMat` back to a NumPy array, as follows:

You might wonder, should you start using `UMat` s everywhere in order to speed up your applications? Probably not. Copying data into and out of GPU memory is an expensive operation and, in many cases, this overhead cost outweighs the benefit

of greater parallelism on the GPU. On the other hand, if you can minimize the number of copies and if you can apply a series of OpenCV functions to a large amount of data on the GPU, then the benefit may outweigh the cost.

Throughout this book, we will mainly use `numpy.array` instead of `cv2.UMat` , but bear in mind that you may want to test the performance of `cv2.UMat` as an alternative for your own practical applications.

Reading/writing a video file

OpenCV provides the `VideoCapture` and `VideoWriter` classes, which support various video file formats. The supported formats vary depending on the operating system and the build configuration of OpenCV, but normally it is safe to assume that the AVI format is supported. Via its `read` method, a `VideoCapture` object may be polled for new frames until it reaches the end of its video file. Each frame is an image in the BGR format.

Conversely, an image may be passed to the `write` method of the `VideoWriter` class, which appends the image to a file in `VideoWriter` . Let's look at an example that reads frames from one AVI file and writes them to another with a YUV encoding:

The arguments to the constructor of the `VideoWriter` class deserve special attention. A video's filename must be specified. Any preexisting file with this name is overwritten. A video codec must also be specified. The available codecs may vary from system to system. The supported options may include the following:

- `0` : This option is an uncompressed raw video file. The file extension should be `.avi` .
- `cv2.VideoWriter_fourcc('I','4','2','0')` : This option is an uncompressed YUV encoding, 4:2:0 chroma subsampled. This encoding is widely compatible but produces large files. The file extension should be `.avi` .
- `cv2.VideoWriter_fourcc('P','I','M','1')` : This option is MPEG-1. The file extension should be `.avi` .
- `cv2.VideoWriter_fourcc('X','V','I','D')` : This option is a relatively old MPEG-4 encoding. It is a good option if you want to limit the size of the resulting video. The file extension should be `.avi` .

- `cv2.VideoWriter_fourcc('M','P','4','V')` : This option is another relatively old MPEG-4 encoding. It is a good option if you want to limit the size of the resulting video. The file extension should be `.mp4` .
- `cv2.VideoWriter_fourcc('X','2','6','4')` : This option is a relatively new MPEG-4 encoding. It may be the best option if you want to limit the size of the resulting video. The file extension should be `.mp4` .
- `cv2.VideoWriter_fourcc('T','H','E','O')` : This option is **Ogg Vorbis** . The file extension should be `.ogv` .

A frame rate and frame size must be specified too. Since we are copying from another video, these properties can be read from the `get` method of the `VideoCapture` class.

Enabling hardware acceleration for video I/O

Often, modern GPUs are capable of accelerating the algorithms that decode (read) and encode (write) some of the common video formats. By default, OpenCV's `VideoCapture` and `VideoWriter` classes do not attempt to use GPU acceleration; they run the decoding and encoding algorithms on the CPU. However, starting in OpenCV 4.5.2 and continuing in OpenCV 5, the support for GPU-accelerated encoding/decoding has been improving. If we modify our video I/O example in the ways **highlighted** below, OpenCV will try to use GPU acceleration on supported systems:

Here, the parameters `cv2.CAP_ANY`, `[cv2.CAP_PROP_HW_ACCELERATION, cv2.VIDEO_ACCELERATION_ANY]` tell the `VideoCapture` instance to use any available capture back-end with any available acceleration back-end. Similarly, the parameter `[cv2.VIDEOWRITER_PROP_HW_ACCELERATION, cv2.VIDEO_ACCELERATION_ANY]` tells the `VideoWriter` instance to use any available acceleration back-end.

Currently, Windows is the best-supported platform, in terms of OpenCV's use of GPU-accelerated encoding/decoding. Some Linux configurations are also supported. Broader support may be available in future releases of OpenCV 5.x. On systems where OpenCV does not support GPU-accelerated encoding/decoding, it should fall back safely to CPU encoding/decoding. Note that the support for GPU acceleration may vary according to the video format, as well as the operating system and hardware.

For more information on the ongoing efforts to support GPU-accelerated encoding/decoding, see the OpenCV Wiki article at <https://github.com/opencv/opencv/wiki/Video-IO-hardware-acceleration> .

Bear in mind, there is no guarantee that enabling hardware acceleration will actually speed up your application; in some cases, the opposite could be true. Typically, high-resolution videos might be a better use case for hardware acceleration than low-resolution videos. Regardless, for any practical application where time is critical, you should conduct tests using the types of data and hardware that your users may have.

Next, let's turn our attention to the task of capturing video input from a camera instead of from a pre-recorded video file.

Capturing camera frames

A stream of camera frames is represented by a `VideoCapture` object too. However, for a camera, we construct a `VideoCapture` object by passing the camera's device index instead of a video's filename. Let's consider the following example, which captures 10 seconds of video from a camera and writes it to an AVI file. The code is similar to the previous section's sample (which captured frames from a video file instead of a camera) but changes are **highlighted in bold** :

For some cameras on certain systems, `cameraCapture.get(cv2.CAP_PROP_FRAME_WIDTH)` and `cameraCapture.get(cv2.CAP_PROP_FRAME_HEIGHT)` may return inaccurate results. To be more certain of the actual image dimensions, you can first capture a frame and then get its height and width with code such as `h, w = frame.shape[:2]` . Occasionally, you might even encounter a camera that yields a few bad frames with unstable dimensions before it starts yielding good frames with stable dimensions. If you are concerned about guarding against this kind of quirk, you may want to read and ignore a few frames at the start of a capture session.

We could, in principle, call `cameraCapture.get(cv2.CAP_PROP_FPS)` to get the camera's frame rate; however, for most cameras on most systems, this does not return an accurate value; it typically returns either `nan` (not a number) or `0` . The official documentation at

https://docs.opencv.org/master/d8/dfe/classcv_1_1VideoCapture.html#aa6480e6972ef4c00d74814ec841a2939 warns of the following:

*"Value 0 is returned when querying a property that is not supported by the backend used by the VideoCapture instance. **Note** Reading / writing properties involves many layers. Some unexpected result might happen along this chain. VideoCapture -> API Backend -> Operating System -> Device Driver -> Device Hardware The returned value might be different from what really is used by the device or it could be encoded using device-dependent rules (for example, steps or percentage). Effective behavior depends on device driver and the API backend."*

To create an appropriate `VideoWriter` instance for the camera, we have to either make an assumption about the frame rate (as we did in the preceding code) or measure it using a timer. The latter approach is better and we will cover it later in this chapter, in the *Abstracting a video stream with managers, CaptureManager* section.

The number of cameras and their order is, of course, system-dependent. Unfortunately, OpenCV does not provide any means of querying the number of cameras or their properties. If an invalid index is used to construct a `VideoCapture` class, the `VideoCapture` class will not yield any frames; its `read` method will return `(False, None)`. To avoid trying to retrieve frames from a `VideoCapture` object that was not opened correctly, you may want to first call the `VideoCapture.isOpened` method, which returns a Boolean.

The `read` method is inappropriate when we need to synchronize either a set of cameras or a **multihead camera** such as a stereo camera. Then, we use the `grab` and `retrieve` methods instead. For a set of two cameras, we can use code similar to the following:

Finally, bear in mind that the `VideoCapture` class is an abstraction, in the sense that it may use various underlying video capture APIs or camera APIs, depending on the operating system and the way we have configured OpenCV. For example, depending on the configuration choices that you made in *Chapter 1, Setting Up OpenCV*, your build of OpenCV may or may not support the use of `VideoCapture` with OpenNI-compatible depth cameras.

Optionally, when we construct a `VideoCapture` instance, we can specify which camera API we want to use as a back-end. For example, on Windows, the default back-end for most camera hardware is **Microsoft Media Foundation (MSMF)**,

and thus our previous code sample uses this default; however, if we want to use the older and more widely compatible **DirectShow** back-end, we can request it as follows:

Alternatively, if we want to change the way OpenCV selects a default camera back-end on a particular system, we can do so via an environment variable. For example, if we want to ensure that OpenCV avoids the MSMF back-end, we can define an environment variable with the name `OPENCV_VIDEOIO_PRIORITY_MSMF` and the value `0` . This means MSMF becomes OpenCV's least-preferred (lowest-priority) camera back-end.

Sometimes, a particular camera proves to be incompatible with a particular back-end. Then, you may want to take steps to change the back-end, as described above. On Windows, OpenCV may give errors such as the following if a camera is not working with the MSMF back-end:

Then, instead of MSMF, you could try DirectShow, as described above.

For a list of other back-end flags besides `cv2.CAP_DSHOW` , see the documentation at https://docs.opencv.org/master/d4/d15/group__videoio__flags__base.html .

Sending a video stream over a network

Perhaps you have heard of a type of camera called an **IP camera** , which continuously sends a stream of images via a specified IP address on a network. The video stream from an IP camera can potentially be viewed in any web browser on any computer connected to the same network. Of course, the network should be secured to prevent unwanted access. IP cameras are used in a wide range of practical applications, including security systems, baby monitors, and livestock monitors.

Using OpenCV and a bit of networking code, we can turn any webcam or any computer vision application into the equivalent of an IP camera. For prototyping purposes, this is a great way to present a demo to multiple users, or to debug an application that is running on a **headless system** (a system with no display capabilities of its own). For example, if your computer vision application runs on a small, mobile robot, then you and other users may find it convenient to access the robot's video stream in a web browser on other local systems, rather than trying to build display capabilities into the robot itself.

The following sample assumes some basic familiarity with concepts relating to web applications, but let's review a few key terms. A **client** application, such as a web browser, typically sends a **request** to a **server** application, such as a streaming video service. The server application sends a **response** to the client; the response may contain content for the client to use. The client and server applications may be running on the same machine or different machines. On a network, a machine is identified by an **IP address** . On a given machine, a particular server application is identified by a **port number** . Thus, a client app can find a server app by the combination of IP address and port number.

Let's build a basic sample of a video streaming server. To capture images from a camera, we will use OpenCV. To serve the images via a network, we will use Flask, a minimalistic web application framework for Python. We can install Flask by running one of the following commands (depending on the environment):

- For any Python environment, using `pip` as a package manager, run:
- Alternatively, on Debian or Ubuntu, using `apt` as a package manager, run:
- Alternatively, on Arch Linux or Manjaro, using `pacman` as a package manager, run:

Having installed Flask, we can begin to write our sample Python script. Import OpenCV and Flask as follows:

Next, we will create an instance of a Flask application. For this purpose, Flask provides a class that is simply called `flask.Flask` . We instantiate it with a module name, which is normally just the name of our script:

Our Flask application has one role: to serve a stream of images. We will use a streaming format called Motion JPEG (MJPEG), which is supported by all popular web browsers and many other stream-reading applications. The stream will consist of an indeterminate number of frames, captured from a camera and converted to JPEG format. The stream will start when a client connects to our server, and it will end when the client disconnects.

Before a function that uses Flask to handle client requests, let's first write a helper function that uses OpenCV to capture and convert frames. Specifically, the helper will be a Python **generator** function, which is a convenient way to encapsulate logic about getting the next element in an indefinitely long series (of frames, in

our case). Our helper begins with familiar code to start capturing low-resolution camera frames:

We will capture and convert frames in a loop until we fail to read a frame from the camera, or until we fail to convert a frame to JPEG format, or until the rest of our application stops calling our generator function. To convert frames, we will use an OpenCV function called `imencode` , which is very similar to `imwrite` , except that `imencode` keeps the encoded image in memory instead of writing it to file. Every time we successfully encode a JPEG image, we will parcel the JPEG data with a header, in a format defined by the MJPEG standard. Also, each packet in the MJPEG stream is separated from the next by a boundary marker, in this case `'--frame'` . Here is the implementation of the loop:

Note the use of Python's `yield` keyword, which is a defining characteristic of a generator function. The `yield` keyword is akin to the `return` keyword, but the next time the generator function is called, it will start from the last `yield` statement instead of from the beginning of the function.

That concludes the implementation of our `mjpeg_generator` helper function. Now, let's see how we can use such a function to make Flask serve a continuous stream of images in response to a client's request. Flask enables us to use a **decorator** pattern to associate certain functions with certain web requests. Here is a function that handles any requests directed at the root (`'/'`) path of streaming video server:

The `@` annotation (as in `@app.route`) denotes a Python **decorator** , a built-in feature of the language that allows the wrapping of a function by another function, normally used to apply user-defined or library-defined behavior in several places in an application. You can find relevant documentation at https://docs.python.org/3/reference/compound_stmts.html#grammar-token-decorator .

Note that the `app` in `@app.route` is the variable name we gave to our `flask.Flask` instance, near the start of our script. The function name `stream_from_camera` has no special significance, as far as Flask is concerned; we could have chosen any function name for this request handler. Finally, note that the role of this function is to return a `flask.Response` object, which represents our server's response to the client's request. Specifically, the response is an MJPEG stream. We initialize the `flask.Response` instance by calling our generator function, which provides the stream's content, and by specifying a

standard media type or **mimetype** , which tells the client what kind of content to expect. Here, the mimetype `'multipart/x-mixed-replace;boundary=frame'` tells the client to expect a series of packets, where each new packet's content (in this case, each JPEG frame) is intended to replace the last, and where the boundary marker between packets is `'--frame'` .

The only thing left to do in this script is to tell our server application to start running. We can do this, with some optional arguments, as follows:

The `host` argument specifies an IP address where the application should accept incoming requests; the special value `'0.0.0.0'` means that Flask should use any and all IP addresses available, including any and all IP addresses that your router has assigned to this computer. (Alternatively, we can manually specify an IP address, but it has to be a valid address for this computer on this network, or else our app will fail to run.) The `port` argument is the port number; we use `8080` , which is a conventional port number for HTTP server apps, other than a public website (which is conventionally port 80). Lastly, with `threaded=True` , we specify that this server app should be multithreaded; this makes it possible for multiple clients to connect at once.

Here, our networking code is a rudimentary sample, it has no security features, and the resulting video stream is accessible to anybody on the same network. Do not use this code on public networks – for instance, in airports or cafés. For practical video streaming applications, always consider using HTTPS, password protection, and other security measures as appropriate.

Let's run our script. Your operating system or firewall software might ask you whether Python can have permission to access the network. Say yes. Now, you should see command line output similar to the following:

Take note of the two URLs. The IP address starting with `127.0` will only work on the same machine. The IP address starting with `192.168` will also work on other machines on the same local network.

Now, on the same computer, open a web browser and enter the first URL (such as `http://127.0.0.1:8080`) in the browser's address bar. The camera should start capturing frames (and its recording light should turn on, if it has one), and you should see the resulting video stream in the browser window.

Now, on a different computer connected to the same local network, open a web browser and navigate to the second URL (such as `http://192.168.2.10:8080`).

Again, you should see a stream of frames from the camera attached to the first computer. The video streams in the two web browsers, on the two machines, should appear to be synchronized or nearly synchronized, unless your local network is very slow.

The following photo shows the camera stream in two web browsers, on two machines. The laptop is capturing the camera frames and streaming them to the web browsers on the same laptop and on the desktop:



Figure 2.3: A photo of an MJPEG streaming application. A server application on the laptop is capturing video frames from the laptop's camera, converting them to JPEG format, and streaming them to two client applications: a web browser on the same laptop and a web browser on the desktop.

When you have finished looking at the streams, close the two web browsers. The camera should stop capturing frames (and its recording light should turn off, if it has one). As the command line output says, you can press Ctrl+C to quit the server app.

Now, let's move on to writing our own basic GUI applications, to show a camera stream on a local machine, without any networking code or web browsers.

Displaying an image in a window

One of the most basic operations in OpenCV is displaying an image in a window. This can be done with the `imshow` function. If you come from any other GUI framework background, you might think it sufficient to call `imshow` to display an image. However, in OpenCV, the window is drawn (or re-drawn) only when you call another function, `waitKey`. The latter function pumps the window's event queue (allowing various events such as drawing to be handled), and it returns the keycode of any key that the user may have typed within a specified timeout. To some extent, this rudimentary design simplifies the task of developing demos that use video or webcam input; at least the developer has manual control over the capture and display of new frames.

Here is a very simple sample script to read an image from a file and display it:

The `imshow` function takes two parameters: the name of the window in which we want to display the image and the image itself. We will talk about `waitKey` in more detail in the next section, *Displaying camera frames in a window*.

The aptly named `destroyAllWindows` function disposes of all of the windows created by OpenCV.

Displaying camera frames in a window

OpenCV allows named windows to be created, redrawn, and destroyed using the `namedWindow`, `imshow`, and `destroyWindow` functions. Also, any window may capture keyboard input via the `waitKey` function and mouse input via the `setMouseCallback` function. Let's look at an example where we show the frames captured from a live camera:

The argument for `waitKey` is a number of milliseconds to wait for keyboard input. By default, it is `0`, which is a special value meaning infinity. The return value is either `-1` (meaning that no key has been pressed) or an ASCII keycode, such as `27` for Esc. For a list of ASCII keycodes, refer to <https://www.asciitable.com/>. Also, note that Python provides a standard function, `ord`, which can convert a character into its ASCII keycode. For example, `ord('a')` returns `97`.

Again, note that OpenCV's window functions and `waitKey` are interdependent. OpenCV windows are only updated when `waitKey` is called. Conversely,

`waitKey` only captures input when an OpenCV window has focus.

The mouse callback passed to `setMouseCallback` should take five arguments, as seen in our code sample. The callback's `param` argument is set as an optional third argument to `setMouseCallback`. By default, it is 0. The callback's event argument is one of the following actions:

- `cv2.EVENT_MOUSEMOVE` : This event refers to mouse movement.
- `cv2.EVENT_LBUTTONDOWN` : This event refers to the left button going down when it is pressed.
- `cv2.EVENT_RBUTTONDOWN` : This event refers to the right button going down when it is pressed.
- `cv2.EVENT_MBUTTONDOWN` : This event refers to the middle button going down when it is pressed.
- `cv2.EVENT_LBUTTONUP` : This event refers to the left button coming back up when it is released.
- `cv2.EVENT_RBUTTONUP` : This event refers to the right button coming back up when it is released.
- `cv2.EVENT_MBUTTONUP` : This event refers to the middle button coming back up when it is released.
- `cv2.EVENT_LBUTTONDBLCLK` : This event refers to the left button being double-clicked.
- `cv2.EVENT_RBUTTONDBLCLK` : This event refers to the right button being double-clicked.
- `cv2.EVENT_MBUTTONDBLCLK` : This event refers to the middle button being double-clicked.

The mouse callback's `flags` argument may be some bitwise combination of the following events:

- `cv2.EVENT_FLAG_LBUTTON` : This event refers to the left button being pressed.
- `cv2.EVENT_FLAG_RBUTTON` : This event refers to the right button being pressed.
- `cv2.EVENT_FLAG_MBUTTON` : This event refers to the middle button being pressed.
- `cv2.EVENT_FLAG_CTRLKEY` : This event refers to the Ctrl key being pressed.
- `cv2.EVENT_FLAG_SHIFTKEY` : This event refers to the Shift key being pressed.
- `cv2.EVENT_FLAG_ALTKEY` : This event refers to the Alt key being pressed.

Unfortunately, OpenCV does not provide any means of manually handling window events. For example, we cannot stop our application when a window's close button is clicked. Due to OpenCV's limited event handling and GUI capabilities, many developers prefer to integrate it with other application frameworks. Later in this chapter, in the *Cameo – an object-oriented design* section, we will design an abstraction layer to help to integrate OpenCV with any application framework.

Project Cameo (face tracking and image manipulation)

OpenCV is often studied through a cookbook approach that covers a lot of algorithms, but nothing about high-level application development. To an extent, this approach is understandable because OpenCV's potential applications are so diverse. To name just a few examples, OpenCV can be used in photo/video editors, motion-controlled games, medical imaging devices, a robot's AI, or psychology experiments where we log participants' eye movements. Across these varied use cases, can we truly study a useful set of abstractions?

The book's authors believe we can, and the sooner we start creating abstractions, the better. Throughout *Chapters 2 to 5*, we will structure several of our OpenCV examples around a single application, but, at each step, we will design a component of this application to be extensible and reusable.

We will develop an interactive application that performs face tracking and image manipulations on camera input in real time. This type of application covers a broad range of OpenCV's functionality and challenges us to create an efficient, effective implementation.

Specifically, our application will merge faces in real time. Given two streams of camera input (or, optionally, prerecorded video input), the application will superimpose faces from one stream atop faces in the other. Filters and distortions will be applied to give this blended scene a unified look and feel. Users should have the experience of being engaged in a live performance where they enter another environment and persona. This type of user experience is popular in amusement parks such as Disneyland.

In such an application, users would immediately notice flaws, such as a low frame rate or inaccurate tracking. To get the best results, we will try several approaches using conventional imaging and depth imaging.

We will call our application Cameo. A cameo (in jewelry) is a small portrait of a person or (in film) a very brief role played by a celebrity.

Cameo – an object-oriented design

Python applications can be written in a purely procedural style. This is often done with small applications, such as our basic I/O scripts, discussed previously. However, from now on, we will often use an object-oriented style because it promotes modularity and extensibility.

From our overview of OpenCV's I/O functionality, we know that all images are similar, regardless of their source or destination. No matter how we obtain a stream of images or where we send it as output, we can apply the same application-specific logic to each frame in this stream. Separation of I/O code and application code becomes especially convenient in an application, such as Cameo, which uses multiple I/O streams.

We will create classes called `CaptureManager` and `WindowManager` as high-level interfaces to I/O streams. Our application code may use `CaptureManager` to read new frames and, optionally, to dispatch each frame to one or more outputs, including a still image file, a video file, and a window (via a `WindowManager` class). A `WindowManager` class lets our application code handle a window and events in an object-oriented style.

Both `CaptureManager` and `WindowManager` are extensible. We could make implementations that did not rely on OpenCV for I/O.

Abstracting a video stream with managers.`CaptureManager`

As we have seen, OpenCV can capture, show, and record a stream of images from either a video file or camera, but there are some special considerations in each case. Our `CaptureManager` class abstracts some of the differences and provides a higher-level interface to dispatch images from the capture stream to one or more outputs—a still image file, video file, or window.

A `CaptureManager` object is initialized with a `VideoCapture` object and has `enterFrame` and `exitFrame` methods that should typically be called on every iteration of an application's main loop. Between a call to `enterFrame` and `exitFrame`, the application may (any number of times) set a `channel` property

and get a `frame` property. The `channel` property is initially 0 and only multihead cameras use other values. The `frame` property is an image corresponding to the current channel's state when `enterFrame` was called.

A `CaptureManager` class also has the `writeImage`, `startWritingVideo`, and `stopWritingVideo` methods that may be called at any time. Actual file writing is postponed until `exitFrame`. Also, during the `exitFrame` method, `frame` may be shown in a window, depending on whether the application code provides a `WindowManager` class either as an argument to the constructor of `CaptureManager` or by setting the `previewWindowManager` property.

If the application code manipulates `frame`, the manipulations are reflected in recorded files and in the window. A `CaptureManager` class has a constructor argument and property called `shouldMirrorPreview`, which should be `True` if we want `frame` to be mirrored (horizontally flipped) in the window but not in recorded files. Typically, when facing a camera, users prefer a live camera feed to be mirrored.

Recall that a `Videowriter` object needs a frame rate, but OpenCV does not provide any reliable way to get an accurate frame rate for a camera. The `CaptureManager` class works around this limitation by using a frame counter and Python's standard `time.perf_counter` function to estimate the frame rate if necessary. This approach is not foolproof. Depending on frame rate fluctuations and the system-dependent implementation of `time.perf_counter`, the accuracy of the estimate might still be poor in some cases. However, if we deploy to unknown hardware, it is better than just assuming that the user's camera has a particular frame rate.

Let's create a file called `managers.py`, which will contain our implementation of `CaptureManager`. This implementation turns out to be quite long, so we will look at it in several pieces:

1. First, let's add imports and a constructor, as follows:
1. Next, let's add the following getter and setter methods for the properties of `CaptureManager`:

Note that most of the member variables are nonpublic, as denoted by the underscore prefix in variable names, such as `self._enteredFrame`. These nonpublic variables relate to the state of the current frame and any file-writing operations. As discussed previously, the application code only needs to configure

a few things, which are implemented as constructor arguments and settable public properties: the camera channel, the window manager, and the option to mirror the camera preview.

Python does not enforce the concept of nonpublic member variables, but in cases where the developer intends a variable to be treated as nonpublic, you will often see the single-underscore prefix (`_`) or double-underscore prefix (`__`). The single-underscore prefix is just a convention, indicating that the variable should be treated as protected (accessed only within the class and its subclasses). The double-underscore prefix actually causes the Python interpreter to rename the variable, such that `MyClass.__myVariable` becomes `MyClass._MyClass__myVariable` . This is called **name mangling** (quite appropriately). By convention, such a variable should be treated as private (accessed only within the class, and *not* its subclasses). The same prefixes, with the same significance, can be applied to methods as well as variables.

1. Continuing with our implementation, let's add the `enterFrame` method to `managers.py` :

Note that the implementation of `enterFrame` only grabs (synchronizes) a frame, whereas actual retrieval from a channel is postponed to a subsequent reading of the `frame` variable.

1. Next, let's add the `exitFrame` method to `managers.py` :

The implementation of `exitFrame` takes the image from the current channel, estimates a frame rate, shows the image via the window manager (if any), and fulfills any pending requests to write the image to files.

1. Several other methods also pertain to file writing. Let's add the following implementations of public methods named `writeImage` , `startWritingVideo` , and `stopWritingVideo` to `managers.py` :

The preceding methods simply update the parameters for file-writing operations, whereas the actual writing operations are postponed to the next call of `exitFrame` .

1. Earlier in this section, we saw that `exitFrame` calls a helper method named `_writeVideoFrame` . Let's add the following implementation of `_writeVideoFrame` to `managers.py` :

The preceding method creates or appends to a video file in a manner that should be familiar from our earlier scripts (refer to the *Reading/writing a video file* section, earlier in this chapter). However, in situations where the frame rate is unknown, we skip some frames at the start of the capture session so that we have time to build up an estimate of the frame rate.

This concludes our implementation of `CaptureManager` . Although it relies on `VideoCapture` , we could make other implementations that do not use OpenCV for input. For example, we could make a subclass that is instantiated with a socket connection, whose byte stream could be parsed as a stream of images. Also, we could make a subclass that uses a third-party camera library with different hardware support than what OpenCV provides. However, for Cameo, our current implementation is sufficient.

Abstracting a window and keyboard with managers.`WindowManager`

As we have seen, OpenCV provides functions that cause a window to be created, be destroyed, show an image, and process events. Rather than being methods of a window class, these functions require a window's name to pass as an argument. Since this interface is not object-oriented, it is arguably inconsistent with OpenCV's general style. Also, it is unlikely to be compatible with other window- or event-handling interfaces that we might eventually want to use instead of OpenCV's.

For the sake of object orientation and adaptability, we abstract this functionality into a `WindowManager` class with the `createWindow` , `destroyWindow` , `show` , and `processEvents` methods. As a property, `WindowManager` has a function object called `keypressCallback` , which (if it is not `None`) is called from `processEvents` in response to any keypress. The `keypressCallback` object must be a function that takes a single argument, specifically an ASCII keycode.

Let's add an implementation of `WindowManager` to `managers.py` . The implementation begins with the following class declaration and `__init__` method:

The implementation continues with the following methods to manage the life cycle of the window and its events:

Our current implementation only supports keyboard events, which will be sufficient for Cameo. However, we could modify `WindowManager` to support mouse events, too. For example, the class interface could be expanded to include a `mouseCallback` property (and optional constructor argument,) but could otherwise remain the same. With an event framework other than OpenCV's, we could support additional event types in the same way by adding callback properties.

Applying everything with `cameo.Cameo`

Our application is represented by the `Cameo` class with two methods: `run` and `onKeypress`. On initialization, a `Cameo` object creates a `WindowManager` object with `onKeypress` as a callback, as well as a `CaptureManager` object using a camera (specifically, a `cv2.VideoCapture` object) and the same `WindowManager` object. When `run` is called, the application executes a main loop in which frames and events are processed.

As a result of event processing, `onKeypress` may be called. The spacebar causes a screenshot to be taken, Tab causes a screencast (a video recording) to start/stop, and Esc causes the application to quit.

In the same directory as `managers.py`, let's create a file called `cameo.py`, where we will implement the `Cameo` class:

1. The implementation begins with the following `import` statements and `__init__` method:
1. Next, let's add the following implementation of the `run` method:
1. To complete the `Cameo` class implementation, here is the `onKeypress` method:
1. Finally, let's add a `__main__` block that instantiates and runs `Cameo`, as follows:

When running the application, note that the live camera feed is mirrored, while screenshots and screencasts are not. This is the intended behavior, as we pass `True` for `shouldMirrorPreview` when initializing the `CaptureManager` class.

Here is a screenshot of Cameo, showing a window (with the title **Cameo**) and the current frame from a camera:

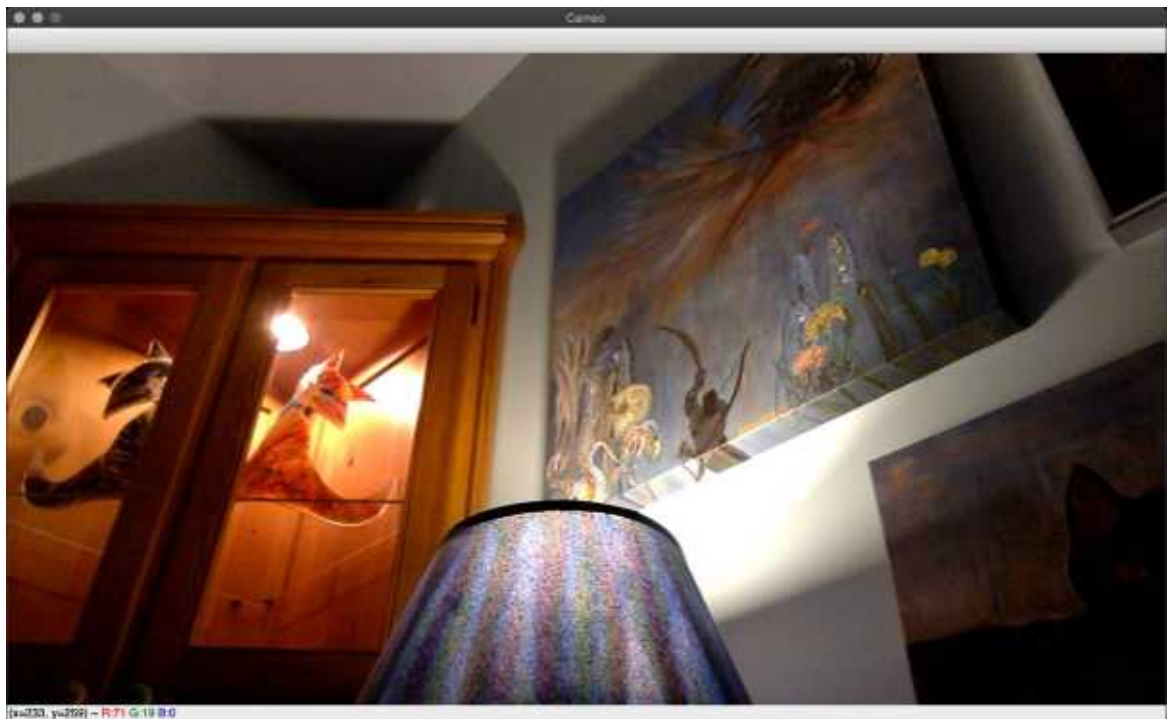


Figure 2.4: A screenshot of the Cameo application, showing a camera frame in a window

So far, we do not manipulate the frames in any way except to mirror them for preview. We will start to add more interesting effects in *Chapter 3 , Processing Images with OpenCV* .

Summary

By now, we should have an application that displays a camera feed, listens for keyboard input, and (on command) records a screenshot or screencast. We are ready to extend the application by inserting some image-filtering code (*Chapter 3 , Processing Images with OpenCV*) between the start and end of each frame. Optionally, we are also ready to integrate other camera drivers or application frameworks besides the ones supported by OpenCV.

We also possess the knowledge to manipulate images as NumPy arrays, UMat objects, or data that we can stream over a network. An understanding of image data and especially of NumPy arrays forms the perfect foundation for our next topic, filtering images.

Join our book community on Discord

<https://discord.gg/djGjeMECxx>



Sooner or later, when working with images, you will find you need to alter them: be it by applying artistic filters, extrapolating certain sections, blending two images, or whatever else your mind can conjure. This chapter presents some techniques that you can use to alter images. By the end of it, you should be able to perform tasks such as sharpening an image, marking the contours of subjects, and detecting crosswalks using a line segment detector. Specifically, our discussion and code samples will cover the following topics:

- Converting images between different color models
- Understanding the importance of frequencies and the Fourier transform in image processing
- Applying **high-pass filters (HPFs)** , **low-pass filters (LPFs)** , edge detection filters, and custom convolution filters
- Detecting and analyzing contours, lines, circles, and other geometric shapes
- Writing classes and functions that encapsulate the implementation of a filter

Technical requirements

This chapter uses Python, OpenCV, NumPy, and SciPy. Please refer to *Chapter 1 , Setting Up OpenCV* , for installation instructions.

The completed code for this chapter can be found in this book's GitHub repository, <https://github.com/PacktPublishing/Learning-OpenCV-5-Computer-Vision-with-Python-Fourth-Edition> , in the chapter03 folder. The sample images are also in this book's GitHub repository, in the images folder.

A subset of the chapter's sample code can be edited and run interactively in Google Colab at <https://colab.research.google.com/github/PacktPublishing/Learning-OpenCV-5-Computer-Vision-with-Python-Fourth-Edition/blob/main/chapter03/chapter03.ipynb> .

Converting images between different color models

OpenCV implements literally hundreds of formulas that pertain to the conversion of color models. Some color models are commonly used by input devices such as cameras, while other models are commonly used for output devices such as televisions, computer displays, and printers. In between input and output, when we apply computer vision techniques to images, we will typically work with three kinds of color models: grayscale, **blue-green-red (BGR)** , and **hue-saturation-value (HSV)** . Let's go over these briefly:

- **Grayscale** is a model that reduces color information by translating it into shades of gray or brightness. This model is extremely useful for the intermediate processing of images in problems where brightness information alone is sufficient, such as face detection. Typically, each pixel in a grayscale image is represented by a single 8-bit value, ranging from 0 for black to 255 for white.
- **BGR** is the blue-green-red color model, in which each pixel has a triplet of values representing the blue, green, and red components or **channels** of the pixel's color. Web developers, and anyone who works with computer graphics, will be familiar with a similar definition of colors, except with the reverse channel order, **red-green-blue (RGB)** . Typically, each pixel in a BGR image is represented by a triplet of 8-bit values, such as `[0, 0, 0]` for black, `[255, 0, 0]` for blue, `[0, 255, 0]` for green, `[0, 0, 255]` for red, and `[255, 255, 255]` for white.
- The **HSV** model uses a different triplet of channels. Hue is the color's tone, saturation is its intensity, and value represents its brightness.

By default, OpenCV uses the BGR color model (with 8 bits per channel) to represent any image that it loads from a file or captures from a camera.

Now that we have defined the color models we will use, let's consider how the default model might differ from our intuitive understanding of color.

Light is not paint

For newcomers to the BGR color space, it might seem that things do not add up properly: for example, the $[0, 255, 255]$ triplet (no blue, full green, and full red) produces the color yellow. If you have an artistic background, you won't even need to pick up paints and brushes to know that green and red paint mix together into a muddy shade of brown. However, the color models that are used in computing are called **additive** models and they deal with lights. Lights behave differently from paints (which follow a **subtractive** color model), and since software runs on a computer whose medium is a monitor that emits light, the color model of reference is an additive one.

Exploring the Fourier transform

Much of the processing you apply to images and videos in OpenCV involves the concept of the Fourier transform in some capacity. Joseph Fourier was an 18th-century French mathematician who discovered and popularized many mathematical concepts. He studied the physics of heat, and the mathematics of all things that can be represented by waveform functions. In particular, he observed that all waveforms are just the sum of simple sinusoids of different frequencies.

In other words, the waveforms you observe all around you are the sum of other waveforms. This concept is incredibly useful when manipulating images because it allows us to identify regions in images where a signal (such as the values of image pixels) changes a lot, and also regions where the change is less dramatic. We can then apply domain-specific logic to mark these regions as noise or regions of interests, background or foreground, and so on. These are the frequencies that make up the original image, and we have the power to separate them to make sense of the image and extrapolate interesting data.

OpenCV implements a number of algorithms that enable us to process images and make sense of the data contained in them, and these are also reimplemented in NumPy to make our life even easier. NumPy has a **fast Fourier transform (FFT)** package, which contains the `fft2` method. This method allows us to compute a **discrete Fourier transform (DFT)** of the image. For more about the Fourier transform and its advanced usage in image analysis, see Joseph Howse's book *OpenCV 4 for Secret Agents*, specifically *Chapter 7, Seeing a Heartbeat with a Motion-Amplifying Camera*.

The Fourier transform can be used to find the **magnitude spectrum** of an image (or other data). The magnitude spectrum represents the original image (or other data) in terms of its changes. Think of it as taking an image and dragging all the brightest pixels to the center. Then, you gradually work your way out to the border where all the darkest pixels have been pushed. Immediately, you will be able to see how many light and dark pixels are contained in your image and the percentage of their distribution.

The Fourier transform is the conceptual basis of many algorithms that are used for common image processing operations, such as edge detection or line and

shape detection. However, the Fourier transform is an expensive operation, so, in practice, many algorithms replace it with simpler functions that process a small region or neighborhood of pixels, instead of computing waveforms or frequencies across the entire image.

Before examining line and shape detection in detail, let's take a look at two kinds of filters that form the foundation of edge detection: HPFs and LPFs.

HPFs and LPFs

An HPF is a filter that examines a region of an image and boosts the intensity of certain pixels based on the difference in the intensity of the surrounding pixels.

Take, for example, the following kernel:

A **kernel** is a set of weights that are applied to a region in a source image to generate a single pixel in the destination image. For example, if we call an OpenCV function with a parameter to specify a kernel size or `ksize` of 7 , this implies that 49 (7 x 7) source pixels are considered when generating each destination pixel. We can think of a kernel as a piece of frosted glass moving over the source image and letting a diffused blend of the source's light pass through.

The preceding kernel gives us the average difference in intensity between the central pixel and all its immediate horizontal neighbors. If a pixel stands out from the surrounding pixels, the resulting value will be high. This type of kernel represents a so-called high-boost filter, which is a type of HPF, and it is particularly effective in edge detection.

Note that the values in an edge detection kernel typically sum up to 0 . We will cover this in the *Custom kernels – getting convoluted* section of this chapter.

Let's go through an example of applying an HPF to an image:

After the initial imports, we define a 3x3 kernel and a 5x5 kernel, and then we load the image in grayscale. After that, we want to convolve the image with each of the kernels. There are several library functions available for such a purpose. NumPy provides the `convolve` function; however, it only accepts one-dimensional arrays. Although the convolution of multidimensional arrays can be

achieved with NumPy, it would be a bit complex. SciPy's `ndimage` module provides another `convolve` function, which supports multidimensional arrays. Finally, OpenCV provides a `filter2D` function (for convolution with 2D arrays) and a `sepFilter2D` function (for the special case of a 2D kernel that can be decomposed into two one-dimensional kernels). The preceding code sample illustrates the `ndimage.convolve` function. We will use the `cv2.filter2D` function in other samples in the *Custom kernels – getting convoluted* section of this chapter.

Our script proceeds by applying two HPFs with the two convolution kernels we defined. Finally, we also implement a different method of obtaining an HPF by applying a LPF and calculating the difference between the original image. Let's see how each filter looks. As input, we start with the following photograph:



Figure 3.1a: A photograph of a statue of an angel, with a clear sky in the background. We will perform high-pass (edge detection) and low-pass (blur) filtering on this image.

Now, here is a screenshot of the output:

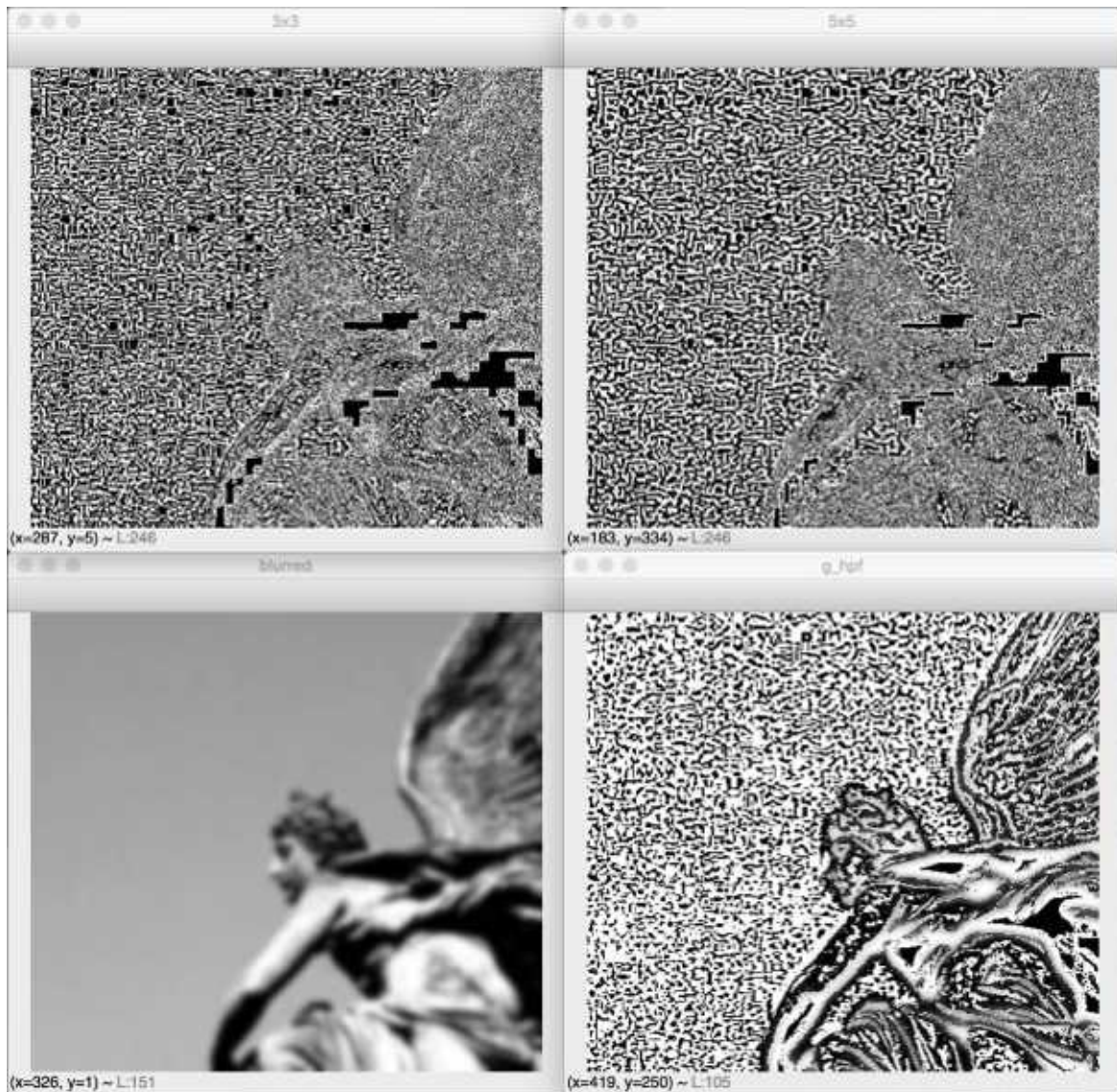


Figure 3.1b: Results of applying high-pass and low-pass filters. Top left: A basic 3x3 high-pass kernel. Top right: A basic 5x5 high-pass kernel. Bottom-left: A Gaussian blur low-pass kernel. Bottom-right: A differential high-pass filter, based on the preceding Gaussian blur result.

You will notice that the differential HPF, as shown in the bottom-right photograph, yields the best edge-finding result. Since this differential method involves a low-pass filter, let's elaborate on that type of filter. If an HPF boosts the intensity of a pixel, given its difference with neighbors, a LPF will smooth or flatten the pixel intensity if the difference from surrounding pixels is lower than a certain threshold. This is used in denoising and blurring. For example, one of the most popular blurring/smoothing filters, the Gaussian blur, is a low-pass filter that

attenuates the intensity of high-frequency signals. The result of the Gaussian blur is shown in the lower-left photograph.

Now that we have tried these filters in a basic example, let's consider how to integrate them into a larger, more interactive application.

Creating modules

Let's revisit the Cameo project that we started in *Chapter 2 , Handling Files, Cameras, and GUIs* . We can modify Cameo so that it applies filters to the captured images in real time. As in the case of our `CaptureManager` and `WindowManager` classes, our filters should be reusable outside of Cameo. Thus, we should separate the filters into their own Python module or file.

Let's create a file called `filters.py` in the same directory as `cameo.py` . We need the following `import` statements in `filters.py` :

Let's also create a file called `utils.py` in the same directory. It should contain the following `import` statements:

We will be adding filter functions and classes to `filters.py` , while more general-purpose math functions will go in `utils.py` .

Edge detection

Edges play a major role in both human and computer vision. We, as humans, can easily recognize many object types and their pose just by seeing a backlit silhouette or a rough sketch. Indeed, when art emphasizes edges and poses, it often seems to convey the idea of an archetype, such as Auguste Rodin's *The Thinker* or Joe Shuster's *Superman*. Software, too, can reason about edges, poses, and archetypes. We will discuss these kinds of reasoning in later chapters.

OpenCV provides many edge-finding filters, including `Laplacian`, `Sobel`, and `Scharr`. These filters are supposed to turn non-edge regions into black and turn edge regions into white or saturated colors. However, they are prone to misidentifying noise as edges. This flaw can be mitigated by blurring an image before trying to find its edges. OpenCV also provides many blurring filters, including `blur` (a simple average), `medianBlur`, and `GaussianBlur`. The arguments for the edge-finding and blurring filters vary but always include `ksize`, an odd whole number that represents the width and height (in pixels) of a filter's kernel.

For blurring, let's use `medianBlur`, which is effective in removing digital video noise, especially in color images. For edge-finding, let's use `Laplacian`, which produces bold edge lines, especially in grayscale images. After applying `medianBlur`, but before applying `Laplacian`, we should convert the image from BGR into grayscale.

Once we have the result of `Laplacian`, we can invert it to get black edges on a white background. Then, we can normalize it (so that its values range from 0 to 1) and then multiply it with the source image to darken the edges. Let's implement this approach in `filters.py`:

Note that we allow kernel sizes to be specified as arguments for `strokeEdges`. The `blurKsize` argument is used as `ksize` for `medianBlur`, while `edgeKsize` is used as `ksize` for `Laplacian`. With a typical webcam, a `blurKsize` value of 7 and an `edgeKsize` value of 5 might

produce the most pleasing effect. Unfortunately, `medianBlur` is expensive with a large `ksize` argument, such as 7 .

If you encounter performance problems when running `strokeEdges` , try decreasing the `blurKsize` value. To turn off the blur effect, set it to a value of less than 3 .

We will see the effect of this filter a little later in this chapter, after we have integrated it into Cameo in the *Modifying the application* section.

Custom kernels – getting convoluted

As we have just seen, many of OpenCV's predefined filters use a kernel. Remember that a kernel is a set of weights that determines how each output pixel is calculated from a neighborhood of input pixels. Another term for a kernel is a **convolution matrix**. It mixes up or convolves the pixels in a region. Similarly, a kernel-based filter may be called a convolution filter.

OpenCV provides a very versatile `filter2D` function, which applies any kernel or convolution matrix that we specify. To understand how to use this function, let's learn about the format of a convolution matrix. It is a 2D array with an odd number of rows and columns. The central element corresponds to a pixel of interest, while the other elements correspond to the neighbors of this pixel. Each element contains an integer or floating-point value, which is a weight that gets applied to an input pixel's value. Consider this example:

Here, the pixel of interest has a weight of 9 and its immediate neighbors each have a weight of -1. For the pixel of interest, the output color will be nine times its input color, minus the input colors of all eight adjacent pixels. If the pixel of interest is already a bit different from its neighbors, this difference becomes intensified. The effect is that the output image looks *sharper*, as the contrast between the neighbors is increased.

Continuing with our example, we can apply this convolution matrix to a source and destination image, respectively, as follows:

The second argument specifies the per-channel depth of the destination image (such as `cv2.CV_8U` for 8 bits per channel). A negative value (such as -1, being used here) means that the destination image has the same depth as the source image.

For color images, note that `filter2D` applies the kernel equally to each channel. To use different kernels on different channels, we would also have to use OpenCV's `split` and `merge` functions.

Based on this simple example, let's add two classes to `filters.py`. One class, `VConvolutionFilter`, will represent a convolution filter in general. A subclass, `SharpenFilter`, will represent our sharpening filter specifically. Let's edit `filters.py` so that we can implement these two new classes, as follows:

Note that the weights sum up to 1. This should be the case whenever we want to leave the image's overall brightness unchanged. If we modify a sharpening kernel slightly so that its weights sum up to 0 instead, we'll have an edge detection kernel that turns edges white and non-edges black. For example, let's add the following edge detection filter to `filters.py`:

Next, let's make a blur filter. Generally, for a blur effect, the weights should sum up to 1 and should be positive throughout the neighborhood. For example, we can take a simple average of the neighborhood as follows:

Our sharpening, edge detection, and blur filters use kernels that are highly symmetric. Sometimes, though, kernels with less symmetry produce an interesting effect. Let's consider a kernel that blurs on one side (with positive weights) and sharpens on the other (with negative weights). It will produce a ridged or *embossed* effect. Here is an implementation that we can add to `filters.py`:

This set of custom convolution filters is very basic. Indeed, it is more basic than OpenCV's ready-made set of filters. However, with a bit of experimentation, you should be able to write your own kernels that produce a unique look.

Modifying the application

Now that we have high-level functions and classes for several filters, it is trivial to apply any of them to the captured frames in Cameo. Let's edit `cameo.py` and add the lines that appear **highlighted in bold** in the following excerpts. First, we need to add our `filters` module to our list of imports, as follows:

Now, we need to initialize any filter objects we will use. An example of this can be seen in the following modified `__init__` method:

Finally, we need to modify the `run` method in order to apply our choice of filters. Refer to the following example:

Here, we have applied two effects: stroking the edges and emulating the colors of a brand of photo film called Kodak Portra. Feel free to modify the code to apply any filters you like.

For details about how to implement the Portra film emulation effect, refer to *Appendix A, Bending Color Space with a Curves Filter*.

Here is a screenshot from Cameo with stroked edges and Portra-like colors:



Figure 3.2: A screenshot from Cameo, with two filters applied: stroked edges and Portra-like colors.

Now that we have sampled some of the visual effects that we can achieve with simple filters, let's consider how we can use other simple functions for analytical purposes – specifically, the detection of edges and shapes.

Edge detection with Canny

OpenCV offers a handy function called `canny` (after the algorithm's inventor, John F. Canny), which is very popular not only because of its effectiveness, but also because of the simplicity of its implementation in an OpenCV program since it is a one-liner:

The result is a very clear identification of the edges:



Figure 3.3: Results of applying Canny edge detection.

The Canny edge detection algorithm is complex but also quite interesting. It is a five-step process:

1. Denoise the image with a Gaussian filter.
2. Calculate the gradients.
3. Apply **non-maximum suppression** (**NMS**) on the edges. Basically, this means that the algorithm selects the best edges from a set of

overlapping edges. We'll discuss the concept of NMS in detail in *Chapter 7 , Building Custom Object Detectors* .

4. Apply a double threshold to all the detected edges to eliminate any false positives.
5. Analyze all the edges and their connection to each other to keep the real edges and discard the weak ones.

After finding Canny edges, we can do further analysis of the edges in order to determine whether they match a common shape, such as a line or a circle. The Hough transform is one algorithm that uses Canny edges in this way. We will experiment with it later in this chapter, in the *Detecting lines, circles, or other shapes* section.

For now, we will examine other ways of analyzing shapes, not based on edge detection but rather on the concept of finding a blob of similar pixels.

Contour detection

A vital task in computer vision is contour detection. We want to detect contours or outlines of subjects contained in an image or video frame, not only as an end in itself but also as a step toward other operations. These operations are, namely, computing bounding polygons, approximating shapes, and generally calculating **regions of interest (ROIs)**. ROIs considerably simplify interaction with image data because a rectangular region in NumPy is easily defined with an array slice. We will be using contour detection and ROIs a lot when we explore the concepts of object detection (including face detection) and object tracking in later chapters.

Let's familiarize ourselves with the contour detection API via an example:

First, we create an empty black image that is 200 x 200 pixels in size. Then, we place a white square in the center of it by utilizing array's ability to assign values on a slice.

Then, we threshold the image and call the `findContours` function. This function has three parameters: the input image, the hierarchy type, and the contour approximation method. The second parameter specifies the type of hierarchy tree returned by the function. One of the supported values is `cv2.RETR_TREE`, which tells the function to retrieve the entire hierarchy of external and internal contours. These relationships may matter if we are searching for smaller objects (or smaller regions) inside larger objects (or larger regions). If you only want to retrieve the most external contours, use `cv2.RETR_EXTERNAL`. This may be a good choice in cases where the objects appear on a plain background and we do not care about finding objects within objects.

Referring back to the code sample, note that the `findContours` function returns two elements: the contours and their hierarchy. We use the contours to draw green outlines on the color version of the image. Finally, we display the image.

The result is a white square with its contour drawn in green – a spartan scene, but effective in demonstrating the concept! Let's move on to more meaningful examples.

Bounding box, minimum area rectangle, and minimum enclosing circle

Finding the contours of a square is a simple task; irregular, skewed, and rotated shapes bring out the full potential of OpenCV's `cv2.findContours` function. Let's take a look at the following image, which is an illustration of a mythical object called Mjölnir or Thor's Hammer:



Figure 3.4a: A drawing of Mjölnir, or Thor's Hammer, a prominent symbol in Norse mythology. We will apply contour detection to this image.

For many applications, we might want to find a simple geometric shape that approximates the bounds of the subject. Three commonly used bounding shapes are the bounding box, the minimum enclosing rectangle, and the enclosing circle. (According to OpenCV's terminology, a bounding box is an upright rectangle, but a minimum enclosing rectangle may be rotated in order to fit the subject more snugly.) The `cv2.findContours` function, in conjunction with a few other OpenCV utilities, makes it very easy to find these bounding shapes. First, the following code reads an image from a file, converts it into grayscale, applies a threshold to the grayscale image, and finds the contours in the thresholded image:

Now, for each contour, we can find and draw the bounding box, the minimum enclosing rectangle, and the minimum enclosing circle, as shown in the following code:

Finally, we can use the following code to draw the contours and show the image in a window until the user presses a key:

Note that contour detection is performed on a thresholded image, so color information is already lost at this stage, but we draw on the original color image and then show the results in color.

Let's go back and look more closely at the steps we performed in the preceding `for` loop, where we process each detected contour. First, we calculate a simple bounding box:

This is a pretty straightforward conversion of contour information into the (x, y) coordinates, width, and height of the rectangle. Drawing this rectangle is an easy task and can be done using the following code:

Next, we calculate the minimum-area rectangle enclosing the subject:

The mechanism being used here is particularly interesting: OpenCV does not have a function to calculate the coordinates of the minimum rectangle vertices directly from the contour information. Instead, we calculate the minimum rectangle area, and then calculate the vertices of this rectangle. Note that the calculated vertices are floats, but pixels are accessed with integers (you can't access a *portion* of a pixel for the purposes of OpenCV's drawing functions), so we need to perform this conversion. Next, we draw the box, which gives us the perfect opportunity to introduce the `cv2.drawContours` function:

This function – like all of OpenCV's drawing functions – modifies the original image. Note that it takes an array of contours in its second parameter so that you can draw a number of contours in a single operation. Therefore, if you have a single set of points representing a contour polygon, you need to wrap these points in an array, exactly like we did with our box in the preceding example. The third parameter of this function specifies the index of the contours array that we want to draw: a value of -1 will draw all contours; otherwise, a contour at the specified index in the contours array (the second parameter) will be drawn.

Most drawing functions take the color of the drawing (as a BGR tuple) and its thickness (in pixels) as the last two parameters.

The last bounding contour we're going to examine is the minimum enclosing circle:

The only peculiarity of the `cv2.minEnclosingCircle` function is that it returns a two-element tuple, of which the first element is a tuple itself, representing the coordinates of the circle's center, and the second element is the radius of this circle. After converting all these values into integers, drawing the circle is quite a trivial operation.

When we apply the preceding code to the original image, the final result looks like this:

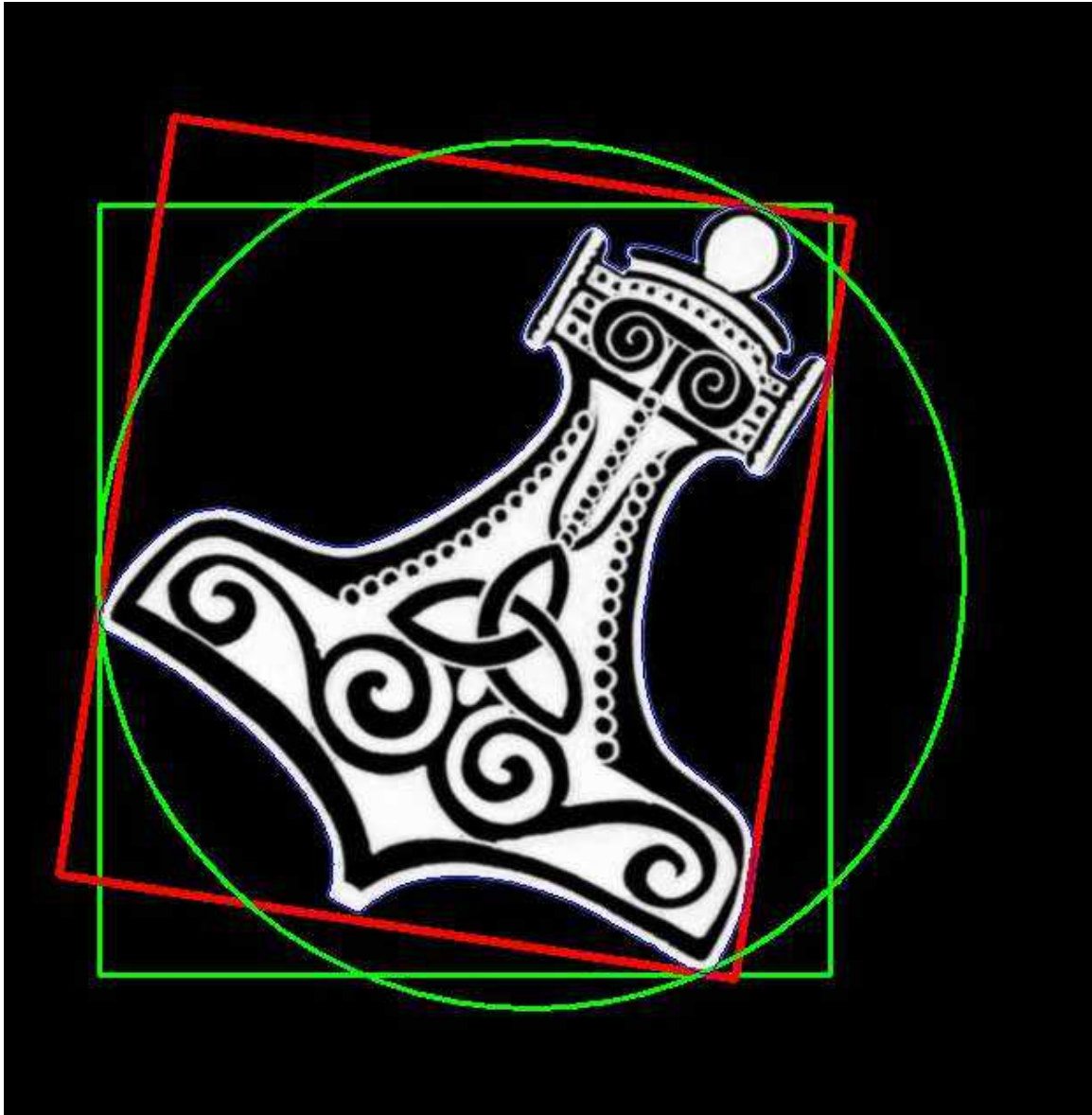


Figure 3.4b: Results of contour detection followed by three kinds of enclosing shape detection: bounding box, minimum area rectangle, and minimum enclosing circle.

This is a good result insofar as the circle and rectangles fit tightly around the object. Obviously, though, the object is not circular or rectangular, so we could achieve a tighter fit with various other shapes. Let's do that next.

Convex contours and the Douglas-Peucker algorithm

When working with contours, we may encounter subjects with diverse shapes, including convex ones. A convex shape is one where there are no two points

within this shape whose connecting line goes outside the perimeter of the shape itself.

The first facility that OpenCV offers to calculate the approximate bounding polygon of a shape is `cv2.approxPolyDP`. The `DP` in the function name stands for the **Douglas-Peucker** algorithm. This function takes three parameters:

- A contour.
- An epsilon value representing the maximum discrepancy between the original contour and the approximated polygon (the lower the value, the closer the approximated value will be to the original contour).
- A Boolean flag. If it is `True`, it signifies that the polygon is closed.

The epsilon value is of vital importance to obtain a useful contour, so let's understand what it represents. Epsilon is the maximum difference between the approximated polygon's perimeter and the original contour's perimeter. The smaller this difference is, the more the approximated polygon will be similar to the original contour.

You may ask yourself why we need an approximate polygon when we have a contour that is already a precise representation. The answer to this is that a polygon is a set of straight lines, and many computer vision tasks become simpler if we can define polygons so that they delimit regions for further manipulation and processing.

Now that we know what an epsilon is, we need to obtain contour perimeter information as a reference value. This can be obtained with the `cv2.arcLength` function of OpenCV:

Effectively, we're instructing OpenCV to calculate an approximated polygon whose perimeter can only differ from the original contour by an epsilon ratio – specifically, 1% of the original arc length.

OpenCV also offers a `cv2.convexHull` function for obtaining processed contour information for convex shapes. This is a straightforward one-line expression:

Let's combine the original contour, approximated polygon contour, and the convex hull into one image to observe the differences between them. To simplify things, we will draw the contours on top of a black background so that the original subject is not visible but its contours are:



Figure 3.4c: Results of contour detection followed by context hull detection and polygon approximation.

As you can see, the convex hull surrounds the entire subject, the approximated polygon is the innermost polygon shape, and in between the two is the original contour, mainly composed of arcs.

By combining all the preceding steps into one script that loads a file, finds contours, approximates the contours as a polygon, finds a convex hull, and displays a visualization, we have the following code:

Code like this works well on simple images, in which we have only one or a few objects, and only a few colors that are easily separated by thresholds.

Unfortunately, color thresholding and contour detection are less effective on complex images that contain several objects or multicolored objects. For these more challenging cases, we will have to look at more complex algorithms.

Detecting lines, circles, and other shapes

Detecting edges and finding contours are not only common and important tasks in their own right; they also form the basis of other complex operations. Line and shape detection walk hand-in-hand with edge and contour detection, so let's examine how OpenCV implements these.

The theory behind line and shape detection has its foundation in a technique called the Hough transform, invented by Richard Duda and Peter Hart, who extended and generalized the work that was done by Paul Hough in the early 1960s. Let's take a look at OpenCV's API for Hough transforms.

Detecting lines

First of all, let's detect some lines. We can do this with either the `HoughLines` function or the `HoughLinesP` function. The former uses the standard Hough transform, while the latter uses the probabilistic Hough transform (hence the `P` in the name). The probabilistic version is so-called because it only analyzes a subset of the image's points and estimates the probability that these points all belong to the same line. This implementation is an optimized version of the standard Hough transform; it is less computationally intensive and executes faster. `HoughLinesP` is implemented so that it returns the two endpoints of each detected line segment, whereas `HoughLines` is implemented so that it returns a representation of each line as a single point and an angle, without information about endpoints.

Let's take a look at a very simple example:

A crucial part of this simple script, as part of the `HoughLines` function call, is setting the minimum line length (shorter lines will be discarded) and the maximum line gap, which is the maximum size of a gap in a line before the two segments start being considered as separate lines.

Also, note that the `HoughLines` function takes a single channel binary image, which is processed through the Canny edge detection filter. Canny is not a strict

requirement, but an image that has been denoised and only represents edges is the ideal source for a Hough transform, so you will find this to be a common practice.

The parameters of `HoughLinesP` are as follows:

- The binary image from Canny or another edge detection filter.
- The resolution or step size to use when searching for lines. `rho` is the positional step size in pixels, while `theta` is the rotational step size in radians. For example, if we specify `rho=1` and `theta=np.pi/180.0`, we search for lines that are separated by as little as 1 pixel and 1 degree.
- The `threshold`, which represents the threshold below which a line is discarded. The Hough transform works with a system of bins and votes, with each bin representing a line, so if a candidate line has at least the `threshold` number of votes, it is retained; otherwise, it is discarded.
- An optional argument, `lines`, which we do not use here and which is not really useful in the function's Python version. The `lines` argument can be used to provide a list or array in which `HoughLinesP` will put the resulting lines; otherwise, it will return a new list.
- `minLineLength` and `maxLineGap`, which we mentioned previously.

The following screenshot shows the output of our line detection script, using an image of a train platform as input:

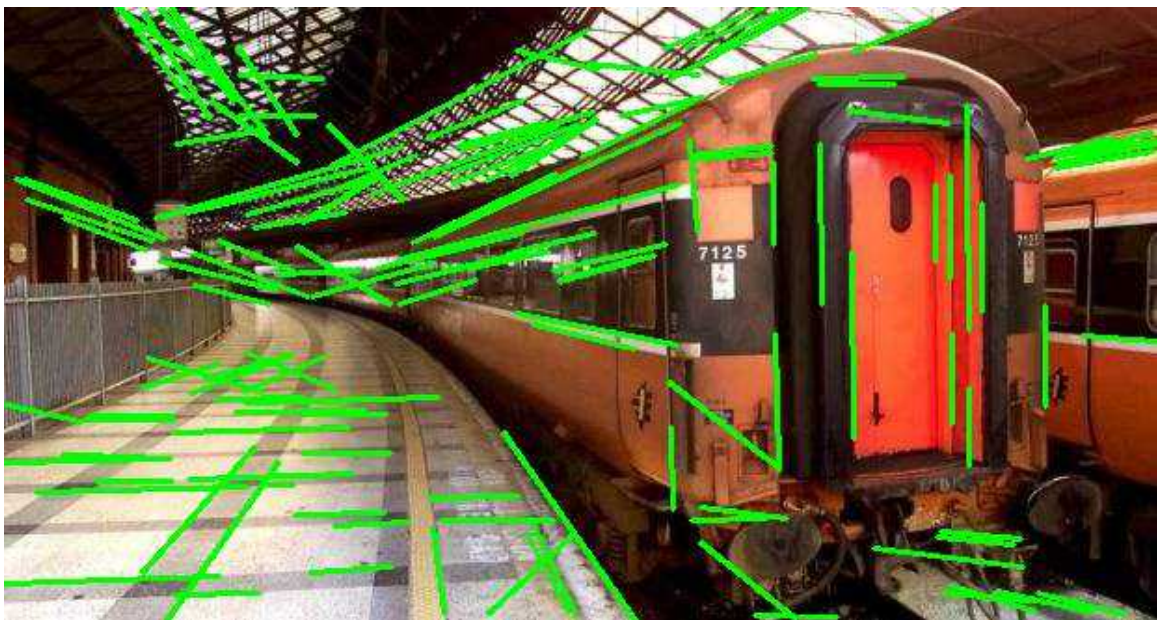


Figure 3.5: Results of Hough line detection on an image of a train platform.

Among other things, the edge of the platform is detected as a line. Here, we begin to get an idea of how a seemingly basic concept such as line detection can have important practical applications.

Now, what about other shapes?

Detecting circles

OpenCV also has a function for detecting circles, called `HoughCircles` . It works in a very similar fashion to `HoughLines` , but where `minLineLength` and `maxLineGap` were the parameters used to discard or retain lines, `HoughCircles` instead lets us specify a minimum distance between a circle's centers, as well as minimum and maximum values for a circle's radius. Here is the obligatory example:

Note that we do not call the `canny` function here because, when we call `HoughCircles` with the `cv2.HOUGH_GRADIENT` option, the latter function already applies the Canny algorithm internally, using the value of `param1` as the first Canny threshold and half this value as the second Canny threshold. Also, with `cv2.HOUGH_GRADIENT` , the value of `param2` is a threshold for weeding out overlapping circle detections; a higher value leads to fewer overlapping detections.

Here is a visual representation of the result:

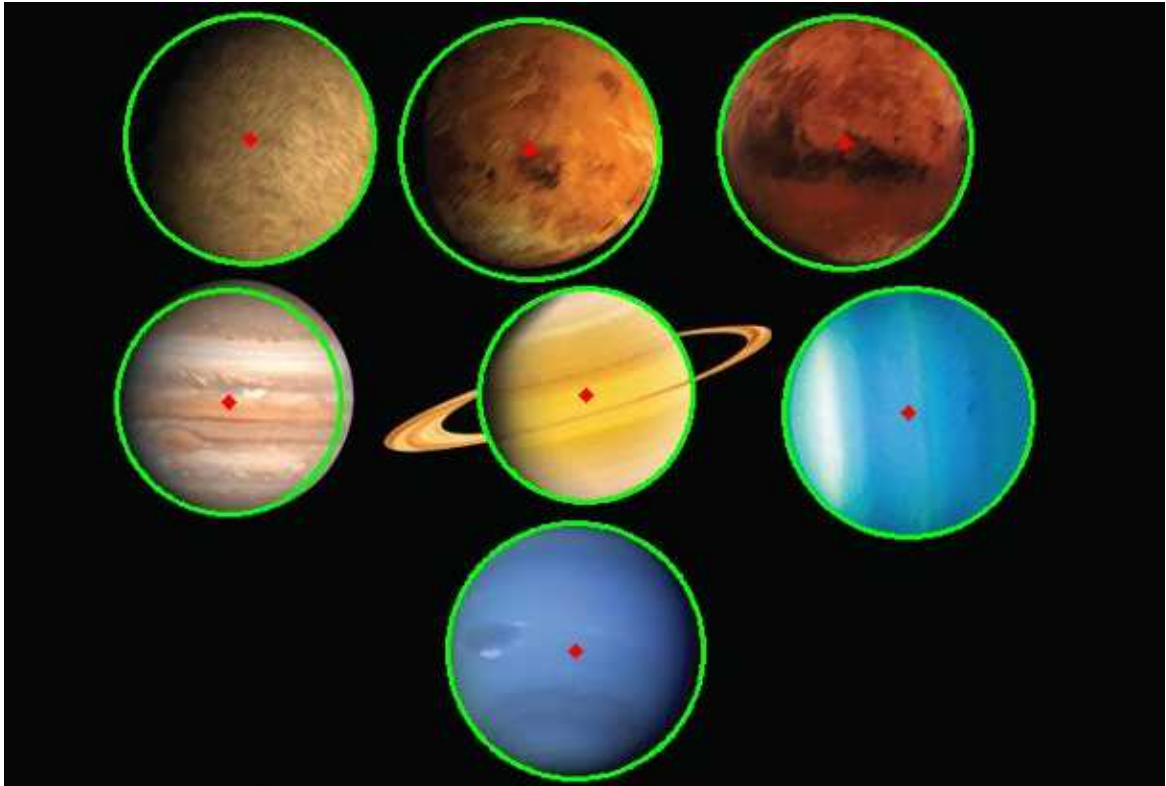


Figure 3.6: Results of Hough circle detection on a stylized image of the planets.

Detecting other shapes

OpenCV's implementations of the Hough transform are limited to detecting lines and circles; however, we already implicitly explored shape detection in general when we talked about `approxPolyDP`. This function allows for the approximation of polygons, so if your image contains polygons, they will be accurately detected through the combined use of `cv2.findContours` and `cv2.approxPolyDP`.

Summary

At this point, you should have gained a good understanding of color models, the Fourier transform, and several kinds of filters that have been made available by OpenCV to process images.

You should also be proficient in detecting edges, lines, circles, and shapes in general. Additionally, you should be able to find contours and exploit the information they provide about the subjects contained in an image. These concepts are complementary to the next chapter's topics – namely, the segmentation of an image according to depth and estimating the distance of a subject in an image.

Join our book community on Discord

<https://discord.gg/djGjeMECxx>



Computer vision makes many futuristic-sounding tasks a reality. Two such tasks are face detection (locating faces in an image) and face recognition (identifying a face as belonging to a specific person). OpenCV implements several algorithms for face detection and recognition. These have applications in all sorts of real-world contexts, from security to entertainment.

This chapter introduces some of OpenCV's face detection and recognition functionality, along with data files that define particular types of trackable objects. Specifically, in this chapter, we'll look at Haar cascade classifiers, which analyze the contrast between adjacent image regions to determine whether or not a given image or sub-image matches a known type. We consider how to combine multiple Haar cascade classifiers in a hierarchy so that one classifier identifies a parent region (for our purposes, a face) and other classifiers identify child regions (such as eyes).

We also take a detour into the humble but important subject of rectangles. By drawing, copying, and resizing rectangular image regions, we can perform simple manipulations on image regions that we are tracking.

We will cover the following topics:

- Understanding Haar cascades
- Finding the pre-trained Haar cascades that come with OpenCV. These include several face detectors
- Using Haar cascades to detect faces in still images and videos
- Gathering images to train and test a face recognizer
- Using several different face recognition algorithms: **Eigenfaces** , **Fisherfaces** , and **Local Binary Pattern Histograms (LBPHs)**
- Copying rectangular regions from one image to another, with or without a mask
- Using a depth camera to distinguish between a face and the background based on depth
- Swapping two people's faces in an interactive application

While this chapter focuses on classic approaches to face detection and recognition, we will go on to explore advanced new models by using OpenCV with a variety of other libraries in *Chapter 11 , Neural Networks with OpenCV – an Introduction* .

By the end of this chapter, we will have integrated face tracking and rectangle manipulations into Cameo, the interactive application that we have developed in previous chapters. Finally, we will have some face-to-face interaction!

Technical requirements

This chapter uses Python, OpenCV, and NumPy. As part of OpenCV, it uses the optional `opencv_contrib` modules, which include functionality for face recognition. Some parts of this chapter use OpenCV's optional support for OpenNI 2 to capture images from depth cameras. Please refer back to *Chapter 1 , Setting Up OpenCV* , for installation instructions.

The complete code for this chapter can be found in this book's GitHub repository, <https://github.com/PacktPublishing/Learning-OpenCV-5-Computer-Vision-with-Python-Fourth-Edition> , in the `chapter05` folder. Sample images are in the repository `images` folder.

A subset of the chapter's sample code can be edited and run interactively in Google Colab at <https://colab.research.google.com/github/PacktPublishing/Learning-OpenCV-5-Computer-Vision-with-Python-Fourth-Edition/blob/main/chapter05/chapter05.ipynb> .

Conceptualizing Haar cascades

When we talk about classifying objects and tracking their location, what exactly are we hoping to pinpoint? What constitutes a recognizable part of an object?

Photographic images, even from a webcam, may contain a lot of detail for our (human) viewing pleasure. However, image detail tends to be unstable with respect to variations in lighting, viewing angle, viewing distance, camera shake, and digital noise. Moreover, even real differences in physical detail might not interest us for classification. Joseph Howse, one of this book's authors, was taught in school that no two snowflakes look alike under a microscope. Fortunately, as a Canadian child, he had already learned how to recognize snowflakes without a microscope, as the similarities are more obvious in bulk.

Hence, having some means of abstracting image detail is useful in producing stable classification and tracking results. The abstractions are called **features**, which are said to be **extracted** from the image data. There should be far fewer features than pixels, though any pixel might influence multiple features. A set of features is represented as a **vector** (conceptually, a set of coordinates in a multidimensional space), and the level of similarity between two images can be evaluated based on some measure of the distance between the images' corresponding feature vectors.

Later, in *Chapter 6, Retrieving Images and Searching Using Image Descriptors*, we will explore several kinds of features, as well as advanced ways of describing and matching sets of features.

Haar-like features are one type of feature that is often applied to real-time face detection. They were first used for this purpose in the paper *Robust Real-Time Face Detection*, by Paul Viola and Michael Jones (*International Journal of Computer Vision* 57(2), 137–154, Kluwer Academic Publishers, 2001). An electronic version of this paper is available at <http://comp3204.ecs.soton.ac.uk/cw/viola04ijcv.pdf>.

Each Haar-like feature describes the pattern of contrast among adjacent image regions. For example, edges, vertices, and thin lines each generate a kind of feature. Some features are distinctive in the sense that they typically occur in a certain class of object (such as a face) but not in other objects. These distinctive features can be organized into a hierarchy, called a **cascade**, in which the highest layers contain features of greatest distinctiveness, enabling a classifier to quickly reject subjects that lack these features. If a subject is a good match for the higher-layer features, then the classifier considers the lower-layer features too in order to weed out more false positives.

For any given subject, the features may vary depending on the scale of the image and the size of the neighborhood (the region of nearby pixels) in which contrast is being evaluated. The neighborhood's size is called the **window size**. To make a Haar cascade classifier **scale-invariant** or, in other words, robust to changes in scale, the window size is kept constant but images are rescaled a number of times; hence, at some level of rescaling, the size of an object (such as a face) may match the window size. Together, the original image and the rescaled images are called an **image pyramid**, and each successive level in this pyramid is a smaller rescaled image. OpenCV provides a scale-invariant classifier that can load a Haar cascade from an XML file in a particular format. Internally, this classifier converts any given image into an image pyramid.

Haar cascades, as implemented in OpenCV, are not robust to changes in rotation or perspective. For example, an upside-down face is not considered similar to an upright face and a face viewed in profile is not considered similar to a face viewed from the front. A more complex and resource-intensive implementation could improve a Haar cascade's robustness to rotation by considering multiple transformations of images as well as multiple window sizes. However, we will confine ourselves to the implementation in OpenCV.

Getting Haar cascade data

Your installation of OpenCV 5 should contain a subfolder called `data` . The path to this folder is stored in an OpenCV variable called `cv2.data.harcascades` .

The `data` folder contains XML files that can be loaded by an OpenCV class called `cv2.CascadeClassifier` . An instance of this class interprets a given XML file as a Haar cascade, which provides a detection model for a type of object such as a face. `cv2.CascadeClassifier` can detect this type of object in any image. As usual, we could obtain a still image from a file, or we could obtain a series of frames from a video file or a video camera.

From the `data` folder, we will use the following cascade files:

As their names suggest, these cascades are for detecting faces and eyes. They require a frontal, upright view of the subject. We will use them later when building a face detector.

If you are curious about how these cascade files are generated, you can find more information in Joseph Howse's book, *OpenCV 4 for Secret Agents* (Packt Publishing, 2019), specifically in *Chapter 3 , Training a Smart Alarm to Recognize the Villain and His Cat* . With a lot of patience and a reasonably powerful computer, you can make your own cascades and train them for various types of objects.

Using OpenCV to perform face detection

With `cv2.CascadeClassifier` , it makes little difference whether we perform face detection on a still image or a video feed. The latter is just a sequential version of the former: face detection on a video is simply face detection applied to each frame. Naturally, with more advanced techniques, it would be possible to track a detected face continuously across multiple frames and determine that the face is the same one in each frame. However, it is good to know that a basic sequential approach also works.

Let's go ahead and detect some faces.

Performing face detection on a still image

The first and most basic way to perform face detection is to load an image and detect faces in it. To make the result visually meaningful, we will draw rectangles around faces in the original image. Remembering that the face detector is designed for upright, frontal faces, we will use an image of a row of people, specifically woodcutters, standing shoulder to shoulder and facing the photographer or viewer.

Let's go ahead and create the following basic script to perform face detection:

Let's walk through the preceding code in small steps. First, we use the obligatory `cv2` import that you will find in every script in this book. Then, we declare a `face_cascade` variable, which is a `CascadeClassifier` object that loads a cascade for face detection:

We then load our image file with `cv2.imread` and convert it into grayscale because `CascadeClassifier` , like many of OpenCV's classifiers, expects grayscale images. (If we try to use a color image, `CascadeClassifier` will internally convert it to grayscale anyway.) The next step, `face_cascade.detectMultiScale` , is where we perform the actual face detection:

The parameters of `detectMultiScale` include `scaleFactor` and `minNeighbors` . The `scaleFactor` argument, which should be greater than 1.0, determines the downscaling ratio of the image at each iteration of the face detection process. As we discussed earlier in the *Conceptualizing Haar cascades* section, this downscaling is intended to achieve scale invariance by matching various faces to the window size. The `minNeighbors` argument is the minimum number of overlapping detections that are required in order to retain a detection result. Normally, we expect that a face may be detected in multiple overlapping windows, and a greater number of overlapping detections makes us more confident that the detected face is truly a face.

The value returned from the detection operation is a list of tuples that represent the face rectangles. OpenCV's `cv2.rectangle` function allows us to draw rectangles at the specified coordinates. `x` and `y` represent the left and top coordinates, while `w` and `h` represent the width and height of the face rectangle. We draw cyan rectangles around all of the faces we find by looping through the `faces` variable, making sure we use the original image for drawing, not the gray version:

Lastly, we call `cv2.imshow` to display the resulting processed image and we call `cv2.imwrite` to save it. As usual, to prevent the image window from closing automatically, we insert a call to `waitKey` , which returns when the user presses any key:

And there we go, three members of the band of woodcutters have been detected in our image, as shown in the following screenshot:



Figure 5.1: Face detection results of a photograph of woodcutters (Image credit: Prokudin-Gorsky)

The photograph in this example is the work of Sergey Prokudin-Gorsky (1863-1944), a pioneer of color photography. Tsar Nicholas II sponsored Prokudin-Gorsky to photograph people and places throughout the Russian Empire as a vast documentary project. Prokudin-Gorsky photographed these woodcutters near the Svir river, in northwestern Russia, in 1909.

Here, we have no false positive detections; all three rectangles really are woodcutters' faces. However, we do have two false negatives (woodcutters whose faces were not detected). Try adjusting the parameters of `face_cascade.detectMultiScale` to see how the results change. Then, let's proceed to a more interactive example.

Performing face detection on a video

We now understand how to perform face detection on a still image. As mentioned previously, we can repeat the process of face detection on each frame of a video (be it a camera feed or a pre-recorded video file).

The next script will open a camera feed, read a frame, examine that frame for faces, and scan for eyes within the detected faces. Finally, it will draw blue rectangles around the faces and green rectangles around the eyes. Here is the script in its entirety:

Let's break up the preceding sample into smaller, more digestible chunks:

1. As usual, we import the `cv2` module. After that, we initialize two `CascadeClassifier` objects, one for faces and another for eyes:
1. As in most of our interactive scripts, we open a camera feed and start iterating over frames. We continue until the user presses any key. Whenever we successfully capture a frame, we convert it into grayscale as our first step in processing it:
1. We detect faces with the `detectMultiScale` method of our face detector. As we have previously done, we use the `scaleFactor` and `minNeighbors` arguments. We also use the `minSize` argument to specify a minimum size of a face, specifically 120x120. No attempt will be made to detect faces smaller than this. (Assuming that our user is sitting close to the camera, it is safe to say that the user's face will be larger than 120x120 pixels.) Here is the call to `detectMultiScale` :
1. We iterate over the rectangles of the detected faces. We draw a blue border around each rectangle in the original color image. Then, within the same rectangular region of the grayscale image, we perform eye detection:

The eye detector is a bit less accurate than the face detector. You might see shadows, parts of the frames of glasses, or other regions of the face falsely detected as eyes. To improve the results, you could try defining `roi_gray` as a smaller region of the face, since we can make a good guess about the eyes' location in an upright face. You could also try using a `maxSize` argument to avoid false positives that are too large to be eyes. Also, you could adjust `minSize` and `maxSize` so that the dimensions are proportional to `w` and `h` ,

the size of the detected face. As an exercise, feel free to experiment with changes to these and other parameters.

1. We loop through the resulting eye rectangles and draw green outlines around them:

1. Finally, we show the resulting frame in the window:

Run the script. If our detectors produce accurate results, and if any face is within the field of view of the camera, you should see a blue rectangle around the face and a green rectangle around each eye, as shown in this screenshot:

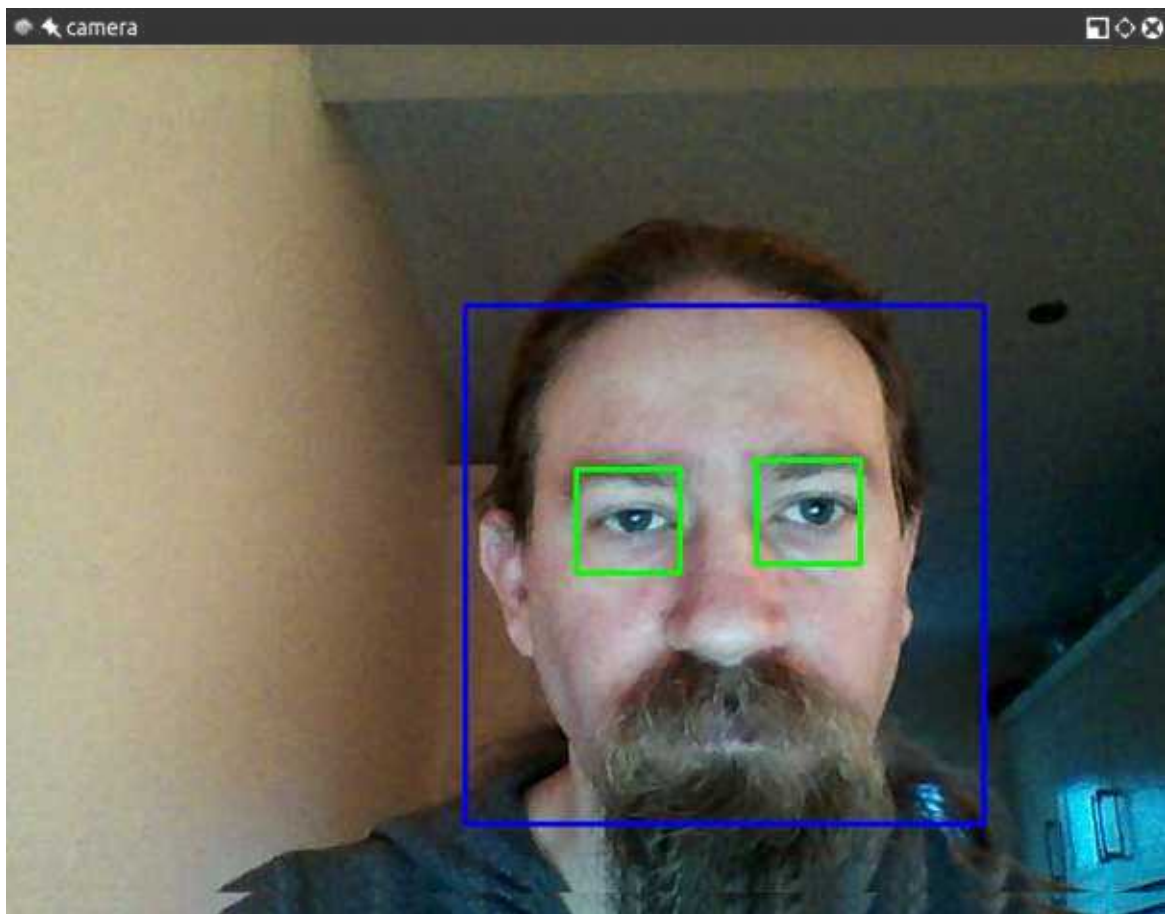


Figure 5.2: The resulting detection of a face and eyes in a photograph

Experiment with this script to see how the face and eye detectors perform under various conditions. Try a brighter or darker room. If you wear glasses, try removing them. Try various people's faces and various expressions. Adjust the detection parameters in the script to see how they affect the results. When you are satisfied, let's consider what else we can do with faces in OpenCV.

Performing face recognition

Detecting faces is a fantastic feature of OpenCV and one that constitutes the basis for a more advanced operation: face recognition. What is face recognition? It is the ability of a program, given an image or a video feed containing a person's face, to identify that person. One of the ways to achieve this (and the approach adopted by OpenCV) is to *train* the program by feeding it a set of classified pictures (a facial database) and perform recognition based on the features of those pictures.

Another important feature of OpenCV's face recognition module is that each recognition has a confidence score, which allows us to set thresholds in real-life applications to limit the incidence of false identifications.

Let's start from the very beginning; to perform face recognition, we need faces to recognize. We fulfill this requirement in two ways: supply the images ourselves or obtain freely available face databases. A large directory of face databases is available online at <http://www.face-rec.org/databases/> . Here are a few notable examples from the directory:

- Yale Face Database (Yalefaces): <http://vision.ucsd.edu/content/yale-face-database>
- Extended Yale Face Database B: <http://vision.ucsd.edu/content/extended-yale-face-database-b-b>
- Database of Faces (from AT&T Laboratories Cambridge): <https://cam-orl.co.uk/facedatabase.html>

If we trained a face recognizer on these samples, we would then have to run face recognition on an image that contains the face of one of the sampled people. This process might be educational, but perhaps not as satisfying as providing images of our own. You probably had the same thought that many computer vision learners have had: I wonder if I can write a program that recognizes my face with a certain degree of confidence. Indeed, you can and soon you will!

Generating the data for face recognition

Let's go ahead and write a script that will generate those images for us. A few images containing different expressions are all that we need, but it is preferable that the training images are square and are all the same size. Our sample script uses a size of 200x200, but most freely available datasets have smaller images than this.

Here is the script itself:

Here, we are generating sample images by building on our newfound knowledge of how to detect a face in a video feed. We are detecting a face, cropping that region of the grayscale-converted frame, resizing it to be 200x200 pixels, and saving it as a PGM file with a name in a particular folder (in this case, `jm`, one of the authors' initials; you can use your own initials). Like many of our windowed applications, this one runs until the user presses any key.

The `count` variable is present because we needed progressive names for the images. Run the script for a few seconds, change your facial expression a few times, and check the destination folder you specified in the script. You will find a number of images of your face, grayed, resized, and named with the format `<count>.pgm`.

Modify the `output_folder` variable to make it match your name. For example, you might choose `'../data/at/my_name'`. Run the script, wait for it to detect your face in a number of frames (say, 20 or more), and then press any key to quit. Now, modify the `output_folder` variable again to make it match the name of a friend whom you also want to recognize. For example, you might choose `'../data/at/name_of_my_friend'`. Do not change the base part of the folder (in this case, `'../data/at'`) because later, in the *Loading the training data for face recognition* section, we will write code that loads the training images from all of the subfolders of this base folder. Ask your friend to sit in front of the camera, run the script again, let it detect your friend's face in a number of frames, and then quit. Repeat this process for any additional people you might want to recognize.

Let's now move on to try and recognize the user's face in a video feed. This should be fun!

Choosing a face recognition algorithm

OpenCV 5 implements three different algorithms for recognizing faces: **Eigenfaces** , **Fisherfaces** , and **Local Binary Pattern Histograms (LBPHs)**. Eigenfaces and Fisherfaces are derived from a more general-purpose algorithm called **Principal Component Analysis (PCA)**. For a detailed description of the algorithms, refer to the following links:

- **PCA** : *A Tutorial on Principal Component Analysis* (2013), by Jonathon Shlens, is available at <http://arxiv.org/pdf/1404.1100v1.pdf> . This algorithm was invented in 1901 by Karl Pearson, and the original paper, *On Lines and Planes of Closest Fit to Systems of Points in Space* , is available at <http://pca.narod.ru/pearson1901.pdf> .
- **Eigenfaces** : The paper *Eigenfaces for Recognition* (1991), by Matthew Turk and Alex Pentland, is available at <http://www.cs.ucsb.edu/~mturk/Papers/jcn.pdf> .
- **Fisherfaces** : The seminal paper *The Use of Multiple Measurements in Taxonomic Problems* (1936), by R. A. Fisher, is available at <http://onlinelibrary.wiley.com/doi/10.1111/j.1469-1809.1936.tb02137.x/pdf> .
- **Local Binary Pattern** : The first paper describing this algorithm is *Performance evaluation of texture measures with classification based on Kullback discrimination of distributions* (1994), by T. Ojala, M. Pietikainen, and D. Harwood. It is available at <https://ieeexplore.ieee.org/document/576366> .

For this book's purposes, let's just take a high-level overview of the algorithms. First and foremost, they all follow a similar process; they take a set of classified observations (our face database, containing numerous samples per individual), train a model based on it, perform an analysis of face images (which may be face regions that we detected in an image or video), and determine two things: the subject's identity and a measure of confidence that this identification is correct. The latter is commonly known as the **confidence score** .

Eigenfaces performs PCA, which identifies principal components of a certain set of observations (again, your face database), calculates the divergence of the current observation (the face being detected in an image or frame) compared to the dataset, and produces a value. The smaller the value, the smaller the difference between the face database and the detected face; hence, a value of 0 is an exact match.

Fisherfaces also derives from PCA and evolves the concept, applying more complex logic. While computationally more intensive, it tends to yield more

accurate results than Eigenfaces.

LBPH instead divides a detected face into small cells and, for each cell, builds a histogram that describes whether the brightness of the image is increasing when comparing neighboring pixels in a given direction. This cell's histogram can be compared to the histogram of the corresponding cell in the model, producing a measure of similarity. Of the face recognizers in OpenCV, the implementation of LBPH is the only one that allows the model sample faces and the detected faces to be of a different shape and size. Hence, it is a convenient option, and the authors of this book find that its accuracy compares favorably to the other two options.

Despite the algorithms' differences, OpenCV provides a similar interface for all three, as we shall soon see.

Loading the training data for face recognition

Regardless of our choice of face recognition algorithm, we can load the training images in the same way. Earlier, in the *Generating the data for face recognition* section, we generated training images and saved them in folders that were organized according to people's names or initials. For example, the following folder structure could contain sample face images of this book's authors, Joseph Howse (`jh`) and Joe Minichino (`jm`):

Let's write a script that loads these images and labels them in a way that OpenCV's face recognizers will understand. To work with the filesystem and the data, we will use the Python standard library's `os` module, as well as the `cv2` and `numpy` modules. Let's create a script that starts with the following `import` statements:

Let's add the following `read_images` function, which walks through a directory's subdirectories, loads the images and shows each of them for user information purposes, resizes them to a specified size, and puts the resized images in a list. The order of this list has no significance in itself but, at the same time, our function builds two corresponding lists that describe the images: first, a corresponding list of people's names or initials (based on the subfolder names), and second, a corresponding list of labels or numeric IDs associated with the loaded images. For example, `jh` could be a name and `0` could be the label for all images that were loaded from the `jh` subfolder. Finally, the function converts the lists of images and labels into NumPy arrays, and it returns three variables: the

list of names, the NumPy array of images, and the NumPy array of labels. Here is the function's implementation:

Let's call our `read_images` function by adding code such as the following:

Edit the `path_to_training_images` variable in the preceding code block to ensure that it matches the base folder of the `output_folder` variables you defined earlier in the code for the section *Generating the data for face recognition* .

So far, we have training data in a useful format but we have not yet created a face recognizer or performed any training. We will do so in the next section, where we continue the implementation of the same script.

Performing face recognition with Eigenfaces

Now that we have an array of training images and an array of their labels, we can create and train a face recognizer with just two more lines of code:

What have we done here? We created the Eigenfaces face recognizer with OpenCV's `cv2.EigenFaceRecognizer_create` function, and we trained the recognizer by passing the arrays of images and labels (numeric IDs). Optionally, we could have passed two arguments to `cv2.EigenFaceRecognizer_create` :

- `num_components` : This is the number of components to keep for the PCA.
- `threshold` : This is a floating-point value specifying a confidence threshold. Faces with a confidence score below the threshold will be discarded. By default, the threshold is the maximum floating-point value so that no faces are discarded.

Having trained this model, we could save it to a file using code such as `model.write('my_model.xml')` . Later (for example, in another script), we could skip the training step and just load the pre-training model from the file using code such as `model.read('my_model.xml')` . However, for the sake of our simple demo, we will just train and test the model in one script without saving the model to a file.

To test this recognizer, let's use a face detector and a video feed from a camera. As we have done in previous scripts, we can use the following line of code to initialize the face detector:

The following code initializes the camera feed, iterates over frames (until the user presses any key), and performs face detection and recognition on each frame:

Let's walk through the most important functionality of the preceding block of code. For each detected face, we convert and resize it so that we have a grayscale version that matches the expected size (in this case, 200x200 pixels as defined by the `training_image_size` variable in the previous section, *Loading the training data for face recognition*). Then, we pass the resized, grayscale face to the face recognizer's `predict` function. This returns a label and confidence score. We look up the person's name corresponding to the numeric label of that face. (Remember that we created the `names` array in the previous section, *Loading the training data for face recognition* .) We draw the name and confidence score in blue text above the recognized face. After iterating over all detected faces, we display the annotated image.

We have taken a simple approach to face detection and recognition, and it serves the purpose of enabling you to have a basic application running and understand the process of face recognition in OpenCV 5. To improve upon this approach and make it more robust, you could take further steps such as correctly aligning and rotating detected faces so that the accuracy of the recognition is maximized.

When you run the script, you should see something similar to the following screenshot:

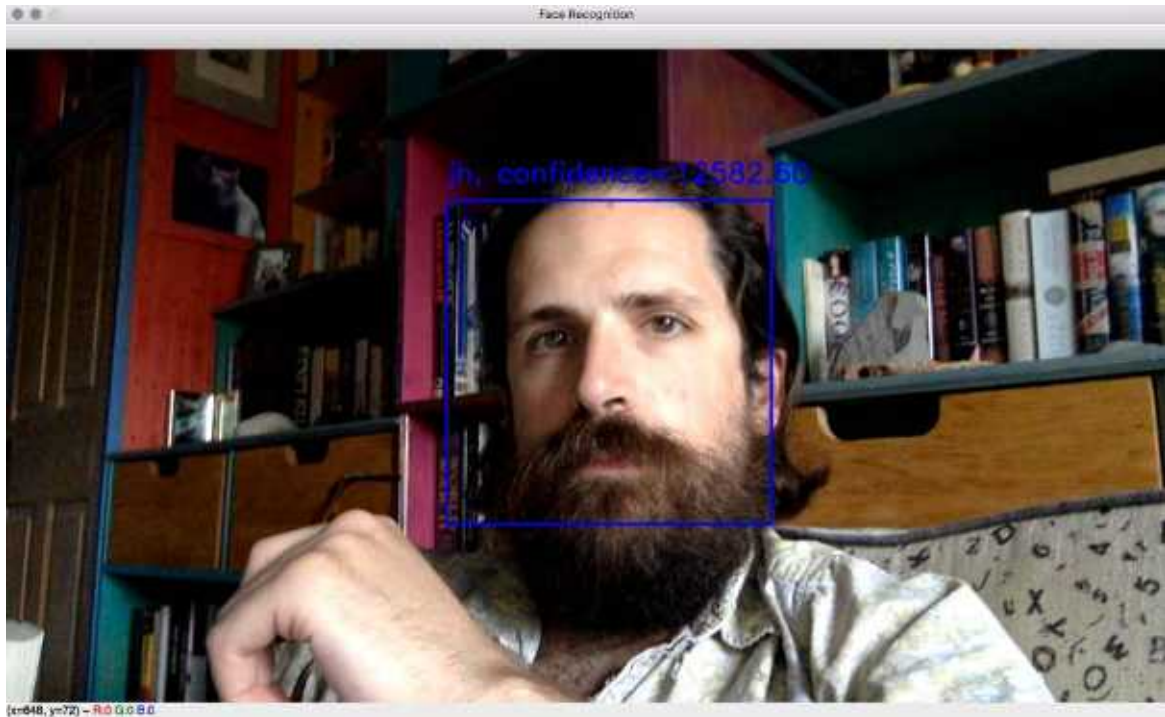


Figure 5.3: A face recognition result using live camera input

Next, let's consider how we would adapt these script to replace Eigenfaces with another face recognition algorithm.

Performing face recognition with Fisherfaces

What about Fisherfaces? The process does not change much; we simply need to instantiate a different algorithm. With default arguments, the declaration of our `model` variable would look like this:

`cv2.face.FisherFaceRecognizer_create` takes the same two optional arguments as `cv2.createEigenFaceRecognizer_create` : the number of principal components to keep and the confidence threshold. As you might guess, these types of `*_create` functions are quite common in OpenCV for situations where several different algorithms share a common interface. Let's look at one more example.

Performing face recognition with LBPH

For the LBPH algorithm, again, the process is similar. However, the algorithm factory takes the following optional parameters (in order):

- `radius` : The pixel distance between the neighbors that are used to calculate a cell's histogram (by default, 1)
- `neighbors` : The number of neighbors used to calculate a cell's histogram (by default, 8)
- `grid_x` : The number of cells into which the face is divided horizontally (by default, 8)
- `grid_y` : The number of cells into which the face is divided vertically (by default, 8)
- `confidence` : The confidence threshold (by default, the highest possible floating-point value so that no results are discarded)

With default arguments, the model declaration would look like this:

Note that, with LBPH, we do not need to resize images as the division into grids allows a comparison of patterns identified in each cell.

Having looked at the available face recognition algorithms, let's now consider how to evaluate a recognition result.

Discarding results based on the confidence score

The `predict` method returns a tuple, in which the first element is the label of the recognized individual and the second is the confidence score. All algorithms come with the option of setting a confidence score threshold, which measures the distance of the recognized face from the original model; therefore, a score of 0 signifies an exact match.

There may be cases in which you would rather retain all recognitions and then apply further processing, so you can come up with your own algorithms to estimate the confidence score of a recognition. For example, if you are trying to identify people in a video, you may want to analyze the confidence score in subsequent frames to establish whether the recognition was successful or not. In this case, you can inspect the confidence score obtained by the algorithm and draw your own conclusions.

The typical range of the confidence score depends on the algorithm. Eigenfaces and Fisherfaces produce values (roughly) in the range of 0 to 20,000, with any score below 4,000-5,000 being a quite confident recognition. For LBPH, a good recognition scores (roughly) below 50, and any value above 80 is considered a poor confidence score.

A normal custom approach would be to hold off drawing a rectangle around a recognized face until we have a number of frames with a good confidence score (where “good” is an arbitrary threshold we must choose, based on our algorithm and use case), but you have total freedom to use OpenCV's face recognition module to tailor your application to your needs. Next, let's see how face detection and recognition work in a specialized use case.

Swapping faces in infrared

Face detection and recognition are not limited to the visible spectrum of light. With a **Near-Infrared (NIR)** camera and NIR light source, face detection and recognition are possible even when a scene appears totally dark to the human eye. This capability is quite useful in security and surveillance applications.

We studied the basic usage of NIR depth cameras, such as the Asus Xtion PRO, in *Chapter 4 , Depth Estimation and Segmentation* . We extended the object-oriented code of our interactive application, Cameo. We captured frames from a depth camera. Based on depth, we segmented each frame into a main layer (such as the user's face) and other layers. We painted the other layers black. This achieved the effect of hiding the background so that only the main layer (the user's face) appeared on-screen in the interactive video feed.

Now, let's modify Cameo to do something that exercises our previous skills in depth segmentation and our new skills in face detection. Let's detect faces and, when we detect at least two faces in a frame, let's swap the faces so that one person's head appears atop another person's body. Rather than copying all pixels in a detected face rectangle, we will only copy the pixels that are part of the main depth layer for that rectangle. This should achieve the effect of swapping faces but not the background pixels surrounding the faces.

Once the changes are complete, Cameo will be able to produce output such as the following screenshot:



Figure 5.4: The result of swapping two faces in a frame from an infrared camera.

Here, we see the face of Joseph Howse swapped with the face of Janet Howse, his mother. Although Cameo is copying pixels from rectangular regions (and this is clearly visible at the bottom of the swapped regions, in the foreground), some of the background pixels within the rectangles are masked based on depth and thus are not swapped, so we do not see rectangular edges everywhere.

You can find all of the relevant changes to the Cameo source code in this book's repository at <https://github.com/PacktPublishing/Learning-OpenCV-5-Computer-Vision-with-Python-Fourth-Edition> , specifically in the `chapter05/cameo` folder. For brevity, we will not discuss all of the changes here in this book, but we will cover some of the highlights in the next two subsections, *Modifying the application's loop* and *Masking a copy operation* .

Modifying the application's loop

To support face swapping, the Cameo project has two new modules called `rects` and `trackers`. The `rects` module contains functions for copying and swapping rectangles, with an optional mask to limit the copy or swap operation to particular pixels. The `trackers` module contains a class called `FaceTracker`, which adapts OpenCV's face detection functionality to an object-oriented style of programming.

As we have covered OpenCV's face detection functionality earlier in this chapter, and we have demonstrated an object-oriented programming style in previous chapters, we will not go into the `FaceTracker` implementation here. Instead, you may look at it in this book's repository.

Let's open `cameo.py` so that we can walk through the overall changes to the application:

1. Near the top of the file, we need to import our new modules, as highlighted in the following code block:
1. Now, let's turn our attention to changes in the `__init__` method of our `CameoDepth` class. Our updated application uses an instance of `FaceTracker`. As part of its functionality, `FaceTracker` can draw rectangles around detected faces. Let's give Cameo's user the option to enable or disable the drawing of face rectangles. We will keep track of the currently selected option via a Boolean variable. The following code block highlights the necessary changes to initialize the `FaceTracker` object and the Boolean variable:

We make use of the `FaceTracker` object in the `run` method of `CameoDepth`, which contains the application's main loop that captures and processes frames. Every time we successfully capture a frame, we call methods of `FaceTracker` to update the face detection result and get the latest detected faces. Then, for each face, we create a mask based on the depth camera's disparity map. (Previously, in *Chapter 4*, *Depth Estimation and Segmentation*, we created such a mask for the entire image instead of a mask for each face rectangle.)

1. Then, we call a function, `rects.swapRects`, to perform a masked swap of the face rectangles. (We will look at the implementation of `swapRects` a little later, in the *Masking a copy operation* section.) Depending on the currently selected option, we might tell `FaceTracker` to draw rectangles around the faces. All of the relevant changes are highlighted in the following code block:

1. Finally, let's modify the `onKeyPress` method so that the user can hit the X key to start or stop displaying rectangles around detected faces. Again, the relevant changes are highlighted in the following code block:

Next, let's look at the implementation of the `rects` module that we imported earlier in this section.

Masking a copy operation

The `rects` module is implemented in `rects.py`. We already saw a call to the `rects.swapRects` function in the previous section. However, before we consider the implementation of `swapRects`, we first need a more basic `copyRect` function.

As far back as *Chapter 2, Handling Files, Cameras, and GUIs*, we learned how to copy data from one rectangular **region of interest (ROI)** to another using NumPy's slicing syntax. Outside the ROIs, the source and destination images were unaffected. Now, we want to apply further limits to this copy operation. We want to use a given mask that has the same dimensions as the source rectangle.

We shall copy only those pixels in the source rectangle where the mask's value is not zero. Other pixels shall retain their old values from the destination image. This logic, with an array of conditions and two arrays of possible output values, can be expressed concisely with the `numpy.where` function.

With this approach in mind, let's consider our `copyRect` function. As arguments, it takes a source and destination image, a source and destination rectangle, and a mask. The latter may be `None`, in which case, we simply resize the content of the source rectangle to match the destination rectangle and then assign the resulting resized content to the destination rectangle. Otherwise, we next ensure that the mask and the images have the same number of channels. We assume that the mask has one channel but the images may have three channels (BGR). We can add duplicate channels to mask using the `repeat` and `reshape` methods of `numpy.array`. Finally, we perform the copy operation using `numpy.where`. The complete implementation is as follows:

We also need to define a `swapRects` function, which uses `copyRect` to perform a circular swap of a list of rectangular regions. `swapRects` has a `masks` argument, which is a list of masks whose elements are passed to the respective `copyRect`

calls. If the value of the `masks` argument is `None` , we pass `None` to every `copyRect` call. The following code shows the full implementation of `swapRects` :

Note that the `mask` argument in `copyRect` and the `masks` argument in `swapRects` both have a default value of `None` . If no mask is specified, these functions copy or swap the entire contents of the rectangle or rectangles.

Summary

By now, you should have a good understanding of how face detection and face recognition work and how to implement them in Python and OpenCV 5.

The accuracy of detection and recognition algorithms heavily depends on the quality of the training data, so make sure you provide your applications with a large number of training images covering a variety of expressions, poses, and lighting conditions. Later in this book, in *Chapter 11 , Neural Networks with OpenCV – an Introduction* , we will look at how to use several robust, pre-trained face detection models that build atop advanced algorithms and large sets of training data.

As human beings, we might be predisposed to think that human faces are particularly recognizable. We might even be overconfident in our own face recognition abilities. However, in computer vision, there is nothing very special about human faces, and we can just as readily use algorithms to find and identify other things. We will begin to do so next in *Chapter 6 , Retrieving Images and Searching Using Image Descriptors* .

Join our book community on Discord

<https://discord.gg/djGjeMECxxw>



Similar to the human eyes and brain, OpenCV can detect the main features of an image and extract them into so-called image descriptors. These features can then be used as a database, enabling image-based searches. Moreover, we can use key points to stitch images together and compose a bigger image. (Think of putting together many pictures to form a 360° panorama.)

This chapter will show you how to detect the features of an image with OpenCV and make use of them to match and search images. Over the course of this chapter, we will take sample images and detect their main features, and then try to find a region of another image that matches the sample image. We will also find the homography or spatial relationship between a sample image and a matching region of another image.

More specifically, we will cover the following tasks:

- Detecting keypoints and extracting local descriptors around the keypoints using any of the following algorithms: **Harris corners** , **SIFT** , **SURF** , or **ORB**

- Matching keypoints using **brute-force algorithms** or the **FLANN algorithm**
- Filtering out bad matches using **KNN** and the **ratio test**
- Finding the homography between two sets of matching keypoints
- Searching a set of images to determine which one contains the best match for a reference image

We will finish this chapter by building a proof-of-concept forensic application. Given a reference image of a tattoo, we will search for a set of images of people in order to find a person with a matching tattoo.

Technical requirements

This chapter uses Python, OpenCV, and NumPy. In regards to OpenCV, we use the optional `opencv_contrib` modules, which include additional algorithms for keypoint detection and matching. To enable the **SURF** algorithm (which is patented and *not* free for commercial use), we must configure the `opencv_contrib` modules with the `OPENCV_ENABLE_NONFREE` flag in CMake. Please refer to *Chapter 1 , Setting Up OpenCV* , for installation instructions. Additionally, if you have not already installed Matplotlib, install it by running `$ pip install matplotlib` (or `$ pip3 install matplotlib` , depending on your environment).

The complete code for this chapter can be found in this book's GitHub repository, <https://github.com/PacktPublishing/Learning-OpenCV-5-Computer-Vision-with-Python-Fourth-Edition> , in the `chapter06` folder. The sample images can be found in the `images` folder.

A subset of the chapter's sample code can be edited and run interactively in Google Colab at <https://colab.research.google.com/github/PacktPublishing/Learning-OpenCV-5-Computer-Vision-with-Python-Fourth-Edition/blob/main/chapter06/chapter06.ipynb> .

Understanding types of feature detection and matching

A number of algorithms can be used to detect and describe features, and we will explore several of them in this section. The most commonly used feature detection and descriptor extraction algorithms in OpenCV are as follows:

- **Harris** : This algorithm is useful for detecting corners.
- **SIFT** : This algorithm is useful for detecting blobs.
- **SURF** : This algorithm is useful for detecting blobs.
- **FAST** : This algorithm is useful for detecting corners.
- **BRIEF** : This algorithm is useful for detecting blobs.
- **ORB** : This algorithm stands for **Oriented FAST and Rotated BRIEF** . It is useful for detecting a combination of corners and blobs.

Matching features can be performed with the following methods:

- Brute-force matching
- FLANN-based matching

Spatial verification can then be performed with homography.

We have just introduced a lot of new terminology and algorithms. Now, we will go over their basic definitions.

Defining features

What is a feature, exactly? Why is a particular area of an image classifiable as a feature, while others are not? Broadly speaking, a feature is an area of interest in the image that is unique or easily recognizable. **Corners** and regions with a high density of textural detail are good features, while patterns that repeat themselves a lot and low-density regions (such as a blue sky) are not. Edges are good features as they tend to divide two regions

with abrupt changes in the intensity (gray or color) values of an image. A **blob** (a region of an image that greatly differs from its surrounding areas) is also an interesting feature.

Most feature detection algorithms revolve around the identification of corners, edges, and blobs, with some also focusing on the concept of a **ridge**, which you can conceptualize as the axis of symmetry of an elongated object. (Think, for example, about identifying a road in an image.)

Some algorithms are better at identifying and extracting features of a certain type, so it is important to know what your input image is so that you can utilize the best tool in your OpenCV belt.

Detecting Harris corners

Let's start by finding corners using the Harris corner detection algorithm. We will do this by implementing an example. If you continue to study OpenCV beyond this book, you will find that chessboards are a common subject of analysis in computer vision, partly because a checkered pattern is suited to many types of feature detection, and partly because chess is a popular pastime, especially in Russia, which is the country of origin of many of OpenCV's developers.

Here is our sample image of a chessboard and chess pieces:

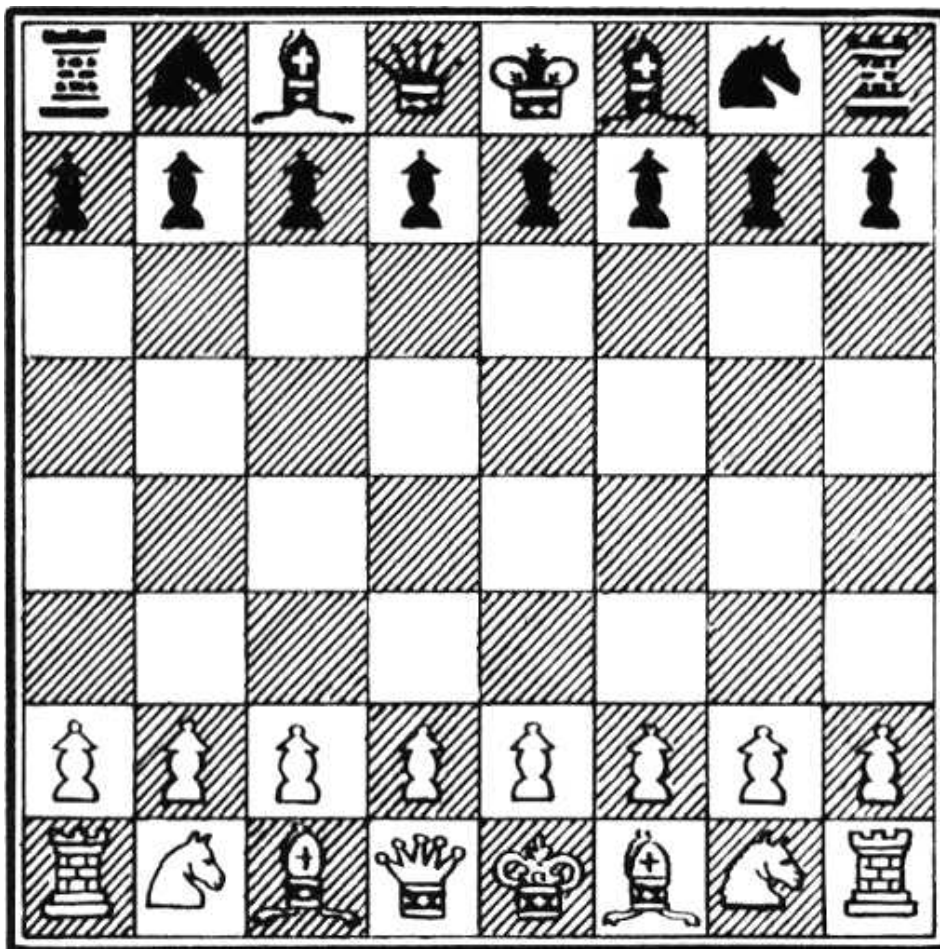


Figure 6.1: A chessboard, to be used for corner detection

OpenCV has a handy function called `cv2.cornerHarris`, which detects corners in an image. We can see this function at work in the following basic example:

Let's analyze the code. After the usual imports, we load the chessboard image and convert it into grayscale. Then, we call the `cornerHarris` function:

The most important parameter here is the third one, which defines the aperture or kernel size of the Sobel operator. The Sobel operator detects edges by measuring horizontal and vertical differences between pixel values in a neighborhood, and it does this using a kernel. The `cv2.cornerHarris` function uses a Sobel operator whose aperture is defined by this parameter. In plain English, the parameters define how sensitive corner detection is. It must be between 3 and 31 and be an odd value. With a low (highly sensitive) value of 3, all those diagonal lines in the black squares of the chessboard will register as corners when they touch the border of the square. For a higher (less sensitive) value of 23, only the corners of each square will be detected as corners.

`cv2.cornerHarris` returns an image in floating-point format. Each value in this image represents a score for the corresponding pixel in the source image. A moderate or high score indicates that the pixel is likely to be a corner. Conversely, we can treat pixels with the lowest scores as non-corners. Consider the following line:

Here, we select pixels with scores that are at least 1% of the highest score, and we color these pixels red in the original image. Here is the result:

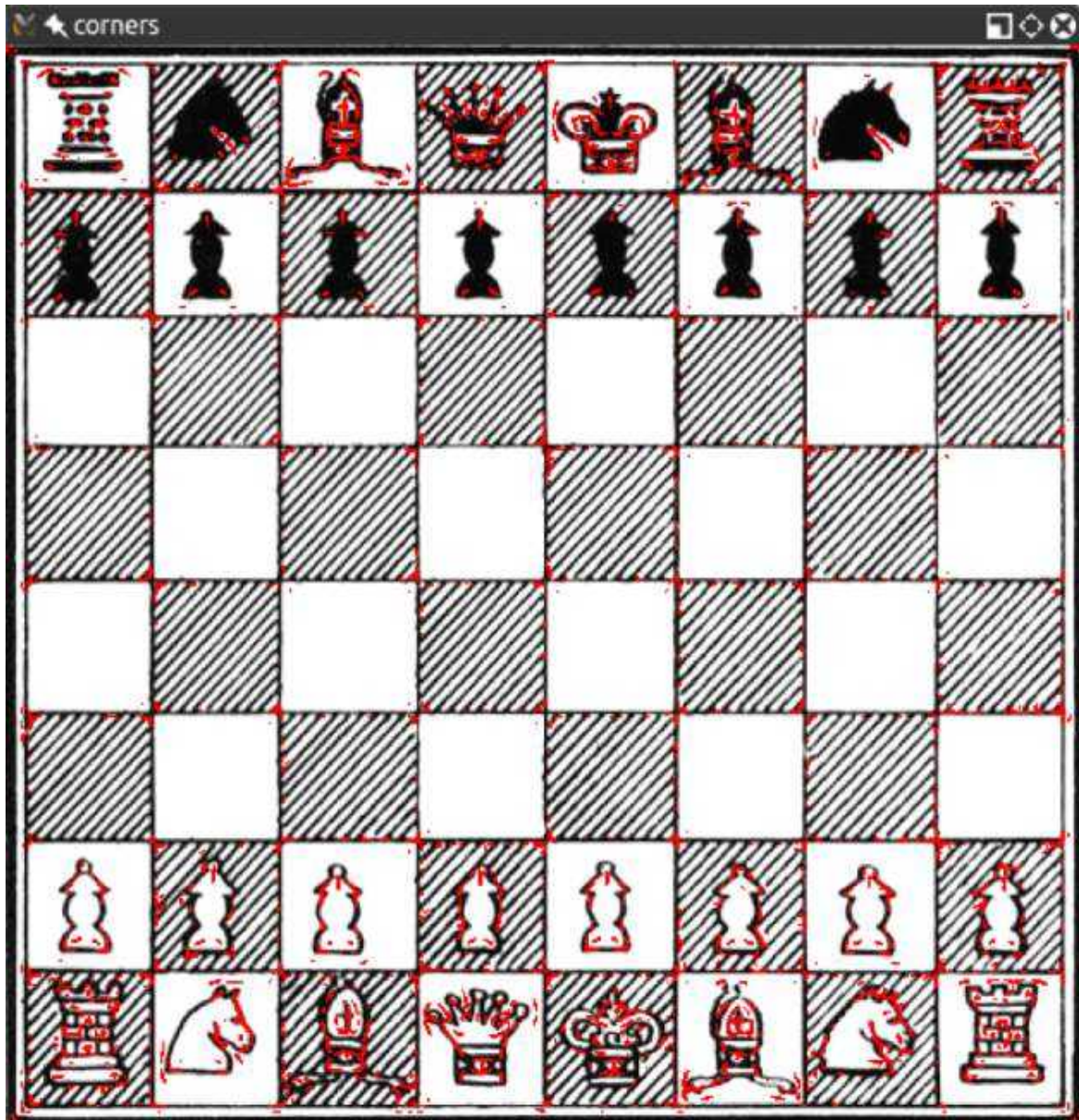


Figure 6.2: Corners detections in a chessboard

Great! Nearly all the detected corners are marked in red. The marked points include nearly all the corners of the chessboard's squares.

If we tweak the second parameter in `cv2.cornerHarris` , we will see that smaller regions (for a smaller parameter value) or larger regions (for a larger parameter value) will be detected as corners. This parameter is called the block size.

Detecting DoG features and extracting SIFT descriptors

The preceding technique, which uses `cv2.cornerHarris`, is great for detecting corners and has a distinct advantage because even if the image is rotated corners are still the corners. However, if we scale an image to a smaller or larger size, some parts of the image may lose or even gain a corner quality.

For example, take a look at the following corner detections in an image of the F1 Italian Grand Prix track:

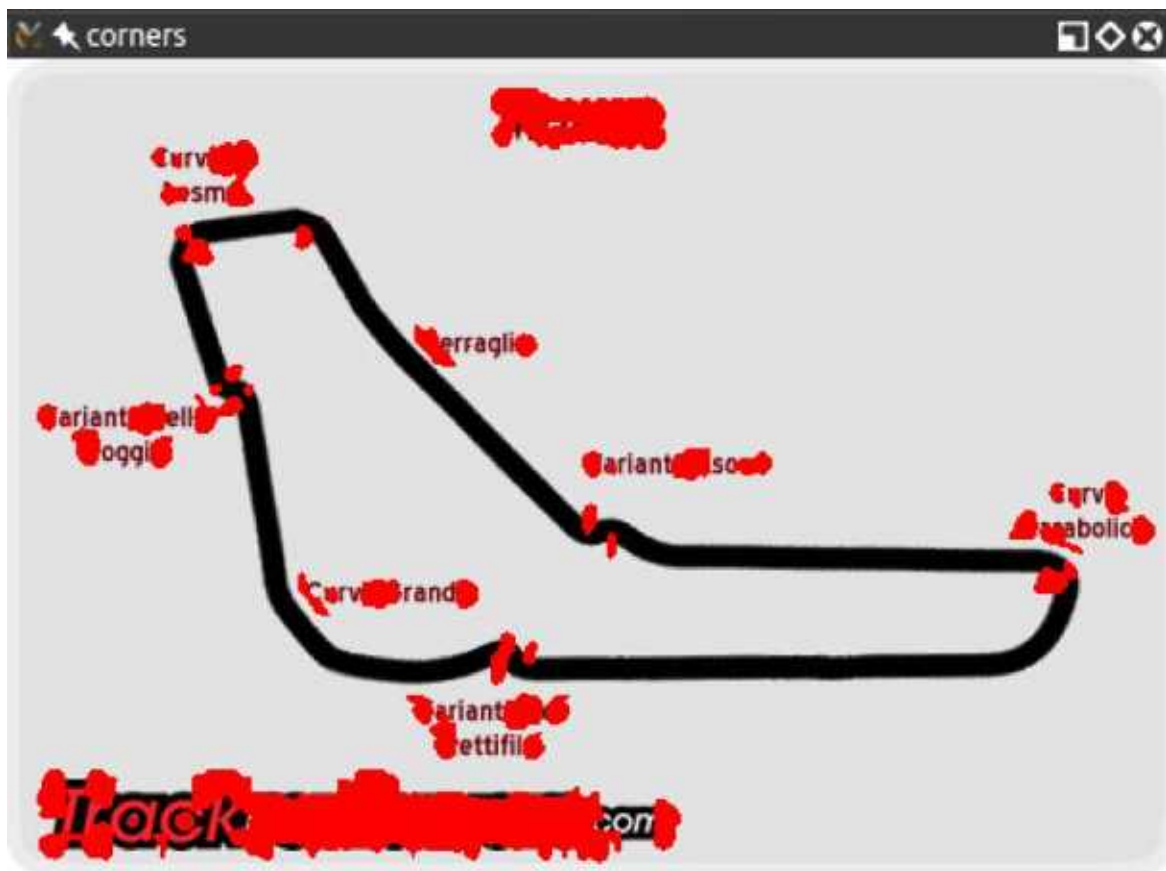


Figure 6.3: Corner detections in an image of the F1 Italian Grand Prix track

Here is the corner detection result with a smaller version of the same image:



Figure 6.4: Corner detections in a smaller image of the F1 Italian Grand Prix track

You will notice how the corners are a lot more condensed; however, even though we gained some corners, we lost others! In particular, let's examine the **Variante Ascari** chicane, which looks like a squiggle at the end of the part of the track that runs straight from northwest to southeast. In the larger version of the image, both the entrance and the apex of the double bend were detected as corners. In the smaller image, the apex is not detected as such. If we further reduce the image, at some scale, we will lose the entrance to that chicane too.

This loss of features raises an issue; we need an algorithm that works regardless of the scale of the image. Enter **Scale-Invariant Feature Transform (SIFT)**. While the name may sound a bit mysterious, now that we know what problem we are trying to solve, it actually makes sense. We need a function (a transform) that will detect features (a feature transform) and will not output different results depending on the scale of the image (a scale-invariant feature transform). Note that SIFT does not detect keypoints. Keypoint detection is done with the **Difference of Gaussians (DoG)**, while SIFT describes the region surrounding the keypoints by means of a feature vector.

A quick introduction to the DoG is in order. Previously, in *Chapter 3 , Processing Images with OpenCV* , we talked about low pass filters and blurring operations, and specifically the `cv2.GaussianBlur` function. DoG is the result of applying different Gaussian filters to the same image. Previously, we applied this type of technique for edge detection, and the idea is the same here. The final result of a DoG operation contains areas of interest (keypoints), which are then going to be described through SIFT.

Let's see how DoG and SIFT behave in the following image, which is full of corners and features:



Figure 6.5: SIFT descriptors for an image of Varese, Lombardy, Italy

Here, the beautiful panorama of Varese (in Lombardy, Italy) gains a new type of fame as a subject of computer vision. Here is the code that produces this processed image:

After the usual imports, we load the image we want to process. Then, we convert the image into grayscale. By now, you may have gathered that many methods in OpenCV expect a grayscale image as input. The next step is to create a SIFT detection object and compute the features and descriptors of the grayscale image:

Behind the scenes, these simple lines of code carry out an elaborate process; we create a `cv2.SIFT` object, which uses DoG to detect keypoints and then computes a feature vector for the surrounding region of each keypoint. As the name of the `detectAndCompute` method clearly suggests, two main operations are performed: feature detection and the computation of descriptors. The return value of the operation is a tuple containing a list of keypoints and another list of the keypoints' descriptors.

Finally, we process this image by drawing the keypoints on it with the `cv2.drawKeypoints` function and then displaying it with the usual `cv2.imshow` function. As one of its arguments, the `cv2.drawKeypoints` function accepts a flag that specifies the type of visualization we want. Here, we specify

`cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINT` in order to draw a visualization of the scale and orientation of each keypoint.

Anatomy of a keypoint

Each keypoint is an instance of the `cv2.KeyPoint` class, which has the following properties:

- The `pt` (point) property contains the *x* and *y* coordinates of the keypoint in the image.
- The `size` property indicates the diameter of the feature.
- The `angle` property indicates the orientation of the feature, as shown by the radial lines in the preceding processed image.
- The `response` property indicates the strength of the keypoint. Some features are classified by SIFT as stronger than others, and `response` is the property you would check to evaluate the strength of a feature.
- The `octave` property indicates the layer in the image pyramid where the feature was found. Let's briefly review the concept of an image pyramid, which we discussed previously in *Chapter 5 , Detecting and Recognizing Faces* , in the *Conceptualizing Haar cascades* section. The SIFT algorithm operates in a similar fashion to face detection algorithms in that it processes the same image iteratively but alters the input at each iteration. In particular, the scale of the image is a parameter that changes at each iteration (`octave`) of the algorithm. Thus, the `octave` property is related to the image scale at which the keypoint was detected.
- Finally, the `class_id` property can be used to assign a custom identifier to a keypoint or a group of keypoints.

Detecting Fast Hessian features and extracting SURF descriptors

Computer vision is a relatively young branch of computer science, so many famous algorithms and techniques have only been invented recently. SIFT is, in fact, only 23 years old, having been published by David Lowe in 1999.

SURF is a feature detection algorithm that was published in 2006 by Herbert Bay. SURF is several times faster than SIFT, and it is partially inspired by it.

Note that SURF is a patented algorithm and, for this reason, is made available only in builds of `opencv_contrib` where the `OPENCV_ENABLE_NONFREE` CMake flag is used. SIFT was formerly a patented algorithm but its patent expired and now SIFT is available in standard builds of OpenCV.

It is not particularly relevant to this book to understand how SURF works under the hood, inasmuch as we can use it in our applications and make the best of it. What is important to understand is that `cv2.SURF` is an OpenCV class that performs keypoint detection with the Fast Hessian algorithm and descriptor extraction with SURF, much like the `cv2.SIFT` class performs keypoint detection with DoG and descriptor extraction with SIFT.

Also, the good news is that OpenCV provides a standardized API for all its supported feature detection and descriptor extraction algorithms. Thus, with only trivial changes, we can adapt our previous code sample to use SURF instead of SIFT. Here is the modified code, with the changes highlighted :

The parameter to `cv2.xfeatures2d.SURF_create` is a threshold for the Fast Hessian algorithm. By increasing the threshold, we can reduce the number of features that will be retained. With a threshold of 8000 , we get the following result:



Figure 6.6: SURF descriptors for an image of Varese, Lombardy, Italy

Try adjusting the threshold to see how it affects the result. As an exercise, you may want to build a GUI application with a slider that controls the value of the threshold. This way, a user can adjust the threshold and see the number of features increase and decrease in an inversely proportional fashion. We built a GUI application with sliders in *Chapter 4 , Depth Estimation and Segmentation* , in the *Depth estimation with a normal camera* section, so you may want to refer back to that section as a guide.

Next, we'll examine the FAST corner detector, the BRIEF keypoint descriptor, and ORB (which uses FAST and BRIEF together).

Using ORB with FAST features and BRIEF descriptors

If SIFT is young, and SURF younger, ORB is in its infancy. ORB was first published in 2011 as a fast alternative to SIFT and SURF.

The algorithm was published in the paper *ORB: an efficient alternative to SIFT or SURF* , available in PDF format at http://www.willowgarage.com/sites/default/files/orb_final.pdf .

ORB mixes the techniques used in the FAST keypoint detector and the BRIEF keypoint descriptor, so it is worth taking a quick look at FAST and BRIEF first. Then, we will talk about brute-force matching – an algorithm used for feature matching – and look at an example of feature matching.

FAST

The **Features from Accelerated Segment Test (FAST)** algorithm works by analyzing circular neighborhoods of 16 pixels. It marks each pixel in a neighborhood as brighter or darker than a particular threshold, which is defined relative to the center of the circle. A neighborhood is deemed to be a corner if it contains a number of contiguous pixels marked as brighter or darker.

FAST also uses a high-speed test, which can sometimes determine that a neighborhood is not a corner by checking just 2 or 4 pixels instead of 16. To understand how this test works, let's take a look at the following diagram, taken from the OpenCV documentation:

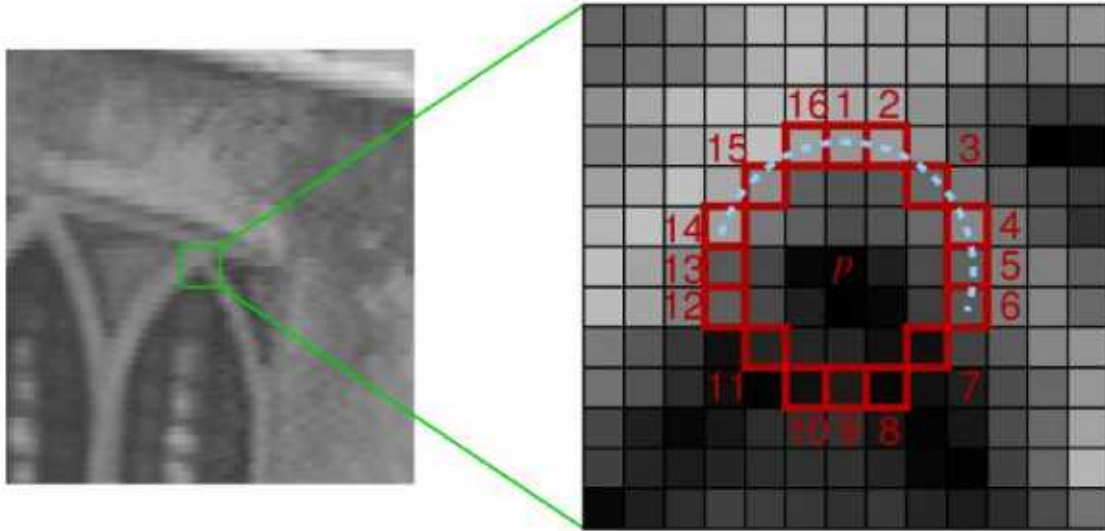


Figure 6.7: How the FAST algorithm analyzes a neighbourhood around a corner

Here, we can see a 16-pixel neighborhood at two different magnifications. The pixels at positions 1, 5, 9, and 13 correspond to the four cardinal points at the edge of the circular neighborhood. If the neighborhood is a corner, we expect that out of these four pixels, exactly three *or* exactly one will be brighter than the threshold. (Another way of saying this is that exactly one *or* exactly three of them will be darker than the threshold.) If exactly two of them are brighter than the threshold, then we have an edge, not a corner. If exactly four or exactly zero of them are brighter than the threshold, then we have a relatively uniform neighborhood that is neither a corner nor an edge.

FAST is a clever algorithm, but it's not devoid of weaknesses, and to compensate for these weaknesses, developers analyzing images can implement a machine learning approach in order to feed a set of images (relevant to a given application) to the algorithm so that parameters such as the threshold are optimized. Whether the developer specifies parameters directly or provides a training set for a machine learning approach, FAST is an algorithm that is sensitive to the developer's input, perhaps more so than SIFT.

BRIEF

Binary Robust Independent Elementary Features (BRIEF), on the other hand, is not a feature detection algorithm, but a descriptor. Let's delve deeper into the concept of what a descriptor is, and then look at BRIEF.

When we previously analyzed images with SIFT and SURF, the heart of the entire process was the call to the `detectAndCompute` function. This function performs two different steps – detection and computation – and they return two different results, coupled in a tuple.

The result of detection is a set of keypoints; the result of the computation is a set of descriptors for those keypoints. This means that OpenCV's `cv2.SIFT` and `cv2.xfeatures2d.SURF` classes implement algorithms for both detection and description. Remember, though, that the original SIFT and SURF are *not* feature detection algorithms. OpenCV's `cv2.SIFT` implements DoG feature detection plus SIFT description, while OpenCV's `cv2.xfeatures2d.SURF` implements Fast Hessian feature detection plus SURF description.

Keypoint descriptors are a representation of the image that serves as the gateway to feature matching because you can compare the keypoint descriptors of two images and find commonalities.

BRIEF is one of the fastest descriptors currently available. The theory behind BRIEF is quite complicated, but suffice it to say that BRIEF adopts a series of optimizations that make it a very good choice for feature matching.

Brute-force matching

A brute-force matcher is a descriptor matcher that compares two sets of keypoint descriptors and generates a result that is a list of matches. It is called brute-force because little optimization is involved in the algorithm. For each keypoint descriptor in the first set, the matcher makes comparisons to every keypoint descriptor in the second set. Each comparison produces a distance value and the best match can be chosen on the basis of least distance.

More generally, in computing, the term **brute-force** is associated with an approach that prioritizes the exhaustion of all possible combinations (for example, all the possible combinations of characters to crack a password of a known length). Conversely, an algorithm that prioritizes speed might skip some possibilities and try to take a shortcut to the solution that seems the most plausible.

OpenCV provides a `cv2.BFMatcher` class that supports several approaches to brute-force feature matching.

Matching a logo in two images

Now that we have a general idea of what FAST and BRIEF are, we can understand why the team behind ORB (composed of Ethan Rublee, Vincent Rabaud, Kurt Konolige, and Gary R. Bradski) chose these two algorithms as a foundation for ORB.

In their paper, the authors set out to achieve the following results:

- The addition of a fast and accurate orientation component to FAST
- The efficient computation of oriented BRIEF features
- Analysis of variance and correlation of oriented BRIEF features
- A learning method to decorrelate BRIEF features under rotational invariance, leading to better performance in nearest-neighbor applications

The main points are quite clear: ORB aims to optimize and speed up operations, including the very important step of utilizing BRIEF in a rotation-aware fashion so that matching is improved, even in situations where a training image has a very different rotation to the query image.

At this stage, though, perhaps you have had enough of the theory and want to sink your teeth into some feature matching, so let's look at some code. The following script attempts to match features in a logo to the features in a photograph that contains the logo:

Let's examine this code step by step. After the usual imports, we load two images (the query image and the scene) in grayscale format. Here is the query image, which is the NASA logo:

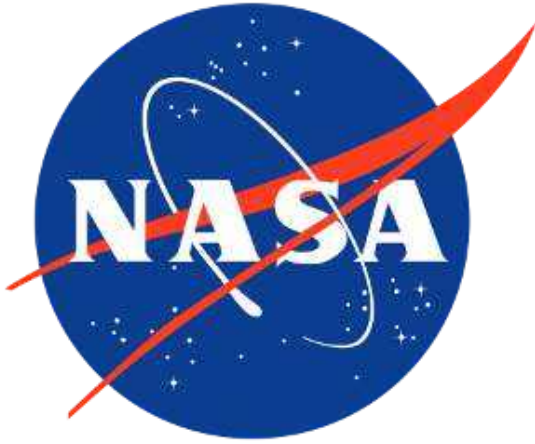


Figure 6.8: The NASA logo, to be used for keypoint descriptor matching

Here is the photo of the scene, which is the Kennedy Space Center:



Figure 6.9: The Kennedy Space Center, with a NASA logo on its wall

Now, we proceed to create the ORB feature detector and descriptor:

In a similar fashion to what we did with SIFT and SURF, we detect and compute the keypoints and descriptors for both images.

From here, the concept is pretty simple: iterate through the descriptors and determine whether they are a match or not, and then calculate the quality of this match (distance) and sort the matches so that we can display the top n matches with a degree of confidence that they are, in fact, matching features on both images. `cv2.BFMatcher` does this for us:

At this stage, we already have all the information we need, but as computer vision enthusiasts, we place quite a bit of importance on visually representing data, so let's draw these matches in a `matplotlib` chart:

Python's slicing syntax is quite robust. If the `matches` list contains fewer than 25 entries, the `matches[:25]` slicing command will run without problems and give us a list with just as many elements as the original.

The result is as follows:

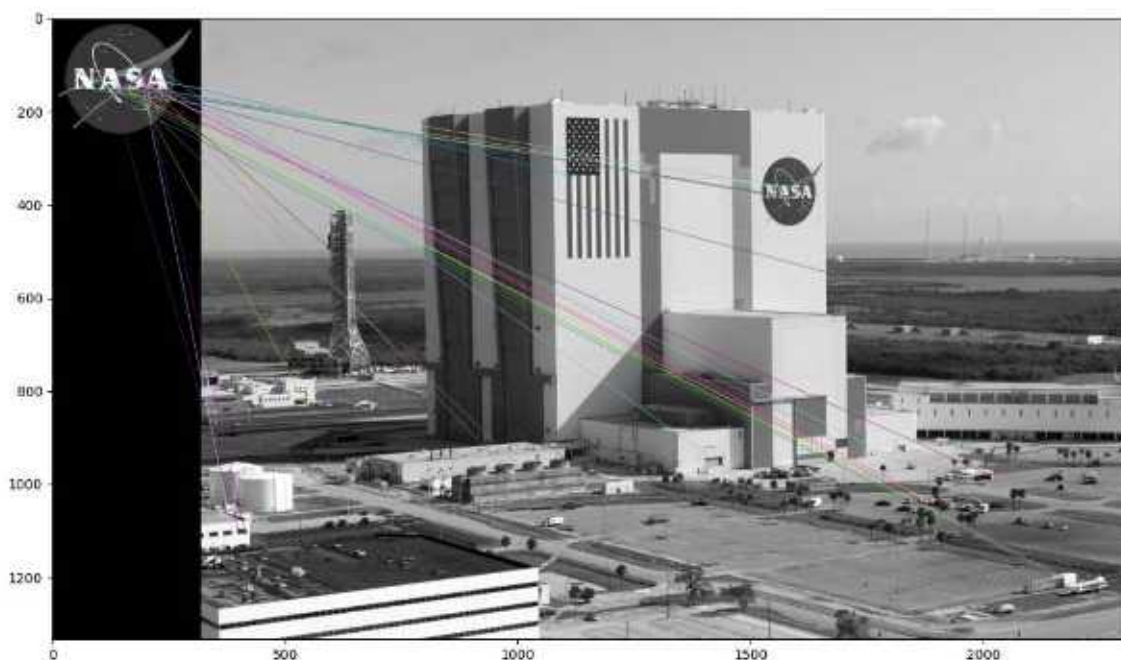


Figure 6.10: Matches between the NASA logo and the Kennedy Space Center, using brute-force matching

You might think that this is a disappointing result. Indeed, we can see that most of the matches are false matches. Unfortunately, this is quite typical. To improve our

results, we need to apply additional techniques to filter out bad matches. We'll turn our attention to this task next.

Filtering matches using K-Nearest Neighbors and the ratio test

Imagine that a large group of renowned philosophers asks you to judge their debate on a question of great importance to life, the universe, and everything. You listen carefully as each philosopher speaks in turn. Finally, when all the philosophers have exhausted all their lines of argument, you review your notes and perceive two things, as follows:

- Every philosopher disagrees with every other
- No one philosopher is much more convincing than the others

From your first observation, you infer that *at most* one of the philosophers is right; however, it is possible that all the philosophers could be wrong. Then, from your second observation, you begin to fear that you are at risk of picking a philosopher who is wrong, *even if* one of the philosophers is correct. Any way you look at it, these people have made you late for dinner. You call it a tie and say that the debate's all-important question remains unresolved.

We can compare our imaginary problem of judging the philosophers' debate to our practical problem of filtering out bad keypoint matches.

First, let's assume that each keypoint in our query image has, at most, one correct match in the scene. By implication, if our query image is the NASA logo, we assume that the other image – the scene – contains, at most, one NASA logo. Given that a query keypoint has, at most, one correct or good match, when we consider all *possible* matches, we are primarily observing bad matches. Thus, a brute-force matcher, which computes a distance score for every possible match, can give us plenty of observations of the distance scores for bad matches. We expect that a good match will have a significantly better (lower) distance score than the numerous bad matches, so the scores for the bad matches can help us pick a threshold for a good match. Such a threshold does not necessarily generalize well across different query keypoints or different scenes, but at least it helps us on a case-by-case basis.

Now, let's consider the implementation of a modified brute-force matching algorithm that adaptively chooses a distance threshold in the manner we have

described. In the previous section's code sample, we used the `match` method of the `cv2.BFMatcher` class in order to get a list containing the single best (least-distance) match for each query keypoint. By doing so, we discarded information about the distance scores of all the worse possible matches – the kind of information we need for our adaptive approach. Fortunately, `cv2.BFMatcher` also provides a `knnMatch` method, which accepts an argument, `k`, that specifies the maximum number of best (least-distance) matches that we want to retain for each query keypoint. (In some cases, we may get fewer matches than the maximum.) **KNN** stands for **k-nearest neighbors**.

We will use the `knnMatch` method to request a list of the two best matches for each query keypoint. Based on our assumption that each query keypoint has, at most, one correct match, we are confident that the second-best match is wrong. We multiply the second-best match's distance score by a value less than 1 in order to obtain the threshold.

Then, we accept the best match as a good match only if its distance score is less than the threshold. This approach is known as the **ratio test**, and it was first proposed by David Lowe, the author of the SIFT algorithm. He describes the ratio test in his paper, *Distinctive Image Features from Scale-Invariant Keypoints*, which is available at <https://www.cs.ubc.ca/~lowe/papers/ijcv04.pdf>. Specifically, in the *Application to object recognition* section, he states the following:

"The probability that a match is correct can be determined by taking the ratio of the distance from the closest neighbor to the distance of the second closest."

We can load the images, detect keypoints, and compute ORB descriptors in the same way as we did in the previous section's code sample. Then, we can perform brute-force KNN matching using the following two lines of code:

`knnMatch` returns a list of lists; each inner list contains at least one match and no more than `k` matches, sorted from best (least distance) to worst. The following line of code sorts the outer list based on the distance score of the best matches:

Let's draw the top 25 best matches, along with any second-best matches that `knnMatch` may have paired with them. We can't use the `cv2.drawMatches` function because it only accepts a one-dimensional list of matches; instead, we must use `cv2.drawMatchesKnn`. The following code is used to select, draw, and show the matches:

So far, we have not filtered out any bad matches – and, indeed, we have deliberately included the second-best matches, which we believe to be bad – so the result looks a mess. Here it is:

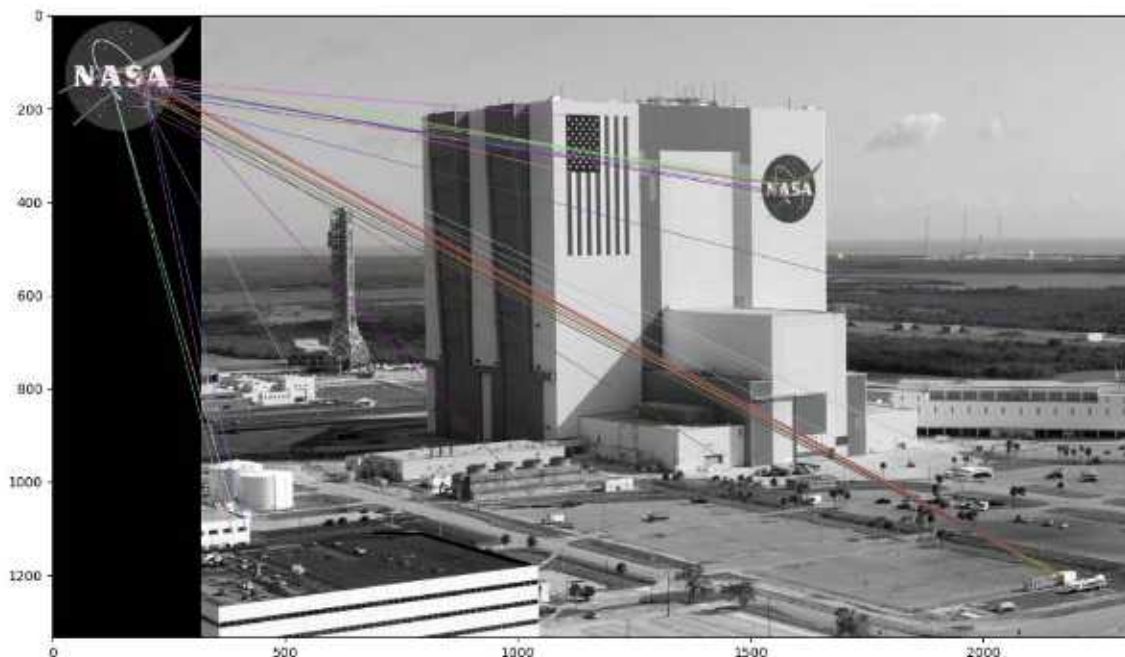


Figure 6.11: Matches between the NASA logo and the Kennedy Space Center, using brute-force KNN matching

Now, let's apply the ratio test. We will set the threshold at 0.8 times the distance score of the second-best match. If `knnMatch` has failed to provide a second-best match, we reject the best match anyway because we are unable to apply the test. The following code applies these conditions and provides us with a list of best matches that passed the test:

Having applied the ratio test, now we are only dealing with best matches (not pairs of best and second-best matches), so we can draw them with `cv2.drawMatches` instead of `cv2.drawMatchesKnn`. Again, we will select the top 25 matches from the list. The following code is used to select, draw, and show the matches:

Here, we can see the matches that passed the ratio test:



Figure 6.12: Matches between the NASA logo and the Kennedy Space Center, using brute-force KNN matching and the ratio test

Comparing this output image to the one in the previous section, we can see that KNN and the ratio test have allowed us to filter out many bad matches. The remaining matches are not perfect but nearly all of them point to the correct region – the NASA logo on the side of the Kennedy Space Center.

We have made a promising start. Next, we will replace the brute-force matcher with a faster matcher called **FLANN**. After that, we will learn how to describe a set of matches in terms of homography – that is, a 2D transformation matrix that expresses the position, rotation, scale, and other geometric characteristics of the matched object.

Matching with FLANN

FLANN stands for **Fast Library for Approximate Nearest Neighbors** . It is an open source library under the permissive 2-clause BSD license and it is available on GitHub at <https://github.com/flann-lib/flann> . There, we find the following description:

"FLANN is a library for performing fast approximate nearest neighbor searches in high dimensional spaces. It contains a collection of algorithms we found to work best for the nearest neighbor search and a system for automatically choosing the best algorithm and optimum parameters depending on the dataset. FLANN is written in C++ and contains bindings for the following languages: C, MATLAB, Python, and Ruby."

In other words, FLANN has a big toolbox, it knows how to choose the right tools for the job, and it speaks several languages. These features make the library fast and convenient. Indeed, FLANN's authors claim that it is 10 times faster than other nearest-neighbor search software for many datasets.

Although FLANN is also available as a standalone library, we will use it as part of OpenCV because OpenCV provides a handy wrapper for it.

To begin our practical example of FLANN matching, let's import NumPy, OpenCV, and Matplotlib, and load two images from files. Here is the relevant code:

Here is the first image – the query image – that our script is loading:



Figure 6.13: Paul Gauguin's painting *Entre les lys* (*Among the lilies*), to be used for keypoint descriptor matching

This work of art is *Entre les lys* (*Among the lilies*), painted by Paul Gauguin in 1889. We will search for matching keypoints in a larger image that contains multiple works by Gauguin, alongside some haphazard shapes drawn by one of the authors of this book. Here is the larger image:



Figure 6.14: Haphazard shapes and various Gauguin paintings, to be used for keypoint descriptor matching

Within the larger image, *Entre les lys* appears in the third column, third row. The query image and the corresponding region of the larger image are not identical; they depict *Entre les lys* in slightly different colors and at a different scale. Nonetheless, this should be an easy case for our matcher.

Let's detect the necessary keypoints and extract our features using the `cv2.SIFT` class:

So far, the code should seem familiar, since we have already dedicated several sections of this chapter to SIFT and other descriptors. In our previous examples, we fed the descriptors to `cv2.BFMatcher` for brute-force matching. This time, we will use `cv2.FlannBasedMatcher` instead. The following code performs FLANN-based matching with custom parameters:

Here, we can see that the FLANN matcher takes two parameters: an `indexParams` object and a `searchParams` object. These parameters, passed in the form of dictionaries in Python (and structs in C++), determine the behavior of the index and search objects that are used internally by FLANN to compute the matches. We have chosen parameters that offer a reasonable balance between accuracy and processing speed. Specifically, we are using a **kernel density tree (kd-tree)** indexing algorithm with five trees, which FLANN can process in parallel. (The FLANN documentation recommends between one tree, which would offer no parallelism, and 16 trees, which would offer a high degree of parallelism if the system could exploit it.)

We are performing 50 checks or traversals of each tree. A greater number of checks can provide greater accuracy but at a greater computational cost.

After performing FLANN-based matching, we apply Lowe's ratio test with a multiplier of 0.7. To demonstrate a different coding style, we will use the result of the ratio test in a slightly different way compared to how we did in the previous section's code sample. Previously, we assembled a new list with just the good matches in it. This time, we will assemble a list called `mask_matches`, in which each element is a sublist of length `k` (the same `k` that we passed to `knnMatch`). If a match is good, we set the corresponding element of the sublist to 1; otherwise, we set it to 0.

For example, if we have `mask_matches = [[0, 0], [1, 0]]`, this means that we have two matched keypoints; for the first keypoint, the best and second-best matches are both bad, while for the second keypoint, the best match is good but the second-best match is bad. Remember, we assume that all the second-best matches are bad. We use the following code to apply the ratio test and build the mask:

Now, it is time to draw and show the good matches. We can pass our `mask_matches` list to `cv2.drawMatchesKnn` as an optional argument, as

Finding homography with FLANN-based matches

First of all, what is homography? Let's read a definition from the internet:

"A relation between two figures, such that to any point of the one corresponds one and but one point in the other, and vice versa. Thus, a tangent line rolling on a circle cuts two fixed tangents of the circle in two sets of points that are homographic."

If you – like the authors of this book – are none the wiser from the preceding definition, you will probably find the following explanation a bit clearer: homography is a condition in which two figures find each other when one is a perspective distortion of the other.

First, let's take a look at what we want to achieve so that we can fully understand what homography is. Then, we will go through the code.

Imagine that we want to search for the following tattoo:



Figure 6.16: An anchor tattoo, to be used for keypoint matching and finding homography

We, as human beings, can easily locate the tattoo in the following image, despite there being a difference in rotation:



Figure 6.17: Fingers, with an anchor tattoo, to be used for keypoint matching and finding homography

As an exercise in computer vision, we want to write a script that produces the following visualization of keypoint matches and the homography:

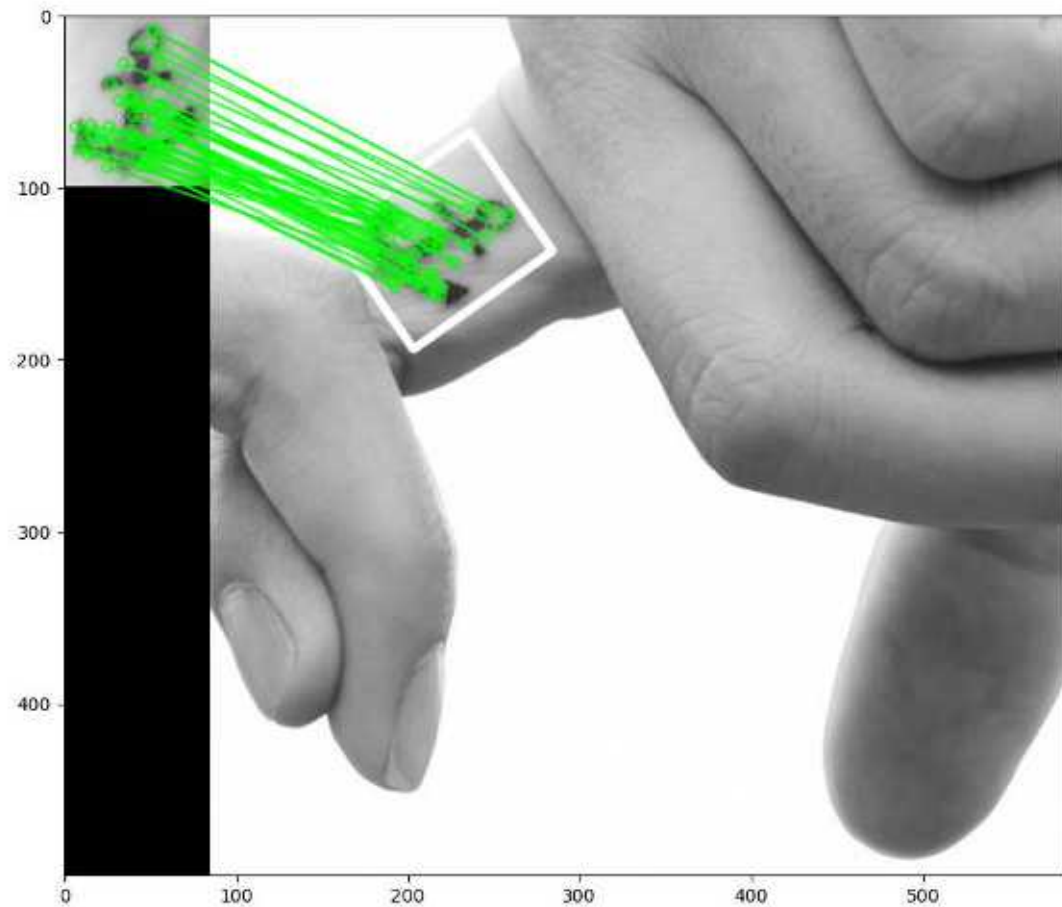


Figure 6.18: Keypoint matches and homography between two images of an anchor tattoo

As shown in the preceding screenshot, we took the subject in the first image, correctly identified it in the second image, drew matching lines between the keypoints, and even drew a white border showing the perspective deformation of the subject in the second image relative to the first image.

You might have guessed – correctly – that the script's implementation starts by importing libraries, reading images in grayscale format, detecting features, and computing SIFT descriptors. We did all of this in our previous examples, so we will omit that here. Let's take a look at what we do next:

1. We proceed by assembling a list of matches that pass Lowe's ratio test, as shown in the following code:
1. Technically, we can calculate the homography with as few as four matches. However, if any of these four matches is flawed, it will throw off the

accuracy of the result. A more practical minimum is 10 . Given the extra matches, the homography-finding algorithm can discard some outliers in order to produce a result that closely fits a substantial subset of the matches. Thus, we proceed to check whether we have at least 10 good matches:

1. If this condition has been satisfied, we look up the 2D coordinates of the matched keypoints and place these coordinates in two lists of floating-point coordinate pairs. One list contains the keypoint coordinates in the query image, while the other list contains the matching keypoint coordinates in the scene:

1. Now, we find the homography:

Note that we create a `mask_matches` list, which will be used in the final drawing of the matches so that only points lying within the homography will have matching lines drawn.

1. At this stage, we have to perform a perspective transformation, which takes the rectangular corners of the query image and projects them into the scene so that we can draw the border:

Then, we proceed to draw the keypoints and show the visualization, as per our previous examples.

A sample application – tattoo forensics

Let's conclude this chapter with a real-life (or perhaps fantasy-life) example. Imagine you are working for the Gotham forensics department and you need to identify a tattoo. You have the original picture of a criminal's tattoo (perhaps captured in CCTV footage), but you don't know the identity of the person. However, you possess a database of tattoos, indexed with the name of the person that the tattoo belongs to.

Let's divide this task into two parts:

- Build a database by saving image descriptors to files
- Load the database and scan for matches between a query image's descriptors and the descriptors in the database

We will cover these tasks in the next two subsections.

Saving image descriptors to file

The first thing we will do is save the image descriptors to an external file. This way, we don't have to recreate the descriptors every time we want to scan two images for matches.

For the purposes of our example, let's scan a folder for images and create the corresponding descriptor files so that we have them readily available for future searches. To create descriptors, we will use a process we have already used a number of times in this chapter: namely, load an image, create a feature detector, detect features, and compute descriptors. To save the descriptors to a file, we will use a handy method of NumPy arrays called `save`, which dumps array data into a file in an optimized way.

The `pickle` module, in the Python standard library, provides more general-purpose serialization functionality that supports any Python

object and not just NumPy arrays. However, NumPy's array serialization is a good choice for numeric data.

Let's break our script up into functions. The main function will be named `create_descriptors` (plural, descriptors), and it will iterate over the files in a given folder. For each file, `create_descriptors` will call a helper function named `create_descriptor` (singular, descriptor), which will compute and save our descriptors for the given image file. Let's get started:

1. First, here is the implementation of `create_descriptors` :

Note that `create_descriptors` creates the feature detector because we only need to do this once, not every time we load a file. The helper function, `create_descriptor`, receives the feature detector as an argument.

1. Now, let's look at the latter function's implementation:

Note that we save the descriptor files in the same folder as the images. Moreover, we assume that the image files have the `png` extension. To make the script more robust, you could modify it so that it supports additional image file extensions such as `jpg` . If a file has an unexpected extension, we skip it because it might be a descriptor file (from a previous run of the script) or some other non-image file.

1. We have finished implementing the functions. To complete the script, we will call `create_descriptors` with a folder name as an argument:

When we run this script, it produces the necessary descriptor files in NumPy's array file format, with the file extension `npz` . These files constitute our database of tattoo descriptors, indexed by name. (Each filename is a person's name.) Next, we'll write a separate script so that we can run a query against this database.

Scanning for matches

Now that we have descriptors saved to files, we just need to perform matching against each set of descriptors to determine which set best matches our query image.

This is the process we will put in place:

1. Load a query image (`query.png`).
2. Scan the folder containing descriptor files. Print the names of the descriptor files.
3. Create SIFT descriptors for the query image.
4. For each descriptor file, load the SIFT descriptors and find FLANN-based matches. Filter the matches based on the ratio test. Print the person's name and the number of matches. If the number of matches exceeds an arbitrary threshold, print that the person is a suspect. (Remember, we are investigating a crime.)
5. Print the name of the prime suspect (the one with the most matches).

Let's consider the implementation:

1. First, the following code block loads the query image:
1. We proceed to assemble and print a list of the descriptor files:
1. We set up our typical `cv2.SIFT` and `cv2.FlannBasedMatcher` objects, and we generate descriptors of the query image:
1. Now, we search for suspects, whom we define as people with at least 10 good matches for the query tattoo. Our search entails iterating over the descriptor files, loading the descriptors, performing FLANN-based matching, and filtering the matches based on the ratio test. We print a result for each person (each descriptor file):

Note the use of the `np.load` method, which loads a specified `npz` file into a NumPy array.

1. In the end, we print the name of the prime suspect (if we found a suspect, that is):

Running the preceding script produces the following output:

If we wanted, we could represent the matches and the homography graphically, as we did in the previous section.

Summary

In this chapter, we learned about detecting keypoints, computing keypoint descriptors, matching these descriptors, filtering out bad matches, and finding the homography between two sets of matching keypoints. We explored a number of algorithms that are available in OpenCV that can be used to accomplish these tasks, and we applied these algorithms to a variety of images and use cases.

If we combine our new knowledge of keypoints with additional knowledge about cameras and perspective, we can track objects in 3D space. This will be the topic of *Chapter 9 , Camera Models and Augmented Reality* . You can skip ahead to that chapter if you are particularly keen to reach the third dimension.

If, instead, you think the next logical step is to round off your knowledge of two-dimensional solutions for object detection, recognition, and tracking, you can continue sequentially with *Chapter 7 , Building Custom Object Detectors* , and then *Chapter 8 , Tracking Objects* . It is good to know of a combination 2D and 3D techniques so that you can choose an approach that offers the right kind of output and the right computational speed for a given application.

Join our book community on Discord

<https://discord.gg/djGjeMECxxw>



This chapter delves deeper into the concept of object detection, which is one of the most common challenges in computer vision. Having come this far in the book, you are perhaps wondering when you will be able to put computer vision into practice on the streets. Do you dream of building a system to detect cars and people? Well, you are not too far from your goal, actually.

We have already looked at some specific cases of object detection and recognition in previous chapters. We focused on upright, frontal human faces in *Chapter 5 , Detecting and Recognizing Faces* , and on objects with corner-like or blob-like features in *Chapter 6 , Retrieving Images and Searching Using Image Descriptors* . Now, in the current chapter, we will explore algorithms that have a good ability to generalize or extrapolate, in the sense that they can cope with the real-world diversity that exists within a given class of object. For example, different cars have different designs, and people can appear to be different shapes depending on the clothes they wear.

Specifically, we will pursue the following objectives:

- Learning about another kind of feature descriptor: the **histogram of oriented gradients (HOG)** descriptor.
- Understanding **non-maximum suppression** , also called **non-maxima suppression (NMS)**, which helps us choose the best of an overlapping set of detection windows.
- Gaining a high-level understanding of **support vector machines (SVMs)**. These general-purpose classifiers are based on supervised machine learning, in a way that is similar to linear regression.
- Detecting people with a pre-trained classifier based on HOG descriptors.
- Training a **bag-of-words (BoW)** classifier to detect a car. For this sample, we will work with a custom implementation of an image pyramid, a sliding window, and NMS so that we can better understand the inner workings of these techniques.

Most of the techniques in this chapter are not mutually exclusive; rather, they work together as components of a detector. By the end of the chapter, you will know how to train and use classifiers that have practical applications on the streets!

Technical requirements

This chapter uses Python, OpenCV, and NumPy. Please refer back to *Chapter 1 , Setting Up OpenCV* , for installation instructions.

The completed code for this chapter can be found in this book's GitHub repository, at <https://github.com/PacktPublishing/Learning-OpenCV-5-Computer-Vision-with-Python-Fourth-Edition> , in the `chapter07` folder. Sample images can be found in the repository in the `images` folder.

Understanding HOG descriptors

HOG is a feature descriptor, so it belongs to the same family of algorithms as **scale-invariant feature transform (SIFT)**, **speeded-up robust features (SURF)**, and **Oriented FAST and rotated BRIEF (ORB)**, which we covered in *Chapter 6 , Retrieving Images and Searching Using Image Descriptors* . Like other feature descriptors, HOG is capable of delivering the type of information that is vital for feature matching, as well as for object detection and recognition. Most commonly, HOG is used for object detection. The algorithm – and, in particular, its use as a people detector – was popularized by Navneet Dalal and Bill Triggs in their paper *Histograms of Oriented Gradients for Human Detection* (INRIA, 2005), which is available online at <https://lear.inrialpes.fr/people/triggs/pubs/Dalal-cvpr05.pdf> .

HOG's internal mechanism is really clever; an image is divided into cells and a set of gradients is calculated for each cell. Each gradient describes the change in pixel intensities in a given direction. Together, these gradients form a histogram representation of the cell. We encountered a similar approach when we studied face recognition with the **local binary pattern histogram (LBPH)** in *Chapter 6 , Detecting and Recognizing Faces* .

Before diving into the technical details of how HOG works, let's first take a look at how HOG sees the world.

Visualizing HOG

Carl Vondrick, Aditya Khosla, Hamed Pirsiavash, Tomasz Malisiewicz, and Antonio Torralba have developed a HOG visualization technique called HOGgles (HOG goggles). For a summary of HOGgles, as well as links to code and publications, refer to Carl Vondrick's MIT web page at <http://www.cs.columbia.edu/~vondrick/ihog/index.html> . As one of their test images, Vondrick et al. use the following photograph of a truck:



Figure 7.1: A truck, to be used as a test image for HOG visualization

Vondrick et al. produce the following visualization of the HOG descriptors, based on an approach from Dalal and Triggs' earlier paper:

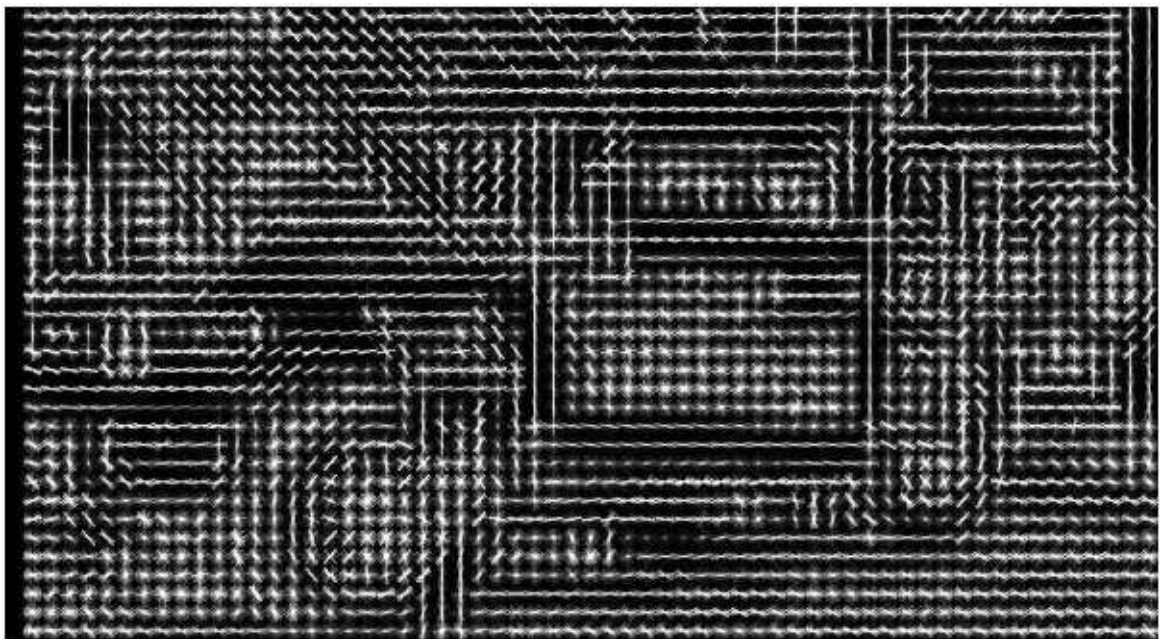


Figure 7.2: HOG cell gradients, visualized as lines

Then, applying HOGgles, Vondrick et al. invert the feature description algorithm to reconstruct the image of the truck as HOG sees it, as shown here:



Figure 7.3: HOG cell gradients, visualized as smooth grayscale transitions

In both of these two visualizations, you can see that HOG has divided the image into cells, and you can easily recognize the wheels and the main structure of the vehicle. In the first visualization, the calculated gradients for each cell are visualized as a set of crisscrossing lines that sometimes look like an elongated star; the star's longer axes represent stronger gradients. In the second visualization, the gradients are visualized as a smooth transition in brightness, along various axes in a cell.

Now, let's give further consideration to the way HOG works, and the way it can contribute to an object detection solution.

Using HOG to describe regions of an image

For each HOG cell, the histogram contains a number of bins equal to the number of gradients or, in other words, the number of axis directions that HOG considers. After calculating all the cells' histograms, HOG processes groups of histograms to produce higher-level descriptors. Specifically, the cells are grouped into larger regions, called blocks. These blocks can be made of any number of cells, but Dalal and Triggs found that 2x2 cell blocks yielded the best results when performing people detection. A block-wide vector is created so that it can be normalized, compensating for local variations in illumination and shadowing. (A single cell is too small a region to detect such variations.) This normalization improves a HOG-based detector's robustness, with respect to variations in lighting conditions.

Like other detectors, a HOG-based detector needs to cope with variations in objects' location and scale. The need to search in various locations is addressed by moving a fixed-size sliding window across an image. The need to search at various scales is addressed by scaling the image to various sizes, forming a so-called image pyramid. We studied these techniques previously in *Chapter 5 , Detecting and Recognizing Faces* , specifically in the *Conceptualizing Haar cascades* section. However, let's elaborate on one difficulty: how to handle multiple detections in overlapping windows.

Suppose we are using a sliding window to perform people detection on an image. We slide our window in small steps, just a few pixels at a time, so we expect that it will frame any given person multiple times. Assuming that overlapping detections are indeed one person, we do not want to report multiple locations but, rather, only one location that we believe to be correct. In other words, even if a detection at a given location has a *good* confidence score, we might reject it if an overlapping detection has a *better* confidence score; thus, from a set of overlapping detections, we would choose the one with the *best* confidence score.

This is where NMS comes into play. Given a set of overlapping regions, we can suppress (or reject) all the regions for which our classifier did not produce a maximal score.

Understanding NMS

The concept of NMS might sound simple. From a set of overlapping solutions, just pick the best one! However, the implementation is more complex than you might initially think. Remember the image pyramid? Overlapping detections can occur at different scales. We must gather up all our positive detections, and convert their bounds back to a common scale before we check for overlap. A typical implementation of NMS takes the following approach:

1. Construct an image pyramid.
2. Scan each level of the pyramid with the sliding window approach, for object detection. For each window that yields a positive detection (beyond a certain arbitrary confidence threshold), convert the window back to the original image's scale. Add the window and its confidence score to a list of positive detections.
3. Sort the list of positive detections by order of descending confidence score so that the best detections come first in the list.
4. For each window, W , in the list of positive detections, remove all subsequent windows that significantly overlap with W . We are left with a list of positive detections that satisfy the criterion of NMS.

Besides NMS, another way to filter the positive detections is to eliminate any subwindows. When we speak of a **subwindow** (or subregion), we mean a window (or region in an image) that is entirely contained inside another window (or region). To check for subwindows, we simply need to compare the corner coordinates of various window rectangles. We will take this simple approach in our first practical example, in the *Detecting people with HOG descriptors* section. Optionally, NMS and suppression of subwindows can be combined.

Several of these steps are iterative, so we have an interesting optimization problem on our hands. A fast sample implementation in MATLAB is provided by Tomasz Malisiewicz at

<http://www.computervisionblog.com/2011/08/blazing-fast-nmsm-from-exemplar-svm.html> . A port of this sample implementation to Python is provided by Adrian Rosebrock at <https://www.pyimagesearch.com/2015/02/16/faster-non-maximum-suppression-python/> . We will build atop the latter sample later in this chapter, in the *Detecting a car in a scene* section.

Now, how do we determine the confidence score for a window? We need a classification system that determines whether a certain feature is present or not, and a confidence score for this classification. This is where **SVMs** come into play.

Understanding SVMs

Without going into details of how an SVM works, let's just try to grasp what it can help us accomplish in the context of machine learning and computer vision. Given labeled training data, an SVM learns to classify the same kind of data by finding an optimal *hyperplane*, which, in plain English, is the plane that divides differently labeled data by the largest possible margin. To aid our understanding, let's consider the following diagram, which is provided by Zach Weinberg under the Creative Commons Attribution-Share Alike 3.0 Unported License:

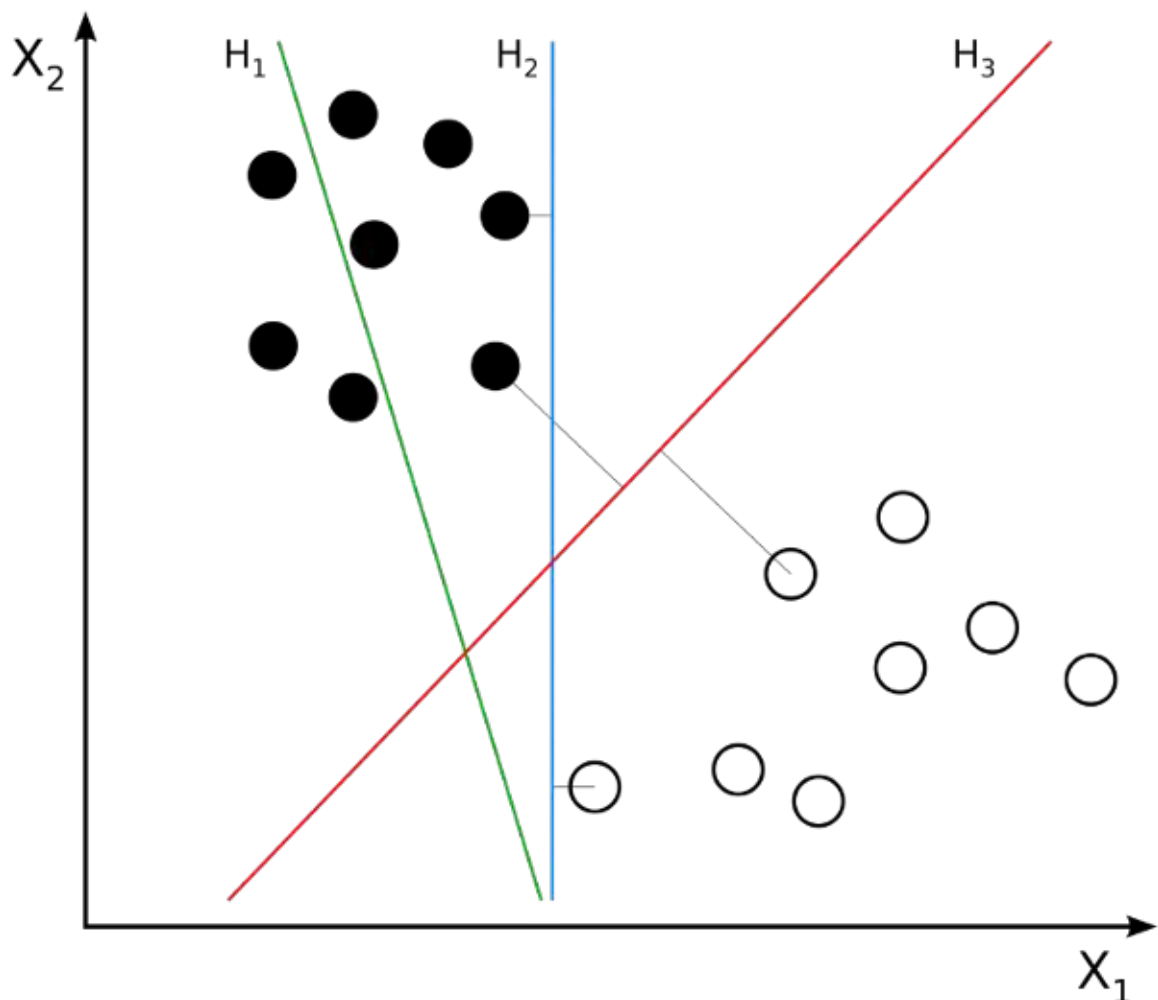


Figure 7.4: Clusters and hyperplanes. An SVM would identify **H3** as the optimal hyperplane.

Hyperplane **H1** (shown as a green line) does not divide the two classes (the black dots versus the white dots). Hyperplanes **H2** (shown as a blue line) and **H3** (shown as a red line) both divide the classes; however, only hyperplane **H3** divides the classes by a maximal margin.

Let's suppose we are training an SVM as a people detector. We have two classes, *person* and *non-person*. As training samples, we provide vectors of HOG descriptors of various windows that do or do not contain a person. These windows may come from various images. The SVM learns by finding the optimal hyperplane that maximally divides the multidimensional HOG descriptor space into people (on one side of the hyperplane) and non-people (on the other side). Thereafter, when we give the trained SVM a vector of HOG descriptors for any other window in any image, the SVM can judge whether the window contains a person or not. The SVM can even give us a confidence value that relates to the vector's distance from the optimal hyperplane.

The SVM model has been around since the early 1960s. However, it has undergone improvements since then, and the basis of modern SVM implementations can be found in the paper *Support-vector networks* (*Machine Learning* , 1995) by Corinna Cortes and Vladimir Vapnik. It is available at <http://link.springer.com/article/10.1007/BF00994018> .

Now that we have a conceptual understanding of the key components we can combine to make an object detector, we can start looking at a few examples. We will start with one of OpenCV's ready-made object detectors, and then we will progress to designing and training our own custom object detectors.

Detecting people with HOG descriptors

OpenCV comes with a class called `cv2.HOGDescriptor` , which is capable of performing people detection. The interface has some similarities to the `cv2.CascadeClassifier` class that we used in *Chapter 5 , Detecting and Recognizing Faces* . However, unlike `cv2.CascadeClassifier` , `cv2.HOGDescriptor` sometimes returns nested detection rectangles. In other words, `cv2.HOGDescriptor` might tell us that it detected one person whose bounding rectangle is located completely inside another person's bounding rectangle. This situation really is possible; for example, a child could be standing in front of an adult, and the child's bounding rectangle could be completely inside the adult's bounding rectangle. However, in a typical situation, nested detections are probably errors, so `cv2.HOGDescriptor` is often used along with code to filter out any nested detections.

Let's begin our sample script by implementing a test to determine whether one rectangle is nested inside another. For this purpose, we will write a function, `is_inside(i, o)` , where `i` is the possible inner rectangle and `o` is the possible outer rectangle. The function will return `True` if `i` is inside `o` ; otherwise, it will return `False` . Here is the start of the script:

Now, we create an instance of `cv2.HOGDescriptor` , and we specify that it will use a default people detector that is built into OpenCV by running the following code:

Note that we specified the people detector with the `setSVMDetector` method. Hopefully, this makes sense based on the previous section of this chapter; an SVM is a classifier, so the choice of SVM determines the type of object our `cv2.HOGDescriptor` will detect.

Now, we proceed to load an image – in this case, an old photograph of women working in a hayfield – and we attempt to detect people in the image by running the following code:

Note that `cv2.HOGDescriptor` has a `detectMultiScale` method, which returns two lists:

1. A list of bounding rectangles for detected objects (in this case, detected people).
2. A list of weights or confidence scores for detected objects. A higher value indicates greater confidence that the detection result is correct.

`detectMultiScale` accepts several optional arguments, including the following:

- `winStride` : This tuple defines the x and y distance that the sliding window moves between successive detection attempts. HOG works well with overlapping windows, so the stride may be small relative to the window size. A smaller value produces more detections, at a higher computational cost. The default stride has no overlap; it is the same as the window size, which is $(64, 128)$ for the default people detector.
- `scale` : This scale factor is applied between successive levels of the image pyramid. A smaller value produces more detections, at a higher computational cost. The value must be greater than 1.0 . The default is 1.5 .
- `groupThreshold` : This value determines how stringent our detection criteria are. A smaller value is less stringent, resulting in more detections. The default is 2.0 .

Now, we can filter the detection results to remove nested rectangles. To determine whether a rectangle is a nested rectangle, we potentially need to compare it to every other rectangle. Note the use of our `is_inside` function in the following nested loop:

Finally, let's draw the remaining rectangles and weights in order to highlight the detected people, and let's show and save this visualization, as follows:

If you run the script yourself, you will see rectangles around people in the image. Here is the result:

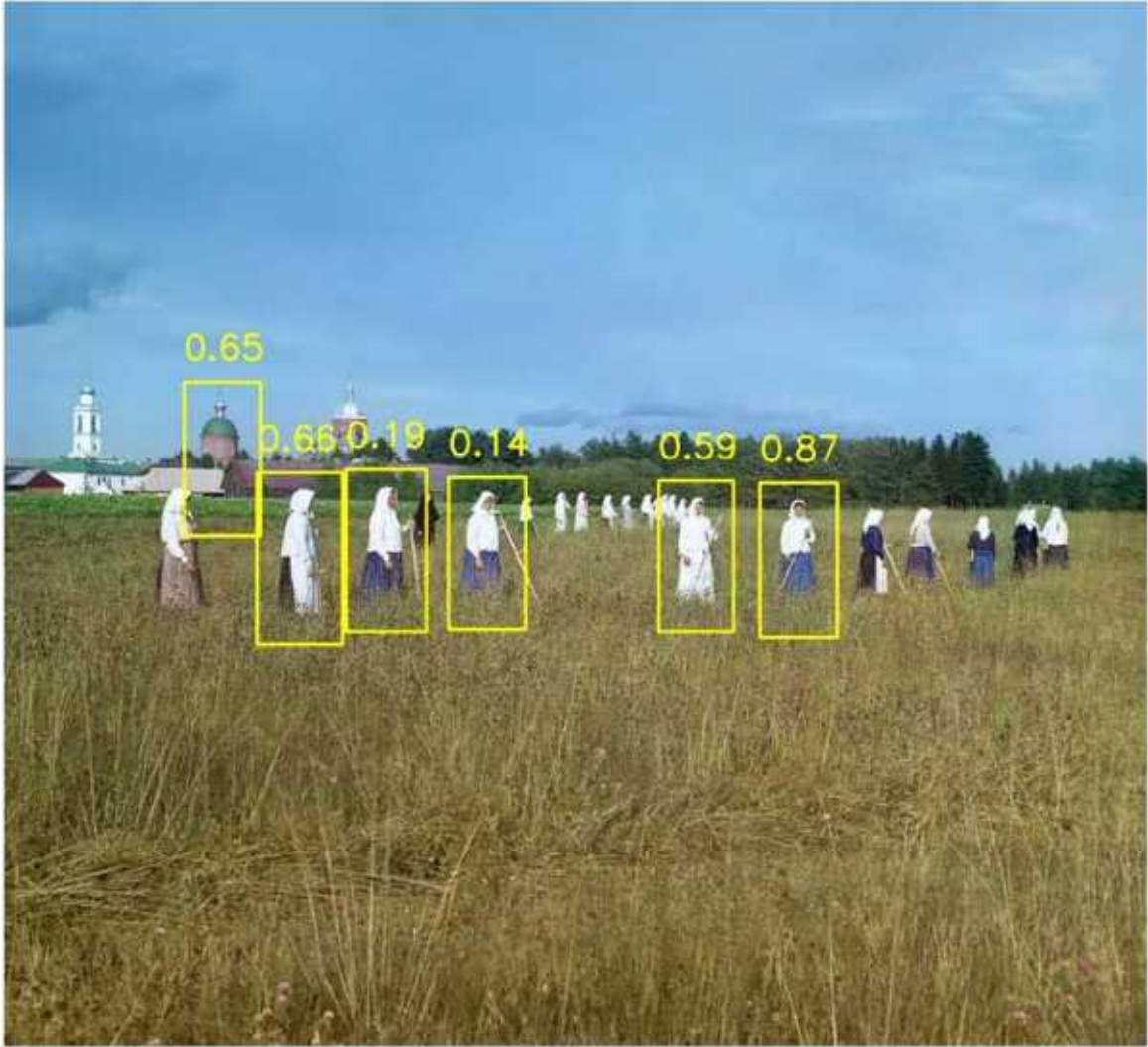


Figure 7.5: People detected using HOG. The photograph is another example of the work of Sergey Prokudin-Gorsky (1863-1944), a pioneer of color photography. Here, the scene is a field at the Leushinskii Monastery, in northwestern Russia, in 1909.

Of the six women who are nearest to the camera, five have been successfully detected. At the same time, a tower in the background has been falsely detected as a person. In many real-world applications, people-detection results can be improved by analyzing a series of frames in a video. For example, imagine that we are looking at a surveillance video of the Leushinskii Monastery's hayfield, instead of a single photograph. We should be able to add code to determine that the tower cannot be a person because it does not move. Also, we should be able to detect additional people in other frames and track each person's movement from frame to frame. We will look at a people-tracking problem in *Chapter 8*, *Tracking Objects*.

Meanwhile, let's proceed to look at another kind of detector that we can train to detect a given class of objects.

Creating and training an object detector

Using a pre-trained detector makes it easy to build a quick prototype, and we are all very grateful to the OpenCV developers for making such useful capabilities as face detection and people detection readily available. However, whether you are a hobbyist or a computer vision professional, it is unlikely that you will only deal with people and faces.

Moreover, if you are like the authors of this book, you will wonder how the people detector was created in the first place and whether you can improve it. Furthermore, you may also wonder whether you can apply the same concepts to detect diverse objects, ranging from cars to goblins.

Indeed, in industry, you may have to deal with problems of detecting very specific objects, such as registration plates, book covers, or whatever thing may be most important to your employer or client.

Thus, the question is, how do we come up with our own classifiers?

There are many popular approaches. Throughout the remainder of this chapter, we will see that one answer lies in SVMs and the BoW technique.

We have already talked about SVMs and HOG. Let's now take a closer look at BoW.

Understanding BoW

BoW is a concept that was not initially intended for computer vision; rather, we use an evolved version of this concept in the context of computer vision. Let's first talk about its basic version, which – as you may have guessed – originally belongs to the field of language analysis and information retrieval.

Sometimes, in the context of computer vision, BoW is called **bag of visual words (BoVW)** . However, we will simply use the term BoW, since this is the term used by OpenCV.

BoW is the technique by which we assign a weight or count to each word in a series of documents; we then represent these documents with vectors of these counts. Let's look at an example, as follows:

- **Document 1** : I like OpenCV and I like Python.
- **Document 2** : I like C++ and Python.
- **Document 3** : I don't like artichokes.

These three documents allow us to build a dictionary – also called a **codebook** or vocabulary – with these values, as follows:

We have eight entries. Let's now represent the original documents using eight-entry vectors. Each vector contains values representing the counts of all words in the dictionary, in order, for a given document. The vector representation of the preceding three sentences is as follows:

These vectors can be conceptualized as a histogram representation of documents or as a descriptor vector that can be used to train classifiers. For example, a document can be classified as *spam* or *not spam* based on such a representation. Indeed, spam filtering is one of the many real-world applications of BoW.

Now that we have a grasp of the basic concept of BoW, let's see how this applies to the world of computer vision.

Applying BoW to computer vision

We are by now familiar with the concepts of features and descriptors. We have used algorithms such as SIFT and SURF to extract descriptors from an image's features so that we can match these features in another image.

We have also recently familiarized ourselves with another kind of descriptor, based on a codebook or dictionary. We know about an SVM, a

model that can accept labeled descriptor vectors as training data, can find an optimal division of the descriptor space into the given classes, and can predict the classes of new data.

Armed with this knowledge, we can take the following approach to build a classifier:

1. Take a sample dataset of images.
2. For each image in the dataset, extract descriptors (with SIFT, SURF, ORB, or a similar algorithm).
3. Add each descriptor vector to the BoW trainer.
4. Cluster the descriptors into k clusters whose centers (centroids) are our visual words. This last point probably sounds a bit obscure, but we will explore it further in the next section.

At the end of this process, we have a dictionary of visual words ready to be used. As you can imagine, a large dataset will help make our dictionary richer in visual words. Up to a point, the more words, the better!

Having trained a classifier, we should proceed to test it. The good news is that the test process is conceptually very similar to the training process outlined previously. Given a test image, we can extract descriptors and **quantize** them (or reduce their dimensionality) by calculating a histogram of their distances to the centroids. Based on this, we can attempt to recognize visual words, and locate them in the image.

This is the point in the chapter where you have built up an appetite for a deeper practical example, and are raring to code. However, before proceeding, let's take a quick but necessary digression into the theory of k - means clustering so that you can fully understand how visual words are created. Thereby, you will gain a better understanding of the process of object detection using BoW and SVMs.

k-means clustering

k -means clustering is a method of quantization whereby we analyze a large number of vectors in order to find a small number of clusters. Given a dataset, k represents the number of clusters into which the dataset is going

to be divided. The term *means* refers to the mathematical concept of the mean or the average; when visually represented, the mean of a cluster is its **centroid** or the geometric center of points in the cluster.

Clustering refers to the process of grouping points in a dataset into clusters.

OpenCV provides a class called `cv2.BOWKMeansTrainer` , which we will use to help train our classifier. As you might expect, the OpenCV documentation gives the following summary of this class:

A "kmeans-based class to train a visual vocabulary using the bag of words approach."

After this long theoretical introduction, we can look at an example, and start training our custom classifier.

Detecting cars

To train any kind of classifier, we must begin by creating or acquiring a training dataset. We are going to train a car detector, so our dataset must contain positive samples that represent cars, as well as negative samples that represent other (non-car) things that the detector is likely to encounter while looking for cars. For example, if the detector is intended to search for cars on a street, then a picture of a curb, a crosswalk, a pedestrian, or a bicycle might be a more representative negative sample than a picture of the rings of Saturn. Besides representing the expected subject matter, ideally, the training samples should represent the way our particular camera and algorithm will see the subject matter.

Ultimately, in this chapter, we intend to use a sliding window of fixed size, so it is important that our training samples conform to a fixed size, and that the positive samples are tightly cropped in order to frame a car without much background.

Up to a point, we expect that the classifier's accuracy will improve as we keep adding good training images. On the other hand, a large dataset can make the training slow, and it is possible to overtrain a classifier such that it fails to extrapolate beyond the training set. Later in this section, we will write our code in a way that allows us to easily modify the number of training images so that we find a good size experimentally.

Assembling a dataset of car images would be a time-consuming task if we were to do it all by ourselves (though it is entirely doable). To avoid reinventing the wheel – or the whole car – we can avail ourselves of ready-made datasets, such as the following:

- **UIUC Image Database for Car Detection:**
<https://cogcomp.seas.upenn.edu/Data/Car/>
- **Stanford Cars Dataset :**
http://ai.stanford.edu/~jkrause/cars/car_dataset.html

Let's use the UIUC dataset in our example. Several steps are involved in obtaining this dataset and using it in a script, so let's walk through them one by one, as follows:

1. Download the UIUC dataset from <https://github.com/gcr/arc-evaluator/raw/master/CarData.tar.gz> . Unzip it to some folder, which we will refer to as `<project_path>` . Now, the unzipped data should be located at `<project_path>/CarData` . Specifically, we will use some of the images in `<project_path>/CarData/TrainImages` and `<project_path>/CarData/TestImages` .
2. Also in `<project_path>` , let's create a Python script called `detect_car_bow_svm.py` . To begin the script's implementation, write the following code to check whether the `CarData` subfolder exists:

If you can run this script and it does not print anything, this means everything is in its correct place.

1. Next, let's define the following constants in the script:

Note that our classifier will make use of two training stages: one stage for the BoW vocabulary, which will use a number of images as samples, and another stage for the SVM, which will use a number of BoW descriptor vectors as samples. Arbitrarily, we have defined a different number of training samples for each stage. At each stage, we could have also defined a different number of training samples for the two classes (*car* and *not car*), but instead, we will use the same number. We have also defined the number of BoW clusters. Note that the number of BoW clusters may be larger than the number of BoW training images, since each image has many descriptors.

1. We will use `cv2.SIFT` to extract descriptors and `cv2.FlannBasedMatcher` to match these descriptors. Let's initialize these algorithms with the following code:

Note that we have initialized SIFT and the **Fast Library for Appropriate Nearest Neighbors (FLANN)** in the same way as we did in *Chapter 6 , Retrieving Images and Searching Using image Descriptors* . However, this time, descriptor matching is not our end goal; instead, it will be part of the functionality of our BoW.

1. OpenCV provides a class called `cv2.BOWKMeansTrainer` to train a BoW vocabulary, and a class called `cv2.BOWImgDescriptorExtractor` to convert some kind of lower-level descriptors – in our example, SIFT descriptors – into BoW descriptors. Let's initialize these objects with the following code:

When initializing `cv2.BOWKMeansTrainer` , we must specify the number of clusters – in our example, 40 , as defined earlier. When initializing `cv2.BOWImgDescriptorExtractor` , we must specify a descriptor extractor and a descriptor matcher – in our example, the `cv2.SIFT` and `cv2.FlannBasedMatcher` objects that we created earlier.

1. To train the BoW vocabulary, we will provide samples of SIFT descriptors for various *car* and *not car* images. We will load the images from the `CarData/TrainImages` subfolder, which contains positive (*car*) images with names such as `pos-x.pgm` , and negative (*not car*) images with names such as `neg-x.pgm` , where `x` is a number starting at 1 . Let's write the following utility function to return a pair of paths to the `i` th positive and negative training images, where `i` is a number starting at 0 :

Later in this section, we will call the preceding function in a loop, with a varying value of `i` , when we need to acquire a number of training samples.

1. For each path to a training sample, we will need to load the image, extract SIFT descriptors, and add the descriptors to the BoW vocabulary trainer. Let's write another utility function to do precisely this, as follows:

If no features are found in the image, then the `keypoints` and `descriptors` variables will be `None` .

1. At this stage, we have everything we need to start training the BoW vocabulary. Let's read a number of images for each class (*car* as the positive class and *not car* as the negative class) and add them to the training set, as follows:
1. Now that we have assembled the training set, we will call the vocabulary trainer's `cluster` method, which performs the *k* -means classification and returns the vocabulary. We will assign this vocabulary to the BoW descriptor extractor, as follows:

Remember that earlier, we initialized the BoW descriptor extractor with a SIFT descriptor extractor and FLANN matcher. Now, we have also given the BoW descriptor extractor a vocabulary that we trained with samples of SIFT descriptors. At this stage, our BoW descriptor extractor has everything it needs in order to extract BoW descriptors from **Difference of Gaussian (DoG)** features.

Remember that `cv2.SIFT` detects DoG features and extracts SIFT descriptors, as we discussed in *Chapter 6 , Retrieving Images and Searching Using Image Descriptors* , specifically in the Detecting DoG features and extracting SIFT descriptors section.

1. Next, we will declare another utility function that takes an image and returns the descriptor vector, as computed by the BoW descriptor extractor. This involves extracting the image's DoG features, and computing the BoW descriptor vector based on the DoG features, as follows:

1. We are ready to assemble another kind of training set, containing samples of BoW descriptors. Let's create two arrays to accommodate the training data and labels, and populate them with the descriptors generated by our BoW descriptor extractor. We will label each descriptor vector with 1 for a positive sample and -1 for a negative sample, as shown in the following code block:

Should you wish to train a classifier to distinguish between multiple positive classes, you can simply add other descriptors with other labels. For example, we could train a classifier that uses the label 1 for *car* , 2 for *person* , and -1 for *background* . There is no requirement to have a negative or background class but, if you do not, your classifier will assume that everything belongs to one of the positive classes.

1. OpenCV provides a class called `cv2.ml_SVM` , representing an SVM. Let's create an SVM, and train it with the data and labels that we previously assembled, as follows:

Note that we must convert the training data and labels from lists to NumPy arrays before we pass them to the `train` method of `cv2.ml_SVM` .

1. Finally, we are ready to test the SVM by classifying some images that were not part of the training set. We will iterate over a list of paths to test images. For each path, we will load the image, extract BoW descriptors, and get the SVM's **prediction** or classification result, which will be either 1.0 (*car*) or -1.0 (*not car*), based on the training labels we used earlier. We will draw text on the image to show the classification result, and we will show the image in a window. After showing all the images, we will wait for the user to hit any key, and then the script will end. All of this is achieved in the following block of code:

Save and run the script. You should see six windows with various classification results. Here is a screenshot of one of the true positive results:



Figure 7.6: An image of a car, correctly classified by our SVM

The next screenshot shows one of the true negative results:



Figure 7.7: A non-car image, correctly classified by our SVM

Of the six images in our simple test, only the following one is incorrectly classified:



Figure 7.8: A non-car image, incorrectly classified as a car by our SVM

Incorrect classification solely depends upon the amount and nature of positive and negative images used for training and the robustness of the learning algorithm. For example, if the car position in every image of our training dataset is in the center of the image, the test image with a car position at the corner of the image may classify incorrectly.

Experiment with adjusting the number of training samples and other parameters, and try testing the classifier on more images, to see what results you can get.

Let's take stock of what we have done so far. We have used a mixture of SIFT, BoW, and SVMs to train a classifier to distinguish between two classes: *car* and *not car*. We have applied this classifier to whole images. The next logical step is

to apply a sliding window technique so that we can narrow down our classification results to specific regions of an image.

Combining an SVM with a sliding window

By combining our SVM classifier with a sliding window technique and an image pyramid, we can achieve the following improvements:

- Detect multiple objects of the same kind in an image.
- Determine the position and size of each detected object in an image.

We will adopt the following approach:

1. Take a region of the image, classify it, and then move this window to the right by a predefined step size. When we reach the rightmost end of the image, reset the x coordinate to 0, move down a step, and repeat the entire process.
2. At each step, perform a classification with the SVM that was trained with BoW.
3. Keep track of all the windows that are positive detections, according to the SVM.
4. After classifying every window in the entire image, scale the image down, and repeat the entire process of using a sliding window. Thus, we are using an image pyramid. Continue rescaling and classifying until we get to a minimum size.

When we reach the end of this process, we have collected important information about the content of the image. However, there is a problem: in all likelihood, we have found a number of overlapping blocks that each yield a positive detection with high confidence. That is to say, the image may contain one object that gets detected multiple times. If we reported these multiple detections, our report would be quite misleading, so we will filter our results using NMS.

For a refresher, you may wish to refer back to the *Understanding NMS* section, earlier in this chapter.

Next, let's look at how to modify and extend our previous script, in order to implement the approach we have just described.

Detecting a car in a scene

We are now ready to apply all the concepts we learned so far by creating a car detection script that scans an image and draws rectangles around cars. Let's create a new Python script, `detect_car_bow_svm_sliding_window.py`, by copying our previous script, `detect_car_bow_svm.py`. (We covered the implementation of `detect_car_bow_svm.py` earlier, in the *Detecting cars* section.) Much of the new script's implementation will remain unchanged because we still want to train a BoW descriptor extractor and an SVM in almost the same way as we did previously. However, after the training is complete, we will process the test images in a new way. Rather than classifying each image in its entirety, we will decompose each image into pyramid layers and windows, we will classify each window, and we will apply NMS to a list of windows that yielded positive detections.

For NMS, we will rely on Malisiewicz and Rosebrock's implementation, as described earlier in this chapter, in the *Understanding NMS* section. You can find a slightly modified copy of their implementation in this book's GitHub repository, specifically in the Python script at `chapter07/non_max_suppression.py`. This script provides a function with the following signature:

As its first argument, the function takes a NumPy array containing rectangle coordinates and scores. If we have N rectangles, the shape of this array is $N \times 5$. For a given rectangle at index i , the values in the array have the following meanings:

- `boxes[i][0]` is the leftmost x coordinate.
- `boxes[i][1]` is the topmost y coordinate.
- `boxes[i][2]` is the rightmost x coordinate.
- `boxes[i][3]` is the bottommost y coordinate.
- `boxes[i][4]` is the score, where a higher score represents greater confidence that the rectangle is a correct detection result.

As its second argument, the function takes a threshold that represents the maximum proportion of overlap between rectangles. If two rectangles have a greater proportion of overlap than this, the one with the lower score will be filtered out. Ultimately, the function will return an array of the remaining rectangles.

Now, let's turn our attention to the modifications to the `detect_car_bow_svm_sliding_window.py` script, as follows:

1. First, we want to add a new import statement for the NMS function, as highlighted in the following code:

1. Let's define some additional parameters near the start of the script, as highlighted here:

Note that we have reduced the number of k -means clusters from 40 to 12 (a number chosen arbitrarily based on experimentation). We will use `SVM_SCORE_THRESHOLD` as a threshold to distinguish between a positive window and a negative window. We will see how the score is obtained a little later in this section. We will use `NMS_OVERLAP_THRESHOLD` as the maximum acceptable proportion of overlap in the NMS step. Here, we have arbitrarily chosen 15%, so we will cull windows that overlap by more than this proportion. As you experiment with your SVMs, you may tweak these parameters to your liking until you find values that yield the best results in your application.

1. We will also adjust the parameters of the SVM, as follows:

With the preceding changes to the SVM, we are specifying the classifier's level of strictness or severity. As the value of the `C` parameter increases, the risk of false positives decreases but the risk of false negatives increases. In our application, a false positive would be a window detected as a car when it is really *not* a car, and a false negative would be a car detected as a window when it really *is* a car.

After the code that trains the SVM, we want to add two more helper functions. One of them will generate levels of the image pyramid, and the other will generate regions of interest, based on the sliding window technique. Besides adding these helper functions, we also need to handle the test images differently in order to make use of the sliding window and NMS. The following steps cover the changes:

1. First, let's look at the helper function that deals with the image pyramid. This function is shown in the following code block:

The preceding function takes an image and generates a series of resized versions of it. The series is bounded by a maximum and minimum image size.

You will have noticed that the resized image is not returned with the `return` keyword but with the `yield` keyword. This is because this function is a so-called generator. It produces a series of images that we can easily use in a

loop. If you are not familiar with generators, take a look at the official Python Wiki at <https://wiki.python.org/moin/Generators> .

1. Next up is the function to generate regions of interest, based on the sliding window technique. This function is shown in the following code block:

Again, this is a generator. Although it is a bit deep-nested, the mechanism is very simple: given an image, return the upper-left coordinates and the sub-image representing the next window. Successive windows are shifted by an arbitrarily sized step from left to right until we reach the end of a row, and from the top to bottom until we reach the end of the image.

1. Now, let's consider the treatment of test images. As in the previous version of the script, we loop through a list of paths to test images, in order to load and process each one. The beginning of the loop is unchanged. For context, here it is:

For each test image, we iterate over the pyramid levels, and for each pyramid level, we iterate over the sliding window positions. For each window or **region of interest (ROI)**, we extract BoW descriptors and classify them using the SVM. If the classification produces a positive result that passes a certain confidence threshold, we add the rectangle's corner coordinates and confidence score to a list of positive detections. Continuing from the previous code block, we proceed to handle a given test image with the following code:

Let's take note of a couple of complexities in the preceding code, as follows:

- To obtain a confidence score for the SVM's prediction, we must run the `predict` method with an optional flag, `cv2.ml.STAT_MODEL_RAW_OUTPUT` . Then, instead of returning a label, the method returns a score as part of its output. This score may be negative, and a low value represents a high level of confidence. To make the score more intuitive – and to match the NMS function's assumption that a higher score is better – we negate the score so that a high value represents a high level of confidence.
- Since we are working with multiple pyramid levels, the window coordinates do not have a common scale. We have converted them back to a common scale – the original image's scale – before adding them to our list of positive detections.

So far, we have performed car detection at various scales and positions; as a result, we have a list of detected car rectangles, including coordinates and scores.

We expect a lot of overlap within this list of rectangles.

1. Now, let's call the NMS function, in order to cherry-pick the highest-scoring rectangles in the case of overlap, as follows:

Note that we have converted our list of rectangle coordinates and scores to a NumPy array, which is the format expected by this function.

At this stage, we have an array of detected car rectangles and their scores, and we have ensured that these are the best non-overlapping detections we can select (within the parameters of our model).

1. Now, let's draw the rectangles and their scores by adding the following inner loop to the code:

As in the previous version of this script, the body of the outer loop ends by showing the current test image, including the annotations we have drawn on it. After the loop runs through all the test images, we wait for the user to press any key; then, the program ends, as shown here:

Let's run the modified script, and see how well it can answer the eternal question: *Dude, where's my car?*

The following screenshot shows a pair of successful detections:

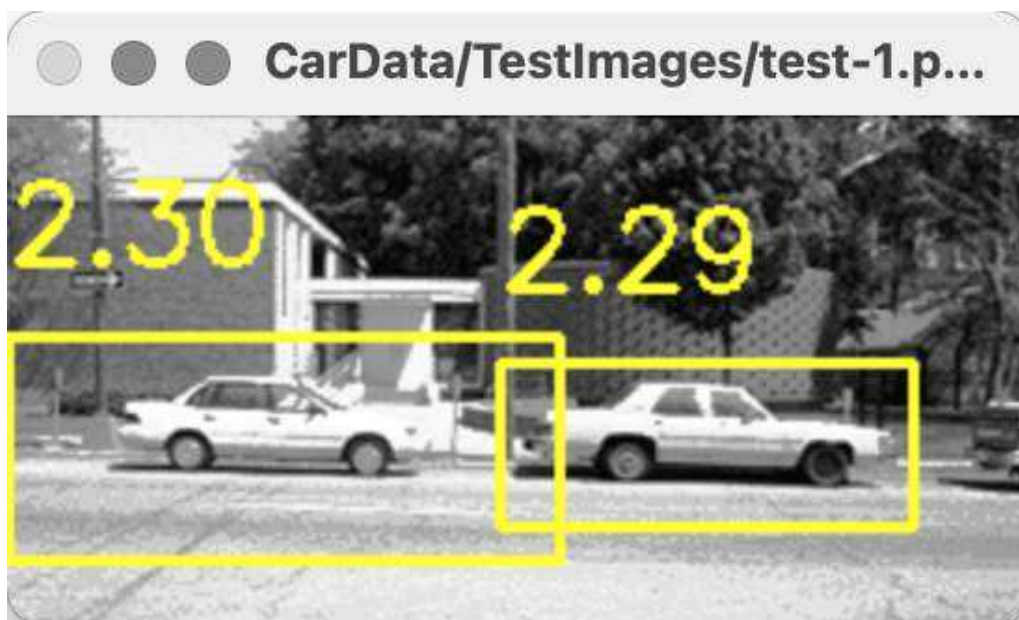


Figure 7.9: A pair of cars, correctly detected by our SVM with a sliding window approach

Among our other 5 test images, we have 2 false negatives (where a car is not detected as such) but no false positives.

Remember that in this sample script, our training sets are small. Larger training sets, with more diverse backgrounds, could improve the results. Also, remember that the image pyramid and sliding window are producing a large number of ROIs. When we consider this, we should realize that a low false *positive* rate is actually a significant accomplishment. Moreover, if we were performing detection on frames of a video, we could mitigate the problem of false *negatives* because we would have a chance to detect a car in multiple frames.

Feel free to experiment with the parameters and training sets of the preceding script. When you are ready, let's wrap up this chapter with a few closing notes.

Saving and loading a trained SVM

A final piece of advice on SVMs: you do not need to train a detector every time you want to use it – and, indeed, you should avoid doing so because training is slow. You can use code such as the following to save a trained SVM model to an XML file:

Subsequently, you can reload the trained SVM, using code such as the following:

Typically, you might have one script that trains and saves your SVM model, and other scripts that load and use it for various detection problems.

Summary

In this chapter, we covered a wide range of concepts and techniques, including HOG, BoW, SVMs, image pyramids, sliding windows, and NMS. We learned that these techniques have applications in object detection, as well as other fields. We wrote a script that combined most of these techniques – BoW, SVMs, an image pyramid, a sliding window, and NMS – and we gained practical experience in machine learning through the exercise of training and testing a custom detector. Finally, we demonstrated that we can detect cars!

Our new knowledge forms the foundation of the next chapter, in which we will utilize object detection and classification techniques on sequences of frames in videos. We will learn how to track objects and retain information about them – an important objective in many real-world applications.

Join our book community on Discord

<https://discord.gg/djGjeMECnw>



In this chapter, we will explore a selection of techniques from the vast topic of object tracking, which is the process of locating a moving object in a movie or a video feed from a camera. Real-time object tracking is a critical task in many computer vision applications, such as surveillance, perceptual user interfaces, augmented reality, object-based video compression, and driver assistance.

Tracking objects can be accomplished in several ways, with the most optimal technique being largely dependent on the task at hand. We will take the following route in our study of this topic:

- Detect moving objects based on differences between the current frame and a frame that represents the background. First, we will try a simple implementation of this approach. Then, we will use OpenCV's implementations of more advanced algorithms, namely, the **Mixture of Gaussians (MOG)** and **k-nearest neighbors (KNN)** background subtractors. We will also consider how to modify our scripts to use any other background subtractor that OpenCV supports, such as the **Godbehere-Matsukawa-Goldberg (GMG)** background subtractor.

- Track a moving object based on a color histogram of the object. This approach involves histogram back-projection, which is the process of computing the similarity between various image regions and a histogram. In other words, the histogram serves as a template of how we expect the object to look. We will use tracking algorithms called **MeanShift** and **CamShift** , which operate on the result of the histogram back-projection.
- Use a Kalman filter to find a trend in an object's motion and to predict where the object is going next.
- Review the manner in which OpenCV favors **object-oriented programming (OOP)** paradigms, and consider how this differs from **functional programming (FP)** paradigms.
- Implement a pedestrian tracker that combines KNN background subtraction, MeanShift, and Kalman filtering.
- Modify the pedestrian tracker to use **Multiple Instance Learning Tracker (MILTrack)** or any of the other machine learning algorithms that OpenCV implements via its `cv2.Tracker` interface.

If you have been reading this book sequentially, then, by the end of this chapter, you will know a lot of ways to describe, detect, classify, and track objects in 2D. At that point, you should be ready to pursue 3D tracking in *Chapter 9 , Camera Models and Augmented Reality* .

Technical requirements

This chapter uses Python, OpenCV, and NumPy. Please refer to *Chapter 1 , Setting Up OpenCV* , for installation instructions.

The complete code and sample videos for this chapter can be found in this book's GitHub

repository, <https://github.com/PacktPublishing/Learning-OpenCV-5-Computer-Vision-with-Python-Fourth-Edition> , in the `chapter08` folder.

The sample videos can be found in the `videos` folder.

Detecting moving objects with background subtraction

To track anything in a video, first, we must identify the regions of a video frame that correspond to moving objects. Many motion detection techniques are based on the simple concept of **background subtraction**. For example, suppose that we have a stationary camera viewing a scene that is also mostly stationary. In addition to this, suppose that the camera's exposure and the lighting conditions in the scene are stable so that frames do not vary much in terms of brightness. Under these conditions, we can easily capture a reference image that represents the background or, in other words, the stationary components of the scene. Then, any time the camera captures a new frame, we can subtract the frame from the reference image and take the absolute value of this difference in order to obtain a measurement of motion at each pixel location in the frame. If any region of the frame is very different from the reference image, we conclude that the given region is a moving object.

Background subtraction techniques, in general, have the following limitations:

- Any camera motion, change in exposure, or change in lighting conditions can cause a change in pixel values throughout the entire scene all at once; therefore, the entire background model (or reference image) becomes outdated.
- A piece of the background model can become outdated if an object enters the scene and then just stays there for a long period of time. For example, suppose our scene is a hallway. Someone enters the hallway, puts a poster on the wall, and leaves the poster there. For all practical purposes, the poster is now just another part of the stationary background; however, it was not part of our reference image, so our background model has become partly outdated.

These problems point to a need to dynamically update the background model based on a series of new frames. Advanced background subtraction techniques attempt to address this need in a variety of ways.

Another general limitation is that shadows and solid objects can affect a background subtractor in similar ways. For instance, we might get an inaccurate

picture of a moving object's size and shape because we cannot differentiate the object from its shadow. However, advanced background subtraction techniques do attempt to distinguish between shadow regions and solid objects using various means.

Background subtractors generally have yet another limitation: they do not offer fine-grained control over the kind of motion that they detect. For example, if a scene shows a subway car that is continuously shaking as it travels on its track, this repetitive motion will affect the background subtractor. For practical purposes, we might consider the subway car's vibrations to be normal variations in a semi-stationary background. We might even know the frequency of these vibrations. However, a background subtractor does not embed any information about frequencies of motion, so it does not offer a convenient or precise way in which to filter out such predictable motions. To compensate for such shortcomings, we can apply preprocessing steps such as blurring the reference image and also blurring each new frame; in this way, certain frequencies are suppressed, albeit in a manner that is not very intuitive, efficient, or precise.

Analyzing the frequencies of motion is beyond the scope of this book. However, for an introduction to this topic in a computer vision context, see Joseph Howse's book, *OpenCV 4 for Secret Agents* (Packt Publishing, 2019), specifically *Chapter 7 , Seeing a Heartbeat with a Motion-Amplifying Camera* .

Now that we have had an overview of background subtraction and understood some of the obstacles that it faces, let's investigate how several implementations of it fare in action. We will start with a simple but not robust implementation that we can handcraft in a few lines of code, and then progress to more sophisticated alternatives that OpenCV provides for us.

Implementing a basic background subtractor

To implement a basic background subtractor, let's take the following approach:

1. Start capturing frames from a camera.
2. Discard the first nine frames so that the camera has time to properly adjust its autoexposure to suit the lighting conditions in the scene.
3. Take the 10th frame, convert it to grayscale, blur it, and use this blurred image as the reference image of the background.

4. For each subsequent frame, blur the frame, convert it to grayscale, and compute the absolute difference between this blurred frame and the reference image of the background. Perform thresholding, smoothing, and contour detection on the differenced image. Draw and show the bounding boxes of the major contours.

The use of a Gaussian blur should make our background subtractor less susceptible to small vibrations, as well as digital noise. We will also use morphological operations for the same purpose.

To blur images, we will use the Gaussian blur algorithm, which we originally discussed in *Chapter 3 , Processing Images with OpenCV* , specifically in the *HPFs and LPFs* section. To smooth thresholded images, we will use morphological erosion and dilation, which we originally discussed in *Chapter 4 , Depth Estimation and Segmentation* , specifically in the *Image segmentation with the Watershed algorithm* section. Contour detection and bounding boxes are also among the topics we introduced in *Chapter 3 , Processing Images with OpenCV* , specifically in the *Contour detection* section.

Expanding the preceding list into smaller steps, we can consider our script's implementation in eight sequential blocks of code:

1. Let's begin by importing OpenCV and defining the size of our kernels for the blur , erode , and dilate operations:
1. Now, let's try to capture 10 frames from a camera:
1. If we were unable to capture 10 frames, we exit. Otherwise, we proceed to convert the 10th frame to grayscale and blur it:
1. At this stage, we have our reference image of the background. Now, let's proceed to capture more frames, in which we may detect motion. Our processing of each frame begins with grayscale conversion and a Gaussian blur operation:
1. Now, we can compare the blurred, grayscale version of the current frame to the blurred, grayscale version of the background image. Specifically, we will use OpenCV's `cv2.absdiff` function to find the absolute value (or the magnitude) of the difference between these two images. Then, we will apply a threshold to obtain a pure black-and-white image, and morphological operations to smooth the thresholded image. Here is the relevant code:

1. At this point, if our technique has worked well, our thresholded image should contain white blobs wherever there is a moving object. Now, we want to find the contours of the white blobs and draw bounding boxes around them. As a further means of filtering out small changes that are probably not real objects, we will apply a threshold based on the area of the contour. If the contour is too small, we conclude that it is not a real moving object. (Of course, the definition of *too small* may vary depending on your camera's resolution and your application; in some circumstances, you might not wish to apply this test at all.) Here is the code to detect contours and draw bounding boxes:

1. Now, let's show the differenced image, the thresholded image, and the detection result with the bounding rectangles:

1. We will continue reading frames until the user presses the Esc key to quit:

There you have it: a basic motion detector that draws rectangles around moving objects. The final result is something like this:

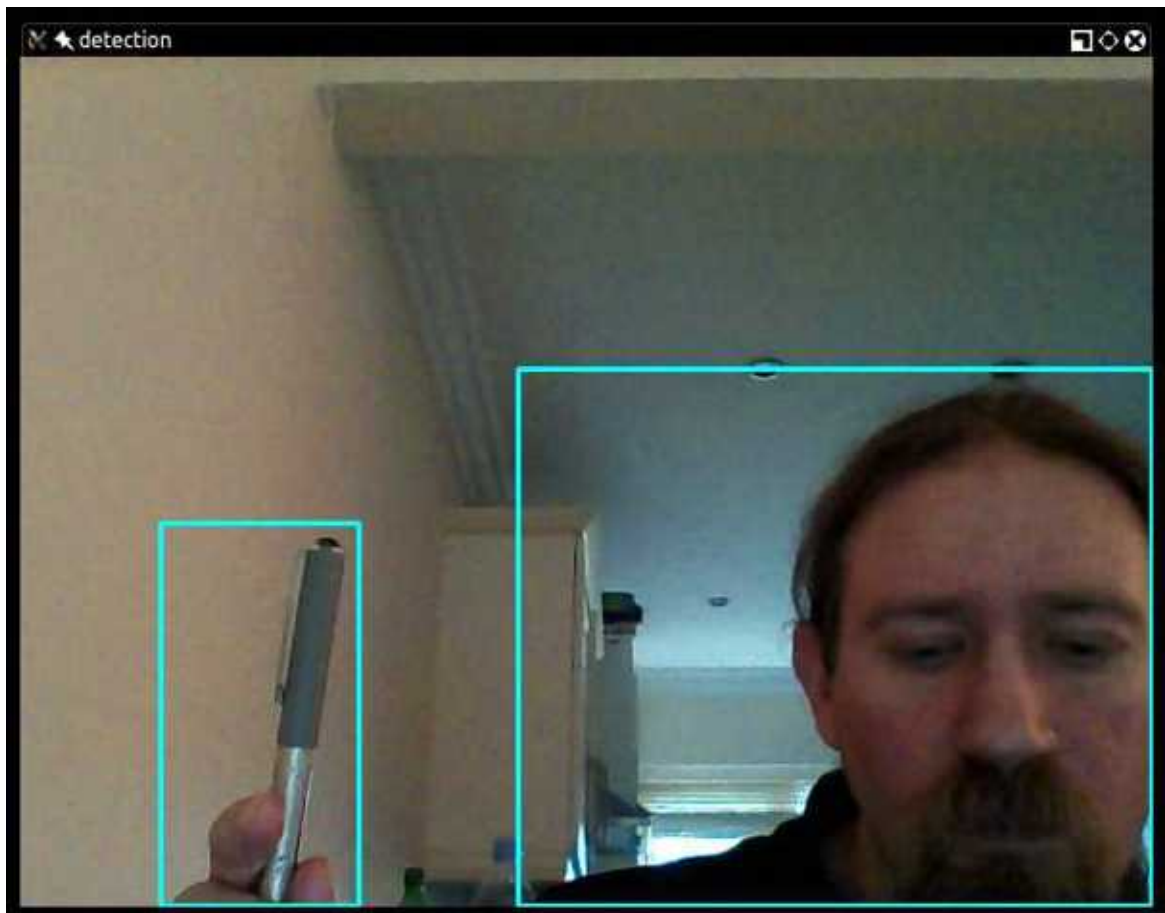


Figure 8.1: A moving pen and moving person, detected using a basic background subtractor

To obtain good results with this script, make sure that you (and other moving objects) do not enter the camera's field of view until after the background image has been initialized.

For such a simple technique, this result is quite promising. However, our script makes no effort to update the background image dynamically, so it will quickly become outdated if the camera moves or the lighting changes. Thus, we should move on to more flexible and intelligent background subtractors. Fortunately, OpenCV provides several ready-made background subtractors for our use. We will start with the one that implements the MOG algorithm.

Using a MOG background subtractor

OpenCV provides a class called `cv2.BackgroundSubtractor`, which has various subclasses that implement various background subtraction algorithms.

You may recall that we previously used OpenCV's implementation of the GrabCut algorithm to perform foreground/background segmentation in *Chapter 4*, *Depth Estimation and Segmentation*, specifically in the *Foreground detection with the GrabCut algorithm* section. Like `cv2.grabCut`, the various subclass implementations of `cv2.BackgroundSubtractor` can produce a mask that assigns different values to different segments of the image. Specifically, a background subtractor can mark foreground segments as white (that is, an 8-bit grayscale value of 255), background segments as black (0), and (in some implementations) shadow segments as gray (127). Moreover, unlike GrabCut, the background subtractors update the foreground/background model over time, typically by applying machine learning to a series of frames. Many of the background subtractors are named after the statistical clustering technique on which they base their approach to machine learning. We will begin by looking at a background subtractor based on the MOG clustering technique.

OpenCV has two implementations of a MOG background subtractor. Perhaps not surprisingly, they are named `cv2.bgsegm.BackgroundSubtractorMOG` and `cv2.BackgroundSubtractorMOG2`. The latter is a more recent and improved implementation, which adds support for shadow detection, so we will use it.

As a starting point, let's take our basic background subtraction script from the previous section. We will make the following modifications to it:

1. Replace our basic background subtraction model with a MOG background subtractor.
2. As input, use a video file instead of a camera.
3. Remove the use of Gaussian blur.
4. Adjust the parameters used in the thresholding, morphology, and contour analysis steps.
5. Get and show the MOG subtractor's model of the background.

These modifications affect a few lines of code, which are scattered throughout the script. Near the top of the script, let's initialize the MOG background subtractor and modify the size of the morphology kernels, as **highlighted** in the following code block:

Note that OpenCV provides a function, `cv2.createBackgroundSubtractorMOG2`, to create an instance of `cv2.BackgroundSubtractorMOG2`. The function accepts a parameter, `detectShadows`, which we set to `True` so that the shadow regions will be marked as such and not marked as part of the foreground.

The remaining changes, including the use of the MOG background subtractor to obtain a foreground/shadow/background mask, are **highlighted** in the following code block:

When we pass a frame to the background subtractor's `apply` method, the subtractor updates its internal model of the background and then returns a mask. As we previously discussed, the mask is white (255) for foreground segments, gray (127) for shadow segments, and black (0) for background segments. For our purposes, we treat shadows as the background, so we apply a nearly white threshold (244) to the mask.

To eliminate the thresholding step, we could configure the background subtractor to color the shadows black (instead of gray). To do this, we would use the subtractor's `setShadowValue` method, as **highlighted** below:

However, for our learning purposes, it is good to see a visualization of the shadows in gray before we filter them out.

The following set of screenshots shows a mask from the MOG detector (the top-left panel), a thresholded and morphed version of this mask (top-right), the model

of the background (bottom-left), and the detection result (bottom-right):



Figure 8.2: A moving person, detected using a MOG background subtractor

This scene contains not only shadows but also reflections due to the polished floor and wall. When shadow detection is enabled (as in the preceding screenshots), we are able to use a threshold to remove the shadows and reflections from our mask, leaving us with an accurate detection rectangle around the man in the hall. For comparison, if we disabled shadow detection by setting `detectShadows=False`, we would get results such as the following set of screenshots:



Figure 8.3: A moving person and moving reflections, detected using a MOG background subtractor

Now, with shadow detection disabled, we have two detections, which are both, arguably, inaccurate. One detection covers the man, his shadow, and his reflection on the floor. The second detection covers the man's reflection on the wall. These are, *arguably*, inaccurate detections because the man's shadow and reflections are not really moving objects, even though they are visual artifacts of a moving object.

So far, we have seen that a background subtraction script can be very concise and that a few small changes can drastically change the algorithm and the results, for better or worse. Continuing in the same vein, let's see how easily we can adapt our code to use another of OpenCV's advanced background subtractors to find another kind of moving object.

Using a KNN background subtractor

By modifying just five lines of code in our MOG background subtraction script, we can use a different background subtraction algorithm, different morphology parameters, and a different video as input. Thanks to the high-level interface that OpenCV provides, even such simple changes enable us to successfully handle a wide variety of background subtraction tasks.

Just by replacing `cv2.createBackgroundSubtractorMOG2` with `cv2.createBackgroundSubtractorKNN`, we can use a background subtractor based on KNN clustering instead of MOG clustering:

Note that despite the change in algorithm, the `detectShadows` parameter is still supported, as is the `setShadowValue` method. Additionally, the `apply` and `getBackgroundImage` methods are still supported, so we do not need to change anything related to the use of the background subtractor later in the script.

Remember that `cv2.createBackgroundSubtractorMOG2` returns a new instance of the `cv2.BackgroundSubtractorMOG2` class. Similarly, `cv2.createBackgroundSubtractorKNN` returns a new instance of the `cv2.BackgroundSubtractorKNN` class. Both of these classes are subclasses of `cv2.BackgroundSubtractor`, which defines common methods such as `apply`.

With the following changes, we can use morphology kernels that are slightly better adapted to a horizontally elongated object (in this case, a car), and we can use a video of traffic as input:

To reflect the change in algorithm, let's change the title of the mask's window from 'mog' to 'knn' :

The following set of screenshots shows the result of motion detection:

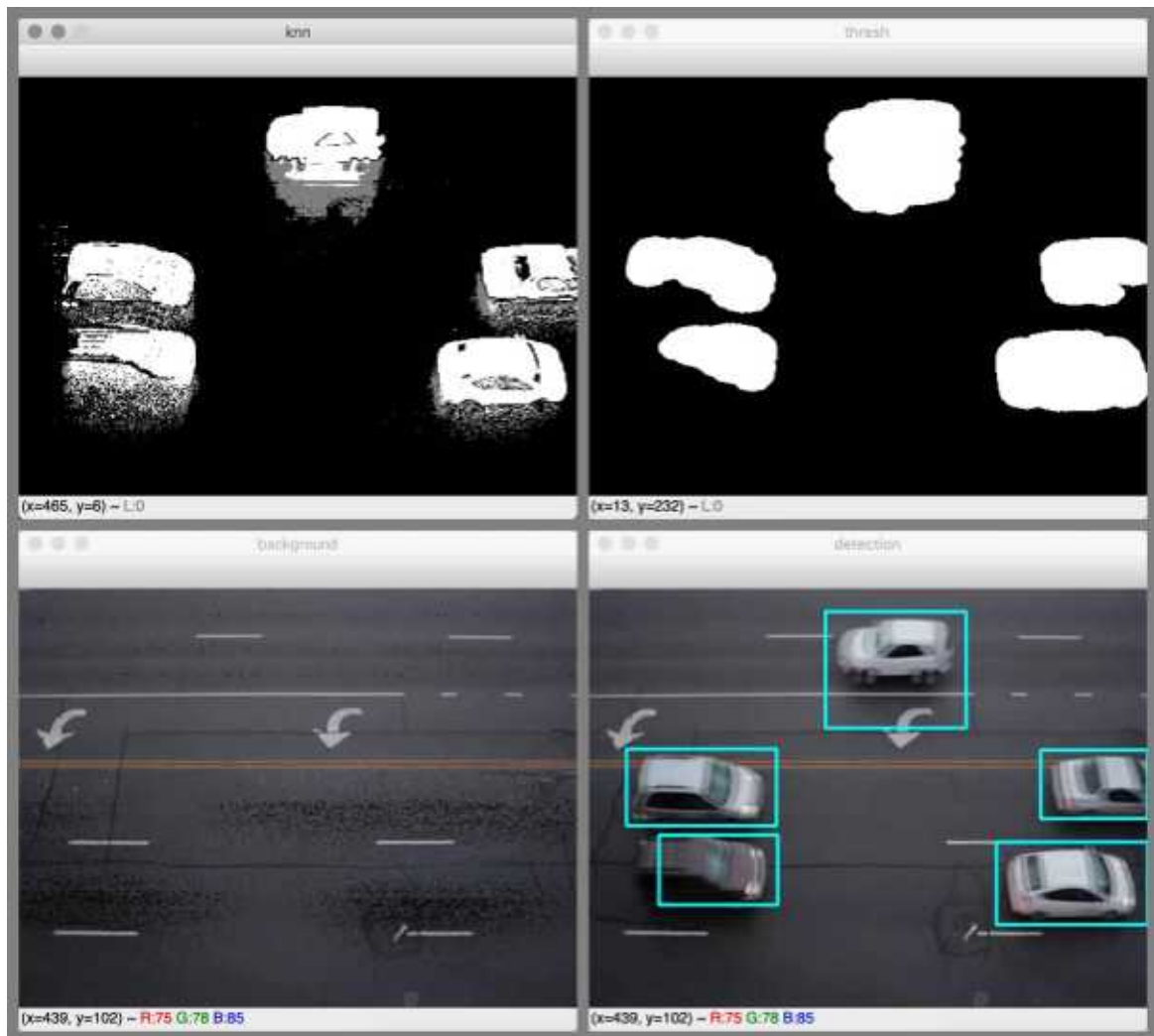


Figure 8.4: Moving cars, detected using a KNN background subtractor

The KNN background subtractor has worked quite well here, thanks in part to its ability to differentiate between objects and shadows. All cars have been individually detected; even though some cars are close to each other, they have not been merged into one detection. For three out of the five cars, the detection rectangles are accurate. For the dark car in the bottom-left part of the video frame, the background subtractor has failed to fully differentiate the rear of the car from the asphalt. For the white car at the top of the frame, the background subtractor has failed to fully differentiate the car and its shadow from the white markings on the road. Such problems could possibly be addressed by fine-tuning the morphology and threshold parameters, or by performing further analysis on the detected contours and surrounding regions based on the expected shape of a vehicle. Nonetheless, overall, this is a useful detection result that could enable us to count the number of cars traveling in each lane.

As we have seen, a few simple variations on a script can produce very different background subtraction results. Let's consider how we could further explore this observation.

Using GMG and other background subtractors

You are free to experiment with your own modifications to our background subtraction script. If you have obtained OpenCV with the optional `opencv_contrib` modules, as described in *Chapter 1 , Setting Up OpenCV* , then several more background subtractors are available to you in the `cv2.bgsegm` module. They can be created using the following functions:

- `cv2.bgsegm.createBackgroundSubtractorCNT`
- `cv2.bgsegm.createBackgroundSubtractorGMG`
- `cv2.bgsegm.createBackgroundSubtractorGSOC`
- `cv2.bgsegm.createBackgroundSubtractorLSBP`
- `cv2.bgsegm.createBackgroundSubtractorMOG`

These functions do not support the `detectShadows` parameter, and they create background subtractors that do not support shadow detection. However, all the background subtractors support the `apply` method.

As these types of background subtractors do not support shadow detection, they do not support the `setShadowValue` method. Moreover, some of them, including the GMG subtractor, do not support the `getBackgroundImage` method.

As an example of how to modify our background subtraction sample to use one of the `cv2.bgsegm` subtractors in the preceding list, let's use a GMG background subtractor. The relevant modifications are **highlighted** in the following block of code:

Note the modifications are similar to the ones we saw in the previous section, *Using a KNN background subtractor* . We simply use a different function to create the GMG subtractor, we adjust the morphology kernels' sizes to values that work better for this algorithm, and we change one of the window titles to `'gmg'` .

The GMG algorithm is named after its authors, Andrew B. Godbehere, Akihiro Matsukawa, and Ken Goldberg. They describe it in their paper, *Visual tracking of human visitors under variable-lighting conditions for a*

responsive audio art installation (ACC, 2012), which is available at <https://ieeexplore.ieee.org/document/6315174> . The GMG background subtractor takes a few frames to initialize itself before it starts producing a mask with white (object) regions.

Compared to the KNN background subtractor, the GMG background subtractor produces worse results with our sample video of traffic. This is partly because OpenCV's implementation of GMG does not differentiate between shadows and solid objects, so the detection rectangles are elongated in the direction of the cars' shadows or reflections. Here is a sample of the output:

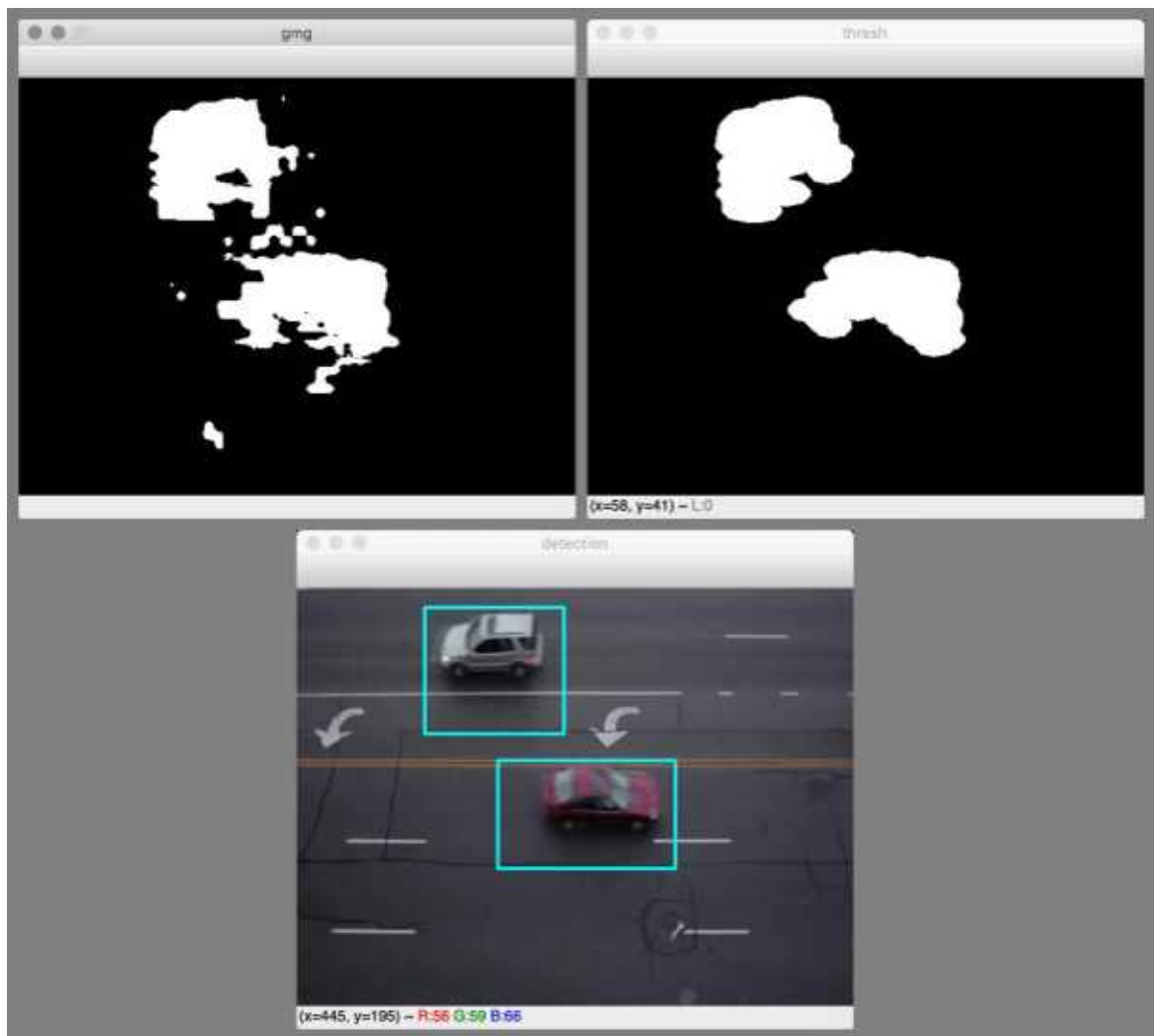


Figure 8.5: Moving cars, detected using a GMG background subtractor

When you are finished experimenting with background subtractors, let's proceed to examine other tracking techniques that rely on a template of an object we are trying to track, rather than a template of the background.

Tracking colorful objects using MeanShift and CamShift

We have seen that background subtraction can be an effective technique for detecting moving objects; however, we know that it has some inherent limitations. Notably, it assumes that the current background can be predicted based on past frames. This assumption is fragile. For example, if the camera moves, the entire background model could suddenly become outdated. Thus, in a robust tracking system, it is important to build some kind of model of foreground objects rather than just the background.

We saw various ways of detecting objects in *Chapter 5 , Detecting and Recognizing Faces* , *Chapter 6 , Retrieving Images and Searching Using Image Descriptors* , and *Chapter 7 , Building Custom Object Detectors* . For object *detection* , we favored algorithms that could deal with a lot of variation within a class of objects, so that our car detector was not too particular about what shape or color of car it would detect. For object *tracking* , our needs are a bit different. If we were tracking cars, we would want a different model for each car in the scene so that a red car and a blue car would not get mixed up. We would want to track the motion of each car separately.

Once we have detected a moving object (by background subtraction or other means), we would like to describe the object in a way that distinguishes it from other moving objects. In this way, we can continue to identify and track the object even if it crosses paths with another moving object. A **color histogram** may serve as a sufficiently unique description. Essentially, an object's color histogram is an estimate of the probability distribution of pixel colors in the object. For example, the histogram could indicate that each pixel in the object is 10% likely to be blue. The histogram is based on the actual colors observed in the object's region of a reference image. For example, the reference image could be the video frame in which we first detected the moving object.

Compared to other ways of describing an object, a color histogram has some properties that are particularly appealing in the context of motion tracking. The histogram serves as a lookup table that directly maps pixel values to probabilities, so it enables us to use every pixel as a feature, at a low computational cost. In this way, we can afford to perform tracking with very fine spatial resolution in real

time. To find the most likely location of an object that we are tracking, we just have to find the region of interest where the pixel values map to the maximum probability, according to the histogram.

Naturally, this approach is leveraged by an algorithm with a catchy name: MeanShift. For each frame in a video, the MeanShift algorithm performs tracking iteratively by computing a centroid based on probability values in the current tracking rectangle, shifting the rectangle's center to this centroid, recomputing the centroid based on values in the new rectangle, shifting the rectangle again, and so on. This process continues until **convergence** is achieved (meaning that the centroid ceases to move or nearly ceases to move) or until a maximum number of iterations is reached. Essentially, MeanShift is a clustering algorithm with applications that extend beyond computer vision. The algorithm was first described in a paper entitled *The estimation of the gradient of a density function, with applications in pattern recognition* (IEEE Transactions on Information Theory, 1975), by K. Fukunaga and L. Hostetler. The paper is available to IEEE subscribers at <https://ieeexplore.ieee.org/document/1055330> .

Before delving into a sample script, let's consider the type of tracking result we want to achieve with MeanShift, and let's learn more about OpenCV's functionality pertaining to color histograms.

Planning our MeanShift sample

For our first demonstration of MeanShift, we are not concerned with the approach to the initial detection of moving objects. We will use a naive approach that simply chooses the central part of the first video frame as our initial region of interest. (The user must ensure that an object of interest is initially located in the center of the video.) We will calculate a histogram of this initial region of interest. Then, in subsequent frames, we will use this histogram and the MeanShift algorithm to track the object.

Visually, the MeanShift demo will resemble many of the object detection samples that we have previously written. For every frame, we will draw a blue outline around the tracking rectangle, as shown here:



Figure 8.6: A toy phone with a distinctive lilac hue, tracked using histogram back-projection and MeanShift

Here, the toy phone has a lilac color that is not present in any other object in the scene. Thus, the phone has a distinctive histogram, making it easy to track. Next, let's consider how such a histogram is calculated and then used as a lookup table of probabilities.

Calculating and back-projecting color histograms

To calculate a color histogram, OpenCV provides a function called `cv2.calcHist`. To apply a histogram as a lookup table, OpenCV provides another function called `cv2.calcBackProject`. The latter operation is known as **histogram back-projection**, and it transforms a given image into a probability map based on a given histogram. Let's first visualize the output of these two functions, and then examine their parameters.

A histogram can use any color model, such as **blue-green-red (BGR)**, **hue-saturation-value (HSV)**, or grayscale. (For an introduction to color models, refer to *Chapter 3 , Processing Images with OpenCV* , specifically the *Converting images between different color models* section.) For our samples, we will use the histograms of only the hue (H) channel of the HSV color model. The following diagram is a visualization of a hue histogram:

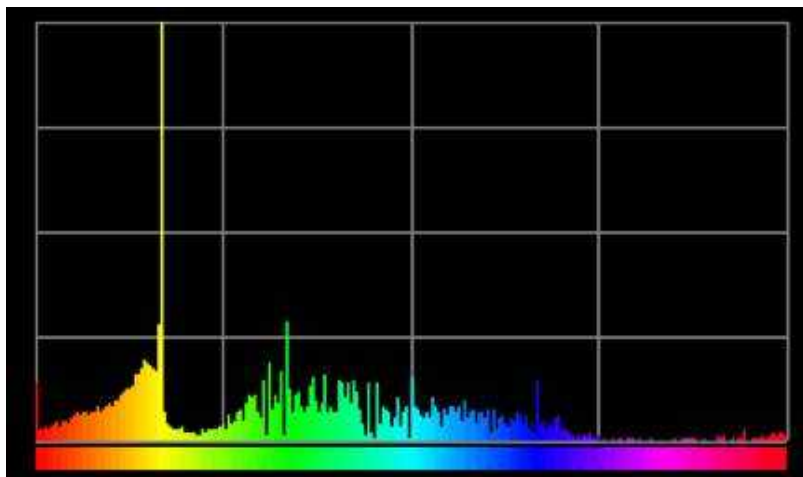


Figure 8.7: A visualization of a hue histogram. This is a sample of the output from an image-viewing application called DPEx (<https://www.rysys.co.jp/dpex/en/>).

On the x axis of this plot, we have the hue, and on the y axis, we have the hue's estimated probability or, in other words, the proportion of pixels in the image that have the given hue. If you are reading the e-book edition of this book, you will see that the plot is color-coded according to hue. From left to right, the plot progresses through the hues of the color wheel: red, yellow, green, cyan, blue, magenta, and, finally, back to red. This particular histogram seems to represent an object with a lot of yellow in it.

OpenCV represents H values with a range from 0 to 179, which, unfortunately, is a tad less precise than a typical HSV representation. Some other systems use a wider (more precise) range from 0 to 359 (like the degrees of a circle) or from 0 to 255 (to match the range of one byte).

Some caution is required in interpreting hue histograms because pure black and pure white pixels do not have a meaningful hue; however, their hue is usually represented as 0 (red).

When we use `cv2.calcHist` to generate a hue histogram, it returns a 1D array that is conceptually similar to the preceding plot. Alternatively, depending on the parameters we provide, we could use `cv2.calcHist` to generate a histogram of a different channel or of two channels at once. In the latter case, `cv2.calcHist` would return a 2D array.

Once we have a histogram, we can back-project the histogram onto any image. `cv2.calcBackProject` produces a back-projection in the format of an 8-bit grayscale image, with pixel values that potentially range from 0 (indicating a low probability) to 255 (indicating a high probability), depending on how we scale the values. For example, consider the following pair of photographs, showing a back-projection and then a visualization of the MeanShift tracking result:

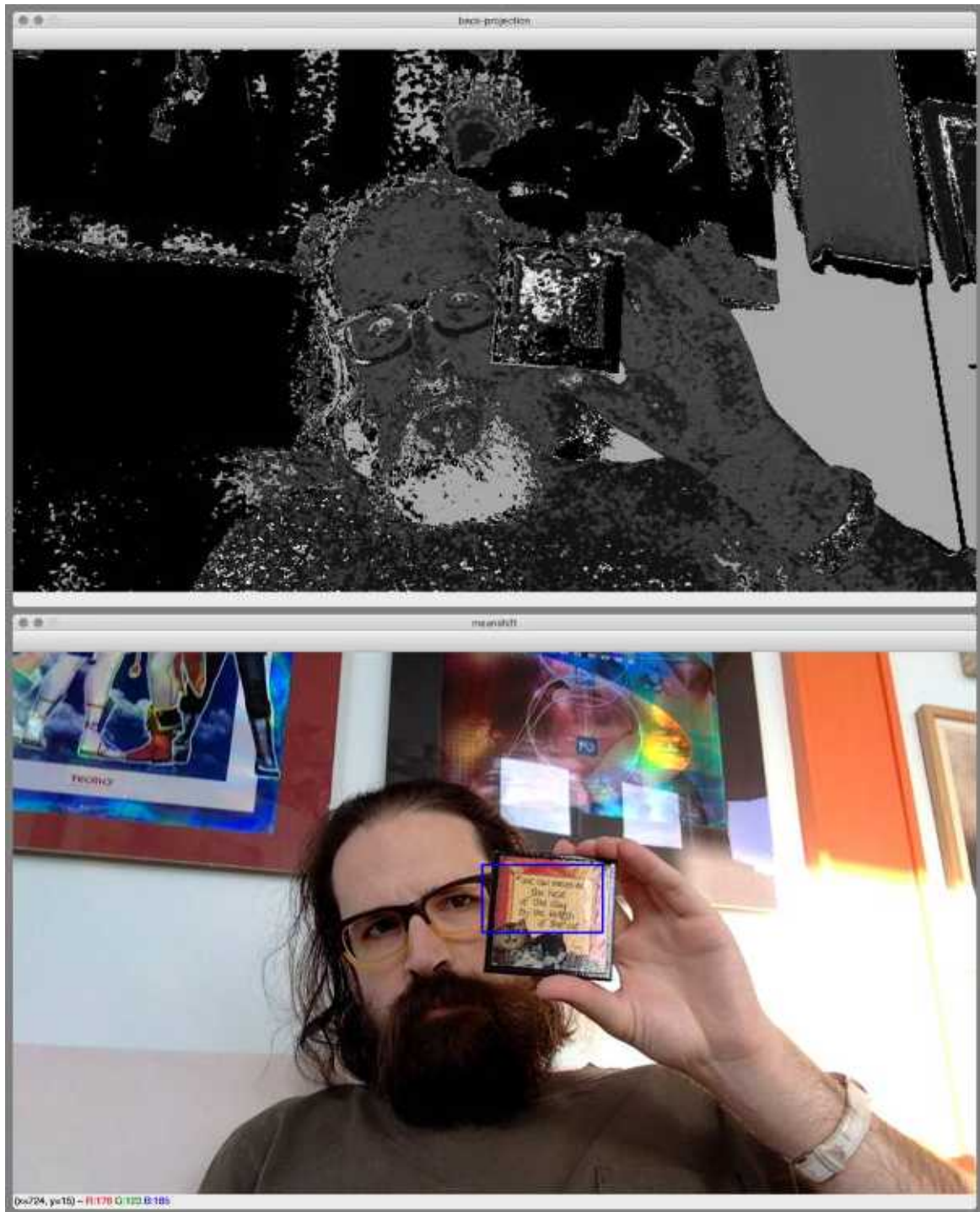


Figure 8.8: A coaster with distinctive yellow, red, and brown hues, tracked using histogram back-projection and MeanShift

Here, we are tracking a small object whose main colors are yellow, red, and brown. The back-projection is brightest in regions that are actually part of the object. The back-projection is also somewhat bright in other regions of similar

color, such as Joseph Howse's brown beard, the yellow rims of his glasses, and the red border of one of the posters in the background.

Now that we have visualized the outputs of `cv2.calcHist` and `cv2.calcBackProject`, let's examine the parameters that these functions accept.

Understanding the parameters of `cv2.calcHist`

The `cv2.calcHist` function has the following signature:

The following table contains descriptions of the parameters (adapted from the official OpenCV documentation):

Parameter	Description
<code>images</code>	This parameter is a list of one or more source images. They should all have the same bit depth (8-bit, 16-bit, or 32-bit) and the same size.
<code>channels</code>	This parameter is a list of the indices of the channels used to compute the histogram. For example, <code>channels=[0]</code> means that only the first channel (that is, the channel with index 0) is used to compute the histogram.
<code>mask</code>	This parameter is a mask. If it is <code>None</code> , no masking is performed; every region of the images is used in the histogram calculation. If it is not <code>None</code> , then it must be an 8-bit array of the same size as each image in <code>images</code> . The mask's nonzero elements mark the regions of the images that should be used in the histogram calculation.
<code>histSize</code>	This parameter is a list of the number of histogram bins to use for each channel. The length of the <code>histSize</code> list must be the same as the length of the <code>channels</code> list. For example, if <code>channels=[0]</code> and <code>histSize=[180]</code> , the histogram has 180 bins for the first channel (and any other channels are unused).
<code>ranges</code>	This parameter is a list that specifies the ranges of values to use for each channel. Each range is inclusive of the lower bound but exclusive of the upper bound. The length of the <code>ranges</code> list must be twice the length of the <code>channels</code> list. For example, if <code>channels=[0]</code> , <code>histSize=[180]</code> , and <code>ranges=[0, 180]</code> , the histogram has 180 bins for the first channel, and these bins are based on values in the range from 0 to 179; in other words, there is one input value per bin.

`hist` This optional parameter is the output histogram. If it is `None` (the default), a new array will be returned as the output histogram.

`accumulate` This optional parameter is the `accumulate` flag. By default, it is `False`. If it is `True`, then the original content of `hist` is not cleared; instead, the new histogram is added to the original content of `hist`. This feature enables you to compute a single histogram from several lists of images, or to update the histogram over time.

In our samples, we will calculate the hue histogram of a region of interest like so:

Next, let's consider the parameters of `cv2.calcBackProject`.

Understanding the parameters of `cv2.calcBackProject`

The `cv2.calcBackProject` function has the following signature:

The following table contains descriptions of the parameters (adapted from the official OpenCV documentation):

Parameter Description

<code>images</code>	This parameter is a list of one or more source images. They all should have the same bit depth (8-bit, 16-bit, or 32-bit) and the same size.
<code>channels</code>	This parameter must be the same as the <code>channels</code> parameter used in <code>calcHist</code> .
<code>hist</code>	This parameter is the histogram.
<code>ranges</code>	This parameter must be the same as the <code>ranges</code> parameter used in <code>calcHist</code> .
<code>scale</code>	This parameter is a scale factor. The back-projection is multiplied by this scale factor.
<code>dst</code>	This optional parameter is the output back-projection. If it is <code>None</code> (the default), a new array will be returned as the back-projection.

In our samples, we will use code similar to the following line to back-project a hue histogram on to an HSV image:

Having examined the `cv2.calcHist` and `cv2.calcBackProject` functions in detail, let's now put them into action in a script that performs tracking with MeanShift.

Implementing the MeanShift example

Let's go sequentially through the implementation of our MeanShift example:

1. Like our basic background subtraction example, our MeanShift example begins by capturing (and discarding) several frames from a camera so that the autoexposure can adjust:
1. By the 10th frame, we assume that the exposure is good; therefore, we can extract an accurate histogram of a region of interest. The following code defines the bounds of the **region of interest** (**ROI**):
1. Then, the following code selects the ROI's pixels and converts them to the HSV color space:
1. Next, we calculate the hue histogram of the ROI:
1. After the histogram is calculated, we normalize the values to a range from 0 to 255:
1. Remember that MeanShift performs a number of iterations before reaching convergence; however, this convergence is not assured. Thus, OpenCV allows us to specify the so-called termination criteria. Let's define the termination criteria as follows:

Based on these criteria, MeanShift will stop calculating the centroid shift after 10 iterations (the **count** criterion) or when the shift is no longer larger than 1 pixel (the **epsilon** criterion). The combination of flags (`cv2.TERM_CRITERIA_COUNT` | `cv2.TERM_CRITERIA_EPS`) indicates that we are using both of these criteria.

1. Now that we have calculated a histogram and defined MeanShift's termination criteria, let's start our usual loop in which we capture and process frames from the camera. With each frame, the first thing we do is convert it to the HSV color space:
1. Now that we have an HSV image, we can perform the long-awaited operation of histogram back-projection:
1. The back-projection, the tracking window, and the termination criteria can be passed to `cv2.meanShift` , which is OpenCV's implementation of the MeanShift algorithm. Here is the function call:

Note that MeanShift returns the number of iterations that it ran, as well as the new tracking window that it found. Optionally, we could compare the number of iterations to our termination criteria in order to determine whether the result converged. (If the actual number of iterations is less than the maximum, the result must have converged.)

1. Finally, we draw and show the updated tracking rectangle:

That is the whole example. If you run the program, it should produce an output that is similar to the screenshots we saw earlier, in the *Calculating and back-projecting color histograms* section.

By now, you should have a good idea of how color histograms, back-projections, and MeanShift work. However, the preceding program (and MeanShift in general) has a limitation: the size of the window does not change with the size of the object in the frames being tracked.

Gary Bradski – one of the founders of the OpenCV project – published a paper in 1998 to improve the accuracy of MeanShift. He described a new algorithm called **Continuously Adaptive MeanShift** (**CAMShift** or **CamShift**), which is very similar to MeanShift but also adapts the size of the tracking window when MeanShift reaches convergence. Let's take a look at an example of CamShift next.

Using CamShift

Although CamShift is a more complex algorithm than MeanShift, OpenCV provides a very similar interface for the two algorithms. The main difference is that a call to `cv2.CamShift` returns a rectangle with a particular rotation that follows the rotation of the object being tracked. With just a few modifications to the preceding MeanShift example, we can instead use CamShift and draw a rotated tracking rectangle. All of the necessary changes are **highlighted** in the following excerpt:

The arguments to `cv2.CamShift` are unchanged; they have the same meanings and the same values as the arguments to `cv2.meanShift` in our previous example.

We use the `cv2.boxPoints` function to find the vertices of the rotated tracking rectangle. Then, we use the `cv2.polylines` function to draw the lines connecting

these vertices. The following screenshot shows the result:



Figure 8.9: A coaster with distinctive yellow, red, and brown hues, tracked using histogram back-projection and CamShift

By now, you should be familiar with two families of tracking techniques. The first family uses background subtraction. The second uses histogram back-projection, combined with either MeanShift or CamShift. Now, let's meet the Kalman filter, which exemplifies a third family; it finds trends or, in other words, predicts future motion based on past motion.

Finding trends in motion using the Kalman filter

The Kalman filter is an algorithm developed mainly (but not exclusively) by Rudolf Kalman in the late 1950s. It has found practical applications in many fields, particularly navigation systems for all sorts of vehicles, from nuclear submarines to aircraft.

The Kalman filter operates recursively on a stream of noisy input data to produce a statistically optimal estimate of the underlying system state. In the context of computer vision, the Kalman filter can smooth the estimate of a tracked object's position.

Let's consider a simple example. Think of a small red ball on a table and imagine you have a camera pointing at the scene. You identify the ball as the subject to be tracked and flick it with your fingers. The ball will start rolling on the table in accordance with the laws of motion.

If the ball is rolling at a constant speed of 1 meter per second in a particular direction, it is easy to estimate where the ball will be in 1 second's time: it will be 1 meter away. The Kalman filter applies laws such as this to predict an object's position in the current video frame based on tracking results gathered in previous frames. The Kalman filter itself is not gathering these tracking results, but it is updating its model of the object's motion based on the tracking results derived from another algorithm – in our case, a visual tracking algorithm such as MeanShift. The tracking algorithm produces results that are (probably) noisy but (hopefully) **unbiased** ; that is to say, its errors do not tend toward any particular direction. Conversely, it is the Kalman filter's role to smooth these results, to make them less noisy, at the risk of biasing the results based by seeking a trend to fit an idealized model of motion.

Naturally, the Kalman filter cannot foresee new forces acting on the ball – such as a collision with a pencil lying on the table – but it can update its model of the ball's motion after the fact, when new tracking results deviate significantly from what the Kalman filter would have predicted. Through the use of the Kalman filter, we can obtain estimates that are more stable and more consistent with the laws of motion than the tracking results alone. This stabilization is usually a good

thing, but be aware that the stabilized results may lag behind reality when a tracked object changes course.

Understanding the predict and update phases

From the preceding description, we gather that the Kalman filter's algorithm has two phases:

- **Predict** : In the first phase, the Kalman filter uses the covariance calculated up to the current point in time to estimate the object's new position.
- **Update** : In the second phase, the Kalman filter records the object's position and adjusts the covariance for the next cycle of calculations.

The update phase is – in OpenCV's terms – a **correction** . Thus, OpenCV provides a `cv2.KalmanFilter` class with the following methods:

For the purpose of smoothly tracking objects, we will call the `predict` method to estimate the position of an object, and then use the `correct` method to instruct the Kalman filter to adjust its calculations based on a new tracking result from another algorithm such as MeanShift. However, before we combine the Kalman filter with a computer vision algorithm, let's examine how it performs with position data from a simple motion sensor.

Tracking a mouse cursor

Motion sensors have been commonplace in user interfaces for a long time. A computer's mouse senses its own motion relative to a surface such as a table. The mouse is a real, physical object, so it is reasonable to apply the laws of motion in order to predict changes in mouse coordinates. We are going to do exactly this as a demo of the Kalman filter.

Our demo will implement the following sequence of operations:

1. Start by initializing a black image and a Kalman filter. Show the black image in a window.
2. Every time the windowed application processes input events, use the Kalman filter to predict the mouse's position. Then, correct the Kalman filter's model based on the actual mouse coordinates. On top of the black image, draw a red line from the old predicted position to the new predicted

position, and then draw a green line from the old actual position to the new actual position. Show the drawing in the window.

3. When the user hits the Esc key, exit and save the drawing to a file.

To begin the script, the following code initializes an 800 x 800 black image:

Now, let's initialize the Kalman filter:

Based on the preceding initialization, our Kalman filter will track a 2D object's position and velocity. We will take a deeper look at the process of initializing a Kalman filter in *Chapter 9, Camera Models and Augmented Reality*, where we will track a 3D object's position, velocity, acceleration, rotation, angular velocity, and angular acceleration. For now, let's just take note of the two parameters in `cv2.KalmanFilter(4, 2)`. The first parameter is the number of variables tracked (or predicted) by the Kalman filter, in this case, 4 : the x position, the y position, the x velocity, and the y velocity. The second parameter is the number of variables provided to the Kalman filter as measurements, in this case, 2 : the x position and the y position. We also initialize several matrices that describe the relationships among all these variables.

Having initialized the image and the Kalman filter, we also have to declare variables to hold the actual (measured) and predicted mouse coordinates. Initially, we have no coordinates, so we will assign `None` to these variables:

Then, we declare a callback function that handles mouse movement. This function is going to update the state of the Kalman filter, and draw a visualization of both the unfiltered mouse movement and the Kalman-filtered mouse movement. The first time we receive mouse coordinates, we initialize the Kalman filter's state so that its initial prediction is the same as the actual initial mouse coordinates. (If we did not do this, the Kalman filter would assume that the initial mouse position was $(0, 0)$.) Subsequently, whenever we receive new mouse coordinates, we correct the Kalman filter with the current measurement, calculate the Kalman prediction, and, finally, draw two lines: a green line from the last measurement to the current measurement and a red line from the last prediction to the current prediction. Here is the callback function's implementation:

The next step is to initialize the window and pass our callback function to the `cv2.setMouseCallback` function:

Since most of the program's logic is in the mouse callback, the implementation of the main loop is simple. We just continually show the updated image until the

user hits the Esc key:

Run the program and move your mouse around. If you make a sudden turn at high speed, you will notice that the prediction line (in red) will trace a wider curve than the measurement line (in green). This is because the prediction is following the momentum of the mouse's movement up to that time. Here is a sample result:

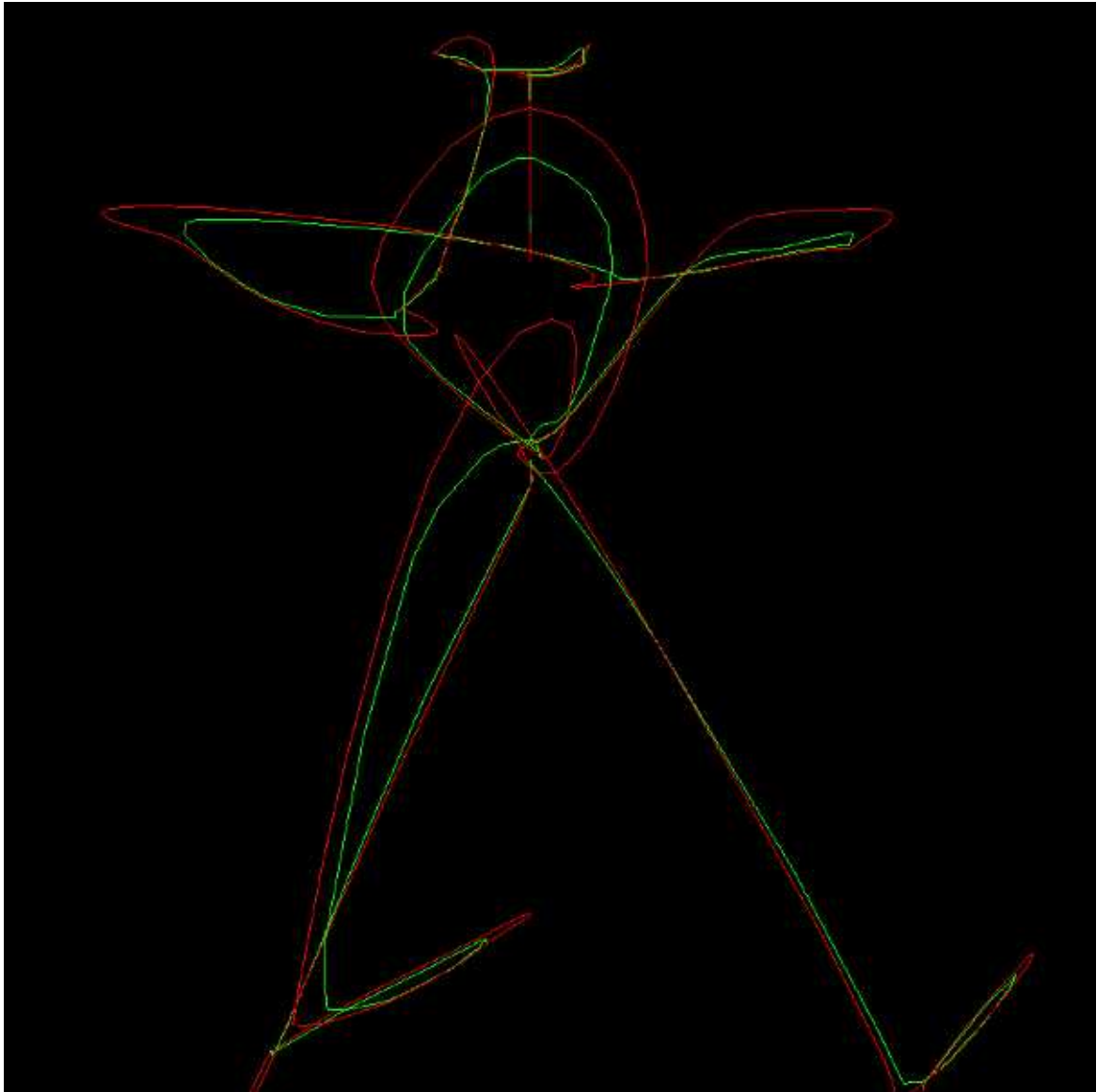


Figure 8.10: A drawing based on the movement of a computer mouse; the green line is actual movement and the red line is a Kalman filter's prediction of the movement

Perhaps the preceding diagram will give us inspiration for our next sample application, in which we track pedestrians.

Tracking pedestrians

Up to this point, we have familiarized ourselves with the concepts of motion detection, object detection, and object tracking. You are probably anxious to put this newfound knowledge to good use in a real-life scenario. Let's do just that by tracking pedestrians in a video from a surveillance camera.

You can find a surveillance video inside the OpenCV repository at `samples/data/vtest.avi`. A copy of this video is located inside this book's GitHub repository at `vidoesvidoes/pedestrians.avi`.

Let's lay out a plan and then implement the application!

Planning the flow of the application

The application will adhere to the following logic:

1. Capture frames from a video file.
2. Use the first 20 frames to populate the history of a background subtractor.
3. Based on background subtraction, use the 21st frame to identify moving foreground objects. We will treat these as pedestrians. For each pedestrian, assign an ID and an initial tracking window, and then calculate a histogram.
4. For each subsequent frame, track each pedestrian using a Kalman filter and MeanShift.

If this were a real-world application, you would probably store a record of each pedestrian's route through the scene so that a user could analyze it later. However, this type of record-keeping is beyond the remit of this example.

Additionally, in a real-world application, you would make sure to identify new pedestrians entering the scene; however, for now, we will focus on

tracking only those objects that are in the scene near the start of the video.

You will find the code for this application inside the book's GitHub repository at `chapter08/track_pedestrians.py` . Before examining the implementation, let's briefly digress to consider programming paradigms and how they relate to our use of OpenCV.

Comparing the object-oriented and functional paradigms

Although most programmers are either familiar (or work on a constant basis) with OOP, another paradigm called FP has, for many years, been gaining support among programmers who favor a purer mathematical foundation.

The works of Samuel Howse demonstrate a specification of a programming language with a pure mathematical foundation. You can find his doctoral thesis, *NummSquared 2006a0 Explained* , at <https://nummist.com/poohbist/NummSquared2006a0Explained.pdf> , and his paper, *NummSquared: a New Foundation for Formal Methods* , at <https://nummist.com/poohbist/NummSquaredFormalMethods.pdf> .

FP treats programs as the evaluation of mathematical functions, allows functions to return functions, and permits functions as arguments in a function. The strength of FP resides not only in what it can do, but also in what it can avoid, or aims at avoiding: for example, side effects and changing states. If the topic of FP has sparked an interest, make sure that you take a look at languages such as **Haskell** , **Clojure** , or **Meta Language (ML)**.

So, what is a side effect in programming terms? A function is said to have side effects if it produces any changes that are accessible outside its local scope, except for its return values. Python, like many other languages, is susceptible to side effects because it allows you to gain access to member variables and global variables – and, sometimes, this access can be accidental!

In languages that are not purely functional, a function's output can vary even when we pass it the same parameters repeatedly. For example, if a function takes an object as an argument, and the computation relies on the internal state of that object, the function will return different results according to changes in the object's state. This is a common occurrence in OOP with languages such as Python and C++.

So, why this digression? Well, it is a good occasion to consider the paradigms used in our own samples and in OpenCV, and how they differ from a purer mathematical approach. Throughout this book, we have often used global variables or object-oriented classes with member variables. The next program is another example of OOP. OpenCV, too, contains many functions with side effects and many object-oriented classes.

For example, any OpenCV drawing function, such as `cv2.rectangle` or `cv2.circle`, modifies the image that we pass to it as an argument. This approach contravenes one of FP's cardinal rules: avoid side effects and changes in state.

As a brief exercise, let's wrap `cv2.rectangle` in another Python function to perform a drawing in an FP style, without any side effects. The following implementation relies on making a copy of the input image, rather than modifying the original:

This approach – while computationally more expensive due to the copy operation – allows for code such as the following to run without side effects:

Here, `frame` and `frame_with_rect` are references to two different NumPy arrays containing different values. If we had used `cv2.rectangle` instead of our FP-inspired `draw_rect` wrapper, then `frame` and `frame_with_rect` would have been references to one and the same NumPy array (containing a drawing of a rectangle on top of the original image).

To conclude this digression, let's note that a variety of programming languages and paradigms can be applied successfully to computer vision problems. It is useful to know about multiple languages and paradigms so that you can choose the right tool for a given job.

Now, let's get back to our program and explore the implementation of a surveillance application, tracking moving objects in a video.

Implementing the Pedestrian class

The nature of the Kalman filter provides the main rationale for creating a `Pedestrian` class. The Kalman filter can predict the position of an object based on historical observations and can correct the prediction based on the actual data, but it can only do this for one object. As a consequence, we need one Kalman filter per object tracked.

Each `Pedestrian` object will act as a holder for a Kalman filter, a color histogram (calculated on the first detection of the object and used as a reference for the subsequent frames), and a tracking window, which will be used by the MeanShift algorithm. Furthermore, each pedestrian has an ID, which we will display so that we can easily distinguish between all of the pedestrians being tracked.

This time, instead of using just a hue histogram, we will use a 2D histogram of hue and value. Hue alone is not a great choice for tracking pedestrians because we do not necessarily expect to find a wide range of rainbow hues in the human form or in typical streetwear. Of course, a pedestrian with purple hair or a neon orange jacket might have a very distinctive hue histogram, yet many pedestrians would not. The addition of a value (or brightness) dimension may help to further distinguish pedestrians from each other and from typical backgrounds such as gray cement.

Let's proceed sequentially through the class's implementation:

1. As arguments, the `Pedestrian` class's constructor takes an ID, an initial frame in HSV format, and an initial tracking window. Here are the declarations of the class and its constructor:
1. To begin the constructor's implementation, we define variables for the ID, the tracking window, and the MeanShift algorithm's termination criteria:

1. We proceed by creating a normalized hue and value histogram of the region of interest in the initial HSV image:

1. Then, we initialize the Kalman filter:

Like in our mouse tracking example, we are configuring the Kalman filter to predict the motion of a 2D point. As the initial point, we use the center of the initial tracking window. This concludes the implementation of the constructor.

1. The `Pedestrian` class also has an `update` method, which we will call once per frame. As arguments, the `update` method takes a BGR frame (to use when we draw a visualization of the tracking result) and an HSV version of the same frame (to use for histogram back-projection). The implementation of the `update` method begins with familiar code for histogram back-projection and MeanShift, as displayed in the following lines:

1. Note that we have extracted the tracking window's center coordinates because we want to perform Kalman filtering on them. We proceed to do exactly this, and then we update the tracking window so that it is centered on the corrected coordinates:

1. To conclude the `update` method, we draw the Kalman filter's prediction as a blue circle, the corrected tracking window as a cyan rectangle, and the pedestrian's ID as blue text above the rectangle:

This is all the functionality and data that we need to associate with an individual pedestrian. Next, we need to implement a program that provides the necessary video frames to create and update `Pedestrian` objects.

Implementing the main function

Now that we have a `Pedestrian` class to maintain data about the tracking of each pedestrian, let's implement our program's `main` function. We will look at the parts of the implementation sequentially:

1. We begin by loading a video file, initializing a background subtractor, and setting the background subtractor's history length (that is, the number of frames affecting the background model):
1. Then, we define morphology kernels:
 1. We define a list called `pedestrians` , which is initially empty. A little later, we will add `Pedestrian` objects to this list. We also set up a frame counter, which we will use to determine whether enough frames have elapsed to fill the background subtractor's history. Here are the relevant definitions of the variables:
 1. Now, we start a loop. At the start of each iteration, we try to read a video frame. If this fails (for instance, at the end of the video file), we exit the loop:
 1. Proceeding with the body of the loop, we update the background subtractor based on the newly captured frame. If the background subtractor's history is not yet full, we simply continue to the next iteration of the loop. Here is the relevant code:
 1. Once the background subtractor's history is full, we do more processing on each newly captured frame. Specifically, we apply the same approach we used with background subtractors earlier in this chapter: we perform thresholding, erosion, and dilation on the foreground mask; and then we detect contours, which might be moving objects:
 1. We also convert the frame to HSV format because we intend to use histograms in this format for MeanShift. The following line of code performs the conversion:
 1. Once we have contours and an HSV version of the frame, we are ready to detect and track moving objects. We find and draw a bounding rectangle for each contour that is large enough to be a pedestrian. Moreover, if we have not yet populated the `pedestrians` list, we do so now by adding a new `Pedestrian` object based on each bounding rectangle (and the corresponding region of the HSV image). Here is

the subloop that handles the contours in the manner we have just described:

1. By now, we have a list of pedestrians whom we are tracking. We call each `Pedestrian` object's `update` method, to which we pass the original BGR frame (for use in drawing) and the HSV frame (for use in tracking with `MeanShift`). Remember that each `Pedestrian` object is responsible for drawing its own information (text, the tracking rectangle, and the Kalman filter's prediction). Here is the subloop that updates the `pedestrians` list:
1. Finally, we display the tracking results in a window, and we allow the user to exit the program at any time by pressing the `Esc` key:

There you have it: `MeanShift` working in tandem with the Kalman filter to track moving objects. All being well, you should see tracking results visualized in the following manner:



Figure 8.8: A visualization of contour detection results (thin green rectangle), Kalman-corrected `MeanShift` results (thick cyan rectangle), and the Kalman prediction (blue dot) in a complex tracker.

In this cropped screenshot, the green rectangle with the thin border is the detected contour, the cyan rectangle with the thick border is the Kalman-corrected `MeanShift` tracking rectangle, and the blue dot is the center position predicted by the Kalman filter.

As usual, feel free to experiment with the script. You may want to adjust the parameters, try a MOG background subtractor instead of KNN, or try CamShift instead of MeanShift. These changes should affect just a few lines of code. When you have finished, next, we will consider other possible modifications that might have a larger effect on the structure of the script.

Considering the next steps

The preceding program can be expanded and improved in various ways, depending on the requirements of a particular application. Consider the following examples:

- You could remove a `Pedestrian` object from the `pedestrians` list (and thereby destroy the `Pedestrian` object) if the Kalman filter predicts the pedestrian's position to be outside the frame.
- You could check whether each detected moving object corresponds to an existing `Pedestrian` instance in the `pedestrians` list, and, if not, add a new object to the list so that it will be tracked in subsequent frames.
- You could train a **support vector machine (SVM)** and use it to classify each moving object. By these means, you could establish whether or not the moving object is something you intend to track. For instance, a dog might enter the scene but your application might require the tracking of humans only. For more information on training an SVM, refer to *Chapter 7 , Building Custom Object Detectors* .
- You could adopt a tracking technique that is based on machine learning, instead of using MeanShift (or CamShift) and a Kalman filter. OpenCV provides easy-to-use implementations of several machine learning trackers, with a common interface called `cv2.Tracker` .

Let's delve a little deeper into this last point. For this chapter's final exercise, we are going to modify our code, especially the `Pedestrian` class, to use MILTrack as the tracking algorithm.

Modifying the pedestrian tracker to use MIL

By its nature, a video of a moving object shows us many variations of that object. For example, in some frames, we might see a person's face and hair, yet in other frames, we might see only the person's hair because she has turned her head away. Later, we might see the person's upper face but not her mouth or chin because she is drinking from a coffee mug. To track this person's head, it would be helpful to build a model of various segments rather than just the whole. Over a series of several frames, the tracker could learn that the features of the eyes, nose, mouth, and hair really are part of this woman's head, yet the coffee mug is not part of her head; it moves separately.

This type of learning is, essentially, what MILTrack tries to do. MILTrack breaks up the contents of the tracking window and nearby regions into several **patches** or **samples** subsections, it detects features in each patch, and it learns which of these local clusters of features do (or do not) move together over time. Specifically, the algorithm uses a variant of Haar-like features, which we previously encountered in *Chapter 5 , Detecting and Recognizing Faces* . MILTrack was first described by B. Babenko, M. Yang, and S. Belongie in their paper, *Visual Tracking with Online Multiple Instance Learning* (CVPR, 2009). A copy can be downloaded from <https://faculty.ucmerced.edu/mhyang/papers/cvpr09a.pdf> .

OpenCV implements MIL in a class called (not surprisingly) `cv2.TrackerMIL` . To create an instance of this class, we can call the `cv2.TrackerMIL_create` function, which has the following signature:

The optional parameters argument is an instance of `cv2.TrackerMIL_Params` , which has the following properties:

- `featureSetNumFeatures` : The maximum number of Haar features to detect in each patch. The default is 250.
- `samplerInitInRadius` : The pixel radius, beyond the tracking window, to search for positive patches during initialization. The default is 3.0.
- `samplerInitMaxNegNum` : The maximum number of negative patches, at the end of initialization. The default is 65.
- `samplerSearchWinSize` : The pixel radius, beyond the tracking window, to search for negative patches. The same value is used during

initialization and updates. The default is 25.0.

- `samplerTrackInRadius` : The pixel radius, beyond the tracking window, to search for positive patches during updates. The default is 4.0.
- `samplerTrackMaxNegNum` : The maximum number of negative patches, at the end of each update. The default is 65.
- `samplerTrackMaxPosNum` : The maximum number of positive patches, at the end of each update. The default is 100,000.

After creating an instance of `TrackerMIL` , we can treat it as a black box; it does not expose any further parameters that are specific to MIL. To use `TrackerMIL` , we simply pass an initial frame and tracking window to its `init` method, and subsequently we pass each new frame to its `update` method, which returns an updated tracking window. The `init` and `update` methods are part of the `cv2.Tracker` interface, which `TrackerMIL` implements.

Let's put MIL into action by modifying our pedestrian tracker. First, make a copy of the script (the one containing the `Pedestrian` class and `main` method) so that later, you can do a before-and-after comparison of the tracking results. Then, modify the constructor of our `Pedestrian` class to remove everything relating to histogram back-projection, CamShift, and the Kalman filter; instead, just create and initialize a `TrackerMIL` . The changes are **highlighted** in the following block of code:

Note that we have modified the constructor's signature by removing the `hsv_frame` argument. HSV data was relevant to our previous example using histogram back-projection and CamShift; however, `TrackerMIL` expects the input to be either BGR or grayscale frames.

Similarly, the `Pedestrian` class's `update` function no longer has any reason to take an HSV-converted frame as an argument. The original BGR frame is the only thing we will need, and we will simply pass it on to the underlying `Tracker` 's `update` method, which returns an updated estimate of the pedestrian's bounding rectangle. The relevant changes to our code are **highlighted** in the following block:

Note that OpenCV's design of a `Tracker` class, with an `init` function and an `update` function, fits very well into our existing object-oriented design of a class for a trackable object, the `Pedestrian`. By using these two similarly-designed classes together, we have simplified the implementation of the `Pedestrian` class.

Next, the `main` function has very minimal changes. We no longer need to convert the frame to HSV, and a `Pedestrian`'s constructor and `update` method no longer take an HSV frame as an argument. Note the **highlighted** changes in the following code:

Run the old and new versions of the script and see how the results compare. Afterward, you may wish to modify the `Pedestrian` class again to try some of the other machine learning trackers that are implemented by OpenCV. All you need to do is replace the call to `cv2.TrackerMIL_create` with one of the following alternatives:

- `cv2.TrackerCSRT_create` : This function returns a `Tracker` that implements the Channel and Spatial Reliability Tracker (CSRT) algorithm. For details of the algorithm, see the paper *Discriminative Correlation Filter with Channel and Spatial Reliability* (IJCV, 2018), by A. Lukežič, T. Vojíř, L. Zajc, J. Matas, and M. Kristan; it is available at https://openaccess.thecvf.com/content_cvpr_2017/papers/Lukezic_Discriminative_Correlation_Filter_CVPR_2017_paper.pdf.
- `cv2.TrackerKCF_create` : This function returns a `Tracker` that implements the **Kernelized Correlation Filter (KCF)** algorithm. For details, see the paper *High-Speed Tracking with Kernelized Correlation Filters* (PAMI, 2015), by J. Henriques, R. Caseiro, P. Martins, and J. Batista; it is available at https://www.robots.ox.ac.uk/~joao/publications/henriques_tpami2015.pdf. Moreover, OpenCV's implementation extends the algorithm to support the kind of color features that are described in the paper *Adaptive Color Attributes for Real-Time Visual Tracking* (CVPR, 2014), by M. Danelljan, F. Khan, M. Felsberg and J. van de Weijer. The latter paper and its sample code are available at

<http://www.cvl.isy.liu.se/research/objrec/visualtracking/colvistrack/index.html> .

These other `Tracker*_create` functions also support an optional `parameters` argument, but with different fields than the ones in `TrackerMIL_Params` . Refer to the OpenCV documentation for details of the supported parameters in each case.

Whatever your needs, hopefully, this chapter has provided you with the necessary knowledge to build 2D tracking applications that satisfy your requirements.

Summary

This chapter has dealt with video analysis and, in particular, a selection of useful techniques for tracking objects.

We began by learning about background subtraction with a basic motion detection technique that calculates frame differences. Then, we moved on to more complex and efficient background subtraction algorithms – namely, MOG and KNN – which are implemented in OpenCV's `cv2.BackgroundSubtractor` class.

We then proceeded to explore the MeanShift and CamShift tracking algorithms, using color histograms and back-projections. We also familiarized ourselves with the Kalman filter and its usefulness in smoothing the results of a tracking algorithm. Then, we put all of our knowledge together in a sample surveillance application, which is capable of tracking pedestrians (or other moving objects) in a video.

Finally, we saw that we can easily replace one set of tracking techniques with another, in our object-oriented implementation of a pedestrian tracker. We learned how to use OpenCV's implementation of the MIL tracker and other advanced trackers that are based on machine learning.

By now, our foundation in OpenCV, computer vision, and machine learning is solidifying. We can look forward to several advanced topics in the next three chapters of this book. We will extend our knowledge of tracking into 3D space in *Chapter 9 , Camera Models and Augmented Reality* , and *Chapter 10 , 3D Reconstruction and Navigation* . Then, we will tackle **artificial neural networks** (**ANNs**) and dive deeper into artificial intelligence in *Chapter 11 , Introduction to Neural Networks with OpenCV* .

Join our book community on Discord

<https://discord.gg/djGjeMECxx>



If you like geometry, photography, or 3D graphics, then this chapter's topics should especially appeal to you. We will learn about the relationship between 3D space and a 2D projection. We will model this relationship in terms of the basic optical parameters of a camera and lens. Finally, we will apply the same relationship to the task of drawing 3D shapes in an accurate perspective projection. Throughout all of this, we will integrate our previous knowledge of image matching and object tracking in order to track 3D motion of a real-world object whose 2D projection is captured by a camera in real time.

On a practical level, we will build an augmented reality application that uses information about a camera, an object, and motion in order to superimpose 3D graphics on top of a tracked object in real time. To achieve this, we will conquer the following technical challenges:

- Modeling the parameters of a camera and lens
- Modeling a 3D object using 2D and 3D keypoints
- Detecting the object by matching keypoints
- Finding the object's 3D pose using the `cv2.solvePnP` function

- Smoothing the 3D pose using a Kalman filter, while avoiding numeric problems (singularities) that are associated with some ways of representing rotations
- Drawing graphics atop the object

Over the course of this chapter, you will acquire skills that will serve you well if you go on to build your own augmented reality engine or any other system that relies on 3D tracking, such as a robotic navigation system.

Technical requirements

This chapter uses Python, OpenCV, and NumPy. Please refer back to *Chapter 1 , Setting Up OpenCV* , for installation instructions.

The completed code and sample videos for this chapter can be found in this book's GitHub repository, <https://github.com/PacktPublishing/Learning-OpenCV-5-Computer-Vision-with-Python-Fourth-Edition> , in the `chapter09` folder.

This chapter's code contains excerpts from an open source demo project called *Visualizing the Invisible* , by Joseph Howse (one of this book's authors). To learn more about this project, please visit its repository at <https://github.com/JoeHowse/VisualizingTheInvisible/> .

Understanding 3D image tracking and augmented reality

We have already solved problems involving image matching in *Chapter 6* , *Retrieving Images and Searching Using Image Descriptors* . Moreover, we have solved problems involving continuous tracking in *Chapter 8* , *Tracking Objects* . Therefore, we are familiar with many of the components of an image tracking system, though we have not yet tackled any 3D tracking problems.

So, what exactly is **3D tracking** ? Well, it is the process of continually updating an estimate of an object's **pose** (its position and orientation) in a 3D space. Typically, the pose is expressed in terms of six variables: three variables to represent the object's 3D **translation** (that is, position) and the other three variables to represent its 3D rotation (that is, orientation).

A more technical term for 3D tracking is **6DOF tracking** – that is, tracking with **6 degrees of freedom** , meaning the 6 variables we just mentioned. With any fewer than 6 variables, it would be impossible to adequately express the pose of a freely moving, freely turning object such as an airplane. Indeed, the practical problems of aeronautics and spaceflight have driven the adoption of 6DOF and higher-dimensional pose representations since the early 20th century.

There are several different ways of representing the 3D rotation as three variables. Elsewhere, you might have encountered various kinds of Euler angle representations, which describe the 3D rotation in terms of three separate 2D rotations around the x , y , and z axes in a particular order. OpenCV does not use Euler angles to represent 3D rotation; instead, it uses a representation called the **Rodrigues rotation vector** . Specifically, OpenCV uses the following six variables to represent the 6DOF pose:

1. t_x : This is the object's translation along the X axis.
2. t_y : This is the object's translation along the Y axis.
3. t_z : This is the object's translation along the Z axis.
4. r_x : This is the first element of the object's Rodrigues rotation vector.
5. r_y : This is the second element of the object's Rodrigues rotation vector.

6. r_z : This is the third element of the object's Rodrigues rotation vector.

Unfortunately, in the Rodrigues representation, there is no easy way to interpret rx , ry , and rz separately from each other. Taken together, as the vector r , they encode both an axis of rotation and an angle of rotation about this axis.

Specifically, the following formulas define the relationship among the r vector; an angle, θ ; a normalized axis vector, $\hat{r}$; and a 3 x 3 rotation matrix, R :

$$\theta = \|r\|$$
$$\hat{r} = \frac{r}{\theta}$$
$$R = (\cos \theta)I + (1 - \cos \theta)\hat{r}\hat{r}^T + (\sin \theta) \begin{bmatrix} 0 & -\hat{r}_z & \hat{r}_y \\ \hat{r}_z & 0 & -\hat{r}_x \\ -\hat{r}_y & \hat{r}_x & 0 \end{bmatrix}$$

For basic usage of OpenCV, we are not obliged to compute or interpret any of these variables directly. OpenCV provides functions that give us a Rodrigues rotation vector as a return value, and we can pass this rotation vector to other OpenCV functions as an argument – without ever needing to manipulate its contents for ourselves.

Unfortunately, if we want to perform custom filtering of the rotation, the Rodrigues vector is generally not an appropriate format because the vector's individual elements lack a meaningful scale. As shown in the formulas above, we need to look at all three elements together in order to understand the rotation's magnitude (the angle) and direction (the axis). On the other hand, if we look at a change in rx (or any element) alone, we cannot say whether this change might have resulted from a change in the rotation's magnitude or in its direction. Thus, a Kalman filter cannot make reliable predictions about such elements individually. Instead, we will convert the rotation to the more common Euler angle format, perform Kalman filtering on it, and then convert it back to the Rodrigues representation. We will cover the details of these conversions later, in the section *Converting between Rodrigues angles and Euler angles* . For now, let's just note that the **Euler rotation vector** represents a 3D rotation by the following three elements:

1. **pitch** or **attitude** : This is the object's rotation around its local X axis.
2. **yaw** or **heading** : This is the object's rotation around its local Y axis.
3. **roll** or **bank** : This is the object's rotation around its local Z axis

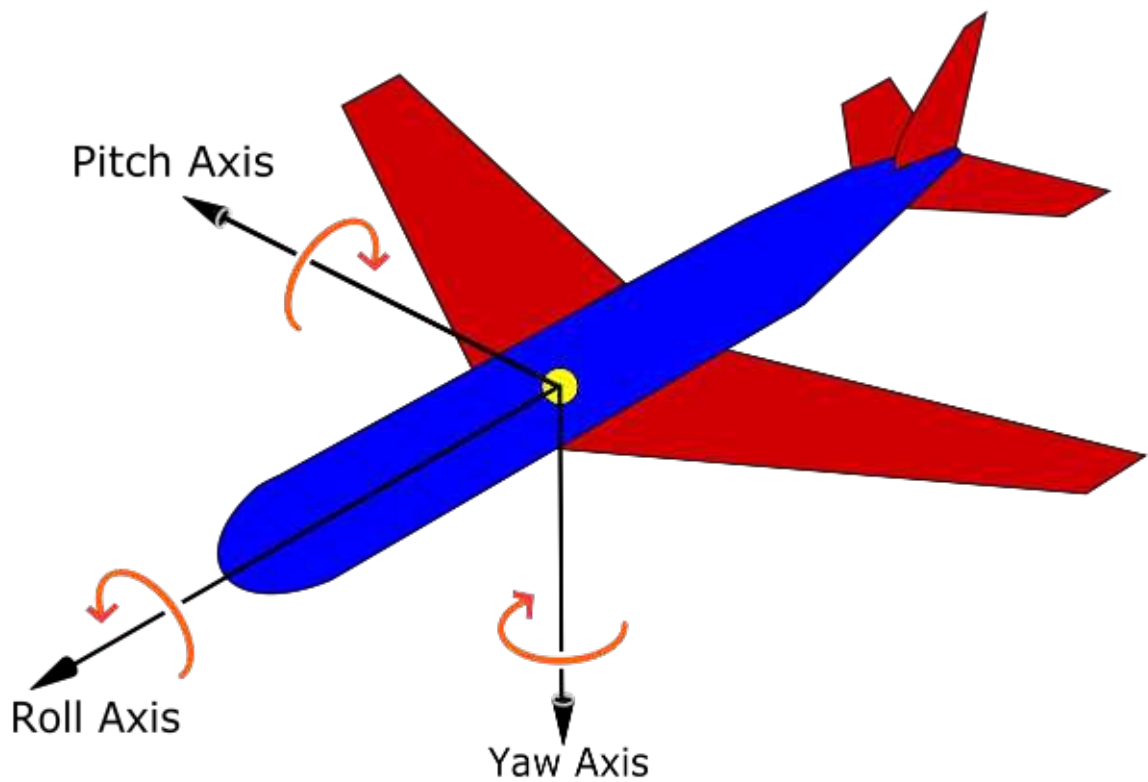


Figure 9.1: Pitch, yaw, and roll axes of an airplane, according to the DIN 9300 aerospace standard. (Image credit: Wikimedia users Auawise and Jrvz. Licensed under CC-BY-SA-3.0.)

We will follow the **yaw-pitch-roll (YPR) Tait-Bryan convention**, whereby yaw is applied first, then pitch, and finally roll. The order matters because each element of the Euler rotation is relative to an object-local axis, which has already been transformed by any previously applied element of the Euler rotation. For example, in the YPR convention, the result of the yaw rotation determines the local axis of the pitch rotation, and the cumulative result of the yaw and pitch rotations determines the local axis of the roll rotation.

The YPR convention refers to the order in which the rotations are applied, not the order in which they are listed in the vector. Despite the YPR convention, the order in the Euler vector is [*pitch* , *yaw* , *roll*].

Euler angles in general, and the YPR Tait-Bryan convention in particular, originate in the field of aeronautics. Imagine that we are describing the 3D rotation of an airplane flying within the atmosphere of the planet Earth. Here are

a few sample values that may help us understand the Euler vector format, [*pitch* , *yaw* , *roll*]:

- [0, 0, 0]: The airplane is level and its nose (its object-local +Z axis) is pointing due North.
- [0, 90°, 0]: The airplane is level and its nose is pointing due East.
- [0, 90°, 90°]: The airplane's nose is pointing due East. Also, the airplane is banked 90° to its right; in other words, the right wing (its object-local +X axis) is pointing toward the Earth's surface.
- [45°, 90°, 0]: The airplane is heading East and climbing at a 45° angle.
- [90°, 0, 0]: The airplane is climbing vertically and its right wing is pointing due East.
- [90°, 90°, 0]: The airplane is climbing vertically and its right wing is pointing due South.
- [90°, 0, 270°]: Again, the airplane is climbing vertically and its right wing is pointing due South.
- [90°, 180°, 90°]: Yet again, the airplane is climbing vertically and its right wing is pointing due South!

As the last three examples show, there are cases where the same real-world rotation has multiple – indeed, infinite – possible representations in Euler angle format. These cases are called **singularities** and they occur where the second step of the rotation – pitch in the case of the YPR convention – is an angle of 90° or 270°. Here, with YPR, a pitch of 90° or 270° transforms the airplane's local Z axis to be a vertical axis, so the roll will be a rotation around a vertical axis – but wait! The yaw's axis of rotation was vertical too! We are now at a singularity, where the distinction between two of our parameters has collapsed; a change in yaw would produce the same effect as a change in roll. Engineers use the term **gimbal lock** to describe a variety of practical problems that can arise from poor handling of such singularities.

When we apply the Kalman filter to Euler angles, we will take care to steer the results away from divergent representations near singularities because such numeric instabilities would spoil our efforts to apply a smoothing filter!

Some of the limitations of the Rodrigues vector and the Euler vector can be resolved by more complex, alternative formats that have at least four dimensions instead of three. The most famous of these is the **quaternion** , which uses imaginary numbers to represent a 3D rotation as a point on a four-dimensional hypersphere. For a discussion of how a Kalman filter can

apply to quaternions and to other alternative formats, see the article *Kalman Filtering for Attitude Estimation with Quaternions and Concepts from Manifold Theory*, by Pablo Bernal-Polo and Humberto Martínez-Barberá; it is available at <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6339217/>.

Now, let's say a few words about the relative nature of coordinates. For our purposes (and, indeed, for many problems in computer vision), the camera is the origin of the 3D coordinate system. Therefore, in any given frame, the camera's current t_x, t_y, t_z, r_x, r_y , and r_z values (using the Rodrigues vector representation of rotation) are defined to be $[0, 0, 0, 0, 0, 1]$. This means zero translation, and zero rotation around the +Z axis (though any axis of rotation would produce the same result in this case, since the angle of rotation is zero). We will endeavor to track other objects relative to the camera's current pose.

Of course, for our edification, we will want to visualize the 3D tracking results. This brings us into the territory of **augmented reality** (**AR**). Broadly speaking, AR is the process of continually tracking relationships between real-world objects and applying these relationships to virtual objects, in such a way that a user perceives the virtual objects as being anchored to something in the real world. Typically, visual AR is based on relationships in terms of 3D space and perspective projection. Indeed, our case is typical; we want to visualize a 3D tracking result by drawing a projection of some 3D graphics atop the object we tracked in the frame.

We will return to the concept of perspective projection in a few moments. Meanwhile, let's take an overview of a typical set of steps involved in 3D image tracking and visual AR:

1. Define the parameters of the camera and lens. We will introduce this topic in this chapter.
2. Initialize a Kalman filter that we will use to stabilize the 6DOF tracking results. For more information about Kalman filtering, refer back to *Chapter 8, Tracking Objects*.
3. Choose a reference image, representing the surface of the object we want to track. For our demo, the object will be a plane, such as a piece of paper on which the image is printed.
4. Create a list of 3D points, representing the vertices of the object. The coordinates can be in any unit, such as meters, millimeters, or something arbitrary. For example, you could arbitrarily define 1 unit to be equal to the object's height.

5. Extract feature descriptors from the reference image. For 3D tracking applications, ORB is a popular choice of descriptor since it can be computed in real time, even on modest hardware such as a basic smartphone. Our demo will use ORB. For more information about ORB, refer back to *Chapter 6 , Retrieving Images and Searching Using Image Descriptors* .
6. Convert the feature descriptors from pixel coordinates to 3D coordinates, using the same mapping that we used in *step 4* .
7. Start capturing frames from the camera. For each frame, perform the following steps:
 1. Extract feature descriptors, and attempt to find good matches between the reference image and the frame. Our demo will use FLANN-based matching with a ratio test. For more information about these approaches for matching descriptors, refer back to *Chapter 6 , Retrieving Images and Searching Using Image Descriptors* .
 2. If an insufficient number of good matches were found, continue to the next frame. Otherwise, proceed with the remaining steps..
 3. Attempt to find a good estimate of the tracked object's 6DOF pose based on the camera and lens parameters, the matches, and the 3D model of the reference object. For this, we will use the `cv2.solvePnPRansac` function.
 4. Apply the Kalman filter to stabilize the 6DOF pose so that it does not jitter too much from frame to frame.
 5. Based on the camera and lens parameters, and the 6DOF tracking results, draw a projection of some 3D graphics atop the tracked object in the frame.

Before proceeding to our demo's code, let's discuss two aspects of this outline a bit further: first, the parameters of the camera and lens; and second, the role of the mysterious function, `cv2.solvePnPRansac` .

Understanding camera and lens parameters

Typically, when we capture an image, at least three objects are involved:

- The **subject** is something we want to capture in the image. Typically, it is an object that reflects light, and we want this object to appear in focus (sharp) in the image.
- The **lens** transmits light and focuses any reflected light from the **focal plane** onto the **image plane** . The focal plane is a circular slice of space that includes the subject (as defined previously). The image plane is a circular

slice of space that includes the image sensor (as defined later). Typically, these planes are perpendicular to the lens's main (lengthwise) axis. The lens has an **optical center**, which is the point where incoming light from the focal plane converges before being projected back toward the image plane. The **focal distance** (that is, the distance between the optical center and the focal plane) varies depending on the distance between the optical center and the image plane. If we move the optical center closer to the image plane, the focal distance increases; conversely, if we move the optical center farther from the image plane, the focal distance decreases. Typically, in a camera system, the focus is adjusted by a mechanism that simply moves the lens back and forth. The **focal length** is defined as the distance between the optical center and the image plane when the focal distance is infinity.

- The **image sensor** is a photosensitive surface that receives light and records it as an image, in either an analog medium (such as film) or a digital medium. Typically, the image sensor is rectangular. Therefore, it does not cover all the extremities of the circular image plane. The image's diagonal **field of view** (**FOV** : the angular extent of the 3D space being imaged) bears a trigonometric relationship to the focal length, the image sensor's width, and the image sensor's height. We shall explore this relationship soon.

Here is a diagram to illustrate the preceding definitions:

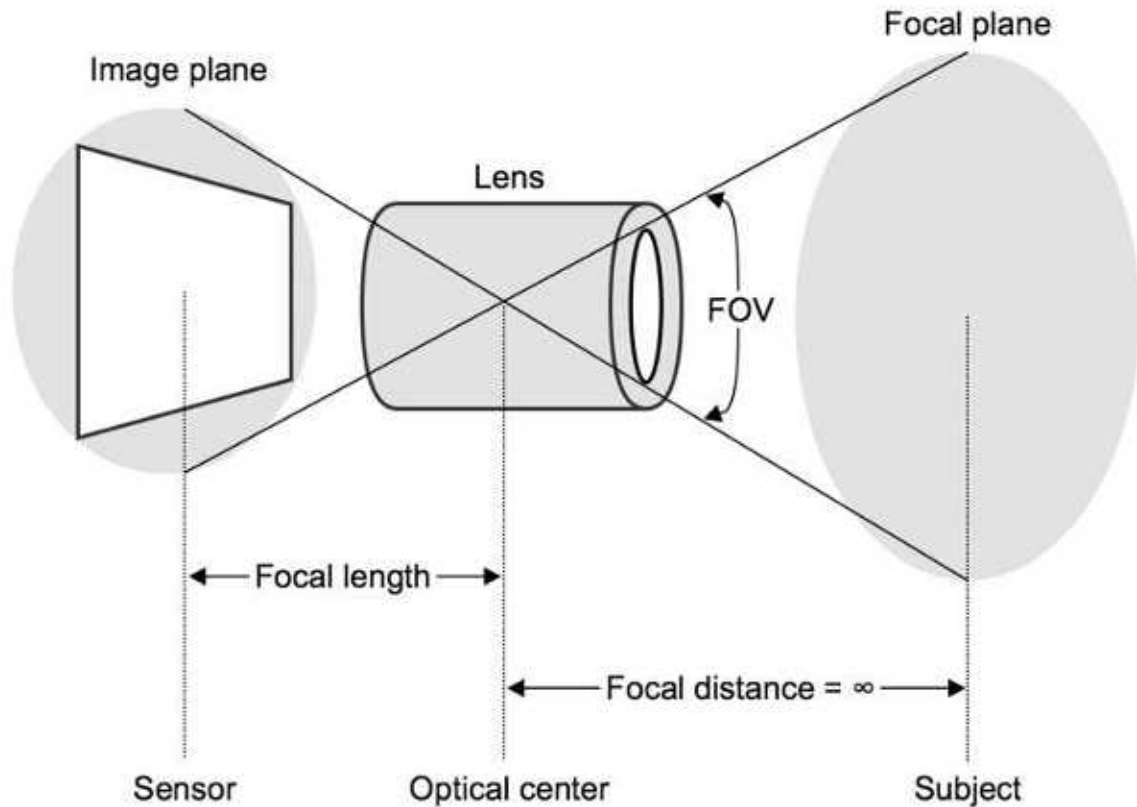


Figure 9.2: Geometric definitions of the basic parameters of a camera and lens

For computer vision, we typically use a lens with a fixed focal length that is optimal for a given application. However, a lens can have a variable focal length; such a lens is called a **zoom lens**. **Zooming in** means increasing the focal length, while **zooming out** means decreasing the focal length. Mechanically, a zoom lens achieves this by moving the optical elements inside the lens.

Let's use the variable f to represent the focal length, and the variables (c_x, c_y) to represent the image sensor's center point within the image plane. OpenCV uses the following matrix, which it calls a **camera matrix**, to represent the basic parameters of a camera and lens:

$$\begin{bmatrix} f & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Assuming that the image sensor is centered in the image plane (as it normally should be), we can calculate c_x and c_y based on the image sensor's width, w , and height, h , as follows:

$$c_x = \frac{w}{2}$$

$$c_y = \frac{h}{2}$$

If we know the diagonal FOV, θ , we can calculate the focal length using the following trigonometric formula:

$$f = \frac{\sqrt{w^2 + h^2}}{2 \left(\tan \frac{\theta}{2} \right)}$$

Alternatively, if we do not know the diagonal FOV, but we know the horizontal FOV, ϕ , and the vertical FOV, ψ , we can calculate the focal length as follows:

$$f = \frac{\sqrt{w^2 + h^2}}{2 \sqrt{\left(\tan \frac{\phi}{2} \right)^2 + \left(\tan \frac{\psi}{2} \right)^2}}$$

You might be wondering how we obtain values for any of these variables as starting points. Sometimes, the manufacturer of a camera or lens provides data on the sensor size, focal length, or FOV in the product's specification sheet. For example, the specification sheet might list the sensor size and focal length in millimeters and the FOV in degrees. However, if the specification sheet is not so informative, we have other ways of obtaining the necessary data. Importantly, the sensor size and focal length do not need to be expressed in real-world units such as millimeters. We can express them in arbitrary units, such as **pixel-equivalent units**.

You may well ask, what is a pixel-equivalent unit? Well, when we capture a frame from a camera, each pixel in the image corresponds to some region of the image sensor, and this region has a real-world width (and a real-world height, which is normally the same as the width, as pixels normally represent square regions).

Therefore, if we are capturing frames with a resolution of 1280 x 720, we can say that the image sensor's width, w , is 1280 pixel-equivalent units and its height, h , is 720 pixel-equivalent units. These units are not comparable across different real-world sensor sizes or different resolutions; however, for a given camera and resolution, they allow us to make internally consistent measurements without needing to know the real-world scale of these measurements.

This trick gets us as far as being able to define w and h for any image sensor (since we can always check the pixel dimensions of a captured frame). Now, to be able to calculate the focal length, we just need one more type of data: the FOV. We can measure this using a simple experiment. Take a piece of paper and tape it to a wall (or another vertical surface). Position the camera and lens so that they are directly facing the piece of paper and the edges of the paper lie along the edges of the frame. (If the paper's aspect ratio does not match the frame's aspect ratio, cut the paper to match.) Measure the diagonal size, s , from one corner of the paper to the diagonally opposite corner. Additionally, measure the distance, d , from the paper to a point halfway down the barrel of the lens. Then, calculate the diagonal FOV, θ , by trigonometry:

$$\theta = 2 \left(\tan^{-1} \frac{s}{2d} \right)$$

Let's suppose that by this experiment, we determine that a given camera and lens have a diagonal FOV of 70 degrees. If we know that we are capturing frames at a resolution of 1280 x 720, then we can calculate the focal length in pixel-equivalent units as follows:

$$f = \frac{\sqrt{w^2 + h^2}}{2 \left(\tan \frac{\theta}{2} \right)} = \frac{\sqrt{1280^2 + 720^2}}{2 \left(\tan \frac{70 * \frac{\pi}{180}}{2} \right)} = 68870.9$$

In addition to this, we can calculate the image sensor's center coordinates:

$$c_x = \frac{w}{2} = \frac{1280}{2} = 640$$

$$c_y = \frac{h}{2} = \frac{720}{2} = 360$$

Therefore, we have the following camera matrix:

$$\begin{bmatrix} 68870.9 & 0 & 640 \\ 0 & 68870.9 & 360 \\ 0 & 0 & 1 \end{bmatrix}$$

The preceding parameters are necessary for 3D tracking, and they correctly represent an ideal camera and lens. However, real equipment may deviate noticeably from this ideal, and the camera matrix alone cannot represent all the possible types of deviations. **Distortion coefficients** are a set of additional parameters that can represent the following kinds of deviations from the ideal model:

- **Radial distortion** : This means that the lens does not magnify all parts of the image equally; thus, it makes straight edges appear curvy or wavy. For radial distortion coefficients, variable names such as k_n (for example, k_1, k_2, k_3 , and so forth) are typically used. If $k_1 < 0$, this usually implies that the lens suffers from **barrel distortion**, meaning that straight edges appear to bend outward toward the borders of the image. Conversely, $k_1 > 0$ usually implies that the lens suffers from **pincushion distortion**, meaning that straight edges appear to bend inward toward the center of the image. If the sign (positive or negative) alternates across the series (for example, $k_1 > 0, k_2 < 0$, and $k_3 > 0$), this might imply that the lens suffers from **mustache distortion**, meaning that straight edges appear wavy.
- **Tangential distortion** : This means that the lens's main (lengthwise) axis is not perpendicular to the image sensor; thus, the perspective is skewed, and the angles between the straight edges appear to be different than in a normal perspective projection. In other words, the lens or the sensor is sitting askew in the camera. This is normally considered a defect; however, specialized equipment may be designed so that a photographer can twist it out of shape to produce tangential distortions or, in other words, a false perspective. (Two such types of equipment are **bellows cameras** and **tilt-shift lenses**.) For tangential distortion coefficients, variable names such as p_n (for example, p_1, p_2 , and so forth) are typically used. The sign of the coefficient depends on the direction of the lens's tilt relative to the image sensor.

The following diagram illustrates some types of radial distortion:

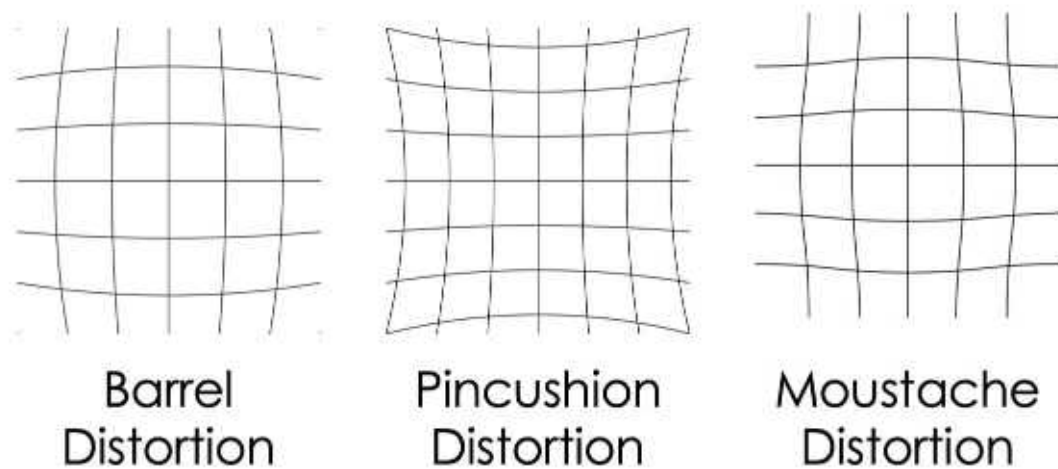


Figure 9.3: Types of radial distortion

OpenCV provides functions to work with as many as five distortion coefficients: k_1 , k_2 , p_1 , p_2 , and k_3 . (OpenCV expects them in this order, as elements of an array.) Rarely, you might be able to obtain official data about distortion coefficients from the vendor of a camera or lens. Alternatively, you can estimate the distortion coefficients, along with the camera matrix, using OpenCV's chessboard calibration process. This involves capturing a series of images of a printed chessboard pattern, viewed from various positions and angles. For further details, you can refer to the OpenCV official tutorial at https://docs.opencv.org/master/dc/dbb/tutorial_py_calibration.html.

For our demo's purposes, we will simply assume that all the distortion coefficients are 0, meaning there is no distortion. Of course, we do not really believe that our webcam lenses are distortion-less masterpieces of optical engineering; we just believe that the distortion is not bad enough to noticeably affect our demo of 3D tracking and AR. If we were trying to build a precise measurement device instead of a visual demo, we would be more concerned about the effects of distortion.

Compared to the chessboard calibration process, the formulas and assumptions that we have outlined in this section produce a more constrained or idealistic model. However, our approach has the advantages of being simpler and more easily reproducible. The chessboard calibration process is more laborious, and each user might execute it differently, producing different (and, sometimes, erroneous) results.

Having absorbed this background information on camera and lens parameters, let's now examine an OpenCV function that uses these parameters as part of a solution to the 6DOF tracking problem.

Understanding `cv2.solvePnPRansac`

Suppose that we know where certain keypoints are located on a 3D model of an object, and we know where the same keypoints are located in a 2D photo of such an object. The `cv2.solvePnPRansac` function implements a solver for the so-called **Perspective-n-Point (PnP)** problem. Given a set of n unique matches between 3D and 2D points, along with the parameters of the camera and lens that generated this 2D projection of the 3D points, the solver attempts to estimate the 6DOF pose of the 3D object relative to the camera. This problem is somewhat similar to finding the homography for a set of 2D-to-2D keypoint matches, as we did in *Chapter 6 , Retrieving Images and Searching Using Image Descriptors* . However, in the PnP problem, we have enough additional information to estimate a more specific spatial relationship – the DOF pose – as opposed to the homography, which just tells us a projective relationship.

So, how does `cv2.solvePnPRansac` work? As the function's name suggests, it implements a Ransac algorithm, which is a general-purpose iterative approach designed to deal with a set of inputs that may contain outliers – in our case, bad matches. Each Ransac iteration finds a potential solution that minimizes a measurement of mean error for the inputs. Then, before the next iteration, any inputs with an unacceptably large error are marked as outliers and discarded. This process continues until the solution converges, meaning that no new outliers are found and the mean error is acceptably low.

For the PnP problem, the error is measured in terms of **reprojection error** , meaning the distance between the observed position of a 2D point and the predicted position according to the camera and lens parameters, and the 6DOF pose that we are currently considering as the potential solution. At the end of the process, we hope to obtain a 6DOF pose that is consistent with most of the 3D-to-2D keypoint matches. Additionally, we want to know which of the matches are inliers for this solution.

Let's consider the function signature of `cv2.solvePnPRansac` :

As we can see, the function has four return values:

- `retval` : This is `True` if the solver converged on a solution; otherwise, it is `False` .
- `rvec` : This array contains r_x , r_y , and r_z – the three rotational degrees of freedom in the 6DOF pose.
- `tvec` : This array contains t_x , t_y , and t_z – the three translational (positional) degrees of freedom in the 6DOF pose.
- `inliers` : If the solver converged on a solution, this vector contains the indices of the input points (in `objectPoints` and `imagePoints`) that are congruous with the solution.

The function also has 12 arguments:

- `objectPoints` : This is an array of 3D points that represent the object's keypoints when there is no translation and no rotation – in other words, when the 6DOF pose variables are all 0.
- `imagePoints` : This is an array of 2D points that represent the object's keypoint matches in the image. Specifically, `imagePoints[i]` is believed to be a match for `objectPoints[i]` .
- `cameraMatrix` : This 2D array is the camera matrix, which we can derive in the manner described in the previous section, *Understanding camera and lens parameters* .
- `distCoeffs` : This is the array of distortion coefficients. If we do not know them, we can assume (for simplicity) that they are all 0, as mentioned in the previous section.
- `rvec` : If the solver converges on a solution, it will put the solution's r_x , r_y , and r_z values in this array (or, if this argument is `None` , the solver will return a new array instead).
- `tvec` : If the solver converges on a solution, it will put the solution's t_x , t_y , and t_z values in this array (or, if this argument is `None` , the solver will return a new array instead).
- `useExtrinsicGuess` : If this is `True` , the solver treats the values in the `rvec` and `tvec` arguments as an initial guess, and then it tries to find a solution that is close to these. Otherwise, the solver takes an unbiased approach in its search for a solution.
- `iterationsCount` : This is the maximum number of iterations that the solver should attempt. If it does not converge on a solution after this number of iterations, it gives up.
- `reprojectionError` : This is the maximum reprojection error that the solver will accept; if a point has a greater reprojection error than this, the solver

treats it as an outlier.

- `confidence` : The solver attempts to converge on a solution that has a confidence score greater than or equal to this value.
- `inliers` : If the solver converges on a solution, it will put the indices of the solution's inlier points in this array (or, if this argument is `None` , the solver will return a new array instead).
- `flags` : The flags specify the solver's algorithm. The default, `cv2.SOLVEPNP_ITERATIVE` , is an approach that minimizes reprojection error and has no special restrictions, so it is generally the best choice. Specifically, `cv2.SOLVEPNP_ITERATIVE` implements an iterative algorithm called **Levenberg–Marquardt (LM)** or **damped least squares (DLS)** . A useful alternative is `cv2.SOLVEPNP_IPPE` – **IPPE** , short for **Infinitesimal Plane-Based Pose Estimation** – but it is restricted to planar objects such as a book cover. Compared to LM, IPPE is a bit faster but a bit less accurate. For a comparison of the two algorithms' results, see the paper *Insights into the robustness of control point configurations for homography and planar pose estimation* , by Raul Acuna and Volker Willert. An electronic version is available at <https://arxiv.org/pdf/1803.03025.pdf>.

Although this function involves a lot of variables, we will see that its usage is a natural extension of the keypoint matching problems we have covered in *Chapter 6 , Retrieving Images and Searching Using Image Descriptors* , and the 3D and projective problems we are introducing in this chapter. With this in mind, let's begin to explore this chapter's sample code.

Implementing the demo application

We are going to implement our demo in a single script, `ImageTrackingDemo.py` , which will contain the following components:

1. Import statements
2. A helper function for a custom grayscale conversion
3. Helper functions to convert keypoints from 2D to 3D space
4. An application class, `ImageTrackingDemo` , which will encapsulate a model of the camera and lens, a model of the reference image, a Kalman filter, 6DOF tracking results (including the translation and both the Rodrigues and Euler representations of the rotation), and an application loop that will track the image and draw a simple AR visualization
5. A `main` function to launch the application

The script will depend on one other file, `reference_image.png` , which will represent the image that we want to track.

By preparing a reference image in advance, and by loading it from file at runtime, we can ensure that its technical qualities are good: it has a high resolution (important for close-up tracking), it is properly focused, it is free of distortions, it is evenly lit and well exposed, the subject is upright and flat and not covered by anyone's thumb...

On the other hand, in some applications, the user might need to gather reference images in real time instead. Then, for example, you could implement a user interface to let the user show any reference image to the camera. For such applications, you may want to read about OpenCV's `selectROI` function, which lets the user click and drag the mouse to select a rectangular region of an image (such as the current camera frame). The selected region could be treated as a new reference image.

Without further ado, let's dive into the script's implementation.

Importing modules

From the Python standard library, we will use the `math` module for trigonometric calculations, and the `timeit` module for accurate time measurement (which will enable us to use a Kalman filter more effectively). As usual, we will also use NumPy and OpenCV. Thus, our implementation of `ImageTrackingDemo.py` begins with the following `import` statements:

Now, let's proceed to the implementation of the helper functions.

Performing grayscale conversion

Throughout this book, we have performed grayscale conversions using code such as the following:

Perhaps a question is long overdue: how exactly does this function map BGR values to grayscale values? The answer is that each output pixel's grayscale value is a weighted average of the corresponding input pixel's B, G, and R values, as follows:

These weights are widely used. They come from a telecommunications industry standard called CCIR 601, which was issued in 1982. They are loosely consistent with a characteristic of human vision; when we see a brightly lit scene, our eyes are most sensitive to yellowish-green light. Moreover, these weights should produce high contrast in scenes with yellowish light and blueish shadows, such as an outdoor scene on a sunny day. Are these good reasons for us to use the CCIR 601 weights? No, they are not; there is no scientific evidence that the CCIR 601 conversion weights yield optimal grayscale input for any particular purpose in computer vision.

Indeed, for the purpose of image tracking, there is evidence in favor of other grayscale conversion algorithms. Samuel Macêdo, Givânio Melo, and Judith Kelner address this topic in their paper, *A comparative study of grayscale conversion techniques applied to SIFT descriptors* (SBC Journal on Interactive Systems, vol. 6, no. 2, 2015). They test a variety of conversion algorithms, including the following types:

- A weighted-average conversion, $\text{gray} = (0.07 * \text{blue}) + (0.71 * \text{green}) + (0.21 * \text{red})$, which is somewhat similar to CCIR 601
- An unweighted-average conversion, $\text{gray} = (\text{blue} + \text{green} + \text{red}) / 3$
- Conversions based on only a single color channel, such as $\text{gray} = \text{green}$

- Gamma-corrected conversions, such as $\text{gray} = 255 * (\text{green} / 255)^{(1/2.2)}$, in which the grayscale value varies exponentially (not linearly) with the inputs

According to the paper, the weighted-average conversion produces results that are relatively unstable – good for finding matches and homography with some images, but bad with others. The unweighted-average conversion and the single-channel conversions yield more consistent results. For some images, the gamma-corrected conversions yield the best results, but these conversions are computationally more expensive.

For our demo's purposes, we will perform grayscale conversion by taking the simple (unweighted) average of each pixel's B, G, and R values. This approach is computationally cheap (which is desirable in real-time tracking), and we expect that it leads to more consistent tracking results than the default weighted-average conversion in OpenCV. Here is our implementation of a helper function to perform the custom conversion:

Note the use of the `cv2.transform` function. This is a well-optimized, general-purpose matrix transformation function, provided by OpenCV. We can use it to perform operations where the values of a pixel's output channels are a linear combination of the values of the input channels. In the case of our BGR-to-grayscale conversion, we have one output channel and three input channels, so our transformation matrix, `m`, has one row and three columns.

Having written our helper function for grayscale conversions, let's go on to consider helper functions for conversions from 2D to 3D space.

Performing 2D-to-3D spatial conversions

Remember that we have a reference image, `reference_image.png`, and we want our AR application to track a print copy of this image. For the purpose of 3D tracking, we can represent this printed image as a plane in 3D space. Let's define the local coordinate system by saying that, normally (when the elements of the 6DOF pose are all 0), this planar object stands upright like a picture hanging on a wall; its front is the side with the image on it;; and its origin is the center of the image.

Now, let's suppose that we want to map a given pixel from the reference image onto this 3D plane. Given the 2D pixel coordinates, the image's pixel dimensions,

and a scaling factor to convert from pixels to some unit of measurement we want to use in 3D space, we can use the following helper function to map a pixel onto the plane:

The scaling factor depends on the real-world size of the printed image and our choice of unit. For example, we might know that our printed image is 20 cm tall – or we might not care about the absolute scale, in which case we could define an arbitrary unit such that the printed image is one unit tall. Anyway, given a list of 2D pixel coordinates, the reference image's size in pixels, and the reference image's real-world height in any unit (absolute or relative), we can use the following helper function to obtain a list of the corresponding 3D coordinates on the plane:

Note that we have a helper function for multiple points, `map_points_to_plane` , and it calls a helper function for every single point, `map_point_to_plane` .

Later, in the *Initializing the tracker* section, we will generate ORB keypoint descriptors for the reference image, and we will use our `map_points_to_plane` helper function in order to convert the keypoint coordinates from 2D to 3D. We will also convert the image's four 2D vertices (that is, its top-left, top-right, bottom-right, and bottom-left corners) to obtain the four 3D vertices of the plane. We will use these vertices when we perform our AR drawing – specifically, in the *Drawing the tracking results* section. Drawing-related functions (in OpenCV and many other frameworks) expect the vertices to be specified in clockwise order (from a frontal perspective) for each face of the 3D shape. To deal with this requirement, let's implement another helper function that is specific to mapping vertices; here it is:

Note that our vertex-mapping helper function, `map_vertices_to_plane` , calls our `map_points_to_plane` helper function, which, in turn, calls `map_point_to_plane` . Therefore, all our mapping functionality shares a common core.

Of course, 2D-to-3D keypoint mapping and vertex mapping can be applied to other kinds of 3D shapes besides planes. To learn how our approach can extend to 3D cuboids and 3D cylinders, please refer to the *Visualizing the Invisible* demo project, by Joseph Howse, which is available at <https://github.com/JoeHowse/VisualizingTheInvisible/>.

We have finished implementing the helper functions. Now, let's proceed to the object-oriented part of the code.

Implementing the application class

We will implement our application in a class named `ImageTrackingDemo` , which will have the following methods:

- `__init__(self, capture, diagonal_fov_degrees, target_fps, reference_image_path, reference_image_real_height)` : The initializer will set up a capture device, a camera matrix, a Kalman filter, and 2D and 3D keypoints for the reference image.
- `run(self)` : This method will run the application's main loop, which captures, processes, and displays frames until the user quits by hitting the *Esc* key. The processing of each frame is performed with the help of other methods, which are mentioned next in this list.
- `_track_object(self)` : This method will perform 6DOF tracking and draw an AR visualization of the tracking result.
- `_init_kalman_transition_matrix(self, fps)` : This method will configure the Kalman filter to ensure that acceleration and velocity are simulated properly for the specified frame rate.
- `_apply_kalman(self)` : This method will stabilize the 6DOF tracking result by applying the Kalman filter.

Let's walk through the methods' implementations one by one, starting with `__init__` .

Initializing the tracker

The `__init__` method involves a lot of steps to initialize the camera matrix, the ORB descriptor extractor, the Kalman filter, the reference image's 2D and 3D keypoints, and other variables related to our tracking algorithm:

1. To begin, let's look at the several arguments that `__init__` accepts. These include: a `cv2.VideoCapture` object, called `capture` (the camera); the camera's diagonal FOV, in degrees; the expected frame rate in **frames per second (FPS)** ; a path to a file containing the reference image; and a measurement of the reference image's real-world height (in any unit):
1. We attempt to capture a frame from the camera in order to determine its pixel dimensions; failing that, we get the dimensions from the camera's properties:

1. Now, given the frame's dimensions in pixels, and the FOV of the camera and lens, we can use trigonometry to calculate the focal length in pixel-equivalent units. (The formula is the one we derived earlier in this chapter, in the *Understanding camera and lens parameters* section.) Moreover, using the focal length and the frame's center point, we can construct the camera matrix. Here is the relevant code:

1. For the sake of simplicity, we assume that the lens does not suffer from any distortion whatsoever:

1. Initially, we are not tracking the object, so we have no estimate of its rotation and position; we just define the relevant variables as if the rotation and position are zero:

1. Now, let's set up a Kalman filter:

As indicated by the preceding code, `cv2.KalmanFilter(18, 6)`, this Kalman filter will track 18 output variables (or predictions), based on 6 input variables (or measurements). Specifically, the input variables are the elements of the 6DOF tracking result: t_x, t_y, t_z, r_x, r_y , and r_z . The output variables are the elements of the stabilized 6DOF tracking result, plus their first-order derivatives (velocity) and their second-order derivatives (acceleration), in the following order: $t_x, t_y, t_z, t_x', t_y', t_z', t_x'', t_y'', t_z'', r_x, r_y, r_z, r_x', r_y', r_z', r_x'', r_y'',$ and r_z'' . The Kalman filter's measurement matrix has 18 columns (representing the output variables) and 6 rows (representing the input variables). Within each row, we put 1.0 in the index that corresponds to the matching output variable; elsewhere, we put 0.0. We also initialize a transition matrix, which defines the relationships among the output variables over time. This part of the initialization is handled by a helper method, `__init_kalman_transition_matrix(target_fps)`, which we will examine later, in the *Initializing and applying the Kalman filter* section.

Not all of the Kalman filter's matrices are initialized by our `__init__` method. The transition matrix is updated every frame during tracking because the actual frame rate (and, thus, the time step) may change. The state matrices are initialized every time we start tracking an object. We will cover these aspects of the Kalman filter's usage in due course, in the *Initializing and applying the Kalman filter* section.

1. We need a Boolean variable (initially, `False`) to indicate whether we successfully tracked the object in the previous frame:
1. We need to define the vertices of some 3D graphics that we will draw every frame as part of our AR visualization. Specifically, the graphics will be a set of arrows representing the object's X , Y , and Z axes. The scale of these graphics will relate to the scale of the real object – that is, the printed image that we intend to track. Remember that, as one of its arguments, the `__init__` method takes the image's scale – specifically, its height – and that this measurement may be in any unit. Let's define the length of the 3D axis arrows to be half the height of the printed image:
1. Using the length that we have just defined, let's define the vertices of the axis arrows relative to the printed image's center, `[0.0, 0.0, 0.0]` :

Note that OpenCV's coordinate system has nonstandard axis directions, as follows:

- + X (the positive X direction) is the object's left-hand direction, or the viewer's right-hand direction in a frontal view of the object.
- + Y is down.
- + Z is the object's backward direction, or the viewer's frontward direction in a frontal view of the object.

We must negate all of the preceding directions in order to obtain the following standard right-handed coordinate system, like the one used in many 3D graphics frameworks such as OpenGL:

- + X is the object's right-hand direction, or the viewer's left-hand direction in a frontal view of the object.
- + Y is up.
- + Z is the object's forward direction, or the viewer's backward direction in a frontal view of the object.

For the purposes of this book, we use OpenCV to draw 3D graphics, so we could simply adhere to OpenCV's nonstandard axis directions, even when we draw visualizations. However, if you do further AR work in the future, you will likely need to integrate your computer vision code with OpenGL and other 3D graphics frameworks using a right-handed coordinate system. To better prepare you for this eventuality, we will convert the axis directions in our OpenCV-centric demo.

1. We will use three arrays to hold three kinds of images: the BGR video frame (where we will do our AR drawing), the grayscale version of the frame (which we will use for keypoint matching), and the mask (where we will draw a silhouette of the tracked object). Initially, these arrays are all None :
1. We will use a `cv2.ORB` object to detect keypoints and compute descriptors for the reference image and, later, for camera frames. We initialize the `cv2.ORB` object as follows:

For a refresher on the ORB algorithm and its usage in OpenCV, refer back to *Chapter 6 , Retrieving Images and Searching Using Image Descriptors* , specifically to the *Using ORB with FAST features and BRIEF descriptors* section.

Here, we have specified several optional parameters for the constructor of `cv2.ORB` . The diameter covered by a descriptor is 31 pixels, our image pyramid has 16 levels with a scale factor of 1.2 between consecutive levels, and we want, at most, 250 keypoints and descriptors per detection attempt. You may want to experiment with these values. For example, if you increase `nfeatures` , you may get more accurate results but the frame rate may drop because there are more descriptors to match.

1. Now, we load the reference image from a file, resize it, convert it to grayscale, and create an empty mask for it:

When resizing the reference image, we have chosen to make it twice as high as the camera frame. The exact number is arbitrary; however, the idea is that we want to perform keypoint detection and description with an image pyramid that covers a useful range of magnifications. The base of the pyramid (that is, the resized reference image) should be larger than the camera frame so that we can match keypoints at an appropriate scale even when the target object is so close to the camera that it cannot all fit into the frame. Conversely, the top level of the pyramid should be smaller than the camera frame so that we can match keypoints at an appropriate scale even when the target object is too far away to fill the whole frame.

Let's consider an example. Suppose that our original reference image is 4000 x 3000 pixels and that our camera frame is 1280 x 720 pixels. We resize the reference image to 1920 x 1440 pixels (twice the height of the frame, and the same aspect ratio as the original reference image). Thus, the base of our image pyramid is also 1920 x 1440 pixels. Since our `cv2.ORB` object is configured to

use 16 pyramid levels and a scale factor of 1.2, the top of the image pyramid has a width of $1920/(1.2^{16-1})=124$ pixels and a height of $1440/(1.2^{16-1})=93$ pixels; in other words, it is 124 x 93 pixels. Therefore, we can potentially match keypoints and track the object even if it is so far away that it fills just 10% of the frame's width or height. Realistically, to perform useful keypoint matching at this scale, we would need a good lens, the object would need to be in focus, and the lighting would need to be good as well.

1. At this stage, we have an appropriately sized reference image in BGR color and in grayscale, and we have an empty mask for this image. We are going to partition the image into 36 equally-sized regions of interest (in a 6 x 6 grid), and for each region, we will attempt to generate as many as 250 keypoints and descriptors (since our `cv2.ORB` object is configured to use this maximum number of keypoints and descriptors). This partitioning scheme helps to ensure that we have some keypoints and descriptors in every region, so we can potentially match keypoints and track the object even if most parts of the object are not visible in a given frame. The following code block shows how we iterate over the regions of interest and, for each region, create a mask, perform keypoint detection and descriptor extraction, and append the keypoints and descriptors to master lists:

1. Now, we draw a visualization of the keypoints atop the grayscale reference image:

1. Next, we save the visualization to a file with `_keypoints` appended to the name. For example, if the filename of the reference image was `reference_image.png`, we save the visualization as `reference_image_keypoints.png`. Here is the relevant code:

1. We proceed to initialize the FLANN-based matcher with custom parameters:

These parameters specify that we are using a multi-probe LSH (locality-sensitive hashing) indexing algorithm with 6 hash tables, a hash key size of 12 bits, and 1 multi-probe level.

For a description of the multi-probe LSH algorithm, refer to the paper *Multi-Probe LSH: Efficient Indexing for High-Dimensional Similarity Search* (VLDB, 2007), by Qin Lv, William Josephson, Zhe Wang, Moses Charikar, and Kai Li. An electronic version is available at https://www.cs.princeton.edu/cass/papers/mplsh_vldb07.pdf.

1. We train the matcher by feeding the reference descriptors to it:
1. We take the 2D coordinates of the keypoints, and we feed these to our `map_points_to_plane` helper function in order to obtain equivalent 3D coordinates on the surface of the object's plane:
1. Similarly, we call our `map_vertices_to_plane` function in order to obtain the 3D vertices and 3D face of the plane:

This concludes the implementation of the `__init__` method. Next, let's take a look at the `run` method, which represents the application's main loop.

Implementing the main loop

As usual, our main loop's primary role is to capture and process frames, until the user hits the *Esc* key. The processing of each frame – including 3D tracking and AR drawing – is delegated to a helper method called `_track_object`, which we will examine later, in the *Tracking the image in 3D* section. The main loop also has a secondary role: that is, to perform timekeeping by measuring the frame rate and updating the Kalman filter's transition matrix accordingly. This update is delegated to another helper method, `_init_kalman_transition_matrix`, which we will examine in the *Initializing and applying the Kalman filter* section. With these roles in mind, we can implement the main loop in the `run` method as follows:

Note the use of the `timeit.default_timer` function from Python's standard library. This function provides a precise measurement of the current system time in seconds (as a floating-point number, so fractions of seconds can be expressed). As the name `timeit` suggests, this module contains useful functionality for situations where you have time-sensitive code and you want to *time it*.

Let's move on to the implementation of `_track_object`, since this helper performs the largest part of the application's work on behalf of `run`.

Tracking the image in 3D

The `_track_object` method is directly responsible for keypoint matching, keypoint visualizations, and solving the PnP problem. Additionally, it calls other methods to deal with Kalman filtering, AR drawing, and masking the tracked object:

1. To begin `_track_object` 's implementation, we call our `convert_to_gray` helper function to convert the frame to grayscale:

1. Now, we use our `cv2.ORB` object to detect keypoints and compute descriptors in a masked region of the grayscale image:

If we were already tracking the object in the previous frame, the mask covers the region where we previously found the object. Otherwise, the mask covers the whole frame because we have no idea where the object might be. We will see how the mask is created later, in the *Drawing the tracking results and masking the tracked object* section.

1. Next, we use our FLANN matcher to find matches between the reference image's keypoints and the frame's keypoints, and we filter these matches according to the ratio test:

For details about FLANN matching and the ratio test, refer back to *Chapter 6 , Retrieving Images and Searching Using Image Descriptors* .

1. At this stage, we have a list of good matches that passed the ratio test. Let's select the subset of the frame's keypoints that correspond to these good matches, and let's draw red circles on the frame to visualize these keypoints:

1. Having found the good matches, we obviously know how many of them there are. If the count is small, then, overall, the set of matches can be considered doubtful and inadequate for tracking. We define two different thresholds for the minimum number of good matches: a higher threshold if we are just starting tracking (that is, we were not tracking the object in the previous frame), and a lower threshold if we are continuing tracking (after already tracking the object in the previous frame):

1. If we fail to meet even the lower threshold, then we note that we did not track the object in this frame, and we reset the mask so that it covers the whole frame:

1. On the other hand, if we have enough matches to satisfy the applicable threshold, we proceed to try to track the object. The first step in this is to select the good matches' 2D coordinates in the frame and their 3D coordinates in the model of the reference object:

1. Now, we are ready to call `cv2.solvePnPRansac` using the kinds of arguments we described near the start of this chapter, in the *Understanding cv2.solvePnPRansac* section. Notably, we use the 3D reference keypoints and the 2D scene keypoints from only the good matches:

Note that at this stage, we have put the new rotation and translation results in local variables, `rodrigues_rotation_vector_temp` and `translation_vector_temp`. We are not yet sure whether the new rotation and translation results are good, so we do not yet want to overwrite the old results in `self._rodrigues_rotation_vector` and `self._translation_vector`.

1. The solver may or may not have converged on a solution to the PnP problem. If it did not converge, we do nothing further in this method. If it did converge, we overwrite the old rotation and translation results. As part of this step, we call a helper method, `_convert_rodrigues_to_euler`, to convert the new rotation result to Euler angles:
1. The next thing we do is to check whether we were already tracking the object in the previous frame. If we were not already tracking it – in other words, if we are starting to track the object afresh in this frame – then we reinitialize the Kalman filter by calling a helper method, `_init_kalman_state_matrices`:
1. Now, in any case, we are tracking the object in this frame, so we can apply the Kalman filter by calling another helper method, `_apply_kalman`:
1. At this stage, we have a Kalman-filtered 6DOF pose. We also have a list of the inlier keypoints from `cv2.solvePnPRansac`. To help the user visualize the result, let's draw the inlier keypoints in green:

Remember that earlier in this method, we drew all the keypoints in red. Now that we have drawn over the inlier keypoints in green, only the outlier keypoints are still red.

1. Finally, we call two more helper methods: the `self._draw_object_axes` to draw the tracked object's 3D axes, and `self._make_and_draw_object_mask` to make and draw a mask of the region that contains the object:

There ends the implementation of our `_track_object` method. By now, we have a mostly complete picture of our tracking algorithm's implementation, but we still

need to implement helper methods relating to the Kalman filter (in the next section, *Initializing and applying the Kalman filter*), and masking and AR drawing (in the section after that, *Drawing the tracking results and masking the tracked object*).

Initializing and applying the Kalman filter

We looked at some aspects of the Kalman filter's initialization earlier, in the *Initializing the tracker* section. However, in that section, we noted that some of Kalman filter's matrices need to be initialized or reinitialized multiple times, as the application runs through various frames and various states of tracking or not tracking. Specifically, the following matrices will change:

- **The transition matrix** : This matrix expresses the temporal relationships among all the output variables. For example, this matrix can model the effects of acceleration on velocity, and of velocity on position. We will reinitialize the transition matrix every frame because the frame rate (and, therefore, the time step between frames) is variable. Effectively, this is a way of scaling the previous predictions of acceleration and velocity to match the new time step.
- **The pre-correction and post-correction state matrices** : These matrices contain the predictions of the output variables. The predictions in the pre-correction matrix only take account of the previous state and the transition matrix. The predictions in the post-correction matrix also take account of new inputs and the Kalman filter's other matrices. We will reinitialize the state matrices whenever we go from a non-tracking state to a tracking state – in other words, when we failed to track the object in the previous frame but now we succeed in tracking it in the current frame. Effectively, this is a way of clearing outdated predictions, and starting afresh from new measurements.

Let's take a look at the transition matrix first. Its initialization method will take one argument, `fps` , that is, the frame rate in frames per second. We can implement the method in three steps:

1. We begin by validating the `fps` argument. If it is not positive, we return immediately without updating the transition matrix:
1. Having determined that `fps` is positive, we proceed to calculate transition rates for velocity and acceleration. We want the velocity transition rate to be

proportional to the time step (that is, the time per frame). Because fps (frames per second) is the inverse of the time step (that is, seconds per frame), the velocity transition rate is inversely proportional to fps . The acceleration transition rate is proportional to the square of the velocity transition rate (and thus, indirectly, the acceleration transition rate is inversely proportional to the square of fps). Choosing 1.0 as a base scale for the velocity transition rate and 0.5 as a base scale for the acceleration transition rate, we can calculate them in code as follows:

1. Next, we populate the transition matrix. Since we have 18 output variables, the transition matrix has 18 rows and 18 columns. First, let's take a look at the content of the matrix, and, afterward, we will consider how to interpret it:

Each row expresses a formula for calculating a new output value based on the previous frame's output values. Let's consider the first row as an example. We can interpret it as follows:

$$t_x \leftarrow 1(t_x) - 0(t_y) + 0(t_z) + v(t'_x) + 0(t'_y) + 0(t'_z) + a(t''_x) + 0(t''_y) + 0(t''_z) = t_x + v(t'_x) + a(t''_x)$$

The new t_x (x position) value depends on the old t_x , t'_x (x velocity), and t''_x (x acceleration) values, along with the velocity transition rate, v , and the acceleration transition rate, a . As we saw earlier in this function, these transition rates may vary because the time step may vary.

That concludes the implementation of the helper method to initialize or update the transition matrix. Remember that we call this function every frame because the frame rate (and thus the time step) may have changed.

We also need a helper function to initialize the state matrices. Remember that we call this method whenever we transition from a non-tracking state to a tracking state. This transition is an appropriate time to clear out any previous predictions; instead, we are starting afresh with the belief that the object's 6DOF pose is exactly what the PnP solver says it is. Moreover, we assume that the object is stationary, with zero velocity and zero acceleration. Here is the helper method's implementation:

Note that the state matrices have one row and 18 columns since we have 18 output variables. Also, note that we are performing Kalman filtering on the Euler representation of the rotation, not the Rodrigues representation.

Now that we have covered the process of initializing and reinitializing the Kalman filter's matrices, let's take a look at how we apply the filter. As we have previously seen in *Chapter 8, Tracking Objects*, we can ask the Kalman filter to estimate the object's new pose (the pre-correction state of the output variables), then we can tell it to take account of the latest unstabilized tracking result (the input variables) in order to adjust its estimate (thereby producing the post-correction state), and, finally, we can extract variables from the adjusted estimate to use as our stabilized tracking result. Compared to our previous work, the only difference this time is that we have more input and output variables. The following code shows how we implement a method to apply the Kalman filter in the context of our 6DOF tracker:

Here, note that `estimate[0:3]` corresponds to t_x , t_y , and t_z , while `estimate[9:12]` corresponds to r_x , r_y , and r_z (or, in other words, pitch, yaw, and roll). The rest of the estimate array corresponds to the first-order derivatives (velocity) and second-order derivatives (acceleration).

Also, note that if we detect an abrupt change of direction in the Euler angle vector, we assume that we passed through a singularity and we reset the rotational motion stabilization by treating the latest rotational tracking result as correct. (Otherwise, if we attempted to apply smoothing to Euler values near a singularity, we would see the object spin wildly.)

Having seen the way we apply a Kalman filter to Euler angles, now let's look at implementations of helper methods to convert a Rodrigues rotation vector to an Euler rotation vector and vice versa.

Converting between Rodrigues vectors and Euler vectors

EuclideanSpace, a mathematics and geometry website, provides an excellent pair of articles on the relationship between the 3x3 rotation matrix and Euler angles, specifically including the yaw-pitch-roll Tait-Bryan variant:

- A conceptual overview is available at <https://www.euclideanspace.com/maths/geometry/rotations/euler/index.htm>.
- Conversion formulas, along with sample code in C++, are available at <http://euclideanspace.com/maths/geometry/rotations/conversions/matrixToEuler/index.htm>.

You may want to open the two preceding links now so that you can compare the explanation and formulas from EuclideanSpace to our OpenCV code, below.

We will adapt the sample code from EuclideanSpace and also make use of an OpenCV function, `cv2.Rodrigues`, which converts between the Rodrigues vector and the 3x3 rotation matrix. Here is our Python implementation of a helper method to convert the Rodrigues vector to an Euler vector:

Conversely, the following method implements the conversion from the Euler vector back to the Rodrigues vector:

Note that `cv2.Rodrigues` is two-way function, which can convert from a Rodrigues rotation vector to a rotation matrix, or vice versa. If we pass it a rotation matrix, it returns a Rodrigues vector (and Jacobian matrix); if we pass it a Rodrigues vector, it returns a rotation matrix (and Jacobian matrix).

By this point, we have almost fully explored the implementation of our 3D tracking algorithm, including the use of the Kalman filter to stabilize the 6DOF pose, as well as the velocity and acceleration. Along the way, we have dealt with conversions between different representations of the rotation. Now, let's turn our attention to two final implementation details of our `ImageTrackingDemo` class: the AR drawing methods and the creation of a mask based on the tracking results.

Drawing the tracking results and masking the tracked object

We will implement one helper method, `_draw_object_axes`, to draw a visualization of the tracked object's *X*, *Y*, and *Z* axes. We will also implement another helper method, `_make_and_draw_object_mask`, to project the object's vertices from 3D to 2D, create a mask based on the object's silhouette, and tint this masked region yellow as a visualization.

Let's start with the implementation of `_draw_object_axes`. We can consider it in three stages:

1. First, we want to take a set of 3D points located along the axes, and project these points to the 2D image space. Remember that we defined the 3D axis points in our `__init__` method, in the *Initializing the tracker* section. They will simply serve as endpoints of the axis arrows that we will draw. Using the `cv2.projectPoints` function, our 6DOF tracking result, and our camera matrix, we can find the 2D projected points as follows:

Besides returning the projected 2D points, `cv2.projectPoints` also returns the **Jacobian matrix**, which represents the partial derivatives (with respect to the input parameters) of the function used to calculate the 2D points. This information is potentially useful for camera calibration, but we do not use it in our example.

1. The projected points are in floating-point format, but we will need integers to pass to OpenCV's drawing functions. Thus, we perform the following conversions to integer format:

1. Having calculated the endpoints, we can now draw three arrowed lines to represent the X, Y, and Z axes:

We have finished implementing `_draw_object_axes`. Now, let's turn our attention to `_make_and_draw_object_mask`, which we can also consider in terms of three steps:

1. Like the previous function, this one begins by projecting points from 3D to 2D. This time, we are projecting the reference object's vertices, which we defined in our `__init__` method, in the *Initializing the tracker* section. Here is the projection code:
1. Again, we convert the projected points from a floating-point format to an integer format (as OpenCV's drawing functions expect integers):
1. The projected vertices form a convex polygon. We can paint the mask black (as a background), and then draw this convex polygon in white:

Remember that our `_track_object` method will use this mask when it processes the next frame. Specifically, `_track_object` will only look for keypoints in the masked region. Therefore, it will attempt to find the object in the region where we recently found it.

Potentially, we could improve this technique by applying a morphological dilation operation to expand the masked region. In this way, we would search for the object, not only in the region where we recently found it, but also in the surrounding region.

1. Now, in the BGR frame, let's highlight the masked region in yellow in order to visualize the shape of the tracked object. To make a region more yellow, we can subtract a value from the blue channel. The `cv2.subtract` function

suits our purpose because it accepts an optional mask argument. Here is how we use it:

When we tell `cv2.subtract` to subtract a single scalar value such as 48 from an image, it subtracts the value only from the image's first channel – in this case (and most cases), the blue channel of a BGR image. This is arguably a bug, but it is convenient for tinting things yellow!

That was the last method in the `ImageTrackingDemo` class. Now, let's bring the demo to life by instantiating this class and calling its `run` method!

Running and testing the application

To complete the implementation of `ImageTrackingDemo.py`, let's write a main function that launches the application with a specified capture device, FOV, and target frame rate:

Here, we are using a capture resolution of 1280 x 720, a diagonal FOV of 70 degrees, and a target frame rate of 25 FPS. You should choose parameters that are appropriate for your camera and lens, and for the speed of your system.

Let's suppose we run the application, and it loads the following image from `reference_image.png` :

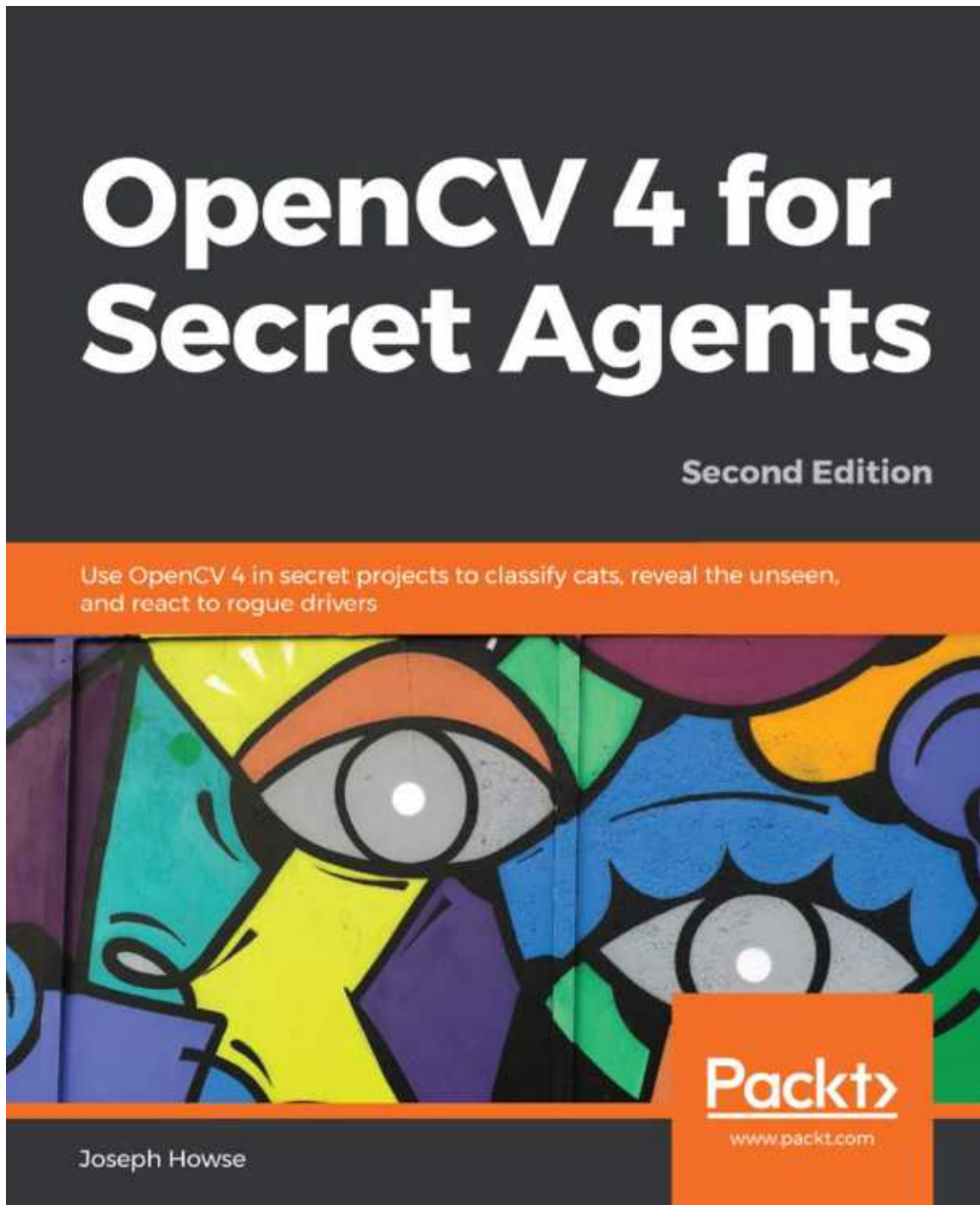


Figure 9.4: A book cover, which we will use as an image to track

This is, of course, the cover of *OpenCV 4 for Secret Agents* (Packt Publishing, 2019), a book by Joseph Howse. Not only is it a vault of secret knowledge, it is also a good target for image tracking. You should buy a print copy!

During initialization, the application saves the following visualization of the reference keypoints to a new file called `reference_image_keypoints.png` :

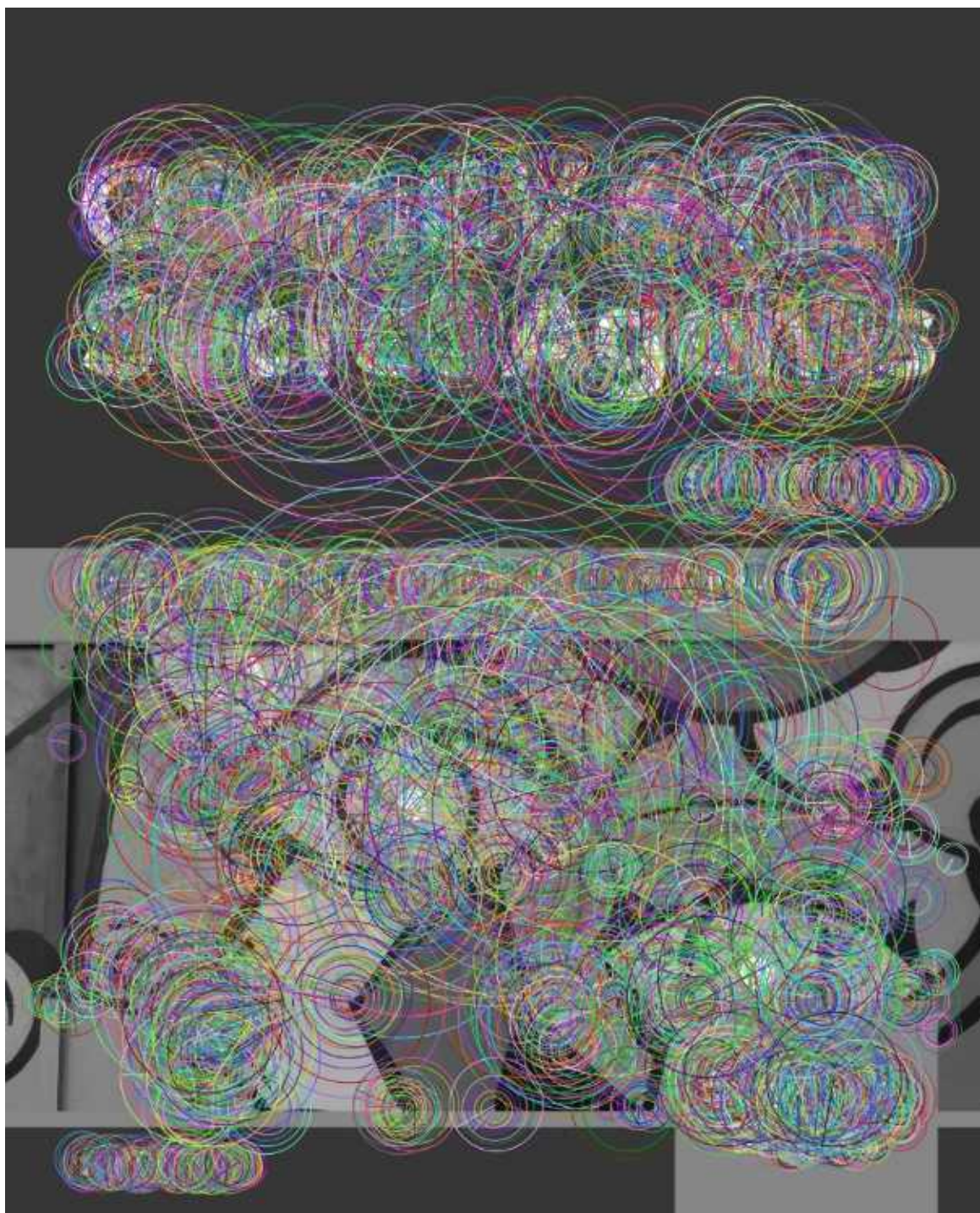


Figure 9.5: Visualization of ORB descriptors for a book cover

We have seen this type of visualization before in *Chapter 6 , Retrieving Images and Searching Using Image Descriptors*. The large circles represent keypoints

that can be matched at a small scale (for example, when we view the printed image from a large distance or with a low-resolution camera). The small circles represent keypoints that can be matched at a large scale (for example, when we view the printed image at a close distance or with a high-resolution camera). The best keypoints are the ones marked by many concentric circles, since these can be matched at various scales. Within each circle, the radial line represents the normal orientation of the keypoint.

Studying this visualization, we can infer that this image's best keypoints are concentrated in the high-contrast text (white against dark gray) in the top part of the image. Other useful keypoints are found in many regions, including the high-contrast lines (black against saturated colors) in the bottom part of the image.

Next, we see a camera feed. When we put a print of the reference image in front of the camera, we see an AR visualization of the tracking results:

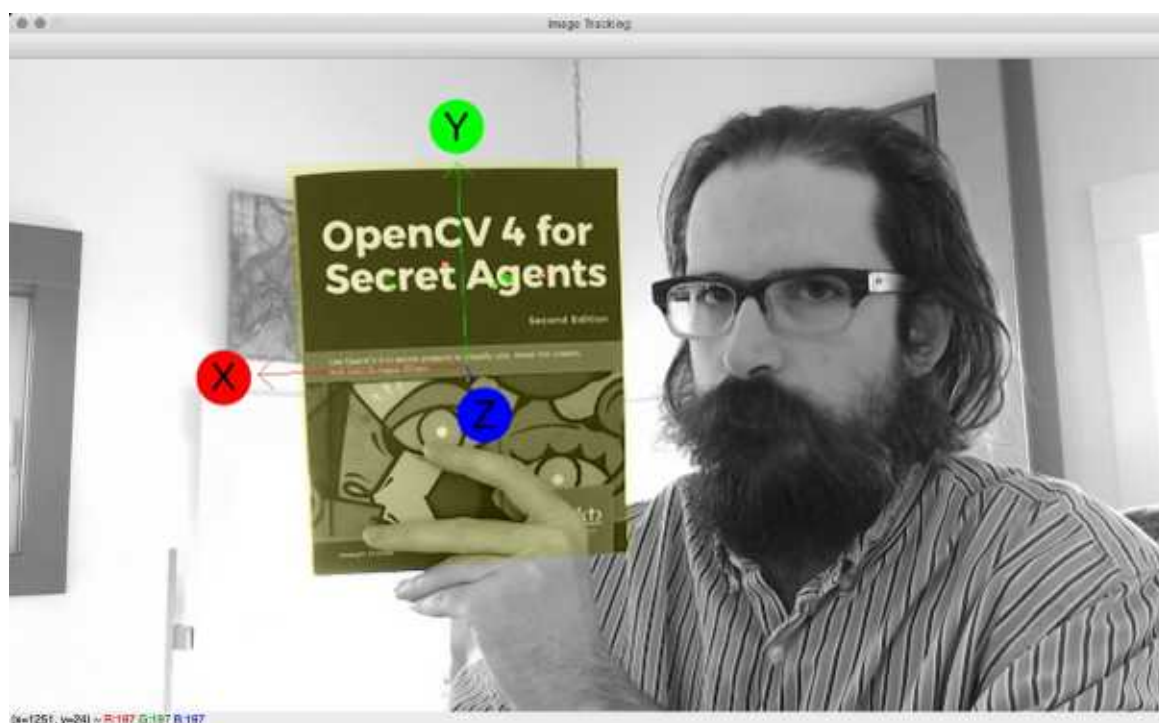


Figure 9.6: Results of image tracking and AR visualization with a book cover

Of course, the preceding screenshot shows a nearly-frontal view of the book cover. The axis directions are drawn as expected. The X axis (in red) points to the book cover's right (the viewer's left). The Y axis (in green) points up. The Z axis (in blue) points forward from the book cover (toward the viewer). As an AR

effect, a semitransparent yellow highlight is superimposed atop the tracked book cover (including the part covered up by Joseph Howse's index finger and middle finger). The positions of the small green and red dots show that in this frame, the good keypoint matches are concentrated in the region of the book's title, and most of these good matches are inliers for `cv2.solvePnPRansac` .

If you are reading the print edition of this book, the screenshots are reproduced in grayscale. To make the X, Y, and Z axes easier to distinguish in a grayscale print, text labels have been added to the screenshots manually; these text labels are not part of the program's output.

Because we took care to find good keypoints in several regions throughout the image, the tracking can succeed even when a large part of the tracked image is in shadow, covered up, or outside the frame. For example, in the following screenshot, the axis directions and highlighted region are correct, even though most of the book cover (including nearly all of the book title's, with the best keypoints) is outside the frame:

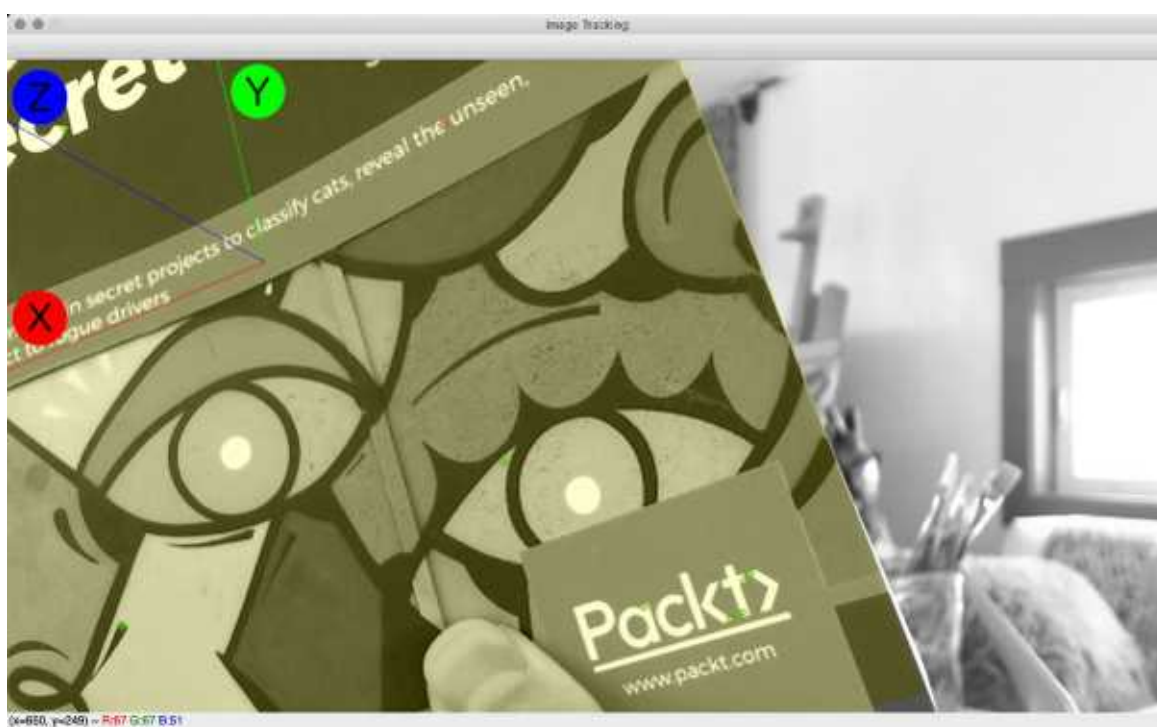


Figure 9.7: Results of image tracking and AR visualization when only part of the book cover is visible

Go ahead and conduct your own experiments with various reference images, cameras, and viewing conditions. Try various resolutions for the reference image

and camera. Remember to measure your camera's FOV and adjust the FOV argument accordingly. Study the keypoint visualizations and the tracking results. What kinds of input yield good (or bad) tracking results in our demo?

If you find it inconvenient to use printed images for tracking, you can instead point your camera at a screen (such as a smartphone screen) where you are displaying the image you want to track. Because a screen is backlit (and it might be glossy, too), it might not give a faithful representation of how a printed image would look in any given scene, but it typically works well for a tracker's purposes.

Once you have experimented to your heart's content, let's consider some of the ways our 3D tracker could be improved.

Improving the 3D tracking algorithm

Essentially, our 3D tracking algorithm combines three approaches:

1. Find a 6DOF pose with a PnP solver, whose inputs depend on FLANN-based matches of ORB descriptors..
2. Use a Kalman filter to stabilize the 6DOF tracking result.
3. If an object was tracked in the previous frame, use a mask to limit the search to the region where the object is now most likely to be found.

Often, commercial solutions for 3D tracking involve additional approaches. We have relied on successfully using a descriptor matcher and a PnP solver for every frame; however, a more complex algorithm may provide some alternatives as fallbacks or as cross-checking mechanisms. This is in case the descriptor matcher and PnP solver miss the object in some frames, or in case they are too computationally expensive to use for every frame. The following alternatives are widely used:

- Update the previous keypoint matches based on optical flow, and update the previous 6DOF pose based on the homography between the keypoints' old and new positions (according to the optical flow).
- Update the rotational component of the 6DOF pose based on an accelerometer, gyroscope, and magnetometer (compass). Typically, even in consumer devices, these sensors together can successfully measure small or large changes in rotation. Often, this set of sensor data is used in combination with a Kalman filter, and this filtering can be done in a way that avoids the singularity problems associated with conventional Euler angles. For example, see the paper *Euler Angle Based Attitude Estimation Avoiding the Singularity Problem* , by Chul Woo Kang and Chan Gook Park; it is available at <https://www.sciencedirect.com/science/article/pii/S1474667016439212>

- Update the positional component of the 6DOF pose based on a barometer and GPS. Typically, in consumer devices, a barometer can measure changes in altitude with an accuracy of approximately 10 cm, while GPS can measure changes in longitude and latitude with an accuracy of approximately 10 m. Depending on the use case, these may or may not be usable levels of accuracy. If we are attempting to perform AR on large and distant features of a landscape – for example, if we want to draw a virtual dragon perched atop a real mountaintop – then 10 m accuracy may be fine. For detailed work – for example, if we want to draw a virtual ring on a real finger – 10 m or even 10 cm accuracy is unusable.
- Update the positional acceleration component of the Kalman filter based on an **accelerometer** (a sensor that measures acceleration). Typically, in consumer devices, accelerometers suffer from drift (a tendency for errors to exhibit runaway growth in one direction or another), so this option should be approached with caution.

These alternative techniques are beyond the scope of this book – and, indeed, some of them are not computer vision techniques – so we leave them to you for independent study.

A final word : Sometimes, significant improvements in tracking results are achievable by altering a preprocessing algorithm rather than the tracking algorithm. Earlier in this chapter, in the *Performing grayscale conversion* section, we mentioned Macêdo, Melo, and Kelner's paper on grayscale conversion algorithms and SIFT descriptors. You may wish to read that paper and conduct your own experiments to determine how the choice of grayscale conversion algorithm affects the number of tracking inliers when using ORB descriptors or other types of descriptors.

Summary

This chapter introduced AR, along with a robust set of approaches to the problem of tracking an image in 3D space.

We began by learning the concept of 6DOF tracking and different ways of representing rotations. We recognized that familiar tools such as ORB descriptors, FLANN-based matching, and Kalman filtering are useful in this kind of tracking, but that we also needed to work with camera and lens parameters in order to solve the PnP problem.

Next, we addressed practical considerations of how best to represent a reference object (such as a book cover or a photo print) in the form of a grayscale image, a set of 2D keypoints, and a set of 3D keypoints.

We proceeded to implement a class that encapsulated a demo of image tracking in 3D space, with a 3D highlighting effect as a basic form of AR. Our implementation dealt with real-time considerations, such as the need to update the Kalman filter's transition matrix based on fluctuations in the frame rate.

Finally, we considered ways to potentially improve the 3D tracking algorithm using additional computer vision techniques, or other sensor-based techniques.

Next, in one of the most advanced chapters of this book, we will tackle even bigger challenges in the real, three-dimensional world; we will scan and navigate a whole 3D environment in real time!!

Join our book community on Discord

<https://discord.gg/djGjeMECxx>



This chapter introduces a family of machine learning models called **artificial neural networks** (**ANNs**), or sometimes just **neural networks** , and we will utilize this knowledge to perform various tasks such as handwritten digit recognition, object detection and recognition, and finally gesture recognition from a video input.

A key characteristic of neural networks is that they attempt to learn relationships among variables in a multi-layered fashion; they learn multiple functions to predict intermediate results before combining these into a single function to predict something meaningful (such as the class of an object). Recent versions of OpenCV contain an increasing amount of functionality related to ANNs—and, in particular, ANNs with many layers, called **deep neural networks** (**DNNs**). We will experiment with both shallower ANNs and DNNs in this chapter.

We have already gained some exposure to machine learning in other chapters—especially in *Chapter 7 , Building Custom Object Detectors* , where we developed a car/non-car classifier using SURF descriptors, a BoW, and an SVM. With this basis for comparison, you might be

wondering, what is so special about ANNs? Why are we devoting this book's final chapter to them?

ANNs aim to provide superior accuracy in the following circumstances:

- There are many input variables, which may have complex, nonlinear relationships to each other.
- There are many output variables, which may have complex, nonlinear relationships to the input variables. (Typically, the output variables in a classification problem are the confidence scores for the classes, so if there are many classes, there are many output variables.)
- There are many hidden (unspecified) variables that may have complex, nonlinear relationships to the input and output variables. DNNs even aim to model *multiple* layers of hidden variables, which are interrelated primarily to each other rather than being related primarily to input or output variables.

These circumstances exist in many – perhaps most – real-world problems. Thus, the promised advantages of ANNs and DNNs are enticing. On the other hand, ANNs and especially DNNs are notoriously opaque models, insofar as they work by predicting the existence of an arbitrary number of nameless, hidden variables that may relate to everything else.

Over the course of this chapter, we will cover the following topics:

Understanding ANNs as a statistical model and as a tool for supervised machine learning.

Understanding ANN topology or, in other words, the organization of an ANN into layers of interconnected neurons. Particularly, we will consider the topology that enables an ANN to act as a type of classifier known as a **multi-layer perceptron (MLP)**.

Training and using ANNs as classifiers in OpenCV.

Building an application that detects and recognizes handwritten digits (0 to 9). For this, we will train an ANN based on a widely used dataset called MNIST, which contains samples of handwritten digits.

Loading and using pre-trained DNNs in OpenCV. We will cover examples of object classification, face detection, and gender classification with DNNs.

By the end of this chapter, you will be in a good position to train and use ANNs in OpenCV, to use pre-trained DNNs from a variety of sources, and to start exploring other libraries that allow you to train your own DNNs.

Technical requirements

This chapter uses Python, OpenCV, and NumPy. Please refer to *Chapter 1 , Setting Up OpenCV* , for installation instructions.

The completed code and sample videos for this chapter can be found in this book's GitHub repository, <https://github.com/PacktPublishing/Learning-OpenCV-5-Computer-Vision-with-Python-Fourth-Edition> , in the `chapter11` folder.

Understanding ANNs

Let's define ANNs in terms of their basic role and components. Although much of the literature on ANNs emphasizes the idea that they are *biologically inspired* by the way neurons connect in a brain, we don't need to be biologists or neuroscientists to understand the fundamental concepts of an ANN.

First of all, an ANN is a **statistical model**. What is a statistical model? A statistical model is a pair of elements, namely the space S (a set of observations) and the probability, P , where P is a distribution that approximates S (in other words, a function that would generate a set of observations that is very similar to S).

Here are two different ways to think of P :

- P is a simplification of a complex scenario.
- P is the function that generated S in the first place, or at the very least a set of observations very similar to S .

Thus, ANNs are models that take a complex reality, simplify it, and deduce a function to (approximately) represent the statistical observations we would expect from that reality, in a mathematical form.

ANNs, like other types of machine learning models, can learn from observations in one of the following ways:

Supervised learning : Under this approach, we want the model's training process to produce a function that maps a known set of input variables to a known set of output variables. We know, *a priori*, the nature of the prediction problem, and we delegate the process of finding a function that solves this problem to the ANN. To train the model, we must provide input samples along with the correct, corresponding outputs. For a classification problem, the output variables may be confidence scores for one or more classes.

Unsupervised learning : Under this approach, the set of output variables is not known *a priori*. The model's training process must yield a set of output variables, as well as a function to map the input variables to these output variables. For a classification problem, unsupervised learning can lead to the discovery of

previously unknown classes, such as previously unknown diseases in the context of medical data. Unsupervised learning may use techniques including (but not limited to) clustering, which we explored in the context of BoW models in *Chapter 7 , Building Custom Object Detectors* .

Reinforcement learning : This approach turns the typical prediction problem upside down. Before training the model, we already have a system that yields values for a known set of output variables when we feed it values for a known set of input variables. We know, *a priori* , a way of scoring a sequence of outputs based on their goodness (desirability) or lack thereof. However, we might not know the real function that maps inputs to outputs—or, even if we do know it, it is so complex that we cannot solve it for optimal inputs. Thus, we want the model's training process to produce a function that predicts the next-in-sequence optimal inputs, based on the last outputs. During training, the model learns from the score that eventually arises from its actions (its chosen inputs). Essentially, the model must learn to become a good decision-maker within the context of a particular system of rewards and punishments.

Throughout the remainder of this chapter, we will confine our discussions to supervised learning, as this is the most common approach to machine learning in the context of computer vision.

The next step in our journey toward comprehending ANNs is to understand how an ANN improves on the concept of a simple statistical model, and on other types of machine learning.

What if the function that generated the dataset is likely to take a large number of (unknown) inputs?

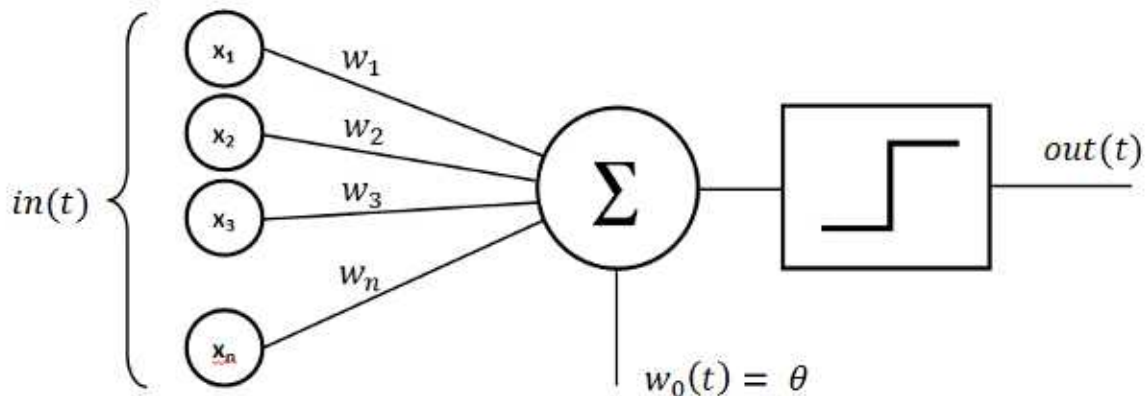
The strategy that ANNs adopt is to delegate work to a number of **neurons** , **nodes** , or **units** , each of which is capable of approximating the function that created the inputs. In mathematics, approximation is the process of defining a simpler function whose output is similar to that of a more complex function, at least for some range of inputs.

The difference between the approximate function's output and the original function's output is called the **error** . A defining characteristic of a neural network is that the neurons must be capable of approximating a nonlinear function.

Let's take a closer look at neurons.

Understanding neurons and perceptrons

Often, to solve a classification problem, an ANN is designed as a **multi-layer perceptron (MLP)**, in which each neuron acts as a kind of binary classifier called a **perceptron**. The perceptron is a concept that dates back to the 1950s. To put it simply, a perceptron is a function that takes a number of inputs and produces a single value. Each of the inputs has an associated weight that signifies its importance in an **activation function**. The activation function should have a nonlinear response; for example, a sigmoid function (sometimes called an S-curve) is a common choice. A threshold function, called a **discriminant**, is applied to the activation function's output to convert it into a binary classification of 0 or 1. Here is a visualization of this sequence, with inputs on the left, the activation function in the middle, and the discriminant on the right:



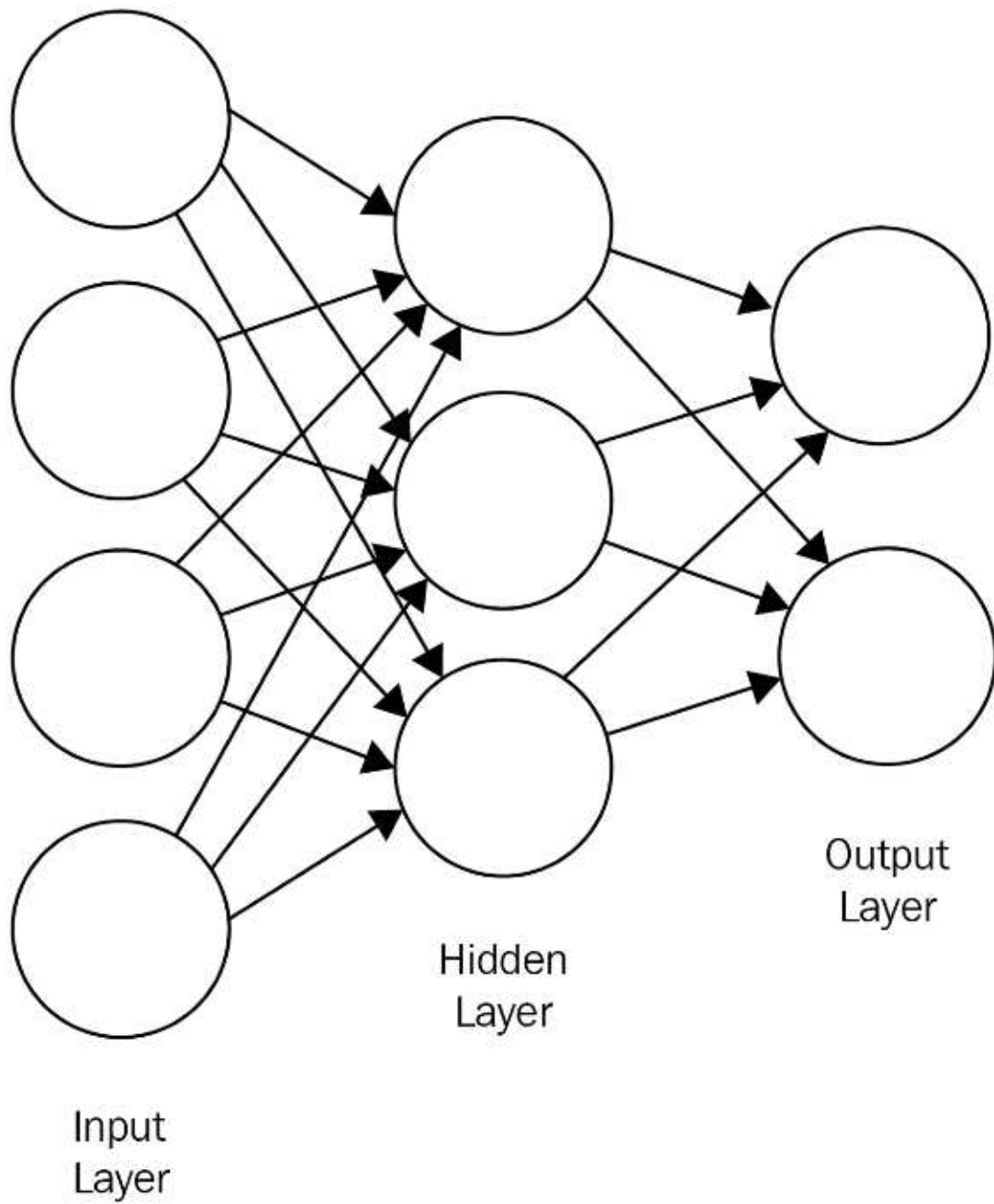
What do the input weights represent, and how are they determined?

Neurons are interconnected, insofar as one neuron's output can be an input for many other neurons. Each input weight defines the strength of the connection between two neurons. These weights are adaptive, meaning that they change in time according to a learning algorithm.

Due to the neurons' interconnectedness, the network has layers. Now, let's examine how these layers are typically organized.

Understanding the layers of a neural network

Here is a visual representation of a neural network:



Circles are "neurons".
Arrows are "connectors".

As the preceding diagram shows, there are at least three distinct layers in a neural network: the **input layer** , the **hidden layer** , and the **output layer** . There can be more than one hidden layer; however, one hidden layer is enough to resolve many real-life problems. A neural network with multiple hidden layers is sometimes called a **deep neural network (DNN)**.

If we are using an ANN as a classifier, then each output node's output value is a confidence score for a class. For a given sample (that is, a given set of input values), we want to know which output node produces the highest output value. This highest-scoring output node corresponds to the predicted class .

How do we determine the network's topology, and how many neurons do we need to create for each layer? Let's make this determination layer by layer.

Choosing the size of the input layer

The number of nodes in the input layer is, by definition, the number of inputs into the network. For example, let's say you want to create an ANN to help you determine an animal's species based on measurements of its physical attributes. In principle, we can choose any measurable attributes. If we choose to classify animals based on weight, length, and number of teeth, that is a set of three attributes, and thus our network needs to contain three input nodes.

Are these three input nodes an adequate basis for species classification? Well, in a real-life problem, surely not—but in a toy problem, it depends on the output we are trying to achieve, and this is our next consideration.

Choosing the size of the output layer

For a classifier, the number of nodes in the output layer is, by definition, the number of classes the network can distinguish. Continuing with the preceding example of an animal classification network, we can use an output layer of four nodes if we know we are going to deal with the following animals: dog, condor, dolphin, and dragon(!). If we try to classify data for an animal that is not in one of these categories, the network will predict the class that is most likely to resemble this unrepresented animal.

Now, we come to a difficult problem—the size of the hidden layer.

Choosing the size of the hidden layer

There are no agreed-upon rules of thumb for choosing the size of the hidden layer; it must be chosen based on experimentation. For every real-world problem where you want to apply ANNs, you will need to train, test, and retrain your ANN until you find a number of hidden nodes that yield acceptable accuracy.

Of course, even when choosing a parameter's value by experimentation, you might wish for the experts to suggest a starting value, or a range of values, for your tests. Unfortunately, there is no expert consensus on these points either. Some experts offer rules of thumb based on the following broad suggestions (these should be taken with a grain of salt):

If the input layer is large, the number of hidden neurons should be between the size of the input layer and the size of the output layer—and, typically, closer to the size of the output layer.

On the other hand, if the input and output layers are both small, the hidden layer should be the largest layer.

If the input layer is small but the output layer is large, the hidden layer should be closer to the size of the input layer.

Other experts suggest that the number of training samples also needs to be taken into account; a greater number of training samples implies that a greater number of hidden nodes might be useful.

One critical factor to keep in mind is **overfitting**. Overfitting occurs when there is such an inordinate amount of pseudo-information contained in the hidden layer, compared to the information actually provided by the training data, that the classification is not very meaningful. The larger the hidden layer, the more training data it requires in order for it to learn properly. Of course, as the size of the training dataset grows, so does the training time.

For some of the ANN sample projects in this chapter, we will use a hidden layer size of 60 as a starting point. Given a large training set, 60 hidden nodes can yield decent accuracy for a variety of classification problems.

Now that we have a general idea of what ANNs are, let's see how OpenCV implements them, and how to put them to good use. We will start with a minimal

code example. Then, we will flesh out the animal-themed classifier that we discussed in the previous two sections. Finally, we will work our way up to a more realistic application, in which we will classify handwritten digits based on image data.

Training a basic ANN in OpenCV

OpenCV provides a class, `cv2.ml_ANN_MLP` , that implements an ANN as a **multi-layer perceptron (MLP)**. This is exactly the kind of model we described earlier, in the *Understanding neurons and perceptrons* section.

To create an instance of `cv2.ml_ANN_MLP` , and to format data for this ANN's training and use, we rely on functionality in OpenCV's machine learning module, `cv2.ml` . As you may recall, this is the same module that we used for SVM-related functionality in *Chapter 7 , Building Custom Object Detectors* . Moreover, `cv2.ml_ANN_MLP` and `cv2.ml_SVM` share a common base class called `cv2.ml_StatModel` . Therefore, you will find that OpenCV provides similar APIs for ANNs and SVMs.

Let's examine a dummy example as a gentle introduction to ANNs. This example will use completely meaningless data, but it will show us the basic API for training and using an ANN in OpenCV:

To begin, we import OpenCV and NumPy as usual:

Now, we create an untrained ANN:

After creating the ANN, we need to configure its number of layers and nodes:

The layer sizes are defined by the NumPy array that we pass to the `setLayerSizes` method. The first element is the size of the input layer, the last element is the size of the output layer, and all the in-between elements define the sizes of the hidden layers. For example, `[9, 15, 9]` specifies 9 input nodes, 9 output nodes, and a single hidden layer with 15 nodes. If we changed this to `[9, 15, 13, 9]` , it would specify two hidden layers with 15 and 13 nodes, respectively.

We can also configure the activation function, the training method, and the training termination criteria, as follows:

Here, we are using a symmetrical sigmoid activation function (`cv2.ml.ANN_MLP_SIGMOID_SYM`) and a backpropagation training method (`cv2.ml.ANN_MLP_BACKPROP`). Backpropagation is an algorithm that calculates errors of predictions at the output layer, traces the sources of the errors backward through previous layers, and updates the weights in order to reduce errors.

Let's train the ANN. We need to specify training inputs (or samples , in OpenCV's terminology), the corresponding correct outputs (or responses), and whether the data's format (or layout) is one row per sample or one column per sample. Here is an example of how we train the model with a single sample:

*Realistically, we would want to train any ANN with a larger dataset that contains far more than one sample. We could do this by extending `training_samples` and `training_responses` so that they contain multiple rows, representing multiple samples and their corresponding responses. Alternatively, we could call the ANN's `train` method multiple times, with new data each time. The latter approach requires some additional arguments for the `train` method, and it is demonstrated in the next section, *Training an ANN- classifier in multiple epochs*.*

Note that in this case, we are training the ANN as a classifier. Each response is a confidence score for a class, and in this case, there are nine classes. We will refer to them by their 0-based indices, as classes 0 to 8. Our training sample in this case has a response of `[0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 0.0]` , meaning that it is an instance of class 5 (with confidence 1.0), and it is definitely not an instance of any other class (as the confidence is 0.0 for every other class).

To complete our minimal tour of the ANN's API, let's make another sample, classify it, and print the result:

This will print the following result:

This means that the provided input was classified as belonging to class 5. Again, this is only a dummy example and the classification is pretty meaningless; however, the network behaved correctly. In the preceding

code, we only provided one training record, which was a sample of class 5, so the network classified a new input as belonging to class 5. (As far as our woefully limited training dataset suggests, other classes besides 5 might never occur.)

As you may have guessed, the output of a prediction is a tuple, with the first value being the class and the second being an array containing the probabilities for each class. The predicted class will have the highest value.

Let's move on to a slightly more believable example—animal classification.

Training an ANN classifier in multiple epochs

Let's create an ANN that attempts to classify animals based on three measurements: weight, length, and number of teeth. This is, of course, a mock scenario. Realistically, no one would describe an animal with just these three statistics. However, our intent is to improve our understanding of ANNs before we start applying them to image data.

Compared to the minimal example in the previous section, our animal classification mock-up will be more sophisticated in the following ways:

We will increase the number of neurons in the hidden layer.

We will use a larger training dataset. For convenience, we will generate this dataset pseudorandomly.

We will train the ANN in multiple epochs, meaning that we will train and retrain it multiple times with the same dataset each time.

The number of neurons in the hidden layer is an important parameter that needs to be tested in order to optimize the accuracy of any ANN. You will find that a larger hidden layer may improve accuracy up to a point, and then it will overfit, unless you start compensating with an enormous training dataset. Likewise, up to a point, a greater number of epochs may improve accuracy, but too many will result in overfitting .

Let's go through the implementation step by step:

First, we import OpenCV and NumPy as usual. Then, from the Python standard library, we import the `randint` function to generate pseudorandom integers and the `uniform` function to generate pseudorandom floating-point numbers:

Next, we create and configure the ANN. This time, we use a three-neuron input layer, a 50-neuron hidden layer, and a four-neuron output layer, as highlighted **in bold** in the following code:

Now, we need some data. We aren't really interested in representing animals accurately; we just require a bunch of records to be used as training data. Thus, we define four functions in order to generate random samples of different classes, along with another four functions to generate the correct classification results for training purposes:

We also define the following helper function in order to convert a sample and classification into a pair of NumPy arrays:

Let's proceed with the creation of our fake animal data. We will create 20,000 samples per class:

Now, let's train the ANN. As we discussed at the start of this section, we will use multiple training epochs. Each epoch is an iteration of a loop, as shown in the following code:

For real-world problems with large and diverse training datasets, an ANN can potentially benefit from hundreds of training epochs. For the best results, you may wish to keep training and testing an ANN until you reach convergence, which means that further epochs no longer produce a noticeable improvement in the accuracy of the results .

Note that we must pass the `cv2.ml.ANN_MLP_UPDATE_WEIGHTS` flag to the ANN's `train` function to update the previously trained model rather than training a new model from scratch. This is a critical point to remember whenever you are training a model incrementally, as we are doing here.

Having trained our ANN, we should test it. For each class, let's generate 100 new random samples, classify them using the ANN, and keep track of the number of correct classifications:

Finally, let's print the accuracy statistics:

When we run the script, the preceding code block should produce the following output:

Since we are dealing with random data, the results may vary each time you run the script. Typically, the accuracy should be high or even perfect because we have set up a simple classification problem with non-overlapping ranges of input data. (The range of random weight values for a dog does not overlap with the range for a dragon, and so forth.)

You may wish to take some time to experiment with the following modifications (one at a time) so that you can see how the ANN's accuracy is affected:

Change the number of training samples by modifying the value of the `RECORDS` variable.

Change the number of training epochs by modifying the value of the `EPOCHS` variable.

Make the ranges of input data partially overlapping by editing the parameters of the `uniform` and `randint` function calls in our `dog_sample` , `condor_sample` , `dolphin_sample` , and `dragon_sample` functions.

When you are ready, we will proceed with an example containing real-life image data. With this, we will train an ANN to recognize handwritten digits.

Recognizing handwritten digits with an ANN

A handwritten digit is any of the 10 Arabic numerals (0 to 9), written manually with a pen or pencil, as opposed to being printed by a machine. The appearance of handwritten digits can vary significantly. Different people have different handwriting, and—with the possible exception of a skilled calligrapher—a person does not produce identical digits every time he or she writes. This variability means that the visual recognition of handwritten digits is a non-trivial problem for machine learning. Indeed, students and researchers in machine learning often test their skills and new algorithms by attempting to train an accurate recognizer for handwritten digits. We will approach this challenge in the following manner:

- Load data from a Python-friendly version of the MNIST database. This is a widely used database containing images of handwritten digits.
- Using the MNIST data, train an ANN in multiple epochs.
- Load an image of a sheet of paper with many handwritten digits on it.
- Based on contour analysis, detect the individual digits on the paper.
- Use our ANN to classify the detected digits.
- Review the results in order to determine the accuracy of our detector and our ANN-based classifier.

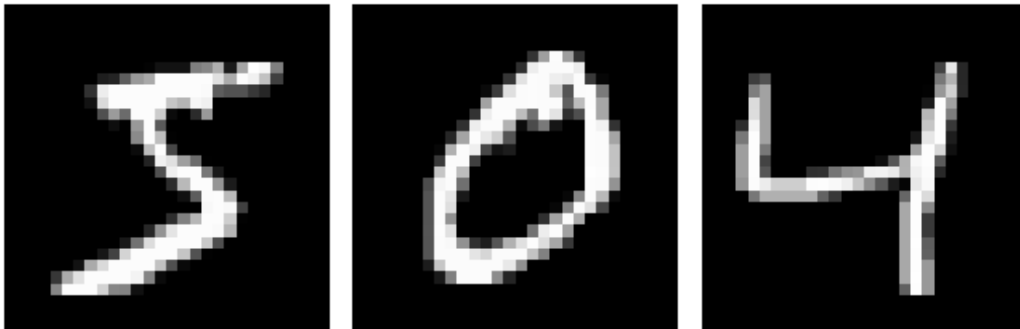
Before we delve into the implementation, let's review some information about the MNIST database.

Understanding the MNIST database of handwritten digits

The **MNIST** database (or **Modified National Institute of Standards and Technology** database) is publicly available at <http://yann.lecun.com/exdb/mnist/>. The database includes a training set of 60,000 images of handwritten digits. Half of these were written by employees of the United States Census Bureau, while the other half were written by high school students in the United States.

The database also includes a test set of 10,000 images, gathered from the same writers. All the training and test images are in grayscale format, with dimensions

of 28 x 28 pixels. The digits are white (or shades of gray) on a black background. For example, here are three of the MNIST training samples:



As an alternative to using MNIST, you could, of course, build a similar database yourself. This would involve collecting a large number of images of handwritten digits, converting the images into grayscale, cropping them so that each image contains a single digit in a standardized position, and scaling the images so that they are all the same size. You would also need to label the images so that a program could read the correct classification for the purpose of training and testing a classifier.

Many authors provide examples of how to use the MNIST database with various machine learning libraries and algorithms—not just OpenCV and not just ANNs. Michael Nielsen, the author of the free online book *Neural Networks and Deep Learning* , devotes a chapter to MNIST and ANNs here:

<http://neuralnetworksanddeeplearning.com/chap1.html> . He shows how to implement an ANN almost from scratch, using only NumPy, and this is excellent reading if you want to deepen your understanding beyond the kind of high-level functionality that OpenCV exposes. His code is freely available on GitHub at <https://github.com/mnielsen/neural-networks-and-deep-learning> .

Nielsen provides a version of MNIST as a PKL.GZ (gzip-compressed Pickle) file, which can be easily loaded into Python. For the purposes of our book's OpenCV sample, we (the authors) have taken Nielsen's PKL.GZ version of MNIST, reorganized it for our purposes, and placed it inside this book's GitHub repository at `chapter10/digits_data/mnist.pkl.gz` .

Now that we know something about the MNIST database, let's consider what ANN parameters are appropriate for this training set.

Choosing training parameters for the MNIST database

Each MNIST sample is an image containing 784 pixels (that is, 28 x 28 pixels). Thus, our ANN's input layer will have 784 nodes. The output layer will have 10 nodes because there are 10 classes of digits (0 to 9).

We are free to choose the values of other parameters, such as the number of nodes in the hidden layer, the number of training samples to use, and the number of training epochs. As usual, experimentation can help us find values that offer acceptable training time and accuracy, without overfitting the model to the training data. Based on some experimentation that the authors of this book have done, we will use 60 hidden nodes, 50,000 training samples, and 10 epochs. These parameters will be good enough for a preliminary test, keeping the training time down to a few minutes (depending on the processing power of your machine).

Implementing a module to train the ANN

Training an ANN based on MNIST is something you might want to do in future projects as well. To make our code more reusable, we can write a Python module that is solely dedicated to this training process. Then (in the next section, *Implementing the main module*), we will import this training module into a main module, where we will implement our demonstration of digit detection and classification.

Let's implement the training module in a file called `digits_ann.py` :

To begin, we will import the `gzip` and `pickle` modules from the Python standard library. As usual, we will also import `OpenCV` and `NumPy`:

We will use the `gzip` and `pickle` modules to decompress and load the MNIST data from the `mnist.pkl.gz` file. We briefly mentioned this file earlier, in the *Understanding the MNIST database of handwritten digits* section. It contains the MNIST data in nested tuples, in the following format:

In turn, the elements of these tuples are in the following format:

`training_images` is a NumPy array of 60,000 images, where each image is a vector of 784-pixel values (flattened from an original shape of 28 x 28 pixels).

The pixel values are floating-point numbers in the range 0.0 (black) to 1.0 (white), inclusive.

`training_ids` is a NumPy array of 60,000 digit IDs, where each ID is a number in the range 0 to 9, inclusive. `training_ids[i]` corresponds to `training_images[i]` .

`test_images` is a NumPy array of 10,000 images, where each image is a vector of 784-pixel values (flattened from an original shape of 28 x 28 pixels). The pixel values are floating-point numbers in the range 0.0 (black) to 1.0 (white), inclusive.

`test_ids` is a NumPy array of 10,000 digit IDs, where each ID is a number in the range 0 to 9, inclusive. `test_ids[i]` corresponds to `test_images[i]` .

Let's write the following helper function to decompress and load the contents of `mnist.pkl.gz` :

Note that in the preceding code, `training_data` is a tuple, equivalent to `(training_images, training_ids)` , and `test_data` is also a tuple, equivalent to `(test_images, test_ids)` .

We must reformat the raw data in order to match the format that OpenCV expects. Specifically, when we provide sample output to train the ANN, it must be a vector with 10 elements (for 10 classes of digits), rather than a single digit ID. For convenience, we will also apply Python's built-in `zip` function to reorganize the data in such a way that we can iterate over matching pairs of input and output vectors as tuples. Let's write the following helper function to reformat the data:

Note that the preceding code calls `load_data` and another helper function, `vectorized_result` . The latter function converts an ID into a classification vector, as follows:

For example, the ID 1 is converted into a NumPy array containing the values `[0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]` . This 10-element array, as you may have guessed, corresponds to the ANN's output layer, and we can use it as a sample of correct output when we train the ANN.

The preceding functions – `load_data` , `wrap_data` , and `vectorized_result` – have been adapted from Nielsen's code for loading his version of `mnist.pkl.gz` .

For more information about Nielsen's work, refer to the Understanding the MNIST database of handwritten digits section of this chapter.

So far, we have written functions that load and reformat MNIST data. Now, let's write a function that will create an untrained ANN:

Note that we have hardcoded the sizes of the input and output layers, based on the nature of the MNIST data. However, we have allowed the caller of this function to specify the number of nodes in the hidden layer.

For further discussion of parameters, refer to the Choosing training parameters for the MNIST database section of this chapter.

Now, we need a training function that allows the caller to specify the number of MNIST training samples and the number of epochs. Much of the training functionality should be familiar from our previous ANN samples, so let's look at the implementation in its entirety and then discuss some details afterward:

Note that we load the data and then train the ANN incrementally by iterating over a specified number of training epochs, with a specified number of samples in each epoch. For every 1,000 training samples that we process, we print a message about the progress of the training. Finally, we return both the trained ANN and the MNIST test data. We could have just returned the ANN, but having the test data on hand is useful in case we want to check the ANN's accuracy.

Of course, the purpose of a trained ANN is to make predictions, so we will provide the following `predict` function in order to wrap the ANN's own `predict` method:

This function takes a trained ANN and a sample image; it performs a minimal amount of data sanitization by making sure the sample image is 28 x 28 and by resizing it if it isn't. Then, it flattens the image data into a vector before giving it to the ANN for classification.

That's all the ANN-related functionality we will need to support our demo application. However, let's also implement a `test` function that measures a trained ANN's accuracy by classifying a given set of test data, such as the MNIST test data. Here is the relevant code:

Now, let's take a short detour and write a minimal test that leverages all the preceding code and the MNIST dataset. After that, we will proceed to implement

the main module of our demo application.

Implementing a minimal test module

Let's make another script, `test_digits_ann.py`, in order to test the functions from our `digits_ann` module. The test script is quite trivial; here it is:

Note that we haven't specified the number of hidden nodes, so `create_ann` will use its default parameter value: 60 hidden nodes. Similarly, `train` will use its default parameter values: 50,000 samples and 10 epochs.

When we run this script, it should print training and test information similar to the following:

Here, we can see that the ANN achieved 95.39% accuracy when classifying the 10,000 test samples in the MNIST dataset. This is an encouraging result, but let's see how well the ANN can generalize. Can it accurately classify data from an entirely different source, unrelated to MNIST? Our main application, which detects digits from our own image of a sheet of paper, will provide this kind of challenge to the classifier.

Implementing the main module

Our demo's main script takes everything we have learned in this chapter about ANNs and MNIST and combines it with some of the object detection techniques that we studied in previous chapters. Thus, in many ways, this is a capstone project for us.

Let's implement the main script in a new file called `detect_and_classify_digits.py`:

To begin, we will import OpenCV, NumPy, and our `digits_ann` module:

Now, let's write a couple of helper functions to analyze and adjust the bounding rectangles of digits and other contours. As we have seen in previous chapters, overlapping detections are a common problem. The following function, called `inside`, will help us determine whether one bounding rectangle is entirely contained inside another:

With the help of the `inside` function, we will be able to easily choose only the outermost bounding rectangle for each digit. This is important because we do not want our detector to miss any extremities of a digit; such a mistake in detection could make the classifier's job impossible. For example, if we detected only the bottom half of a digit, 8, the classifier might reasonably see this region as a 0.

To further ensure that the bounding rectangles meet the classifier's needs, we will use another helper function, called `wrap_digit`, to convert a tightly-fitting bounding rectangle into a square with padding around the digit. Remember that the MNIST data contains 28 x 28 pixel square images of digits, so we must rescale any region of interest to this size before we attempt to classify it with our MNIST-trained ANN. By using a padded bounding square instead of a tightly-fitting bounding rectangle, we ensure that skinny digits (such as a 1) and fat digits (such as a 0) are not stretched differently.

Let's look at the implementation of `wrap_digit` in multiple stages. First, we modify the rectangle's smaller dimension (be it width or height) so that it equals the larger dimension, and we modify the rectangle's x or y position so that the center remains unchanged:

Next, we add 5-pixel padding on all sides:

At this point, our modified rectangle could possibly extend outside the image.

To avoid out of bounds problems, we crop the rectangle so that it lies entirely within the image. This could leave us with non-square rectangles in these edge cases, but this is an acceptable compromise; we would prefer to use a non-square region of interest rather than having to entirely throw out a detected digit just because it is at the edge of the image. Here is the code for bounds-checking and cropping the rectangle:

Finally, we return the modified rectangle's coordinates:

This concludes the implementation of the `wrap_digit` helper function.

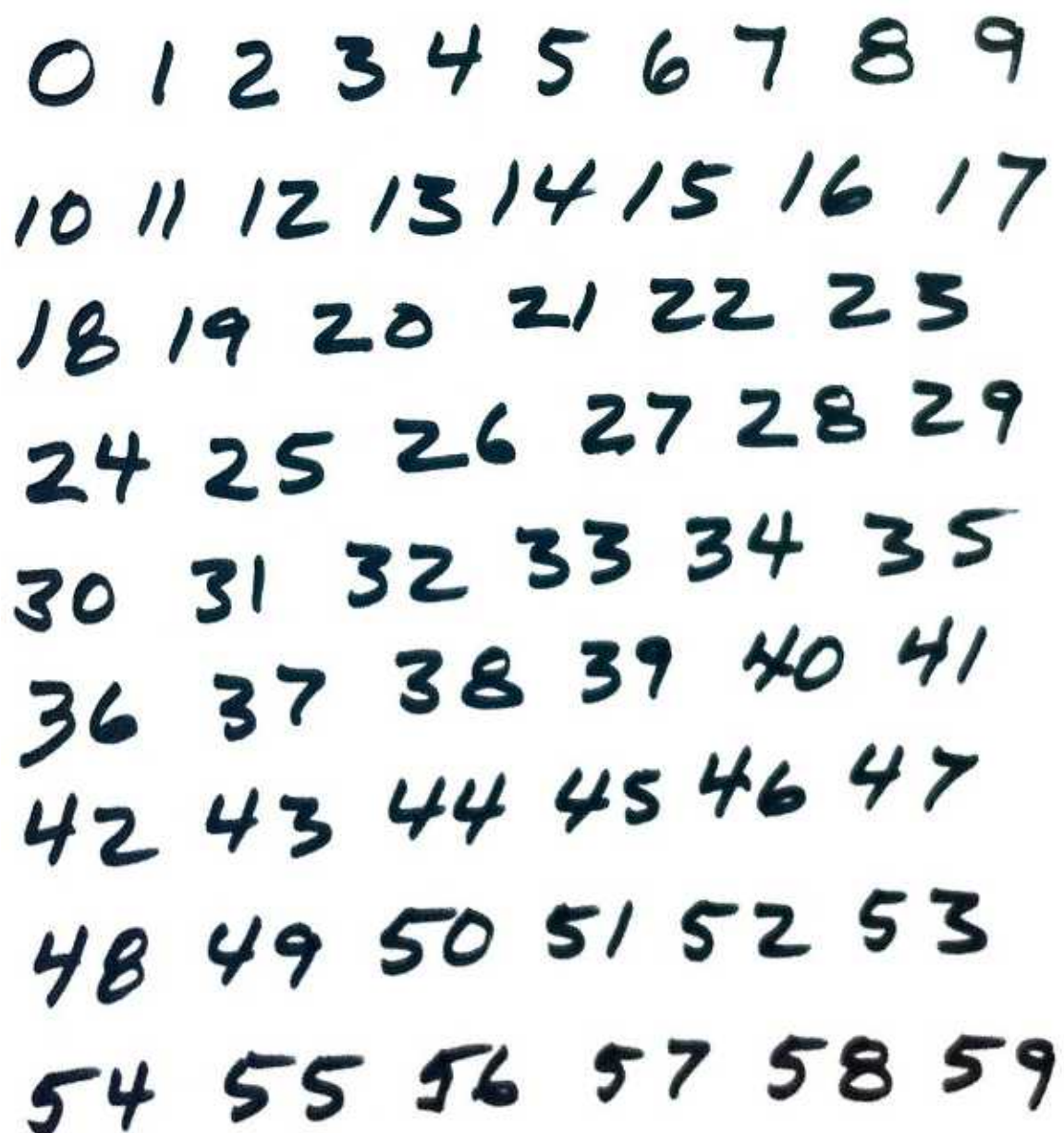
Now, let's proceed to the main part of the program. Here, we start by creating an ANN and training it on MNIST data:

Note that we are using the `create_ann` and `train` functions from our `digits_ann` module. As we mentioned earlier (in the *Choosing parameters for the MNIST database* section), we are using 60 hidden nodes, 50,000 training

samples, and 10 epochs. Although these are the default parameter values for our functions, we specify them here anyway so that they are easier to see and modify later, in case we want to experiment with other values.

Now, let's load a test image that contains many handwritten digits on a white sheet of paper:

We are using the following image of Joe Minichino's handwriting (but, of course, you could substitute another image if you prefer):



A handwritten list of digits from 0 to 59, arranged in eight rows. The digits are written in a casual, cursive style on a white background. The first row contains digits 0 through 9. The second row contains 10 through 17. The third row contains 18 through 23. The fourth row contains 24 through 29. The fifth row contains 30 through 35. The sixth row contains 36 through 41. The seventh row contains 42 through 47. The eighth row contains 48 through 53. The final row contains 54 through 59.

0	1	2	3	4	5	6	7	8	9
10	11	12	13	14	15	16	17		
18	19	20	21	22	23				
24	25	26	27	28	29				
30	31	32	33	34	35				
36	37	38	39	40	41				
42	43	44	45	46	47				
48	49	50	51	52	53				
54	55	56	57	58	59				

Let's convert the image into grayscale and blur it in order to remove noise and make the darkness of the ink more uniform:

Now that we have a smoothened grayscale image, we can apply a threshold and some morphology operations to ensure that the numbers stand out from the background and that the contours are relatively free of irregularities, which might throw off the prediction. Here is the relevant code:

Note the threshold flag, `cv2.THRESH_BINARY_INV` , which is for an inverse binary threshold. Since the samples in the MNIST database are white on black (and not black on white), we turn the image into a black background with white numbers. We use the thresholded image for both detection and classification.

After the morphology operation, we need to separately detect each digit in the picture. As a step toward this, first, we need to find the contours:

Then, we iterate through the contours and find their bounding rectangles. We discard any rectangles that we deem too large or too small to be digits. We also discard any rectangles that are entirely contained in other rectangles. The remaining rectangles are appended to a list of good rectangles, which (we believe) contain individual digits. Let's look at the following code snippet:

Now that we have a list of good rectangles, we can iterate through them, sanitize them using our `wrap_digit` function, and classify the image data inside them:

Moreover, after classifying each digit, we draw the sanitized bounding rectangle and the classification result:

After processing all the regions of interest, we save the thresholded image and the fully annotated image and display them until the user hits any key to end the program:

That is the end of the script. When running it, we should see the thresholded image as well as a visualization of detection and classification results. (The two windows may overlap initially, so you might need to move one to see the other.) Here is the thresholded image:

0 1 2 3 4 5 6 7 8 9
10 11 12 13 14 15 16 17
18 19 20 21 22 23
24 25 26 27 28 29
30 31 32 33 34 35
36 37 38 39 40 41
42 43 44 45 46 47
48 49 50 51 52 53
54 55 56 57 58 59

Here is the visualization of the results:



This image contains 110 sample digits: 10 digits in the single-digit numbers from 0 to 9, plus 100 digits in the double-digit numbers from 10 to 59. Out of these 110 samples, the bounds are correctly detected for 108 samples, meaning that the detector's accuracy is 98.18%. Then, out of these 108 correctly detected samples, the classification result is correct for 80 samples, meaning that the ANN classifier's accuracy is 74.07%. This is a lot better than a random classifier, which would correctly classify a digit only 10% of the time.

Thus, the ANN is evidently capable of learning to classify handwritten digits in general, not just the ones in the MNIST training and test datasets. Let's consider

some ways to improve its learning.

Trying to improve the ANN's training

We could apply a number of potential improvements to the problem of training our ANN. We have already mentioned some of these potential improvements, but let's review them here:

You could experiment with the size of your training dataset, the number of hidden nodes, and the number of epochs until you find a peak level of accuracy.

You could modify our `digits_ann.create_ann` function so that it supports more than one hidden layer.

You could also try different activation functions. We have used `cv2.ml.ANN_MLP_SIGMOID_SYM` , but it isn't the only option; the others include `cv2.ml.ANN_MLP_IDENTITY` , `cv2.ml.ANN_MLP_GAUSSIAN` , `cv2.ml.ANN_MLP_RELU` , and `cv2.ml.ANN_MLP_LEAKYRELU` .

Similarly, you could try different training methods. We have used `cv2.ml.ANN_MLP_BACKPROP` . The other options include `cv2.ml.ANN_MLP_RPROP` and `cv2.ml.ANN_MLP_ANNEAL` .

For more information about ANN-related parameters in OpenCV, refer to the official documentation at

https://docs.opencv.org/master/d0/dce/classcv_1_1ml_1_1ANN__MLP.html .

Aside from experimenting with parameters, think carefully about your application requirements. For example, where and by whom will your classifier be used? Not everyone draws digits the same way. Indeed, people in different countries tend to draw numbers in slightly different ways.

The MNIST database was compiled in the United States, where the digit 7 is handwritten like the typewritten character 7. However, in Europe, the digit 7 is often handwritten with a small horizontal line halfway through the diagonal portion of the number. This stroke was introduced to help distinguish the handwritten digit 7 from the handwritten digit 1.

For a more detailed overview of regional handwriting variations, check the Wikipedia article on the subject, which is a good introduction, available at

https://en.wikipedia.org/wiki/Regional_handwriting_variation.

This variation means that an ANN trained on the MNIST database may be less accurate when applied to the classification of European handwritten digits. To avoid such an outcome, you may choose to create your own training dataset instead. In almost all circumstances, it is preferable to utilize training data that belongs to the current application domain.

Finally, remember that once you are happy with the accuracy of your classifier, you can always save it and reload it later so that it can be utilized in applications without having to train the ANN every time.

The interface for this is just like the interface we saw in the *Saving and loading a trained SVM* section, near the end of *Chapter 7 , Building Custom Object Detectors* . Specifically, you can use code such as the following to save a trained ANN to an XML file:

Subsequently, you can reload the trained ANN using code such as the following:

Now that we have learned how to create a reusable ANN for handwritten digit classification, let's think about the use cases for such a classifier.

Finding other potential applications

The preceding demonstration is only the foundation of a handwriting recognition application. You could readily extend the approach to videos and detect handwritten digits in real-time, or you could train your ANN to recognize the entire alphabet for a full-blown **optical character recognition (OCR)** system.

Detection and recognition of car registration plates would be another useful extension of the lessons we have learned up to this point. The characters on registration plates have a consistent appearance (at least, within a given country), and this should be a simplifying factor in the OCR part of the problem.

You could also try applying ANNs to problems where we have previously used SVMs, or vice versa. This way, you could see how their accuracy compares for different kinds of data. Recall that in *Chapter 7 , Building Custom Object Detectors* , we used SIFT descriptors as inputs for SVMs. Likewise, ANNs are capable of handling high-level descriptors and not just plain old pixel data.

As we have seen, the `cv2.ml_ANN_MLP` class is quite versatile, but in truth, it covers only a small subset of the ways an ANN can be designed. Next, we will learn about OpenCV's support for more complex **deep neural networks (DNNs)** that can be trained with a variety of other frameworks.

Using DNNs from other frameworks in OpenCV

OpenCV can load and use DNNs that have been trained in any of the following frameworks:

- Caffe (<http://caffe.berkeleyvision.org/>)
- TensorFlow (<https://www.tensorflow.org/>)
- Torch (<http://torch.ch/>)
- Darknet (<https://pjreddie.com/darknet/>)
- ONNX (<https://onnx.ai/>)
- DLDT (<https://github.com/opencv/dldt/>)

*The **Deep Learning Deployment Toolkit (DLDT)** is part of Intel's OpenVINO Toolkit (<https://software.intel.com/openvino-toolkit/>) for computer vision. DLDT provides tools for optimizing DNNs from other frameworks and for converting them into a common format. A collection of DLDT-compatible models is freely available in a repository called the Open Model Zoo (https://github.com/opencv/open_model_zoo/). DLDT, the Open Model Zoo, and OpenCV have some of the same people on their development teams; all three of these projects are sponsored by Intel.*

These frameworks use various file formats to store trained DNNs. Several of these frameworks use a combination of a pair of file formats: a text file to describe the model's parameters, plus a binary file to store the model itself. The following code snippet shows the file types and OpenCV functions that are relevant to loading a model from each framework:

After we load a model, we need to preprocess the data we will use with the model. The necessary preprocessing is specific to the way the given DNN was designed and trained, so any time we use a third-party DNN, we must read about how that DNN was designed and trained. OpenCV provides a function, `cv2.dnn.blobFromImage` , that can perform some common preprocessing steps, depending on the parameters we pass to it. We can

perform other preprocessing steps manually before passing data to this function.

*A neural network's input vector is sometimes called a **tensor** or **blob** – hence the function's name, `cv2.dnn.blobFromImage` .*

Let's proceed to a practical example where we'll see a third-party DNN in action.

Detecting and classifying objects with third-party DNNs

For this demo, we are going to capture frames from a webcam in real-time and use a DNN to detect and classify 20 kinds of objects that may be in any given frame. Yes, a single DNN can do all this in real-time on a typical laptop that a programmer might use!

Before delving into the code, let's introduce the DNN that we will use. It is a Caffe version of a model called MobileNet-SSD, which uses a hybrid of a framework from Google called **MobileNet** and another framework called **Single Shot Detector (SSD)** MultiBox. The latter framework has a GitHub repository at <https://github.com/weiliu89/caffe/tree/ssd/> . The training technique for the Caffe version of MobileNet-SSD is provided by a project on GitHub at <https://github.com/chuanqi305/MobileNet-SSD/> . Copies of the following MobileNet-SSD files can be found in this book's repository, in the `chapter10/objects_data` folder:

- `MobileNetSSD_deploy.caffemodel`: This is the model.
- `MobileNetSSD_deploy.prototxt`: This is the text file that describes the model's parameters.

This model's capabilities and proper usage will soon become clear as we progress through our sample code:

As usual, we begin by importing OpenCV and NumPy:

We proceed to load the Caffe model with OpenCV in the same manner that we described in the previous section:

We need to define some preprocessing parameters that are specific to this model. It expects the input image to be 300 pixels high. Also, it expects the pixel values in the image to be on a scale from -1.0 to 1.0. This means that, relative to the usual scale from 0 to 255, it is necessary to subtract 127.5 and then divide by 127.5. We define the parameters as follows:

We also define a confidence threshold, representing the minimum confidence score that we require in order to accept a detection as a real object:

The model supports 20 classes of objects, with IDs from 1 to 20 (not 0 to 19). The labels for these classes can be defined as follows:

Later, when we use class IDs to look up labels in our list, we must remember to subtract 1 from the ID in order to obtain an index in the range 0 to 19 (not 1 to 20).

With the model and parameters at hand, we are ready to start capturing frames.

For each frame, we begin by calculating the aspect ratio. Remember that this DNN expects the input to be based on an image that is 300 pixels high; however, the width can vary in order to match the original aspect ratio. The following code snippet shows how we capture a frame and calculate the appropriate input size:

At this point, we can simply use the `cv2.dnn.blobFromImage` function, with several of its optional arguments, to perform the necessary preprocessing, including resizing the frame and converting its pixel data into a scale from -1.0 to 1.0:

We feed the resulting blob to the DNN and get the model's output:

The results are an array, in a format that is specific to the model we are using.

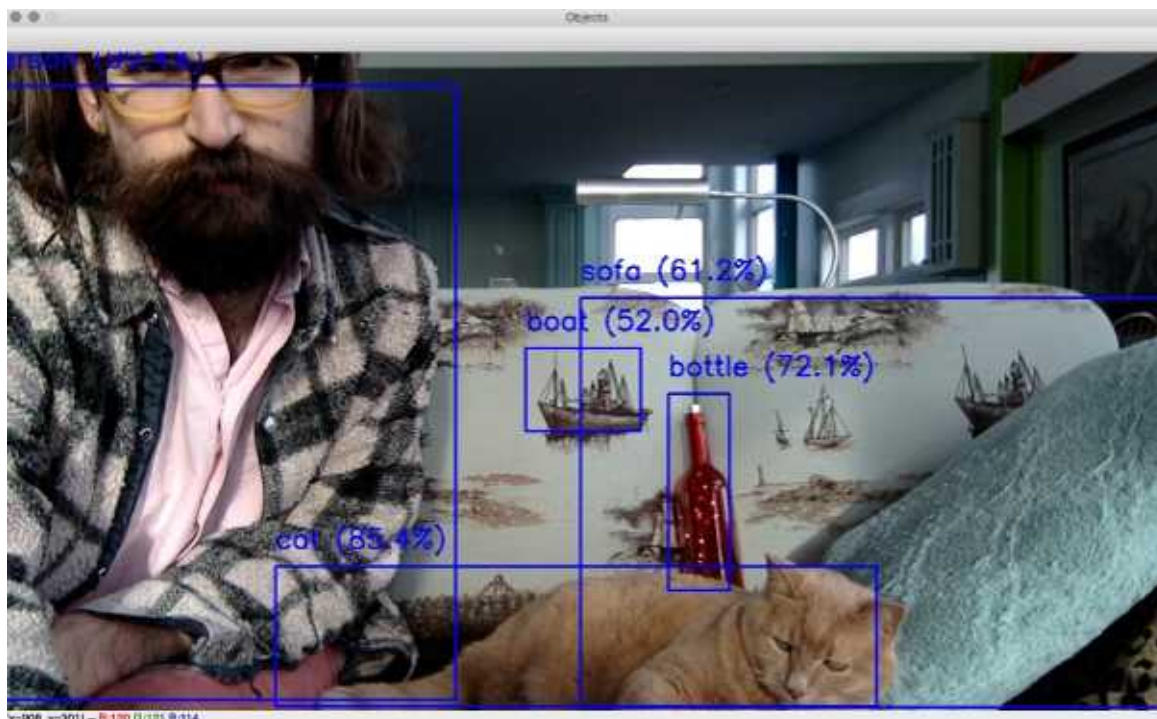
For this object detection DNN—and for other DNNs trained with the SSD framework—the results include a subarray of detected objects, each with its own confidence score, rectangle coordinates, and class ID. The following code shows how to access these, as well as how to use an ID to look up a label in the list we defined earlier:

As we iterate over the detected objects, we draw the detection rectangles, along with the classification labels and confidence scores:

The last thing we do with the frame is show it. Then, if the user has hit the *Esc* key, we exit; otherwise, we capture another frame and continue to the next iteration of the loop:

If you plug in a webcam and run the script, you should see a visualization of detection and classification results, updated in real-time. Here is a screenshot

showing Joseph Howse and Sanibel Delphinium Andromeda (a mighty, great, and righteous cat) in their living room in a Canadian fishing village:



The DNN has correctly detected and classified a human *person* (with 99.4% confidence), a *cat* (85.4%), a decorative *bottle* (72.1%), part of a *sofa* (61.2%), and a woven picture of a *boat* (52.0%). Evidently, this DNN is well equipped to classify the contents of living rooms in nautical settings!

This is only a first taste of the things that DNNs can do—and do in real time! Next, let's see what we can achieve by combining three DNNs in one application.

Detecting and classifying faces with third-party DNNs

For this demonstration, we are going to use one DNN to detect faces and two other DNNs to classify the age and gender of each detected face. Specifically, we will use pre-trained Caffe models that are stored in the following files in the `chapter10/faces_data` folder of this book's GitHub repository.

Here is an inventory of the files in this folder, and of the files' origins:

`detection/res10_300x300_ssd_iter_140000.caffemodel` : This is the DNN for face detection. The OpenCV team has provided this file at https://github.com/opencv/opencv_3rdparty/blob/dnn_samples_face_detector_20170830/res10_300x300_ssd_iter_140000.caffemodel . This Caffe model was trained with the SSD framework (<https://github.com/weiliu89/caffe/tree/ssd/>). Thus, its topology is similar to the MobileNet-SSD model that we used in the previous section's example.

`detection/deploy.prototxt` : This is the text file that describes the parameters of the preceding DNN for face detection. The OpenCV team provides this file at https://github.com/opencv/opencv/blob/master/samples/dnn/face_detector/deploy.prototxt .

The `chapter10/faces_data/age_gender_classification` folder contains the following files, which are all provided by Gil Levi and Tal Hassner in their GitHub repository (<https://github.com/GilLevi/AgeGenderDeepLearning/>) and on their project page (https://talhassner.github.io/home/publication/2015_CVPR) for their work on age and gender classification:

- `age_net.caffemodel` : This is the DNN for age classification.
- `age_net_deploy.prototxt` : This is the text file that describes the parameters of the preceding DNN for age classification.
- `gender_net.caffemodel` : This is the DNN for gender classification.
- `gender_net_deploy.prototxt` : This is the text file that describes the parameters of the preceding DNN for age classification.
- `average_face.npy` and `average_face.png` : These files represent the average faces in the classifiers' training dataset. The original file from Levi

and Hassner is called `mean.binaryproto` , but we have converted it into a NumPy-readable format and a standard image format, which are more convenient for our purposes.

Let's see how we can use all these files in our code:

To begin the sample program, we load the face detection DNN, define its parameters, and define a confidence threshold. We do this in much the same way as we did for the object detection DNN in the previous section's sample:

We do not need to define labels for this DNN because it does not perform any classification; it just predicts the coordinates of face rectangles.

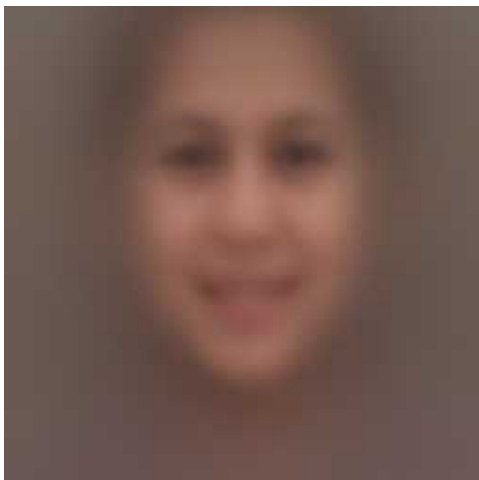
Now, let's load the age classifier and define its class labels:

Note that in this model, the age labels have gaps between them. For example, '0-2' is followed by '4-6' . Thus, if a person is actually 3 years old, the classifier has no proper label for this case; at best, it can pick either of the neighboring ranges, '0-2' or '4-6' . Presumably, the model's authors deliberately chose disconnected ranges, in an effort to ensure that the classes are **separable** with respect to the inputs. Let's consider the alternative. Based on data from facial images, is it possible to separate a group of people who are 4 years old from a group of people who are 4-years-less-a-day? Surely it isn't; they look the same. Thus, it would be wrong to formulate a classification problem based on contiguous age ranges. A DNN could be trained to predict age as a continuous variable (such as a floating-point number of years), but this would be altogether different than a classifier, which predicts confidence scores for various classes.

Now, let's load the gender classifier and define its labels:

The age and gender classifiers use the same blob size and the same average. Rather than using a single color as the average, they use an average facial image, which we will load (as a NumPy array in floating-point format) from an NPY file. Later, we will subtract this average facial image from an actual facial image before we perform classification. Here are the definitions of the blob size and average image:

If you want to see what the average face looks like, open the file at `chapter10/faces_data/age_gender_classification/average_face.png` , which contains the same data in a standard image format. Here it is:



Of course, this is only the average face for a particular training dataset; it is not necessarily representative of the true average face in the world population, or in any particular nation or community. Even so, here, we can see a face that is a blurry composite of many faces, and it contains no obvious clues about age or gender. Note that the image is square, it is centered around the tip of the nose, and it extends vertically from the top of the forehead to the base of the neck. To obtain accurate classification results, we should take care to apply this classifier to facial images that are cropped in the same manner.

Having set up our models and their parameters, let's proceed to capture and process frames from a camera. With each frame, we begin by creating a blob that is the same aspect ratio as the frame, and we feed this blob to the face detection DNN:

Like the object detector that we used in the previous section's sample, the face detector provides confidence scores and rectangle coordinates as part of its results. For each detected face, we need to check whether the confidence score is acceptably high, and, if it is, we'll get the coordinates of the face rectangle:

This face detection DNN produces rectangles that are taller than they are wide. However, the age and gender classification DNNs expect square faces. Let's widen the detected face rectangle to make it a square:

Note that if part of the square falls outside the bounds of the image, we skip this detection result and continue to the next one.

At this point, we can select the square **region of interest (ROI)**, which contains the image data that we will use for age and gender classification. We proceed by

scaling the ROI to the classifiers' blob size, converting it into floating-point format, and subtracting the average face. From the resulting scaled and normalized face, we create the blob:

We feed the blob to the age classifier, pick the class ID with the highest confidence score, and then take note of the label and confidence score for this ID:

Similarly, we classify the gender:

We draw a visualization of the detected face rectangle, the expanded square ROI, and the classification results:

To conclude, we show the annotated frame, and we keep capturing more frames until the user hits the *Esc* key:

What does this program report about Joseph Howse? Let's take a look:



Without vanity, Joseph Howse is going to write a couple of paragraphs about this result.

First, let's consider the detection of the face and the selection of the ROI. The face has been accurately detected. The ROI has been correctly expanded to a square

region that includes the neck—or, in this case, the full beard, which could be an important region for the purposes of classifying age and gender.

Second, let's consider the classification. The truth is that Joseph Howse is male and is approximately 35.8 years old at the time of this picture. Other human beings who see Joseph Howse's face are able to judge with perfect confidence that he is male; however, their estimates of his age vary widely. The gender classification DNN says with perfect confidence (100.0%) that Joseph Howse is male. The age classification DNN says with high confidence (96.6%) that he is 25-32 years old. Perhaps it is tempting to take the midpoint of this range, 28.5, and say that the prediction has an error of -7.3 years, which is subjectively a big underestimate, being -20.4% of the true age. However, this type of assessment is a stretch of the prediction's meaning.

Remember that this DNN is an age classifier, not a predictor of continuous age values, and that the DNN's age classes are labeled as disconnected ranges; the next one after '25-32' is '38-43'. Thus, the model has a gap around Joseph Howse's true age, but at least it managed to choose one of the two classes that border this gap.

This demonstration concludes our introductory tour of ANNs and DNNs. Let's review what we have learned and done.

Using OpenCV with MediaPipe and Tensorflow to classify gestures and actions

In the previous sections of the chapter you have familiarized with the idea of Artificial Neural Networks, and how they can be used in a computer vision context, coupled with OpenCV.

This wonderful match of Computer Vision and ML/AI, coupled with the exponential increase in computing power, enabled engineers to resolve really difficult challenges, such as general object detection and recognition, tracking etc.

Like every other area of software engineering, at first some tasks are difficult to accomplish, then they become run of the mill, then a plethora of libraries and frameworks becomes available in many programming languages, often hiding the underlying logic.

In many respects, it's like driving a car: you don't have to know how an engine works to drive a car, and if all you want to do is to drive the car from point A to point B respecting limits and rules of the road, then knowing how an engine works will add nothing to your experience.

OpenCV is your car for computer vision tasks: you don't necessarily need to know how OpenCV detects shapes to be able to do so, you just need to know OpenCV's API.

When it comes to detecting poses, faces, hands and gestures, we have another car, it is called MediaPipe! I could not be more relieved that a library so efficient and so easy to use exists: I don't know about you but I really do not like re-inventing the wheel, I'd rather take a library and use it to achieve goals that the library alone cannot achieve for me.

Let's take a look at how we can use MediaPipe, OpenCV and Tensorflow to build a ML classifier that will recognize a number of hand gestures such as an open hand, an "OK" sign, a "peace" sign and a "thumb up" sign.

What is MediaPipe

MediaPipe is a library developed by Google that makes the detection of body poses, faces, hands, iris and lots more (to do with a human body) a trivial task. It is highly optimized and performs really well even on commodity hardware like normal development machines.

MediaPipe's website is available at this address:
<https://google.github.io/mediapipe/>

MediaPipe does a lot more than simply detect body parts so I encourage you to check it out and learn how to integrate it in your programs.

Installing MediaPipe is as easy as “pip install mediapipe”

TensorFlow

TensorFlow should be familiar to you as one of the available backends for OpenCV's DNN module. If you're not familiar with it, and specifically with Keras, feel free to visit <https://keras.io/about/> where you can learn all about it. Keras is a user-friendly API running on top of TensorFlow which makes the building of deep learning models simple and easy.

Installing TensorFlow is also as easy as “pip install tensorflow”.

Hand Recognition

Hand recognition is a task made easy by MediaPipe, let's take a look at how the documentation shows that you can perform it using webcam input:

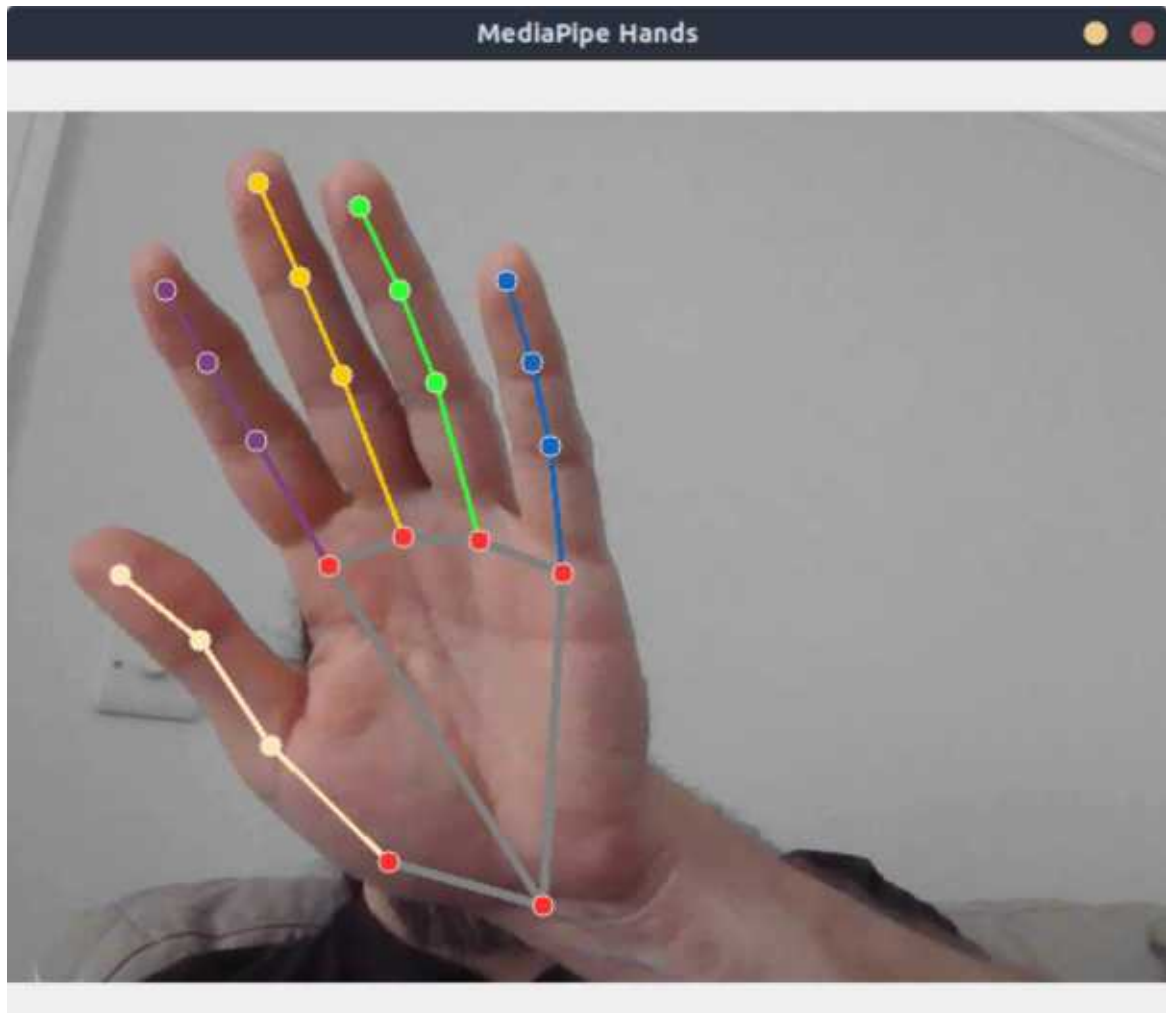
That's for the entire code, but if we strip some of the boilerplate and the loop to continuously display the processed input, there are very few lines of real importance (aside for the necessary imports at the beginning):

These lines read the input and create the hand detector, then

We perform hand detection and lastly

We draw the landmarks.

If you run the full code above, you should get an output similar to this



Hand Gesture Recognition

Now let's take the precious information that MediaPipe generates for us and use it to build a gesture classifier.

Here is the high-level overview of the process, which should follow a very common pattern:

- Generate labeled training data
- Build a model
- Train the model
- Test it

The two biggest challenges are the generation of data (it's a custom gesture recognition classifier, we presume such data does not exist at all) and creating a deep learning model that's accurate enough (upwards of 98%).

Generating Labeled Training Data

Let's tackle the first challenge by creating data. I am going to offer my approach to this but clearly you can modify it to suit your needs.

The approach I offer is as follows: start capturing video input, and for as long as you keep pressing one number key (0 to 3), the landmark coordinates (normalized) will be stored in a csv file with a label corresponding to the number key you are pressing.

To increase the model accuracy, make sure to move the hand around, tilt it and turn it, all while keeping the gesture "intact" so to speak. Even more importantly, don't break the gesture or you will be polluting the training dataset and introducing noise. Lastly, make sure to be consistent with the gestures and keys, don't mix number keys and gestures up.

The algorithm looks like this:

So let's take a look at the training data generation program:

Now let's take a closer look at the more important parts.

The first crucial part is the introduction of a "number" variable that allows us to store the class (gesture) we are capturing, so we use the familiar "waitKey" function to do that, and since the keys 0 to 9 have code 48 to 56, we subtract "48" to obtain the actual number key that we pressed:

I tailored the script to train only 4 classes, so I spare the program from performing any hand detection unless we are generating training data, which means that the number variable needs to be a value between 0 and 3, like so:

You can edit the value in the array to your use case but I would still put that condition so to ensure you only store data when you have the correct hand position being captured.

Now for the beefiest parts of our code. We create a module called "landmark_utils" where we store two functions that will help us correct,

normalize and store the landmarks coordinates.

This function creates an array of landmark points, and populates it by taking the values of the landmark and ensuring that the landmark coordinate is within the boundaries of the image.

This is one way of doing it, for even more accurate data you could potentially discard any hand detection whose landmark points lie outside the frame.

The second function takes the landmark points (which are stored as two-element arrays symbolizing x and y), re-calculates the x and y values as coordinates relative to the first landmark coordinates, and then normalizes the values.

You should be very familiar with the notion of data normalization at this point, but if you are wondering why we are doing it, it's because a hand closer to the camera appears bigger than one that is further away, and normalizing values means storing values that keep proportion into consideration but ignore size.

We also squash the list into a one-dimensional array, so it can be stored in a csv file.

The next function is the csv logging:

Nothing too special here. If the number is not in the desired range, we don't store anything, otherwise we simply store the flat array that we created in the previous utility function, putting the label (the number) as the first element in the row.

When we go to train the model we will separate this first column from the rest of the data.

Great, we are ready to give it a go! Run "python train.py", put your hand in front of the camera in the gesture you want to generate data for, press the key you are going to associate to that gesture and move the hand around. Each frame in which a key is pressed and a hand is detected will generate a row in the csv training file.

If everything worked as planned you should have a file in the folder model/keypoint_classifier called keypoint.csv with a content similar to this:

Granted, it doesn't look very informative, but you will notice that there are 43 fields (42 commas). Why 43? So close to the "42" of "Hitchhiker's Guide to the

Galaxy” fame (and for that reason often used as a value for random seed in machine learning models).

No, unfortunately Adams’s novel series has nothing to do with it. The reason why there are 43 entries is actually quite simple:

1. The first field is the label
2. MediaPipe detects 21 landmarks (4 per finger/thumb + 1 for the base of the palm), each of which has an x value and a y value, making it 42 values.

If your generated csv looks like this, you are on a roll and you can move on to training the model and then testing it.

Building and Training the Classifier with Jupyter Notebooks

Now let’s build our classifier. As we’ve seen for the handwritten digits, a deep learning model will do perfectly well in this case.

We know the following:

1. Each hand is represented by 42 features
2. We are classifying 4 gestures

So the input layer will have 42 nodes, the output 4. We will put another 2 layers in between, one with 20 and one with 10 nodes, but let’s not rush ahead and go through the entire notebook step by step.

First we import the necessary libraries and functions:

The first one is the path to the training data, the second is the path to where the hdf5 model will be saved and the third where the tflite model will be saved.

What is tflite? Tensorflow Lite is a framework that allows you to run deep learning models on devices and hardware that may be constrained in terms of performance, like mobile or the Raspberry Pi. The idea here is to train a TensorFlow model, save it, then convert it to tflite for a smaller yet fast deep learning model that can be deployed anywhere.

Next we set the constant NUM_CLASSES

Which specifies the number of gestures we are recognizing.

Now we load the training data. Note that in the “usecols” option of “loadtxt” we skip the first column (which would have index 0), because the first column is the labels.

Then we operate the classic X, y / train,test split so the model can test accuracy on part of the data that has not been used for training.

The Deep Learning model

Here’s our Keras model:

Nothing too exotic here: we have a Dense input layer with 42 nodes, and we gradually reduce the hidden layers (20 and 10) interspersed with Dropout layers to help reduce overfitting.

Next, we set up checkpoints and early stopping:

Once the model has finished training (which - depending on the size of the training data you accumulated - might take some time), we evaluate its accuracy:

The we save the model:

The notebook code also contains a cell for the display of a confusion matrix if you are interested in it, but that’s not particularly relevant here.

Finally, we convert the model to tflite:

And that’s it, your model is ready for testing.

Testing the model on camera input

All that’s left to do now is to capture some video input, create a classifier that uses the model we just saved and do real-time predictions on any hand detected by MediaPipe.

For this we need, first of all, to create a classifier python object. We do this in the model/keypoint_classifier folder, in a file called keypoint_classifier.py, and here is the code for it:

Although some of the lines in this code may look a bit obscure, it really does not do much other than loading the model file and specifying how to invoke the

prediction.

If you are unsure, please consult the documentation for the TFLite interpreter at https://www.tensorflow.org/api_docs/python/tf/lite/Interpreter

Of particular note are the constructor (which specifies which file to use to load the model), the “allocate_tensors()” call which is required for execution, the “set_tensor” call which loads the features for prediction, the “invoke” call which performs the prediction and the “get_tensor” call which retrieves the result of the last prediction.

Now that we have a classifier, we only need to include it into our normally video capture loop, which I have done like so:

In the first line we create a classifier, then we create a dictionary of gestures for display purposes.

After that, all we do is to detect hands, flatten the landmarks into a one-dimensional 42-size array and pass it into our classifier, and use the detected class index to print the gesture’s name in the top left corner of the screen.

If all goes well, you will have very satisfactory results like so:



Summary

This chapter scratched the surface of the vast and fascinating world of ANNs. We learned about the structure of ANNs, and how to design a network topology based on application requirements. Then, we focused on OpenCV's implementation of MLP ANNs, as well as on OpenCV's support for diverse DNNs that have been trained in other frameworks.

We applied neural networks to real-world problems: notably, handwritten digit recognition; object detection and classification; and a combination of face detection, age classification, and gender classification in real time. We saw that even in these introductory demos, neural networks show a lot of promise in terms of versatility, accuracy, and speed. Hopefully, this encourages you to try out pre-trained models from various authors, and to learn to train advanced models of your own in various frameworks. Finally, we have learned how to use MediaPipe for hand detection and trained a classifier with TensorFlow, and utilized OpenCV for video capture and image manipulation.

I hope you enjoyed it and that you have learned useful techniques that you can apply in real-life scenarios or your personal projects!

Join our book community on Discord

<https://discord.gg/djGjeMECxx>



In this chapter we will take OpenCV to the cloud, by exploring technologies and methodologies that will allow you to package your application and deploy it on AWS so it can be made available to the public even at large scale, without having to learn lengthy and complex technologies to maintain such applications.

We will provide a brief introduction to containers and containerization tools like Docker, an overview of the AWS cloud provider and AWS Lambda, and finally we will demonstrate how to develop a simple application and deploy it.

NOTE: even if you don't want to use AWS, the knowledge you will acquire about containerization should seamlessly transfer to other cloud providers or cloud-agnostic solutions such as Kubernetes (which can be deployed on any cloud).

IMPORTANT NOTE: to follow the examples in this chapter you will need an AWS account. While you have to provide credit card credentials, if you follow the examples in this chapter you will only incur in charges when you

exceed free-tier quotas, something you would normally only do if you are working with revenue-generating levels of loads.

What are Containers?

We mentioned containers and Docker. Let's see what they are and why they are so important in our day and age.

First of all, containers are a form of virtualization, that is, making a single physical machine run on or more “machines”, which are called virtual.

But while Virtual Machines (or VMs) are an abstraction of the hardware layer and simulate the entire operating system, in fact VM images are huge, in the order of GBs.

Containers however, are an abstraction at the application layer, and container images can be as small as tens of MBs, and they package code and dependencies into a single image that can be deployed in several ways.

The most popular technology for container deployment and orchestration is Kubernetes, and in AWS you can do this with a Kubernetes equivalent (Elastic Kubernetes Service or EKS), or three separate tools called ECS (Elastic Container Service), Fargate and AWS Lambda.

We will limit ourselves to the simplest and easiest (from a management point of view) of these tools: AWS Lambda, but all in good time.

Traditional VMs can be very slow to boot, and because they are so cumbersome, not very agile to work with. It is no wonder they are only in use in specific use cases (and mainly to do with security), but other than niche use cases, they have been supplanted by containers.

The main advantage of containers from a software development cycle point of view is portability.

This means that you can develop an application, package it into a container for local testing and use in your local development environment and then deploy as it is, and it will work because the abstraction is operated by the Docker Engine, not the underlying OS.

It is quite literally that simple. Some say containers resolved the age old rebuttal

“it works on my machine”

developers often blurt when things don’t seem to work as expected. If a container works on your machine and not in a production environment there are a few and very precise reasons for it and it will not be due to complex and annoying dependency issues. Which is where the “12factor app” deserves a brief mention.

The Twelve-Factor App

There is a website called <https://12factor.net/> that illustrates a simple methodology that is perfectly suited to the delivery of an application as a web app or as a “Software as a Service (SaaS)”.

If you are reading this chapter (and not happily skipping) then you either want to deploy your OpenCV app in the cloud or you intend to acquire the knowledge to do so, therefore you might as well learn a tried-and-tested methodology that will greatly simplify your life especially when it comes to preventing problems from happening at all, and then resolving them in the easiest and quickest way possible.

Let’s take a quick look at the factors as they are illustrated on the website and see how they fit in our container-based work.

1. Codebase

One codebase tracked in revision control, many deploys

1. Dependencies

Explicitly declare and isolate dependencies

1. Config

Store config in the environment

1. Backing services

Treat backing services as attached resources

1. Build, release, run

Strictly separate build and run stages

1. Processes

Execute the app as one or more stateless processes

1. Port binding

Export services via port binding

1. Concurrency

Scale out via the process model

1. Disposability

Maximize robustness with fast startup and graceful shutdown

1. Dev/prod parity

Keep development, staging, and production as similar as possible

1. Logs

Treat logs as event streams

1. Admin processes

Run admin/management tasks as one-off processes

Let's focus on a few that stand out as particularly relevant, or that containers help making easier:

1. Dependencies: you package the dependencies of your code in your container, and you have to declare such dependencies in your container build file (called a Dockerfile) so they are downloaded and installed before your code can be built in the container itself. This achieves total isolation and explicit declaration.
2. Config: you declare configuration parameters in container-based development in environment variables. Things like database string connections, stage, names of resources etc. are all declared as environment variables.
3. Disposability: containers are so light they are destroyed and recreated rather than moved.
4. Dev/prod parity: the fact that your container image deployed locally for testing only differs from the production image only by the environment variables injected into it means you have complete dev/prod parity which is incredibly important.

Docker Basics

We talked enough about containers and Docker, let's move onto an example so the whole idea becomes clearer for those unfamiliar with containerization technologies.

NOTE: Our overview of Docker will be minimal because it is far removed from the scope of this tutorial, which is not to teach you docker but to teach how to deploy OpenCV applications to scale. We will explore basic concepts and refer you to the Docker documentation available at <https://docs.docker.com/> for anything else you need to know.

First let's see how to install the Docker engine. The official page on the docker website where you can obtain downloads is at <https://docs.docker.com/engine/install/>

Docker engine used to be a CLI-only tool that lacked a bit in user friendliness. Nowadays what you download from Docker is a tool called Docker-Desktop which also includes the CLI client that Docker old-timers and CLI aficionados are used to (and probably prefer to use).

Throughout the chapter we will use the CLI, not just because I'm used to it, but because the overwhelming majority of support material including documentation and StackOverflow questions will be using the CLI, so it's simply the right thing to do. When all is said and done, the CLI is the more difficult tool to learn and the GUI the intuitive, easy one, so if we learn the difficult parts I'm sure you will be flying with the easy ones.

Docker Version

So go ahead and install docker-desktop, you should then be able to verify your docker client is installed from a terminal window:

Docker Run

Docker run allows you to launch a container. If you have never done so, try the obligatory "hello-world":

This will test that the installation is working correctly and will produce the following output

The output itself describes step by step what the command did, which in short meant contacting the local daemon, failing to find the “image” called hello-world, therefore downloading it from Docker Hub (a publicly available repository of container images), then creating the container and running it, and printing the output to your terminal.

Clearly the container itself does not do much, but it was enough to showcase the workflow adopted by the docker client.

So the first subcommand we learned is “run”, which is what we use to launch a container. If you “run” a container it will immediately execute a command that you specified in the build file for the container (Dockerfile), which we will explore soon enough.

Docker Pull

Next up, we want to actually do something with containers, so we want to download an image that will allow you to write and execute a Python program, which is a prerequisite of any Python OpenCV application.

To do so, try the following:

Which will produce an output similar to the following:



```
joe@chapsdescends:~$ docker pull python:3.9
3.9: Pulling from library/python
1339eaac5b67: Downloading [=====>] 25.83MB/55.01MB
4c78fa1b9799: Download complete
14f0d2bd5243: Download complete
76e5964a957d: Downloading [=====>] 31.78MB/54.58MB
cc4bb1a04a94: Downloading [=====>] 35.51MB/196.8MB
3dee34a94cb0: Waiting
14451d1a3feb: Pulling fs layer
b39eb9ce1023: Pulling fs layer
e90c569049a2: Waiting
```

This is deceptively important to understand: containers leverage “file system layers” which allows caching of parts of the image so when you apply changes to it (and go to upload the image to a remote repository) you don’t have to re-upload the entire image but only the parts that changed. If the standard image has something like 9 layers, a simple code change (and no dependency changes) may

involve only 1 or 2 layer updates, making your data transfer lighter and faster, and future downloads quicker too.

This python:3.9 image that you downloaded does not do anything, but you can launch it as a service and then log into it, much like “SSH’ing” into a remote machine, but this time without the hassle of authentication since it’s running on your development machine. We are going to see how to do that but before we can do this, we need to learn about Dockerfile.

Dockerfile

Dockerfile is the docker equivalent of a Makefile, and it is probably not a coincidence they sound so similar.

It is a build file that specifies:

1. FROM what base image will your container be derived
2. What files to COPY or volumes to mount on your image
3. What preliminary operations to RUN, such as downloading dependencies and building an artifact
4. What network ports to EXPOSE
5. What command (CMD) to run once everything is ready

Notice where I put the words in capital letters. That’s not a coincidence: I used Dockerfiles directives to explain the steps involved in creating a container image, and they follow a precise order. There are a lot more directives in Dockerfile but these will get us going.

Now let’s create an extremely simple “hello world”-type Python application and then containerize it by writing a Dockerfile and then using Docker “build”.

Let’s write a main.py file with the following code:

If you run it on your machine it should output something like

(OpenCV version may vary of course depending on your installation).

Now, if we run the base image “python3:9” in docker, but with the option to make the container interactive and allocate a terminal, we will notice that we are sent straight into the python shell:

That “-it” is a double flag, where “i” stands for “--interactive” and “t” for “AllocateTTY”, in other words it allows you to interact with the container through a terminal.

If you then run, in another terminal window, the following command:

You will notice that the container’s “COMMAND” is “python3”, which adds up to the behavior we just witnessed where launching the container and interacting with it brought us into the python shell.

So what we want to do now is replace the “python3” command with the execution of our little script.

To do so we need to perform a few operations:

1. Create a requirements.txt file for our script
2. Write a Dockerfile that tells Docker to build a container image which contains a copy of main.py and requirements.txt and installs the dependencies in requirements.txt
3. Instruct the Dockerfile to run “python3 main.py” rather than just “python3”
4. Build the image and run it to test it works

NOTE: you need to create the requirements.txt file. I normally use a handy tool called pipreqs, you can use pip freeze or any other tool that allows you to create a requirements.txt file which is necessary to install dependencies in the container.

Let’s jump straight into Dockerfile to check how it matches with the above description of the Dockerfile directives and the order of steps to be taken during the image build process.

The above Dockerfile substantially says “create a container image that contains the main.py and requirements.txt files from the current directory, run pip install and then run the program main.py”.

To actually build the container we use docker build. The minimal version of docker build is extremely simple and you only need to specify two elements

1. The tag of the container (basically its name, for future reference)
2. The path to the Dockerfile (normally “.” to represent the current directory)

Let’s go ahead and do that

As you can see I called the container “opencv4.6-python3.9:latest”.

Now we can run it with docker run:

That’s terrible, we have a missing dependency. Not to worry this is all very much calculated to give you a complete perspective of the Docker workflow. In reality, when installing OpenCV through pip, there are a lot of dependencies already satisfied by your local operating system. This does not happen in a container and all dependencies need to be installed from scratch. So we’d have to resort to the lengthy version of installing OpenCV, as if you just opened your laptop for the first time and wanted to get going with OpenCV.

Fortunately, the Docker community is wide and resourceful, and some kind and generous individuals have already done this work for you. In my case I used the Dockerfile at <https://github.com/janza/docker-python3-opencv/blob/master/Dockerfile>

I downloaded it on my machine in a separate location then ran

To ensure you are able to download this image I also created an updated version of it, which you can pull with the docker client using:

This will ensure that if the original maintainer removes it, you will still be able to find it under my account.

The above docker build allowed me to create a local image with the name specified after the -t flag. And this is where we use some Docker magic. In our original Dockerfile (the one that failed to build our app) we can now replace the base python3:9 image with our shiny new OpenCV one

Now, when we go to run our previously failing command, we get a successful output:

Docker Summary

Now that we possess the basics for developing with OpenCV, we can go to create an application that will leverage container technology and deploy it with some simple steps.

AWS Serverless (Fargate, Lambda) and AWS SAM

As mentioned, the reason why we are focusing on Lambda functions is their serverless nature.

Serverless clearly does not mean “without servers”, more “without the need of provisioning and managing servers”. Which, if you ask me, is a great selling point for all computer vision developers out there who want to put their code to production in a web context without having to learn more than simple basics of web development (really, just how to handle a web request).

This idea of developing applications without having to provision the underlying resources isn't new, but the flexibility and stability provided by containers pushes this aspect of development and deployment into another level.

Moreover, even in existing products that encapsulate your application in an environment which makes deployment easy, scalability remains an issue.

AWS Fargate and AWS Lambda instead are resolving exactly this problem. They will take care of scalability for you, and if you reach a global scale you can easily configure your lambda deployment to be geographically distributed for faster response time to your users.

Let's take a look at these technologies individually and understand why they are a big deal.

AWS Lambda

Think about a small OpenCV application, such as one that detects the coordinates of faces in a picture. This is a few lines of code in python, along these lines:

This is so simple it doesn't even draw the rectangles around the faces like we've been doing in the whole book, it simply gives the x and y coordinates of the top left corner of the face, its width and its height.

If you were to deploy this as a web application, you would need to provision servers, configure them, provision and configure ancillary services such as web servers, load balancers etc that's before we talk about the boilerplate code necessary to finally be able to wrap your 6 lines of code into an endpoint that can return such simple information.

This is precisely what Lambda resolves. Lambda takes its name from Lambda functions, inline functions that take an input and produce an output, because AWS wants you to think about your code exactly like that: a self-contained, isolated, piece of code that can be deployed as is and executed on demand.

You, as a computer vision engineer, need only focus on the part you love (the computer vision coding) and AWS will take care of the rest.

To learn how to deploy a simple piece of code in a Lambda, let's take a look at Serverless Application Model (SAM).

AWS SAM

If Lambda resolves the problem of being free of provisioning and configuring servers to deploy your applications, SAM makes it a piece of cake.

Although a complex and complete tool capable of provisioning the most complex of architectures and related infrastructure, SAM is particularly useful when you want to create applications from templates, and AWS generously provides you with ready-made configurations that cover the most common use cases from a single, stand-alone Lambda, to a REST API, to a Machine Learning inference application.

To install AWS SAM, we need to install two tools: the aws-cli (your swiss-army knife for all things AWS) and sam-cli. Both are very straightforward to install, and only require minimal configuration.

The documentation for the installation of these tools on the various operating systems is available at the following addresses:

AWS CLI:

<https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>

AWS SAM:

<https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/serverless-sam-cli-install.html>

Both links contain specific installations for Windows, Linux and Mac OS.

The last step before you are good to use either tool is to configure it with your account credentials. There are two different files that can be used to configure your command line tools, the `.config` file and the `.credentials` file.

In the `.config` file we store configuration preferences such as the region where you are going to operate and the output format of the commands. In the `.credentials` file we store actual credentials needed to connect to your AWS account and perform operations on it.

The full guide on configuration is here: <https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/serverless-getting-started-set-up-credentials.html>

Bear in mind that a region, an access key and a secret key are all that is really needed to start working, the rest is entirely optional and down to preference.

If everything works as expected, you will have AWS SAM installed on your machine.

So what can SAM do for us? First of all, SAM interactively guides you through the creation of a project, by asking questions that have a multiple choice answer. AWS were feeling really helpful when they created SAM because if you just accepted all default options it all just works.

However in our case we are deploying OpenCV python applications so this blind acceptance of default options won't work, but not to worry, it is still pretty simple.

The second thing SAM takes care of is the deployment of our application. Again, if you just accepted all default choices it would work like a charm (I know, I have done it multiple times!). All you need to do is answer interactive questions about your deployment preferences.

So let's go ahead and create a simple Python project and later evolve it into a face detection application.

First of, initialize the application:

Here we told sam-cli to choose between one of the ready-made templates. We are new to SAM so we most definitely do not have custom templates of our own.

SAM proceeds to download these templates' definitions:

Once again we choose the first option because we want to start from scratch and are completely unfamiliar with Serverless. The Hello World example creates a SAM application for a single Lambda function that will perform some kind of work for us.

Use the most popular runtime and package type? (Nodejs and zip) [y/N]: N

Now we are asked about the runtime to use: we reject using the defaults (because it's Node.js packaged as a Zip) and instead choose python3.9 for our runtime...

And "Image" for the package type. Image means we are going to use a container image, so yes, we are going to put our newfound Docker knowledge to good use.

SAM subsequently creates the skeleton of the application which we are going to use as a basis for our further OpenCV work.

Let's move over to the newly created sam-opencv folder and start working from there.

The structure of the project is as follows:

```
├─ events
|   └─ event.json
├─ hello_world
|   └─ app.py
|   └─ Dockerfile
|   └─ __init__.py
|   └─ requirements.txt
```

```
├── __init__.py
├── README.md
├── template.yaml
├── tests
│   ├── __init__.py
│   └── unit
├── __init__.py
└── test_handler.py
```

4 directories, 11 files

Let's go through it. The main file that defines the resources in the project is `template.yaml`, which is a CloudFormation template.

For those unfamiliar, CloudFormation is an infrastructure management tool that allows you to declare resources in code (Infrastructure as Code), and to deploy, update and destroy infrastructure from the command line.

The `sam-cli` will take care of all of that.

The second main element of interest is the `hello-world` folder, which contains the code resources for the Lambda function. For simplicity, we will leave the function name as `HelloWorld`, but clearly in production scenarios you may want to rename this to more meaningful identifiers such as `FaceDetectionFunction`, for example.

Within the `hello-world` folder, we have 4 files:

1. `app.py` containing the actual code
2. `Dockerfile` which will be needed for the container image for the Lambda
3. The `__init__.py` is only there to make sure the current folder is treated as a module
4. The `requirements.txt` file is there because the `Dockerfile` will run `pip install` during the build phase

So let's take a look at the `app.py` script:

It does not do much but it reveals important information about how to structure our code. Global imports at the top of the file, a main function called “lambda_handler” which will perform the actual work, and since the hello-world function is a simple web request, we see that we can just return a dictionary.

With this information in mind, we move to rewrite our code to do something extremely simple but that proves we have OpenCV correctly installed and running in the cloud in a Lambda function.

Great, at this point we only want to focus on learning how to take this simple code and deploy it, nothing more.

Once this part works, adding complexity to the code will not affect the deploy process in any way.

Before moving onto building the Dockerfile, we need to create the requirements file. As I mentioned, there are a variety of ways (not least typing one manually but I would not advise to do so). I use pipreqs, you can use pip freeze or whatever tool you prefer for the task.

With the requirements.txt file created and stored in the same folder as the Dockerfile, we can go to edit the Dockerfile itself to make sure we leverage the above opencv-python container image.

There are two steps we need to take to be in a position to deploy our micro application:

1. Specify the files to be copied into the container image
2. Add the Lambda Runtime Interface Client (RIC) to our container which is needed for containers to run in Lambda functions.

NOTE: The Lambda RIC is only required for Lambda containers. Fargate containers don't need it.

The default Dockerfile does pretty much what you would expect:

Note the “app.lambda_handler” command. This means that the Lambda function's handler is contained in the “app.py” (the “app” part) and in the “lambda_handler” function of that script (the “lambda_handler” part).

We're going to vastly discard this Dockerfile as it does not do all we need it to do. The installation of the RIC isn't complex but requires that we modify the Dockerfile and also that we save a shell script that slightly modifies the container to enable the execution of the Lambda handler.

So in order, save the following small shell script into the hello-world folder as entry.sh

Then let's modify the Dockerfile with the following:

```
RUN apt-get update && \  
apt-get install -y \  
g++ \  
make \  
cmake \  
unzip \  
libcurl4-openssl-dev  
RUN pip install \  
--target ${FUNCTION_DIR} \  
awslambdaric
```

The above may seem a little confusing but if you break it down, it does only a couple of things more than a standard SAM-generated Dockerfile:

1. It installs some dependencies needed to retrieve and install the package awslambdaric (the aforementioned Lambda RIC)
2. It copies the entry.sh file we created and uses it as an entrypoint to the container. This is to enable the Runtime Interface Emulator so you can test your container locally.

You may have noticed a strange “multi-stage build” going on: this is only an optimization trick to keep the size of the image down, you can explore documentation about multi-stage builds here:

<https://docs.docker.com/develop/develop-images/multistage-build/> . For now it should suffice to say that the above file takes the content of our hello-world folder, copies into the container, adds the RIC and builds the final image.

So to build the Lambda (which is a required step before we can deploy it) we run:

Let's examine some of the output (I purposely omitted some as it was unnecessarily verbose):

Up to this point the build process is simply collecting Docker metadata from the template file and using the Dockerfile to start building the container image. At this point the package management will try to download the dependencies we are attempting to install:

Now that dependencies are installed, we can move onto copying the content:

The Lambda RIC is installed, we move on to the second step of the multi-stage build:

The entry.sh file is copied into the new container

And then used as an entrypoint:

Great, the container image is built, now we have to deploy it, and SAM is so kind as to actually show us the command for the purpose:

Let's stop for a second though. We don't want to go through the hassle of deployment, which takes a little time, without verifying that things are working properly.

You may have noted that SAM suggests you use "sam local invoke", which is a command that allows us to test the lambda locally. Naturally if our Lambda was connecting to resources it would be a little trickier to test but our code does not rely on external resources so it should work.

Let's run the local invoke:

Perfect! Our Lambda returned a message which correctly detects the version of OpenCV installed on the container.

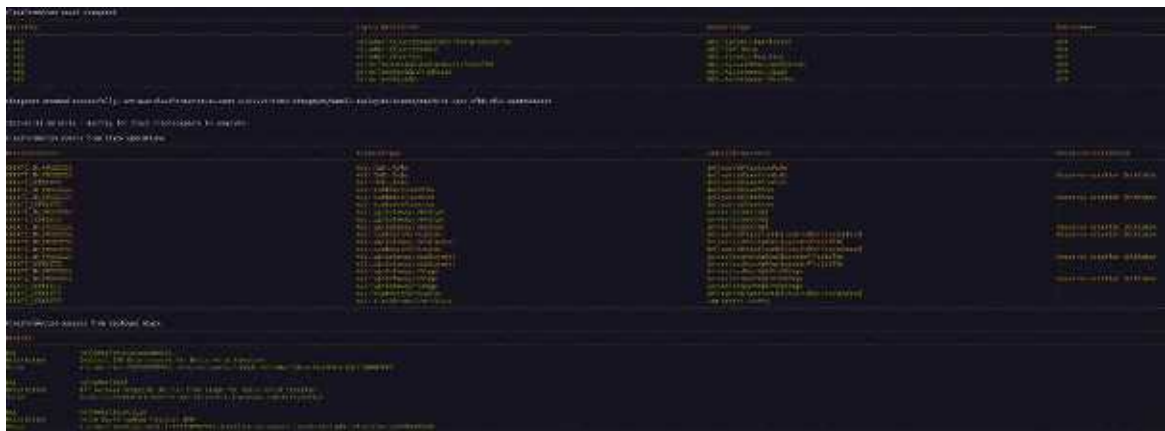
We are now ready to deploy the function, and as mentioned earlier, we will make SAM do all the hard work, by selecting the "--guided" option:

As you can see it's just a series of questions regarding deployment preferences and options. You do have to answer “y” to the question on the lack of authorization. Without it (meaning “no, you are not ok with the Lambda not having an authorization defined”), the deployment process will be halted.

I highly recommend you let SAM create all IAM Roles to execute the Lambda and the ECR repository needed to store your container images, unless you know what you are doing.

Once the resources are created, SAM will start “pushing” your container image into the ECR repository (it's just AWS's equivalent of Docker Hub), and when that's completed it will deploy the container into a Lambda function. At that point we are able to test our mini-application in production.

When SAM is done deploying you will have a list of the changes that were applied to your AWS account:



Now we want to test that the Lambda is still working correctly, even in production. You can do this in 2 ways: you can either hit the endpoint listed in the output of the deploy operation (that long URL in the first of the three output groups in the image), or you can go to your AWS web console, navigate to “Lambda”, select your newly deployed function and click on “Test”. You need to create a test event (you can just hit save on the default “hello-world” event), and then click test. The output will be exactly like the one you had locally:

Note that the “slrnph6l6j” part of the URL will be different for you as this is uniquely generated at deployment time.

Great, you now know how to create a container image that has all the necessary dependencies to run an OpenCV application on it. We will soon move to running

a slightly more complex example.

Is this really necessary?

You may be wondering if this trip down container lane is really necessary. Are we complicating our lives unnecessarily, and adding moving parts to an already complex landscape?

The answer is yes, it is necessary. Or to better quantify, it is necessary, but it's also good.

It's necessary because the Zip deployment method (so working without container images) only prevents *you (the developer)* from dealing with containers, but the underlying Lambda function will simply be a container image with your code baked into it; if you do that, the base image used by Lambda will be a base python image, which will not contain OpenCV, and as you have seen, you cannot simply install the OpenCV python library for it to work, you need the actual underlying libraries to be present. So your only choice is to use custom-built containers.

The second part of the answer is that I wholeheartedly encourage you to develop using containers, they are fun, you can learn a lot about computers and above all, they make your development lifecycle extremely reliable. Containers made deploying OpenCV applications very feasible, and now you can leverage very powerful computing resources (those made available by AWS) to run your compute intensive CV-applications. The uses you can do at an absolutely unfathomable scale of containers running OpenCV applications are simply limitless. Think real time object detection on streaming, or face recognition, or cataloging image metadata on GBs of incoming images.

So yes, it's necessary, and it's good for you.

In fact, let's see how we can use Lambda to analyze images and return coordinates for detected faces.

Example Application: Face Detection

Let's now extend our fairly useless mini-application into a function that does something. Let's take a run-of-the-mill task such as detecting faces in an image.

A short premise before proceeding: it's important that you understand that this is a web endpoint, so while you would normally draw rectangles on an image, in this case you want to just return the coordinates of the faces, so whatever client application calls your endpoint will use that information (and potentially draw rectangles on an image in the client).

As for the input image, we are not going to pass this as an actual image, but as a URL. So then the Lambda function will download the image at the provided URL, perform face detection and return the coordinates.

So let's take a look at the code:

A few points of note:

1. We need the "requests" package to operate the URL download
2. We need to include the cascade XML file and copy into the container
3. The URL is accessed through the "url" property of the "queryStringParameters" property of the event object. In practical terms, the URL will be passed into the function with the format ?url=<url of the image>
4. Lambda functions allow you to write data to the /tmp folder for a maximum of 500MB, and that's why we are using that particular location as the download location of the input image.

So let's start from the beginning and include the XML cascade file which we have used multiple times in this book so you should simply place a copy of it in the hello-world folder.

After that, let's refresh the requirements.txt file by re-running pipreqs or pip freeze so the "requests" package is included.

So the new Dockerfile really only has one edit:

And now we can re-run "sam build --use-container" to rebuild the image.

Now we can test the application. Again we can either deploy and test, or - as it is advisable - first run a local test.

This time however, we can't just run "sam local invoke" and hope it works, because the function expects a URL for the input image. So we need to create an event that conforms to the expected input. You can read how to create events at

this address: <https://docs.aws.amazon.com/serverless-application-model/latest/developerguide/sam-cli-command-reference-sam-local-generate-event.html>

However, to simplify your life I created an event for you and saved it into the project's repository at `events/awsproxy.json`

The notable part of the event's JSON is this block:

You can replace the actual url with whatever image from the internet containing faces you prefer.

Now we can run `"sam local invoke -e events/awsproxy.json"` which specifies the event to use as an input.

Let's take a look at the output:

A successful run! Now we just need to deploy the function. If you remember, the first time you ran the guided deployment, SAM saved a deployment configuration in the file `"samconfig.toml"`.

Now we can run a deployment with the same parameters by simply specifying a configuration file, and avoid re-answering the interactive questions:

This will update the Lambda which you can reach at the same url we tested above, but this time we need to remember to add the url query string parameter to obtain the identical output as the local test.

A Note on AWS Fargate

What you have learned so far in this chapter is entirely transferable to AWS Fargate, in fact, the process is simpler in that you don't need to install the Lambda RIC to turn your standard container image into a runnable workload on the cloud.

The reason why we concentrated on Lambda is that Lambdas are far the most suitable tool to perform small and specific jobs, since Lambdas have a maximum execution time of 15 minutes. Fargate can launch long-running jobs and is more suited to large and complex workloads such as batch jobs. So if you are processing one image at a time (or a small number), Lambda is the way to go. If you are tackling processing of GBs of image or video data then Fargate is more suited.

The tool to deploy jobs on AWS Fargate is called AWS Copilot and it is extremely user friendly and easy to operate. You can read about it at this address: <https://aws.amazon.com/blogs/containers/introducing-aws-copilot/> and have fun deploying containers.

Conclusion

In this chapter we familiarized with the idea of containers and created OpenCV containers that can be deployed on the cloud, then we learned about AWS Lambda which enables the deployment of OpenCV applications that can be automatically scaled, and finally we showed how to develop a face detection web service that returns face location information about an input image.

You are now ready to take your OpenCV knowledge and build massively scalable applications with it!